А. В. Фрунзе

Микроконтроллеры? Это же просто!

Том 1

МОСКВА
Издательский дом «Додэка-XXI»

УДК 621.316.544.1(075)
ББК 31.264.5я7

Ф93 **Фрунзе А. В.** Микроконтроллеры? Это же просто! Т. 1 (+CD). — М.: Издательский дом «Додэка-XXI». — 312 с.: ил.

ISBN 978-5-94120-140-2

Первая в отечественной литературе книга, написанная на доступном вплоть до начинающих уровне, охватывающая абсолютно все аспекты использования микроконтроллеров. На примере ставшего «де-факто» промышленным стандартом микроконтроллерного семейства x51 рассмотрены внутреннее устройство микроконтроллера, его система команд, схемы его сопряжения с периферийными устройствами и программы, осуществляющие это сопряжение, техника написания и трансляции ассемблера программ, анализ сообщений компилятора об ошибках, техника занесения программы в микроконтроллер и последующей отладки занесенных программ. Рассмотрен ряд полезных программ (многобайтного умножения, деления, преобразования из одного представления в другое и т. д.). Отдельно даны примеры полностью законченных разработок. Приведено большое количество обзорных и справочных материалов, охватывающих практически все микроконтроллеры, программно и аппаратно совместимые с семейством x51, существующие на момент выпуска книги.

Книга адресована специалистам в области микроконтроллерной техники, студентам, ее изучающим, а также тем, кто желает самостоятельно разобраться в этой области.

УДК 621.316.544.1(075)
ББК 31.264.5я7

ISBN 978-5-94120-140-2

© Издательский дом «Додэка-XXI»
® Серия «Программируемые системы»

Оглавление

Вступительная статья 5
Предисловие к второму изданию 7
Предисловие 9

Глава 1. **Первое знакомство** 12

1.1. Память микроконтроллера 12
1.2. Особенности включения микроконтроллера и назначение выводов 15
1.3. Двоичные и шестнадцатеричные числа 19
1.4. Краткие выводы 22

Глава 2. **Сопряжение МК с программно-управляемыми ИС** 24

2.1. Сопряжение с параллельным АЦП 24
2.2. Программа работы с параллельным АЦП 28
2.3. Ассемблер: основные понятия и приемы 34
2.4. Ассемблер: особенности трансляции 40
2.5. Ассемблер: ошибки трансляции 47
2.6. Сопряжение с последовательным АЦП 50
2.7. Краткие выводы 57

Глава 3. **Регистры микроконтроллера** 60

3.1. Регистры общего назначения и слово состояния программы 61
3.2. Аккумулятор, расширитель аккумулятора, указатель стека и подпрограммы 65
3.3. Работа МК с внешней памятью данных 70
3.4. Регистр-указатель данных 79
3.5. Пример: подпрограмма, использующая регистры МК 80
3.6. Краткие выводы 92

Глава 4. **Сопряжение МК с индикаторами различных типов** 94

4.1. Сопряжение с ЖКИ на основе контроллера HT1611 фирмы HOLTEK 95
4.2. Сопряжение со светодиодными индикаторами типа АЛС318 104

4.3. Сопряжение с ЖКИ на основе контроллера HD44780 фирмы HITACHI 115
4.4. Сопряжение с 4-разрядными светодиодными матричными индикаторами 136
4.5. Краткие выводы 163

Глава 5. **Система команд микроконтроллеров x51** 166

5.1. Общие сведения о системе команд 168
5.2. Группа команд передачи данных 172
5.3. Группа команд арифметических операций 180
5.4. Группа команд логических операций 185
5.5. Группа команд операций с битами 187
5.6. Флаги результата 190
5.7. Группа команд передачи управления 191
5.8. Краткие выводы 200

Глава 6. **Таймеры-счетчики и система прерываний МК x51** 202

6.1. Таймеры-счетчики микроконтроллеров семейства x51 202
6.2. Система прерываний микроконтроллеров семейства x51 209
6.3. Программа, использующая таймер-счетчик и прерывание 216
6.4. Краткие выводы 222

Глава 7. **Практические примеры разработки устройств на МК x51** 225

7.1. Противоугонное устройство на микроконтроллере семейства x51 225
7.2. Простой термостабилизатор на микроконтроллере AT89C2051 234
7.3. Подпрограммы целочисленного многобайтного умножения и деления 240
7.4. Милливольтметр постоянного тока на АЦП AD7894 и МК семейства x51 253
7.5. Милливольтметр постоянного тока на АЦП AD7714 и МК семейства x51 264
7.6. Краткие выводы 284

Глава 8. **Использование приемопередатчика для связи с ПК** 286

8.1. Как связывать микроконтроллер и компьютер по каналу RS-232 287
8.2. Описание микросхем 289
8.3. Режим работы микроконтроллера
с последовательным каналом 290
8.4. Основные подпрограммы для микроконтроллера 290
8.5. Общая программа для микроконтроллера. Диаграмма состояний устройства ... 293
8.6. Общая программа для ПК. Диаграмма состояний ПК 299
8.7. Основные подпрограммы для ПК 301
8.8. Пользовательское описание программы для ПК 305
8.9. Заключение 308

Литература 310

Вступительная статья

Как известно, уже набила оскомину фраза про «все возрастающий интерес инженерной общественности к применению микроконтроллеров в задачах управления и обработки сигналов». Литературы по упомянутой тематике вроде бы стало издаваться достаточно. Тем не менее, выход данной книги — событие. Почему же? Так потому, что — ПРОСТО! Лично меня трудно удивить литературной новинкой в электронной области, но когда я взял в руки первую статью из цикла, на базе которого написана эта книга, то испытал приятный шок — наконец-то на просторах 1/6 части суши появилась книга, написанная на хорошем русском языке, не лишенная чувства юмора и вместе с тем технически исключительно цельная и грамотная. В общем, это и неудивительно. Александр Виленович Фрунзе — известный и авторитетный специалист, и хочется надеяться, что книгу, которую вы сейчас держите в руках, ждет долгая и счастливая судьба. Я почти уверен, что и через 10...15 лет основные ее положения будут оставаться актуальными. Это важное отличие данной книги от великого множества изданий по микроконтроллерной тематике, которые в огромном большинстве представляют собой либо справочники, либо компиляцию с фирменных руководств по конкретным семействам.

Материал излагается на примере старого доброго 51-го контроллера. Предвижу возмущение некоторых специалистов, готовых похоронить этот контроллер из-за «неперспективности и устаревшей архитектуры и системы команд». Тем не менее, на основе этой архитектуры выпускается огромное число различных контроллеров, содержащих на борту широкую гамму переферийных устройств — от портов до АЦП, ЦАП и интерфейса CAN. Кроме того — как известно, «кадры решают все». В стране выросло не одно поколение разработчиков, активно применяющих 51 контроллеры, его изучение введено в учебные планы вузов, наконец в классическом исполнении его еще выпускает отечественная промышленность. Из личного опыта могу сказать только одно — используй ту элементную базу, которой владеешь. К сожалению, внедрение новых семейств связано с необходимостью их освоения — а это время и, конечно же, деньги.

Материал книги изложен методически исключительно грамотно. Она самодостаточна — начинающему не придется безуспешно искать в библиотеках издания, на которые обычно ссылаются при рассмотрении смежных вопросов (как то машинная арифметика, представление чисел и т. п.), в то же время опытный разработчик легко может пропустить знакомые разделы без ущерба для понимания. Изложение настолько последовательно и выверено, что книга читается как хорошее художественное произведение. Крайне разумно используются приложения — они позволяют не перегрузить основной текст и в то же время это не сухие справочные данные, а разделы, имеющие самостоятельную ценность. Тут и описание работы с микроконверторами от Analog Devices, и исследование влияния на рынок средств поддержки разработки, и пути модернизации систем — охват тем широчайший! Причем и в основном тексте, и в приложениях чувствуется авторский стиль.

Отдельно следует упомянуть о примерах из книги. Не поленился и прогнал первый попавшийся наугад — РАБОТАЕТ! Все примеры тщательно протестированы и помогут начинающему в освоении нелегкого искусства разработки систем на микроконтроллерах.

Конечно, как и в любой столь масштабной работе, не обошлось без недостатков. Практически не рассматриваются вопросы проектирования систем с несколькими параллельно работающими контроллерами.

Особо следует сказать, что эту книгу можно и нужно рекомендовать студентам. Она стимулирует самостоятельность мышления и дает представление о логике и методах проектирования, что зачастую гораздо важнее технических подробностей.

Доцент кафедры
«Автономные информационные
и управляющие системы»
МГТУ им. Н.Э.Баумана,
к.т.н. В.Б.СТЕШЕНКО

Предисловие к второму изданию

Переиздание этой книги — событие для меня, безусловно, приятное. Я не сомневался, что книга будет переиздаваться, но одно дело — мои соображения по тому или иному вопросу, а другое — события, их подтверждающие.

Второе издание практически полностью повторяет первое. Я лишь исправил несколько досадных «очепяток», имевших место преимущественно в шестой главе, да добавил абзац во второй главе в то место, где описывается, как с помощью моего ассемблера получить hex-файл (этот момент в первом издании был сформулирован нечетко, и некоторые читательские письма содержали просьбы о дополнительном разъяснении). Все остальное в книге оставлено без изменений.

С момента выхода книги я получил от ее читателей более двух сотен писем. Самыми ценными для меня являются те пять, в которых авторы просто благодарят меня за книгу, без приложения к благодарности того или иного вопроса. Ведь их писать заставила не нужда получить от меня дополнительно ту или иную подсказку, а именно благодарность.

Но и тем письмам, которые вместе с благодарностью содержали какие-либо вопросы, я был рад. Анализ этих вопросов показал, что и где мною было сформулировано недостаточно точно, и куда вкрались опечатки. Так что и эти письма тоже были важны — именно они позволили исправить упомянутые недостатки.

Помимо конкретных технических вопросов, авторы писем спрашивали и о том, почему в этом томе столь мало внимания уделено периферийным устройствам, интерфейсам и т. д. Ответ, в общем-то, прост. Главное назначение этой книги — помочь тем, кто только начинает осваивать микроконтроллерную технику. А для них подробное описание I^2C-интерфейса или аналоговой периферии новейших LPC-микроконтроллеров — это китайская грамота, излишняя на первом этапе знакомства. Формируя книгу, я старался оставить в ней лишь то, без чего научиться работать с микроконтроллерами действительно невозможно. И, судя по отзывам читателей, мне это

вполне удалось. Именно поэтому она оставлена практически без изменений.

Когда готовилось к выпуску первое издание первых двух томов книги, я еще не предполагал, что у нее будет продолжение. Однако безусловный читательский интерес к ней стимулировал работу над третьим и четвертым томами. Материалы, помещенные в этих томах, очень полезны не только для формирования правильного стиля программирования, но и для расширения кругозора разработчика.

А. В. Фрунзе, 2007 г.

Предисловие

Идея написания этой книги возникла у меня довольно неожиданно. Так сложилось, что в течение одной-двух недель сразу четверо авторов из тех, кто регулярно публикует свои материалы в журнале «СХЕМОТЕХНИКА», в разговоре со мной коснулись темы полного отсутствия литературы, по которой люди, не имеющие опыта работы с микроконтроллерами, смогли бы освоить их. Примерно в это же время я услышал подобные сетования и от двух моих знакомых инженеров-электронщиков, специалистов в аналоговой электронике — они тоже столкнулись с тем, что хотели бы освоить работу с микроконтроллерами, но не представляют, где найти литературу, рассчитанную на новичков, самостоятельно начинающих почти с нуля. Мне казалось, что подобной литературы если не навалом, то, во всяком случае, очень много, и просто нужно пару раз съездить в книжные магазины, торгующие научно-технической литературой — от обилия предлагаемых ими книг просто рябит в глазах, и не может быть, чтобы выбрать было не из чего. Но когда я сам посмотрел на появившиеся в последние несколько лет книги по микроконтроллерной тематике, а также на публикации в журналах, я понял, что практически все они ориентированы на тех, кто уже освоил эту предметную область. Статей и книг, рассчитанных на новичков, и позволяющих шаг за шагом освоить микроконтроллеры, не перегружая, раньше времени важными, но необязательными в первый момент подробностями, увы, нет. Так родилась идея написать для начинающих цикл статей по микроконтроллерам, знакомство с которым позволило бы им осознать, что такое микроконтроллеры, как они устроены, как функционируют, как писать, отлаживать и заносить в них программы, и т. д. Тем более что в свое время подобные книги были — достаточно вспомнить, например, замечательную книгу Дж. Коффрона «Технические средства микропроцессорных систем», вышедшую в 1983 году в издательстве «Мир», — с ней знакомы практически все отечественные специалисты по микроконтроллерной технике, начинавшие в 80-х с незабвенного КР580ИК80...

Первоначально я планировал просто написать цикл статей для журнала «СХЕМОТЕХНИКА», полагая, что вряд ли стоит делать книгу из материала, в основе которого лежат тысячу раз описанные микроконтроллеры разработанного еще в конце 80-х годов прошлого столетия семейства x51. Однако несколько десятков отзывов, которые я получил от читателей в ходе публикации первых частей подготовленного материала, и отклики авторов, имеющих опыт издания книг, убедили меня в том, что подобная книга может быть интересна широкому кругу читателей. Поэтому после публикации в «СХЕМОТЕХНИКЕ» первых трех глав я решил, что этот материал должен появиться в виде книги, и готовил его далее уже с учетом принятого решения.

Название настоящей книги «слизано» из серии популярных в семидесятые годы книжек, посвященных радио и телевидению, и рассчитанных, также как и настоящая, на начинающих. Любой, даже самый сложный вопрос, можно объяснить просто и доступно, и авторам тех книжек это вполне удалось. Постарался это сделать и я.

Несмотря на большой опыт написания самых разнообразных статей, задачу подготовки книги для начинающих я поставил перед собой впервые. Тем более что книг аналогичного содержания, как я уже отмечал, у нас пока еще нет. Поэтому мне было довольно сложно принимать решения, какой материал может быть опущен при первом знакомстве с микроконтроллерами, а без какого не обойтись. Кроме того, по собственному опыту я знаю, что если процесс освоения новых знаний идет успешно, то предлагаемого материала никогда не бывает достаточно, и начинаешь искать дополнительную литературу. Поскольку вследствие необходимости упрощения материала основной части книги многие особенности рассматриваемых микроконтроллеров мне пришлось опустить, я счел необходимым снабдить книгу большим количеством приложений, куда и перенес опущенное. Эти приложения выпущены отдельно (Фрунзе А. В. Микроконтроллеры? Это же просто! Т. 2. — М.: ООО «ИД СКИМЭН», 2002). Знакомство с приложениями не является необходимым при первом чтении книги, но, безусловно, окажется полезным для тех, кто решит идти дальше. При этом знакомиться с материалами приложений можно в любой последовательности, по мере понимания приведенной там информации и появления интереса к ней. Обратите также внимание и на список рекомендуемой литературы — некоторые вопросы в этих книгах освещены гораздо полнее, чем у меня, хотя в силу особенности стиля, которым они написаны, начинающим я рекомендовал бы знакомиться с этими книгами лишь после того, как они разберутся с существом рассматриваемой темы по материалам настоящей книги.

Отдельно хочу сказать о программном обеспечении к настоящей книге. Я в своей работе (пока еще) в основном использую старый DOS'овский ас-

семблер TASM. В связи с этим все, что говорится в главе 2 о процедуре ассемблирования, относится именно к этому ассемблеру. Соответственно, фрагменты программ и сами программы также написаны на этом ассемблере. Все это (TASM и программы) вы можете найти на сайте: www.pyrometer.ru .

Для тех, кто уже работает с каким-либо другим ассемблером, при использовании моих программ и подпрограмм может понадобиться некоторое редактирование исходных ассемблерных текстов с учетом особенностей их ассемблеров.

Далее необходимо отметить следующее. Все программы и их фрагменты, приведенные в главах 1—8 книги, взяты из реальных, написанных мной в разные годы программ. Однако при переносе их в книгу и некотором «причесывании» могли возникнуть ошибки. Я тщательно проверял полученные тексты, но 100-процентно гарантировать то, что в приведенных материалах ошибок нет, я все же не рискнул бы. Поэтому если вы обнаружите в моих программах какую-либо ошибку, я очень прошу сообщить об этом мне (alex.fru@mtu-net.ru). Информация об ошибках, если таковые найдутся, будет оперативно отражаться в материалах, помещенных на вышеупомянутых сайтах.

Ряд материалов, вошедших в книгу, подготовлен не мной, а другими авторами. Приложения 5 и 9 написаны Ю. Зобниным и Ш. Кобахидзе, приложение 6 — В. Мясниковым (все они — специалисты московской фирмы «Фитон»). Материал главы 8 и приложения 7 подготовлен моим сыном, Алексеем Фрунзе. Обзор микроконтроллерных семейств фирмы CYGNAL (первая часть приложения 10) подготовлен О. Николайчуком (АО InformInstrument, Кишинев, Молдова), информация по ADuC812 и ADuC816 (приложение 11) — А. Соловьевым и А. Соболевым (фирма «Аргуссофт», Москва). Все перечисленное я только отредактировал. Эти материалы включены в книгу с устного согласия авторов, и я хочу поблагодарить их за проделанную работу.

И в заключение я хотел бы отметить терпение моей жены Татьяны, с пониманием относившейся к более чем полугодовым моим вечерне-ночным бдениям над рукописью, без чего появление книги, которую вы держите сейчас в руках, вряд ли было бы возможным.

А. В. Фрунзе, 2002 г.

Глава 1. **ПЕРВОЕ ЗНАКОМСТВО**

Как уже упоминалось в предисловии, в качестве объекта изучения я выбрал микроконтроллеры *семейства x51.* Почему именно их? Во-первых потому, что мне, автору, легче объяснять материал на основе того, что я знаю лучше всего (AVR или PIC, к примеру, я знаю намного хуже). Во-вторых, все, кто разобрался хотя бы с одним контроллером, после этого всегда в состоянии самостоятельно разобраться с любым иным — было бы время и желание (или необходимость). А в-третьих, с этими контроллерами по-прежнему работает не меньше разработчиков, чем с AVR или PIC-контроллерами, не говоря уже о любых других, причем SiLabs, Atmel и Analog Devices в последнее время предоставили в наше с вами распоряжение еще более совершенные образцы контроллеров этого семейства. Да и не только они — чтобы убедиться в этом, достаточно заглянуть в приведенные в приложении (том 2) обзоры, посвященные x51-совместимым микроконтроллерам, выпускаемым более чем десятком лидеров мировой микроэлектроники. После знакомства с ними становится очевидно, что слухи о кончине славного 51-го семейства оказались явно преувеличенными, и еще добрый десяток лет эти изделия будут вполне конкурентоспособными в семействе наиболее распространенных 8-битных микроконтроллеров. Так что отбросим снобизм и начнем наше знакомство со старым и добрым семейством x51.

1.1. Память микроконтроллера

Первое, с чего стоит начать — это со слов, что при всей кажущейся сложности ничего непостижимого в микроконтроллерах нет. Микроконтроллеры представляют собой микросхемы, которые всего лишь скрупулезно выполняют программы, записанные в них программистами. Последние, зная, что из себя представляет микроконтроллер, какие команды он может выполнить, составили и отладили программы (последовательность этих самых команд) и записали их в микроконтроллеры, которые при подаче питания выполняют все то, что было предусмотрено программистами.

Наверное, у вас уже возникли вопросы. Например: микроконтроллер может выполнять какие-то команды. Какие? Далее: программа пишется программистом и заносится в микроконтроллер. Как? А заодно, где она там хранится?

Начнем с последнего. У большинства микроконтроллеров имеется *память программ*, представляющая из себя некоторое количество ячеек (от тысячи до десятка тысяч и более). Она находится внутри самой микросхемы (говорят — «на кристалле»). Каждая ячейка имеет свой порядковый номер, или, как говорят программисты, адрес. В этих ячейках хранятся числа, значения которых могут изменяться от 0 до 255. Совокупность этих чисел и есть та самая программа, которую выполняет микроконтроллер, когда мы подадим на него питание.

Удивлены? На самом деле все довольно просто, хотя на первый взгляд может показаться непривычным. Программа — это последовательность выполняемых микроконтроллером команд. Каждой команде в памяти программ соответствует свое число (корректнее сказать — код). При включении питания микроконтроллер один за другим считывает эти коды, осуществляет их дешифрацию (другими словами, определяет, что же нужно сделать), а затем исполняет одну за другой эти дешифрированные команды. Кстати, отмечу, что в контроллерах семейства x51 первой выполняется команда, код которой расположен в самой первой ячейке памяти программ с адресом 0.

Главная особенность памяти программ — занесенные в нее коды сохраняются неизменными при отсутствии питания микроконтроллера. Действительно — уж коль скоро мы написали и отладили программу, она не должна самопроизвольно изменяться с течением времени.

Память программ — это не единственный вид памяти, имеющийся внутри микроконтроллера. Любой микроконтроллер имеет еще *память данных*. Ее принципиальное отличие от памяти программ состоит в том, что микроконтроллер может не только читать содержимое ее ячеек, но и определенными своими командами содержимое их изменять (записывать в них данные), в то время как менять содержимое памяти программ ему «не по зубам». Память данных еще иногда называют оперативной памятью (оперативным запоминающим устройством или ОЗУ), в отличие от памяти программ, именуемой постоянным запоминающим устройством (ПЗУ).

Записанные в ОЗУ коды теряются (т. е. изменяются произвольным образом) при выключении питания.

Помимо ОЗУ и ПЗУ, микроконтроллер содержит еще один вид встроенной памяти, без знакомства с которым мы не сможем двигаться дальше — это так называемая *регистровая память* или регистры. Они представляют из себя ячейки оперативной памяти, обращение к которым контроллер осуществляет более короткими и быстровыполнимыми командами, чем к

ОЗУ. В остальном же регистры и ячейки оперативной памяти идентичны — содержимое их теряется при выключении питания и может быть считано или изменено при выполнении микроконтроллером некоторых из своих команд. Более подробно о регистрах (их названиях, адресах, командах обращения к ним) мы будем говорить в одной из следующих глав.

Для пояснения всего сказанного приведу следующий пример. На **Рис. 1.1** показано содержимое последовательно расположенных восьми ячеек памяти программ микроконтроллера с занесенным в него фрагментом пользовательской программы.

Рис. 1.1. Пример содержимого последовательно расположенных восьми ячеек памяти программ микроконтроллера

На первый взгляд, это сущая бессмыслица. Но, на самом деле, это — вполне разумный фрагмент реальной программы. Код 238 предписывает микроконтроллеру перенести (говорят «прочитать») число из регистра R6 в главный регистр, называемый аккумулятором. Код 36 предписывает контроллеру прибавить к содержимому аккумулятора число, размещенное в памяти программ непосредственно за этим кодом (т. е. в данном случае — единицу). Код 254 предписывает вернуть полученное в результате суммирования число из регистра-аккумулятора в регистр R6. Следующий код (239) вынуждает микроконтроллер прочитать в аккумулятор число из регистра R7. Код 52 — прибавить к содержимому аккумулятора число, расположенное в программной памяти за этим кодом (т. е. 0), и еще так называемый бит переноса. (Для любопытных — бит переноса равен 1, если предыдущее суммирование дало результат больше или равный 256, и 0 — если меньше; вспомните единичку, которая «идет на ум» при суммировании на бумажке «столбиком», когда сумма оказывается больше 9. Бит переноса — это и есть та самая единичка.) Последний код — 255 — предписывает вернуть число из аккумулятора в регистр R7.

Не будем обсуждать, зачем нужно выполнять указанную последовательность команд — поверьте на слово, иногда нужно. Смысл приведенного

фрагмента — показать, что между числами в памяти программ и конкретными командами есть взаимно однозначное соответствие. Программы, которые пишутся пользователями, переводятся в коды при помощи важной и нужной программы, именуемой *транслятором с языка ассемблера* или просто *ассемблером* (далее мы с ним обязательно познакомимся). После этого полученные коды заносятся в память программ микроконтроллера. Для этой цели служат специально выпускаемые рядом фирм устройства, именуемые *программаторами*. Микроконтроллер вставляется в панельку программатора (только предварительно нужно убедиться, что используемый программатор рассчитан на программирование вашего контроллера), запускается соответствующая программа, в которой указывается тип контроллера и имя файла, в котором находятся коды вашей программы, и программатор заносит их в микроконтроллер. Если, конечно, последний исправен...

Кстати, существуют программы, называемые *дизассемблерами*, которые осуществляют обратную операцию — переводят коды программы в понятные человеку команды микроконтроллера.

1.2. Особенности включения микроконтроллера и назначение выводов

Оставим теперь на время внутреннее устройство микроконтроллера и обратимся к его внешним цепям. Стандартные микроконтроллеры семейства x51 выпускаются в 40-выводных DIP-корпусах с расстоянием между рядами выводов 15 мм, а между самими выводами — 2.54 мм. Их цоколевка и стандартная схема включения приведена на **Рис. 1.2**.

Вывод 20 — GND (ЗЕМЛЯ). Он, очевидно, соединяется с общим проводом. Вывод 40 (V_{CC}) соединяется с шиной питания (+3...5 В). К выводам 18 (XTAL2) и 19 (XTAL1) подключается кварцевый резонатор. Наиболее часто используемые кварцы — на 11.0592 МГц и 12 МГц, хотя на практике микроконтроллеры семейства x51 работают с кварцами и с более низкими частотами (например, 1 МГц), и с более высокими (я встречал микроконтроллер x51 от Philips, работающий на 40 МГц). Для более стабильного запуска выводы кварцевого резонатора соединены с общим проводом через конденсаторы C1 и C2 емкостью от 15 до 30 пФ.

Кстати, в англоязычной литературе выводы микросхем называются пинами (от английского слова *pin* — булавка, шпилька, штифт).

Вывод 9 — это вход RESET или СБРОС. Единичный уровень на этом входе в течение нескольких десятков периодов тактового генератора приводит к сбросу в начальное состояние регистров микроконтроллера и к началу исполнения программы с нулевого адреса. Сброс обязателен при по-

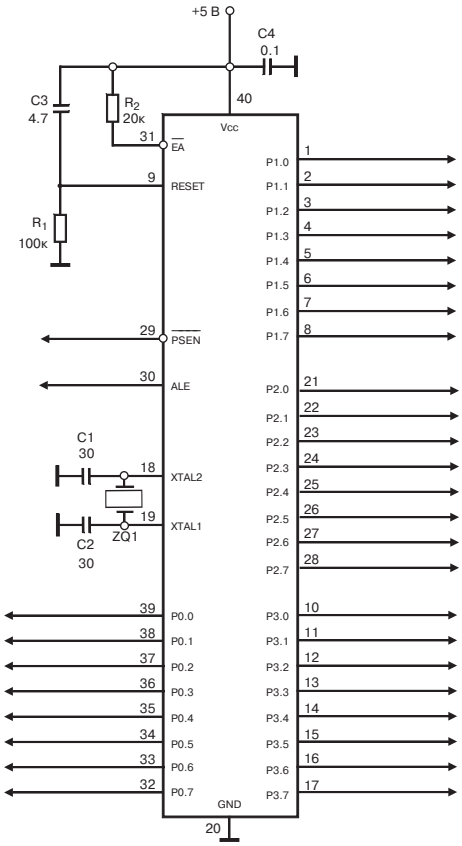


Рис. 1.2. Цоколевка и стандартная схема включения микроконтроллера семейства x51

даче напряжения питания на микроконтроллер. С этой целью вход RESET соединяют с шиной питания через конденсатор C3 емкостью несколько микрофарад, и с общим проводом — через резистор R1 сопротивлением порядка сотни килоом. В момент включения питания конденсатор разряжен, и вход сброса оказывается под потенциалом, близким к напряжению питания. Несмотря на снижение этого потенциала вследствие заряда C3, в течение нескольких десятков миллисекунд уровень сигнала на входе сбро-

са остается единичным, и осуществляется корректный запуск микроконтроллера.

Как-то раз в одной из моих плат вследствие ошибки в разводке вход сброса оказался «висящим в воздухе» — упомянутая *RC*-цепь была соединена с соседним выводом. В результате мне понадобилось несколько дней для того, чтобы понять, почему при включении питания контроллер то запускался нормально, то «зависал», не подавая никаких признаков жизни. Я запрограммировал второй контроллер, вставил вместо первого — то же самое, разве что после этого один нормальный старт стал приходиться не на четыре «зависания», а на три. В общем, если микроконтроллер с отлаженной программой то нормально стартует, то ведет себя кое-как, начните с проверки цепи сброса.

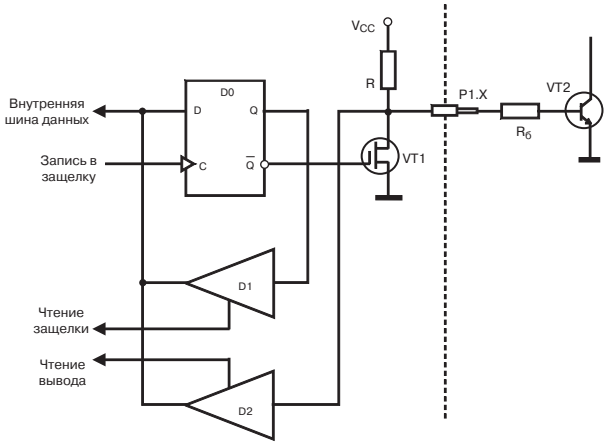
Следующий важный вход — $\overline{EA}$, вывод 31. Если на него подана логическая единица, то микроконтроллер работает с уже упоминавшейся памятью программ, расположенной на кристалле. Нуль на входе заставит микроконтроллер выполнять программу из внешней памяти (такое возможно). О том, как организовывается связь между микроконтроллером и дополнительной микросхемой, содержащей эту внешнюю память, мы расскажем в одной из следующих глав. На первых же порах мы будем работать только с памятью программ на кристалле, поэтому на входе должна быть установлена логическая единица. Избегайте плавающего потенциала на этом входе — если он окажется «висящим в воздухе», контроллер будет работать нестабильно, постоянно сбоить и «зависать».

На выводе 30 (ALE) обычно присутствует непрерывная последовательность прямоугольных импульсов с частотой, в 6 раз ниже, чем у кварцевого резонатора, соединенного с выводами 18 и 19. Для 12-мегагерцового кварца она, очевидно, составит 2 МГц. В этой последовательности длительность единицы на выводе ALE примерно вдвое меньше длительности нуля, т. е. скважность составляет 33%. Этот сигнал можно использовать для тактирования микросхем, требующих для работы внешний источник тактового сигнала.

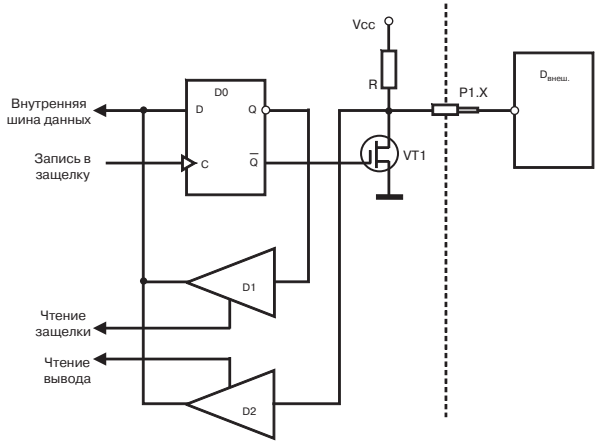
Назначение вывода 29 ($\overline{PSEN}$) будет рассмотрено в разделе, где мы будем говорить о подключении к микроконтроллеру внешней памяти.

Оставшиеся 32 вывода — это линии ввода/вывода информации. Они сгруппированы по 8 в четыре так называемых порта ввода/вывода (P0, P1, P2 и P3). Каждая линия любого из них может использоваться либо как вход, либо как выход, независимо от использования остальных линий. Для этого их оконечные каскады выполнены соответствующим образом. На **Рис. 1.3** приведена упрощенная схема одной из линий ввода/вывода порта P1.

Как видно из **Рис. 1.3, а**, вывод микросхемы P1.x соединен со стоком выходного полевого транзистора VT1, «подтянутого» к потенциалу питания при помощи внутреннего нагрузочного резистора R. С этим же выво-



а)



б)

Рис. 1.3. Упрощенная схема одной из линий ввода/вывода порта P1

дом микросхемы соединен вход буфера ввода D2. Если мы присоединим к этому выводу микросхемы через резистор $R_б$ базу внешнего транзистора VT2, то, занося в триггер-защелку D0 логические 1 или 0, мы будем открывать или закрывать VT2, используя, таким образом, выбранную линию в качестве линии вывода информации.

Использование линии в качестве линии ввода информации иллюстрируется **Рис. 1.3, б**. Вывод микроконтроллера, а, следовательно, и вход буфера D2, соединены с выходом микросхемы $D_{внеш}$, состояние которого мы хотим проанализировать (иными словами, «ввести» его в микроконтроллер, или «прочитать»). Но прежде, чем читать содержимое буфера D2, необходимо закрыть транзистор VT1, записав в триггер D0 этой линии единицу. В самом деле, если VT1 будет открыт, он попросту шунтирует анализируемый выход микросхемы $D_{внеш}$. В лучшем случае, этот конфликт на выводе просто исказит вводимую информацию, когда выход микросхемы $D_{внеш}$ будет «тянуть» потенциал вверх, а выход VT1 — вниз. В худшем же варианте, победа сильнейшей из противоборствующих сторон приведет к сгоранию слабейшей. Так что запомним, что если какие-то линии порта мы собираемся использовать в качестве линий ввода, то перед этим обязательно в соответствующие выходные триггеры нужно записать единицы.

По схеме, приведенной на **Рис. 1.3**, выполнены линии портов P1, P2 и P3. Порт P0 оформлен несколько иначе — сток его транзистора VT1 вместо обычного нагрузочного резистора соединен с динамической нагрузкой (источником тока). Это сделано для того, чтобы линии порта P0 при занесении в их триггеры-защелки единичек оказывались в так называемом высокоимпедансном («сером») состоянии, характеризующимся очень высоким выходным сопротивлением. В остальном же функционирование линий порта P0 похоже на работу линий остальных трех портов.

1.3. Двоичные и шестнадцатеричные числа

Приведенной выше информации по цоколевке и назначению выводов вполне достаточно для рассмотрения первых примеров. Однако прежде, чем перейти к ним, нам необходимо познакомиться с некоторыми особенностями записи программ и чисел.

При написании программ нам с вами придется пользоваться так называемыми двоичным и шестнадцатеричным представлением чисел. Поначалу привыкнуть к ним довольно трудно. Но после привыкания числа в двоичном и шестнадцатеричном представлении дают вам при анализе и написании программ гораздо больше информации, чем привычные десятичные числа.

В двоичном представлении числа записываются при помощи всего двух цифр — 0 и 1. ***Числам 0 и 1*** в двоичном представлении соответствуют, как и обычно, ***цифры 0 и 1***. А вот с двойкой уже иначе — ***число 2*** в двоичной системе записывается как ***10b*** (буква b на конце служит признаком того, что число записано в двоичном представлении; просто 10 без буквы на конце или 10d — это десять, а 10b — это двойка).

Почему двойка в двоичном представлении записывается таким образом? Да вот почему. В обычной арифметике для первых десяти чисел (от нуля до девяти) есть десять цифр — 0, 1, 2, ..., 9. Для следующего числа, десятки, самостоятельной цифры уже нет. Поэтому для нее мы снова используем самую младшую цифру 0, но ставим слева перед ней цифру 1 — число становится двузначным. Так и с двойкой в двоичном представлении — на нуль и единицу есть свои цифры, а на двойку уже нет (напомню, есть только цифры 0 и1). Поэтому, как и десятка в обычном представлении, двойка в двоичном представлении записывается при помощи младшей из двух возможных цифр— 0, слева перед которым ставим 1.

Тройка в двоичной системе представляется как 11b — это очевидно, 11 — это число, на 1 больше, чем 10. А вот число четыре двумя цифрами в двоичной системе не представить. Действительно, 00b (или 0b) соответствует нулю, 01b (или 1b) соответствует единице, 10b соответствует двойке, 11b — тройке, а пятого двузначного числа, используя только цифры 0 и 1, записать нельзя. Как быть? Элементарно. Коль скоро все двузначные числа кончились, четверка будет трехзначной, причем правые две цифры должны быть нулями, а крайняя слева — 1. Т. е. четыре в двоичном представлении — это 100b, пять — соответственно 101b, шесть — 110b, семь — 111b. Для восьмерки и трех цифр уже не хватает— значит, она должна быть четырехзначной, три правых цифры — нули, крайняя слева какая? Правильно, 1. Итого, восемь — это 1000b.

Надеюсь, принцип представления чисел в двоичной системе я вам объяснил. Если нет — вам придется обратиться к примерам, приведенным в приложении 1 (см. том 2). Ну, а чтобы легко переводить числа из двоичного представления в десятичное и наоборот, проще всего использовать стандартную программу Калькулятор из Windows (кнопка «Пуск» — меню «Программы» — меню «Стандартные» — «Калькулятор»). Запустив «Калькулятор», щелкните мышью заголовок меню «Вид», выберите «Инженерный». Слева над цифрами вы увидите форму представления числа — Hex (шестнадцатеричное, это у нас впереди), Dec (обычное десятичное, оно всегда выбрано при первом запуске программы Калькулятор), Oct (восьмеричное, мы им не будем пользоваться) и Bin — двоичное или бинарное (отсюда и буква b на конце обозначения двоичных цифр). Выбрав Dec — десятичное и набрав 8, кликните мышкой на Bin и вы увидите на экране 1000 (не забудьте, что это — 1000 в системе Bin, т.е. 1000b). Не выходя из

системы Bin, наберите 11000011 и перейдите в Dec — вы увидите на экране 195, т. е. числу 11000011 в двоичной системе соответствует 195 в привычной нам десятичной. И так далее...

В шестнадцатеричном представлении числа записываются при помощи 16 цифр. Десять из них вам хорошо знакомы — 0, 1, 2, ..., 9. В качестве остальных шести цифр используют (не удивляйтесь!) буквы A, B, C, D, E, F. То есть ***для числа десять*** есть своя ***цифра (A)***, для числа ***одиннадцать — цифра B***, для числа ***двенадцать — цифра C***, для числа ***тринадцать — цифра D***. Числам ***четырнадцать*** и ***пятнадцать*** соответствуют, как вы уже догадались, ***цифры E*** и ***F***. А вот для числа шестнадцать цифры уже нет, поэтому его, как и двойку в двоичном представлении, запишем в виде младшей цифры — 0, перед которой поставим цифру 1: таким образом, число ***шестнадцать*** — это ***10h***. Буква h в конце обозначает, что число записано в шестнадцатеричной системе счисления (Hex). Переводить числа из нее в десятичную систему и обратно можно с помощью все того же Windows-Калькулятора. Например, 112Bh — это 4395 десятичное, а 8190 десятичное — это 1FFEh (не поленитесь, проверьте!).

Зачем все это нужно? Так, память программ микроконтроллера AT89C1051 содержит 1024 ячейки, AT89C2051 содержит 2048 ячеек, AT89C51 — 4096 ячеек, а AT89C52 — 8192 ячейки. В шестнадцатеричном представлении это будет соответственно 400h, 800h, 1000h, 2000h. Для процессора 8086, на котором собирались первые IBM-PC, допустимо использование памяти объемом 1048576 ячеек или 100000h. Согласитесь, что в шестнадцатеричном представлении приведенные цифры легче запоминаемы.

А теперь вернемся чуть-чуть назад, к фрагменту памяти программ, который мы анализировали в первом разделе. Обычно адреса ячеек и коды команд представляют именно в шестнадцатеричной, а не в десятичной системе счисления. В Hex-представлении этот фрагмент будет выглядеть так, как показано на **Рис. 1.4**.

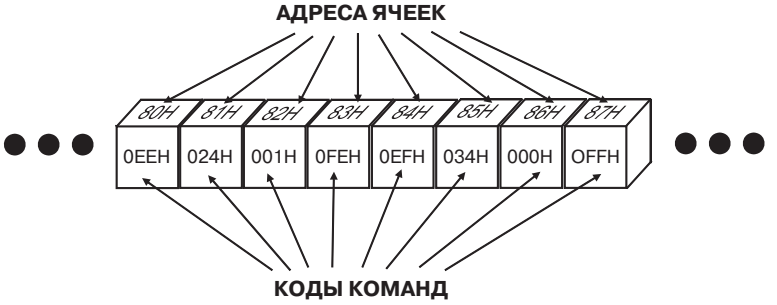


Рис. 1.4. Пример содержимого последовательно расположенных восьми ячеек памяти программ микроконтроллера

На самом деле это ровно то же самое, что было приведено на **Рис. 1.1** (проверьте при помощи Windows-Калькулятора). И хотя подобная запись поначалу выглядит не так привычно, как на **Рис. 1.1**, нам придется привыкать к таким записям. Дело в том, что программа ассемблер, о которой мы раньше уже упоминали и которая преобразовывает написанные нами команды в понятные микроконтроллеру коды, все «результаты своей деятельности» выводит в соответствующие файлы именно в шестнадцатеричном представлении.

1.4. Краткие выводы

Итак, подведем первые итоги. Микроконтроллеры представляют собой микросхемы, которые всего-навсего «от корки до корки» выполняют программы, записанные в них программистами. Последние, зная, что из себя представляет микроконтроллер, какие команды и каким образом он может выполнить, составили и отладили программы (последовательность этих команд), оттранслировали их при помощи ассемблера и записали их в микроконтроллеры при помощи специальных программаторов. Такой программатор вы можете либо позаимствовать у кого-то из знакомых, либо приобрести в одной из фирм, реклама которых размещена в журнале «Компоненты и технологии». Стоить он может от 15...20 до 200...400 у.е., в зависимости от его универсальности. Ни в коем случае не приобретайте программатор, не убедившись в том, что он «прошивает» интересующие нас микроконтроллеры (в частности, фирмы Atmel семейства AT89 — AT89C1051, AT89C2051, AT89C51, AT89C52; они самые доступные и дешевые). Для начала, конечно, лучше не покупать, а одолжить программатор у кого-либо из знакомых. Эти люди к тому же смогут вам что-то подсказать и помогут совершить первые шаги в освоении микроконтроллеров, что очень важно для начинающих.

У большинства микроконтроллеров память программ (ПЗУ) находится на кристалле и состоит из нескольких тысяч ячеек. Содержимое их не изменяется при включении/выключении питания. В каждой ячейке хранятся коды команд. Они представляют собой числа от 0 до 255 (в двоичной системе — от 00000000b до 11111111b). Каждой команде соответствует свое число (корректнее сказать — код). При включении питания микроконтроллер один за другим читает эти коды, осуществляет их дешифрацию, а затем выполняет дешифрованные команды. Первой выполняется команда, код которой расположен в самой первой ячейке памяти программ, имеющей нулевой адрес.

Помимо памяти программ, на кристалле расположена и память данных (ОЗУ). В отличие от ПЗУ, при выключении питания содержимое ее теряется (искажается). Информация из памяти данных может быть не только

прочитана микроконтроллером, но и изменена (т. е. осуществлена запись данных в ОЗУ).

К некоторым ячейкам ОЗУ микроконтроллер может обращаться (читать и записывать информацию) при помощи коротких быстро выполняющихся команд. Такие ячейки называются регистрами. При выключении питания содержимое их также теряется.

Стандартные микроконтроллеры семейства x51 выпускаются преимущественно в 40-выводных DIP-корпусах. К выводам 18 и 19 обычно подсоединяют кварцевый резонатор с частотой от 1 до 12 МГц. Для корректного запуска микроконтроллера при включении питания вход сброса 9 через конденсатор соединяют с шиной питания, а через резистор — с общим проводом. С шиной питания через 20-килоомный резистор нужно соединить также вывод 31.

32 вывода микроконтроллера организованы в четыре 8-битных порта ввода-вывода. Все линии одного и того же порта в схемотехническом исполнении совершенно идентичны. В схемотехнике разных портов есть небольшие различия. Каждая линия каждого порта независимо от остальных может работать как на вывод информации (как выход, устанавливающийся в нулевой или в единичный уровень), так и на ввод (как вход). В последнем случае при запуске микроконтроллера написанная вами программа должна занести в выходной триггер этой линии единичку, иначе возможен конфликт на линии.

На этом первое знакомство с микроконтроллером будем считать завершенным. Оно ужасающе поверхностно — приведенная выше информация, наверное, составляет лишь один-два процента от того, что знает профессионал среднего уровня. Но и этого уже достаточно, чтобы начать знакомиться с тем, как на практике работают схемы с микроконтроллерами, как пишутся программы, транслируются, заносятся в микроконтроллеры, отлаживаются. На конкретных примерах мы посмотрим, как заставить микроконтроллер работать с реальным АЦП или как организовать вывод информации из микроконтроллера на семисегментный индикатор. Этому будут посвящены следующие три главы. Разобравшись в них, вы сможете сделать первые шаги к самостоятельной разработке системы на микроконтроллере.

Глава 2. СОПРЯЖЕНИЕ МК С ПРОГРАММНО-УПРАВЛЯЕМЫМИ ИС

Подобно тому, как нельзя научиться плавать без практических уроков в бассейне, также невозможно освоить работу с микроконтроллерами без разбора практических примеров реализации тех или иных устройств. В настоящей главе мы с вами рассмотрим, как нужно аппаратно сопрягать микроконтроллеры с микросхемами, имеющими, как иногда принято говорить, интерфейс, ориентированный на микроконтроллеры. Также нам с вами предстоит составить программы, которые «оживят» эти схемы. Параллельно с этим мы начнем знакомиться с командами, входящими в эти программы. Естественно, мы ближе познакомимся с конкретной версией программы ассемблер и с его помощью оттранслируем написанные программы. В результате этого у вас должны появиться некие минимальные навыки написания и трансляции программ. Кроме того, вы будете располагать некоторыми конкретными и полностью понятными вам программами. На этой базе вы сможете приступить к написанию самостоятельных программ, пусть для начала и примитивных.

2.1. Сопряжение с параллельным АЦП

Одна из микросхем, наиболее часто используемых совместно с микроконтроллером — это АЦП (аналого-цифровой преобразователь). Ничего удивительного в этом нет — прежде чем как-то обработать и отобразить какой-то результат, его нужно ввести в микроконтроллер в цифровой форме.

В настоящее время АЦП производят десятки фирм. Они различаются по принципу преобразования, быстродействию, разрешению, точностным параметрам, питающим напряжениям, диапазонам входных напряжений, количеству каналов — список этих параметров может быть продолжен еще на десяток строк. Однако из всего их многообразия в данный момент нас интересуют лишь те, которые имеют отношение к связи АЦП с микроконтроллером. Вот их-то не так уж и много.

Во-первых, с точки зрения интерфейса АЦП делятся на параллельные и последовательные. Первые после преобразования передают микроконтроллеру все биты результата одновременно, каждый по своей индивидуальной линии. Это означает, что с 12-битным АЦП микроконтроллер должен быть связан минимум 12-ю проводниками (реально — на 3...5 больше упомянутого числа за счет сигналов управления).

Последовательные АЦП связаны с микроконтроллером всего тремя-четырьмя проводниками, независимо от их разрешения (разрядности). Биты результата оцифровки они передают по одному проводнику, друг за другом (последовательно). Управление передачей микроконтроллер осуществляет по второму проводнику, третий, как правило, дает АЦП команду на начало преобразования. Ясно, что последовательные АЦП работают медленнее параллельных, но достигнутые ими предельные скорости преобразования и передачи информации достаточно высоки (многим 12...14-битным АЦП требуется менее 10 мкс на весь цикл преобразования/передачи данных).

Далее, АЦП могут содержать некоторые внутренние регистры (ячейки памяти размером от 4 до 24 бит), в которые микроконтроллер должен предварительно занести информацию. К таким АЦП относятся, к примеру, многоканальные преобразователи — микроконтроллер должен сообщить АЦП, какой из каналов последнего должен преобразовывать информацию. При работе с такими микросхемами микроконтроллер не только читает информацию, но и записывает ее в АЦП при помощи соответствующих сигналов. Подобные преобразователи, естественно, сложнее простых, не требующих записи в них управляющих слов (заносимую микроконтроллером в подобные микросхемы информацию программисты обычно называют управляющими словами), поэтому на первом этапе мы исключим подобные сложные микросхемы из нашего рассмотрения и вернемся к ним попозже.

Остальные различия не столь принципиальны — какими сигналами управляется АЦП, какова полярность этих сигналов, каковы привязки к их фронтам и спадам, задержки и т. д. Эту информацию, имеющуюся в datasheet'е на конкретную микросхему, вы должны держать перед собой при разработке аппаратного сопряжения и программы для микроконтроллера.

Перейдем теперь к конкретному примеру, в качестве которого у меня заготовлено сопряжение микроконтроллера с 12-битным параллельным АЦП AD7880 фирмы Analog Devices (**Рис. 2.1**). В середине 90-х годов 20-го столетия это был чуть ли не единственный АЦП, требовавший всего одно питающее напряжение (5 В) и потреблявший при этом относительно небольшой ток (< 10 мА). Почему я выбрал именно этот АЦП? Да просто когда-то я с ним работал, и программы, которые мы с вами будем анализировать, были тогда отлажены и не содержат ошибок.

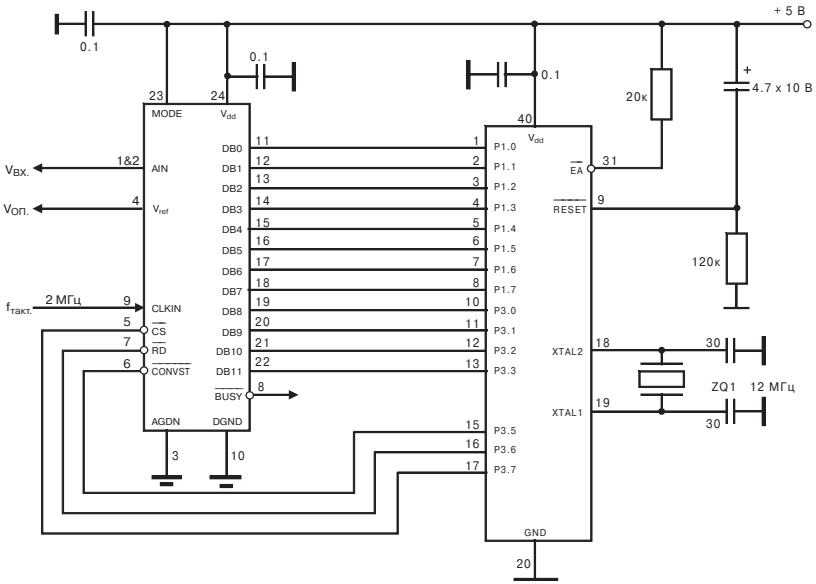


Рис. 2.1. Сопряжение микроконтроллера с 12-битным параллельным АЦП AD7880

Коль скоро АЦП параллельный 12-битный, с микроконтроллером он должен быть соединен 12 линиями данных, по которым в последний будут одновременно переданы все 12 бит результата. Добавлю, что для организации обмена АЦП AD7880 использует еще 4 сигнала управления — $\overline{\text{CONVST}}$, $\overline{\text{CS}}$, $\overline{\text{RD}}$ (входы АЦП) и $\overline{\text{BUSY}}$ (выход; кстати, его можно и не использовать, в чем мы убедимся чуть ниже). Итого, контроллер с АЦП должны соединяться 15-ю или 16-ю проводниками. Кроме того, на вход CLKIN АЦП нужно подать тактовую последовательность с частотой до 2.5 МГц и отношением длительности единичного уровня к длительности нулевого в пределах от 0.4 : 0.6 до 0.6 : 0.4. К сожалению, хотя рассмотренный нами ранее сигнал ALE микроконтроллера имеет подходящую для данного случая частоту (в 6 раз меньшую, чем частота кварцевого резонатора, т. е. 2 МГц для 12-мегагерцового кварца), скважность этого сигнала составляет 33%. Поэтому на вход CLKIN АЦП нужно подать 1...2-мегагерцовую тактовую последовательность с отдельного генератора.

Как отмечалось, АЦП с микроконтроллером должны связывать 15 или 16 проводников. Фактически мы будем вынуждены целиком задействовать два из четырех 8-битных порта ввода/вывода микроконтроллера. В

принципе мы можем использовать для этого любые линии любых портов, но удобнее взять линии портов P1 и P2 или P1 и P3. Как видно из **Рис. 2.1**, я остановился на втором варианте, поскольку в имеющихся у меня аппаратных средствах отладки (об этом — позже) порты P0 и P2 заняты — они используются для связи с компьютером.

Работает AD7880 следующим образом. В начальный момент времени на всех выводах, соединенных со входами управления АЦП, микроконтроллер должен установить единицы. Для запуска преобразования на $\overline{\text{CONVST}}$ необходимо подать отрицательный импульс. Перепад на нем из 0 в 1 при единичных уровнях сигналов на $\overline{\text{CS}}$ и $\overline{\text{RD}}$ запускает цикл преобразования. В течение примерно 20 мкс АЦП (при тактовой частоте 2 МГц) оцифровывает сигнал и заносит его в свой выходной регистр. В это время он удерживает нулевой уровень на своем выходе $\overline{\text{BUSY}}$, и по состоянию этого сигнала наш микроконтроллер может определить, завершил ли АЦП преобразование, или нет. Кстати, если мы по каким-либо причинам не можем анализировать сигнал $\overline{\text{BUSY}}$, можно просто отсчитать 20 мкс после подачи сигнала старта преобразования — за это время оно завершится.

После завершения преобразования, когда $\overline{\text{BUSY}}$ вернется в 1, данные могут быть считаны из АЦП. С этой целью микроконтроллер должен установить в 0 сначала сигнал на входе $\overline{\text{CS}}$, а затем — на $\overline{\text{RD}}$. Примерно через 75 нс после установки $\overline{\text{RD}}$ в 0 на выходах данных DB0...DB11 АЦП появится результат преобразования. Микроконтроллер должен считать его, после чего вернуть вначале $\overline{\text{RD}}$, а затем $\overline{\text{CS}}$, в исходное единичное состояние, и на этом цикл завершается. Сказанное иллюстрируется **Рис. 2.2.**

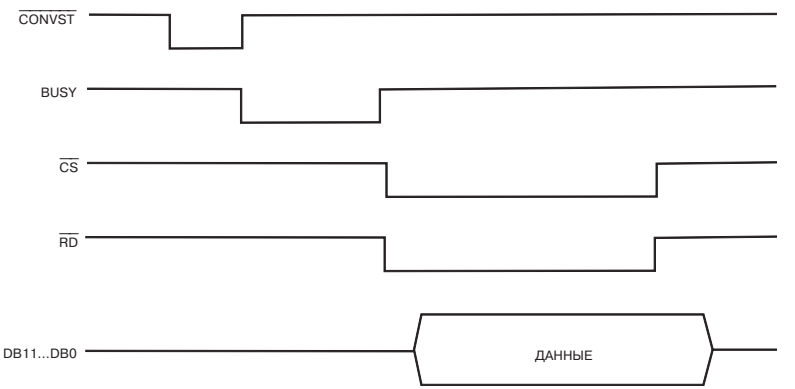


Рис. 2.2. Сигналы обмена микроконтроллера с 12-битным параллельным АЦП AD7880

2.2. Программа работы с параллельным АЦП

Итак, схема соединения АЦП AD7880 с микроконтроллером составлена и описан алгоритм работы. Теперь самое время перейти к рассмотрению программы, которая и реализует описанный алгоритм.

Программа работы микроконтроллера с параллельным АЦП AD7880

```
; НАЧАЛО ПРОГРАММЫ ЧТЕНИЯ АЦП AD7880,
; РАБОТАЕМ С ПОРТАМИ P1 И P3, CS = P3.7,
; RD = P3.6 , CONVST = P3.5.
;                          *****
CS      .EQU   P3.7
RD      .EQU   P3.6
CONVST  .EQU   P3.5
;
        MOV    P1,#11111111B;НАЧАЛЬНАЯ УСТАНОВКА
        MOV    P3,#11111111B
;
L7880:                     ;СОБСТВЕННО ЧТЕНИЕ
;
        CLR    CONVST      ;ИМПУЛЬС СТАРТА ПРЕОБРАЗОВАНИЯ
        SETB   CONVST
;
        NOP                ;ЗАДЕРЖКА НА ВРЕМЯ ПРЕОБРАЗОВАНИЯ
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
;
        CLR    CS          ;CS=0
        CLR    RD          ;RD=0
;
        MOV    A,P1        ;ЧИТАЕМ ИЗ ПОРТА P1 МЛ.И СР. ТЕТРАДЫ
        MOV    R4,A        ;СОХРАНЯЕМ ИХ В R4
```

```
;
        MOV    A,P3        ;ЧИТАЕМ ИЗ ПОРТА P3 СТ. ТЕТРАДУ
        MOV    R5,A        ;В R5R4 - РЕЗУЛЬТАТ
;
        SETB   RD          ;УСТАНОВКА RD В 1
        SETB   CS          ;УСТАНОВКА CS В 1
;
        SJMP   L7880       ;ЗАЦИКЛИВАНИЕ
;
```

Давайте разберемся с тем, что в ней присутствует. Первое, на что я рекомендую обратить внимание — это на то, что некоторые строки начинаются с точки с запятой (;). Запомните — во всех ассемблерах все, что записано после точки с запятой вплоть до конца текущей строки — это комментарии, пропускаемые ассемблером при трансляции вашей программы в контроллерные коды. Зачем же они тогда нужны? Для нас. В тот момент, когда мы пишем программу, мы знаем, зачем мы поставили ту или иную команду или последовательность команд. Но, отложив ее в сторону, спустя месяц мы с удивлением обнаруживаем, что не можем вспомнить, что и зачем мы понаписали в лежащей перед глазами программе. Именно для этого и нужны комментарии — понять, где начало программы, что она делает, что делает в ней та или иная команда. Настоятельно рекомендую не лениться и писать комментарии как можно подробнее — это сэкономит вам немало времени и нервов.

Иногда после точки с запятой ставят пустую строку, без комментариев — наличие пустых строк позволяет разбить программу на логически самостоятельные куски, что повышает читаемость текста программы. Кстати, обратите внимание, что точка с запятой (и с последующим комментарием) может стоять не только в начале строки, но и в середине ее после той или иной команды — такой комментарий относится к конкретной команде, а не к последовательности команд (последнюю я называю фрагментом программы).

После такого вступления перейдем к описанию команд, входящих в анализируемую программку.

```
;
CS     .EQU    P3.7
RD     .EQU    P3.6
CONVST .EQU    P3.5;
```

Эти три строки сообщают ассемблеру, что везде, где он встретит имена CS, RD и CONVST, он должен подставлять вместо них линии портов P3.7, P3.6 и P3.5 соответственно. В принципе, мы могли бы этого не делать, и везде в программе писать имена выбранных линий (P3.7, P3.6, P3.5). Но я не рекомендую вам привыкать работать таким образом — положим, завтра придется перенести этот фрагмент в другую программу, а там линия P3.7, будет уже занята другим устройством, и сигналом $\overline{CS}$

должна будет управлять другая линия порта, например P3.2. В таком, случае вам придется в этой программе самостоятельно найти все места, где упомянута P3.7, и везде заменить ее на P3.2. Поверьте на слово, подобная операция — один из основных источников ошибок. В то же время, если в начале программы у вас есть строки, подобные трем вышеупомянутым, то достаточно заменить P3.7 на P3.2 в первой из них, и дальше ассемблер осуществит требуемую замену самостоятельно, причем без ошибок. Есть еще аргументы в пользу работы с использованием подобных строк, но даже и без них, как мне кажется, ясна выгода использования в программе имен (`CS, RD, CONVST`) вместо явного указания закрепленных за ними линий.

```
;
  MOV   P1,#11111111b
  MOV   P3,#11111111b
;
```

Приведенные команды записывают в выходные буферы всех линий портов P1 и P3 единички. При этом, первая цифра, идущая после символа #, соответствует информации, заносимой в буфер линии (бит) P1.7 (или P3.7 соответственно), следующая за ним — в буфер P1.6 (P3.6) и т. д. Естественно, последние цифры соответствуют буферам младших битов (P1.0 и P3.0). Если бы нам нужно было занести в P1.7, P1.5, P3.3 и P3.1 нули, а в остальные — единицы, то соответствующие команды выглядели бы так:

```
;
  MOV   P1,#01011111b
  MOV   P3,#11110101b
;
```

Кстати, ассемблер вполне бы понял, если бы мы вместо #01011111b написали бы #5Fh или #95 — все три записи соответствуют одному и тому же числу (равно как и #11110101b, #0F5h и #245). Но шестнадцатеричное и десятичное представления числа менее наглядны, чем двоичное — в последнем случае сразу видно, какой бит установится в 0, какой — в 1. Думаю, этот пример развеет ваш скептицизм относительно необходимости применения двоичных чисел (если этот скептицизм у вас все еще наличествует).

Еще один момент, на который обязательно нужно обратить внимание — по правилам ассемблера для микроконтроллера x51 перед числом обязательно должен стоять знак #.

```
MOV   P1,#00101101b
```

Запись означает, что в порт P1 нужно занести ***число 00101101b*** (45d), в то время как

```
MOV   P1,00101101b
```

(то же, но без знака #) — что в P1 нужно занести ***содержимое ячейки памяти с адресом 00101101b*** (45d). В ячейке же может храниться любое число, от 0 до 255, и поэтому в порту P1 после выполнения команды `MOV P1,00101101b` все линии, к примеру, могут оказаться зануленными (если в ячейке хранится нуль), хотя мы подразумевали, что в результате выполнения команды записи в порт числа 45d занулятся только его первый, четвертый, шестой и седьмой биты, а остальные окажутся в единичном состоянии. Так что старайтесь не забывать ставить знак # перед числами, а если в какой-то момент обнаружите, что забыли — не переживайте очень уж сильно, это одна из самых распространенных ошибок у программистов, пишущих на ассемблере для x51.

Зачем мы установили в 1 выходные буферы портов P1 и P3? Все линии порта P1 и младшие 4 порта P3 мы используем как входы для ввода в микроконтроллер результата оцифровки, поэтому их, как я отмечал выше, нужно установить в единицы для предотвращения конфликтов на линиях. Линии P3.7, P3.6, P3.5 в соответствии с приведенным выше алгоритмом работы мы также до начала работы должны установить в 1. Ну а P3.4 мы устанавливаем в 1 за компанию с остальными.

Идем далее.

```
;
  CLR   CONVST              ;ИМПУЛЬС СТАРТА ПРЕОБРАЗОВАНИЯ
  SETB  CONVST
;
```

Напомню, что чуть ранее в строке `CONVST .EQU P3.5` мы сообщили ассемблеру, что пятый бит порта P3 мы назвали `CONVST` (поскольку он управляет одноименным входом АЦП). Команда `CLR CONVST` предписывает микроконтроллеру установить в 0 (сбросить, стереть) этот бит. Соответственно `SETB CONVST` предписывает установить его в 1. Последовательно идущие друг за другом, эти две команды сформируют на выводе P3.5 импульс отрицательной полярности. Собственно, именно это нам и нужно было для того, чтобы запустить цикл преобразования АЦП.

Дальше идут одна за другой 20 команд `NOP`. Команда `NOP` — это так называемая пустая операция, которая предписывает процессору ничего не делать и переходить в выполнению следующей за ней команды. На первый взгляд может показаться, что она абсолютно бессмысленна. Однако примем во внимание, что она выполняется (если так можно сказать о команде, при исполнении которой, простите за каламбур, ничего не выполняется) в течение ровно 1 мкс при тактовой частоте микроконтроллера 12 МГц. 20 идущих подряд команд `NOP` микроконтроллер будет выполнять, как нетрудно сосчитать, ровно 20 мкс, в течение которых состояние ни одного из его портов и ячеек памяти не изменится. Другими словами, этот

фрагмент — задержка на 20 мкс, в течение которых АЦП AD7880 гарантированно осуществит преобразование. Использование команды NOP для реализации задержек — одно из наиболее распространенных ее применений. Правда, много идущих одна за другой команд NOP — не самый красивый способ создания задержек, но он безотказно работает, и на первых порах, пока вы не научитесь более изящным способом формировать задержки нужной длительности, смело используйте команду NOP.

```
;
  CLR   CS     ;CS=0
  CLR   RD     ;RD=0
;
```

Этот фрагмент, надеюсь, очевиден — установка в 0 сигнала $\overline{CS}$, а за ним и $\overline{RD}$ (соответственно линий P3.7 и P3.6).

Через 75 нс после того, как сигнал на входе АЦП установился в 0, на его выходах DB11...DB0 появится результат преобразования. Микроконтроллер считает результат следующим образом:

```
;
  MOV   A,P1              ;ЧИТАЕМ ИЗ ПОРТА P1 МЛ.И СР. ТЕТРАДЫ
  MOV   R4,A              ;СОХРАНЯЕМ ИХ В R4
;
  MOV   A,P3              ;ЧИТАЕМ ИЗ ПОРТА P3 СТ. ТЕТРАДУ
  MOV   R5,A              ;В R5R4 РЕЗУЛЬТАТ
;
```

Первая команда предписывает микроконтроллеру перенести данные из порта P1 в аккумулятор (мы уже упоминали как-то о нем). Вторая переносит эти данные из аккумулятора в еще один регистр, который называется R4. В результате этого младшие 8 бит результата «осядут» в регистре R4. Ну а после выполнения следующих двух команд старшие биты результата попадут, как вы наверное, уже догадались, в регистр R5.

Прервемся затем, чтобы сделать два лирических отступления. Первое. Пока мы подробно не знакомились с внутренним устройством микроконтроллера, поэтому вы еще почти ничего не знаете о его регистрах. Это знакомство у нас еще впереди, хотя уже и не за горами. Сейчас же, чтобы не терять нить обсуждения, постарайтесь запомнить, что ***внутри микроконтроллера есть 8-битный регистр-аккумулятор (или просто аккумулятор) и восемь 8-битных регистров общего назначения, которые называются R0, R1, R2, ..., R6, R7.*** (8-битные— это значит, что в них можно записывать числа длиной до 8 двоичных разрядов, т. е. от 00000000b до 11111111b или от 0 до 255d.) Команды позволяют осуществлять с данными, находящимися в упомянутых регистрах, разнообразные действия. Конкретно какие — мы узнаем в ходе постепенного знакомства с системой команд микроконтроллера.

Второе. Если вы обратили внимание, в вышеупомянутых командах перемещения данных после самой команды `MOV` записаны через запятую и тот регистр, куда нужно перенести данные, и тот, откуда их нужно взять. Так вот, во всех ассемблерах существуют свои правила, какой из регистров (приемник данных или их источник) должен быть записан первым, а какой — вторым. Запомните, что ***в ассемблере x51 после самой команды всегда первым записывается регистр, куда нужно поместить данные (регистр-приемник), а за ним через запятую — регистр, откуда их нужно извлечь (регистр-источник).*** Перенести данные из регистра R2 в аккумулятор — это `MOV A,R2` , но никак не `MOV R2,A`. Постарайтесь это хорошо запомнить.

Итак, вернемся к нашим баранам. Данные прочитаны из АЦП в регистры R4 и R5 микроконтроллера. Теперь с ними можно сделать все, что нам взбредет в голову — преобразовать, отобразить, вывести в ЦАП и т. д. Правда, сигналы $\overline{CS}$ и $\overline{RD}$ все еще сохраняют нулевые уровни. Для завершения операции чтения их нужно вернуть в единичное состояние. Делается это, как нетрудно догадаться, при помощи команд

```
;
  SETB  RD                ;УСТАНОВКА RD В 1
  SETB  CS                ;УСТАНОВКА CS В 1
;
```

Переходим к последней команде

```
;
  SJMP  L7880             ;ЗАЦИКЛИВАНИЕ
;
```

Она возвращает микроконтроллер к выполнению той части программы, которая идет после метки `L7880`. Меткой называется слово из обычно не более чем 8 символов, начинающееся с буквы и кончающееся двоеточием. Такая метка (`L7880:`) присутствует в программе перед командами, формирующими импульс старта преобразования. Следовательно, рассматриваемая команда вернет микроконтроллер к формированию импульса старта преобразования, выполнению задержки 20 мкс и т. д. — словом, к повторному исполнению оцифровки сигнала и считывания результата микроконтроллером до того момента, пока мы не отключим от схемы питание. Другими словами, мы таким образом зацикливаем программу, что и отражено в соответствующем комментарии.

Итак, мы разобрали первый пример — как связать микроконтроллер с АЦП параллельного типа, и какими командами заставить его запустить преобразование в АЦП и считать его результат во внутренние регистры микроконтроллера. Как видите, в этом нет ничего недоступного. Сформировать на входе $\overline{CONVST}$ АЦП AD7880 отрицательный импульс, выждать 20 мкс и сформировать после этого еще 2 импульса на входах $\overline{CS}$ и $\overline{RD}$ ока-

залось при помощи микроконтроллера гораздо проще, чем сделать узел, формирующий все эти сигналы в нужной последовательности и с нужными длительностями, на дискретных элементах. Для формирования на выводах микроконтроллера импульсов есть чудесные команды `CLR x.y` и `SETB x.y` (x = 0...3, y = 0...7). Выполняя первую, мы сбрасываем соответствующую линию соответствующего порта в 0, а второй командой — устанавливаем ее в 1. Вставляя между ними некоторое количество команд `NOP`, мы можем изменять длительность этого импульса. И все, никаких мультивибраторов, времязадающих емкостей, инверторов для согласования фазировки сигналов. Словом, вы не зря захотели научиться работать с микроконтроллерами.

2.3. Ассемблер: основные понятия и приемы

Итак, мы ознакомились с некоторыми из команд микроконтроллера, которые были необходимы для написания программы связи микроконтроллера и АЦП. Команды эти мы записали в соответствии с требованиями, предъявляемыми ассемблером для x51. Давайте теперь более подробно познакомимся с ассемблером — с его требованиями и правилами работы с ним. В результате этого знакомства мы должны будем освоить не только то, как писать тексты программ, но и как их транслировать в коды, записываемые впоследствии в память программ микроконтроллера.

Программ-ассемблеров для x51 известно довольно много. Все они слегка отличаются друг от друга, но основные правила, установленные Intel для ассемблеров x51, у них едины. К моменту написания книги большинство из них были DOS'овские, ибо они были разработаны еще в те времена, когда DOS безраздельно господствовала в компьютерном мире. Конечно, в последние годы стали доступными и Windows-версии ассемблера для x51, но программисты — народ довольно консервативный, и большинство из них с неохотой переходят на новые программные продукты. Я уже много лет использую DOS-ассемблер TASM. Он, возможно, не столь удобен, как свежие Windows-ассемблеры, но про него я точно знаю, что он не содержит ошибок, и поэтому не горю желанием с ним расставаться. К тому же, именно под него в 1996 г. моим сыном был разработан хороший программный отладчик EMF-52, описание которого вы найдете в приложении 7 (см. том 2), и в совокупности эти программы составляют хороший и удобный программный комплекс, работавший без сбоев на любом из компьютеров, где бы его ни запускали, и требующий минимального количества ресурсов.

После такого вступления перейдем собственно к ассемблеру. Начнем с того, что в тексте программы на ассемблере у нас всегда будут присутствовать две группы команд. Одна из них — это команды микроконтроллера,

которые ассемблер и будет транслировать в коды. Другая — это так называемые директивы ассемблера, т. е. команды, которые управляют работой самого ассемблера. Давайте с ними познакомимся поближе.

Начнем мы со знакомства с тремя основными директивами — `ORG`, `EQU` и `END`. Обращаю ваше внимание на то, что в ассемблере TASM перед всеми директивами всегда стоит точка — мы должны писать `.ORG`, `.EQU` и `.END` (в других ассемблерах точка необязательна). Что означают эти директивы?

Первая из них предписывает ассемблеру транслировать идущий вслед за ней фрагмент программы (или всю программу) с того адреса, который указан после слова `.ORG`. Например, `.ORG 0` означает, что код команды, идущей вслед за директивой, должен располагаться в ячейке памяти программ с нулевым адресом. Как мы уже говорили раньше, микроконтроллеры семейства x51 при включении питания начинают выполнять программу именно с этой команды. Поэтому очевидно, что директива `.ORG 0` должна стоять перед самой первой командой нашей программы.

Отметим далее, что для x51 первые 256 байт памяти программ лучше не занимать под основную программу — в них нужно располагать так называемые подпрограммы обработки прерываний (об этом позже). Поэтому обычно начало программы должно выглядеть следующим образом:

```
   .ORG  0
   LJMP  START
   .ORG100h
START:
   MOV   P1,#11111111b
         …
```

Здесь директива `.ORG 0` указывает ассемблеру разместить коды записанной после нее команды `LJMP START` в ячейках памяти программ, начиная с адреса 0. Сама эта команда предписывает микроконтроллеру перейти к выполнению команды, стоящей после метки `START:` т. е. `MOV P1,#11111111b`. Место расположения в памяти программ кодов последней — начиная с адреса 100h (или 256d), т. к. метка расположена сразу за директивой `.ORG 100h`. Все это будет хорошо видно, когда мы познакомимся чуть ниже с листингом программы — текстовым файлом, в котором ассемблер проставит рядом с этими командами их коды и адреса ячеек памяти программ, где эти коды будут располагаться.

Еще одно замечание — метки (в данном случае `START:`) при написании программ на ассемблере должны начинаться с первой позиции в строке, без каких-либо пробелов перед именем метки. Сами команды и начинающиеся с точки директивы ассемблера должны идти после одного или нескольких пробелов. Если при написании программы вы будете использовать редактор типа Turbo Pascal для DOS, то перед написанием команды

или директивы нажмите клавишу табуляции Tab — она поставит перед ними 8 пробелов. А когда вы, работая в этом редакторе, после написания строки с командой нажмете клавишу Enter, курсор не только переместится на следующую строку, но и расположится под первым символом строчки, расположенной выше, т. е. ввод 8 пробелов осуществится автоматически.

Следующая директива — `.END`. Она информирует ассемблер о том, что команда, стоящая перед ней — последняя в этой программе, и что на этом месте нужно завершить трансляцию написанных команд в понятные микроконтроллеру коды. Перед `.END` также нужно поставить один или несколько пробелов (или нажать Tab).

Очень полезной является третья из рассматриваемых директив — `.EQU`. С ней мы уже встречались в предыдущем разделе — вспомним, например, строку

```
CS      .EQU   P3.7
```

или

```
RD      .EQU   P3.6
```

Одна из особенностей нашего мозга состоит в том, что мы очень плохо запоминаем неинформативные, унылые и однообразные надписи типа P3.6 или 0B2h, в то время как запомнить надпись `CS` или `RD`, явно ассоциирующуюся с сигналом $\overline{CS}$ или $\overline{RD}$ АЦП, для нас не представляет труда. В самом деле, поди сообрази сразу, какому импульсу соответствуют идущие друг за другом команды `CLR P1.5` и `SETB P1.5`. А вот если вместо P1.5 поставить имя `CONVST`, то вопросов не возникнет. А для того, чтобы и ассемблер понял, к какой линии порта или ячейке памяти мы обращаемся, до первого упоминания имени `CONVST` мы должны разместить строку

```
CONVST  .EQU   P3.5
```

Отметим, что в этой директиве имя переменной (в данном случае `CONVST`) записывается, подобно метке, с самого начала строки без предшествующих ему пробелов.

При написании программ рекомендуется использовать подобные символические имена везде, где это возможно. Помимо легкости понимания, это дает нам ряд дополнительных удобств. В самом деле, если вы, к примеру, по причине упрощения разводки платы вынуждены будете соединить вход АЦП $\overline{CONVST}$ не с линией P3.5 микроконтроллера, а с P3.2, вам нужно будет везде в тексте вашей программы заменить P3.5 на P3.2. Конечно, если программа умещается на одной-двух страницах, это несложно. А если в программе около 2000 строк, и она занимает 30 страниц? Поверьте, найти все без исключения места, где упоминается P3.5, не пропустив ни одного из них, крайне затруднительно. А если вы линии P3.5 присвоили имя

$\overline{\text{CONVST}}$, и в начале вашей программы поставили строку `CONVST .EQU P3.5`, то вам достаточно заменить линию P3.5 на P3.2 только в этой строке — все остальные замены ассемблер осуществит самостоятельно и при этом, что крайне важно, ничего не пропустит.

Далее нам необходимо ознакомиться с одной особенностью, присущей именно ассемблеру TASM. Дело в том, что этот ассемблер весьма универсален — он может быть адаптирован для очень многих типов микроконтроллеров. Для большинства из них принципы построения их ассемблеров идентичны, различия заключаются в адресах линий портов, регистров, ячеек оперативной памяти, в кодах команд и в словах, которыми мы описываем эти команды (упомянутые слова называются мнемониками: `MOV`, например, — это мнемоника команды пересылки данных). Для каждого семейства микроконтроллеров у TASM'а есть таблица, в которой записаны мнемоники всех их команд и коды, им соответствующие, а для тех семейств, для которых эти таблицы отсутствуют, их при желании можно сделать самостоятельно.

Такая гибкость TASM'а имеет свою оборотную сторону. Он «не знает» адресов регистров и линий портов микроконтроллеров семейства x51. Поэтому в самом начале программы ему необходимо указать упомянутые адреса при помощи уже описанной директивы `.EQU`. В принципе, это не такая уж проблема — написать этот фрагмент нужно лишь однажды, и затем переносить его из одной программы в другую. Когда-то я эту работу сделал, и все эти строчки вы найдете в приводимых далее программах. Пользуйтесь ими на здоровье. А тех, кого раздражает необходимость размещать перед текстом программы строки с директивой `.EQU`, я успокою — в новых микроконтроллерах, например в ADuC8xx, появились дополнительные регистры, имен которых не знают не только TASM, но и ни один из остальных ассемблеров, за исключением специально разработанных под эти микроконтроллеры. Так что чем бы вы ни пользовались, без подобных строк вы вряд ли обойдетесь.

После всего сказанного мы уже готовы к тому, чтобы программу работы микроконтроллера с АЦП, приведенную на стр. 28, записать с учетом всех требований ассемблера.

Программа работы микроконтроллера с параллельным АЦП AD7880 (файл par_adc.a51)

```
; ПРОГРАММА ЧТЕНИЯ АЦП AD7880,
; РАБОТАЕМ С ПОРТАМИ P1 И P3, CS = P3.7,
; RD = P3.6 , CONVST = P3.5.
;                          *****
;
R7      .EQU   7           ;АДРЕСА РЕГИСТРОВ R0-R7
```

```
R6      .EQU  6
R5      .EQU  5
R4      .EQU  4
R3      .EQU  3
R2      .EQU  2
R1      .EQU  1
R0      .EQU  0
ACC     .EQU  0E0H        ;АДРЕС АККУМУЛЯТОРА
B       .EQU  0F0H        ;АДРЕС РЕГИСТРА B
PSW     .EQU  0D0H        ;АДРЕС РЕГИСТРА (СЛОВА) СОСТОЯНИЯ
SP      .EQU  81H         ;АДРЕС УКАЗАТЕЛЯ СТЕКА
DPL     .EQU  82H         ;АДРЕС МЛАДШЕЙ ПОЛОВИНЫ DPTR
DPH     .EQU  83H         ;АДРЕС СТАРШЕЙ ПОЛОВИНЫ DPTR
P0      .EQU  80H         ;АДРЕС РЕГИСТРА ПОРТА P0
P1      .EQU  90H         ;АДРЕС РЕГИСТРА ПОРТА P1
P2      .EQU  0A0H        ;АДРЕС РЕГИСТРА ПОРТА P2
P3      .EQU  0B0H        ;АДРЕС РЕГИСТРА ПОРТА P3
B.0     .EQU  0F0H        ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА B
B.1     .EQU  0F1H
B.2     .EQU  0F2H
B.3     .EQU  0F3H
B.4     .EQU  0F4H
B.5     .EQU  0F5H
B.6     .EQU  0F6H
B.7     .EQU  0F7H
ACC.0   .EQU  0E0H        ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ АККУМУЛЯТОРА
ACC.1   .EQU  0E1H
ACC.2   .EQU  0E2H
ACC.3   .EQU  0E3H
ACC.4   .EQU  0E4H
ACC.5   .EQU  0E5H
ACC.6   .EQU  0E6H
ACC.7   .EQU  0E7H
PSW.0   .EQU  0D0H        ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА PSW
PSW.1   .EQU  0D1H
PSW.2   .EQU  0D2H
PSW.3   .EQU  0D3H
PSW.4   .EQU  0D4H
PSW.5   .EQU  0D5H
PSW.6   .EQU  0D6H
PSW.7   .EQU  0D7H
P0.0    .EQU  080H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P0
P0.1    .EQU  081H
P0.2    .EQU  082H
P0.3    .EQU  083H
P0.4    .EQU  084H
P0.5    .EQU  085H
P0.6    .EQU  086H
P0.7    .EQU  087H
P1.0    .EQU  090H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P1
P1.1    .EQU  091H
```

```
P1.2    .EQU   092H
P1.3    .EQU   093H
P1.4    .EQU   094H
P1.5    .EQU   095H
P1.6    .EQU   096H
P1.7    .EQU   097H
P2.0    .EQU   0A0H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P2
P2.1    .EQU   0A1H
P2.2    .EQU   0A2H
P2.3    .EQU   0A3H
P2.4    .EQU   0A4H
P2.5    .EQU   0A5H
P2.6    .EQU   0A6H
P2.7    .EQU   0A7H
P3.0    .EQU   0B0H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P3
P3.1    .EQU   0B1H
P3.2    .EQU   0B2H
P3.3    .EQU   0B3H
P3.4    .EQU   0B4H
P3.5    .EQU   0B5H
P3.6    .EQU   0B6H
P3.7    .EQU   0B7H
;
CS      .EQU   P3.7
RD      .EQU   P3.6
CONVST  .EQU   P3.5
;
  .ORG  0                  ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 0
;
  LJMP  START              ;НА КОМАНДУ ПОСЛЕ МЕТКИ START
;
  .ORG  100H               ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 100H
;
START:
  MOV   P0,#11111111B      ;НАЧАЛЬНАЯ УСТАНОВКА
  MOV   P1,#11111111B
  MOV   P2,#11111111B
  MOV   P3,#11111111B
;
L7880:                     ;СОБСТВЕННО ЧТЕНИЕ
;
  CLR   CONVST             ;ИМПУЛЬС СТАРТА ПРЕОБРАЗОВАНИЯ
  SETB  CONVST
;
  NOP                      ;ЗАДЕРЖКА НА ВРЕМЯ ПРЕОБРАЗОВАНИЯ
  NOP
  NOP
  NOP
  NOP
  NOP
  NOP
```

```
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
    NOP
;
    CLR   CS                ;CS=0
    CLR   RD                ;RD=0
;
    MOV   A,P1              ;ЧИТАЕМ ИЗ ПОРТА P1 МЛ.И СР. ТЕТРАДЫ
    MOV   R4,A              ;СОХРАНЯЕМ ИХ В R4
;
    MOV   A,P3              ;ЧИТАЕМ ИЗ ПОРТА P3 СТ. ТЕТРАДУ
    MOV   R5,A              ;В R5R4 - РЕЗУЛЬТАТ
;
    SETB  RD                ;УСТАНОВКА RD В 1
    SETB  CS                ;УСТАНОВКА CS В 1
;
    SJMP  L7880             ;ЗАЦИКЛИВАНИЕ
;
    .END
```

Все, что написано в программе, вам уже знакомо — комментарии после точки с запятой, фрагмент присвоения директивой `.EQU` адресов основным регистрам микроконтроллера, их битам и линиям портов, метки, оканчивающиеся двоеточиями, директивы `.ORG`, команды пересылок, установки и сброса линий портов и т. д. Эту программу уже можно оттранслировать при помощи ассемблера TASM, и получить как файл для загрузки в память программ микроконтроллера, так и уже упоминавшийся листинговый файл с кодами команд и адресами расположения этих кодов.

2.4. Ассемблер: особенности трансляции

Прежде чем приступить к трансляции приведенной выше программы в коды, нам необходимо кратко познакомиться с двумя основными форматами представления результатов трансляции.

Первый формат, именуемый бинарным или объектным, попросту содержит коды программы в том самом виде, в котором они будут записаны

в память программ микроконтроллера. Большинство ассемблеров при создании файла в этом формате присваивают ему расширение *.obj. Я предполагаю, читатели знают, что такое DOS, директории, файлы, их имена и расширения, bat-файлы, умеют смотреть содержимое файла при помощи вьювера оболочки Norton Commander или ей аналогичной, редактировать текст тем или иным DOS-редактором. Тем, кто со всем этим не знаком, придется найти какую-нибудь книжку по DOS или расспросить пользователей со стажем.

При трансляции файла par_adc.a51 в объектный формат TASM сформирует нам файл par_adc.obj. Просмотреть его можно вьювером Norton Commander'а, установив на него курсорную плашку и нажав привычное пользователям DOS F3, а затем — F4. Мой программатор понимает этот формат файлов, поэтому я транслирую написанные программы в коды таким образом, чтобы получать именно obj-файлы.

Второй формат носит название Intel Hex-Format или, по-простому, hex-формат. При трансляции файла par_adc.a51 в hex-формат TASM сформирует нам файл par_adc.hex. Просмотреть его можно все тем же вьювером Norton Commander'а, установив на файл курсорную плашку и нажав F3. Правда, в отличие от предыдущего случая, F4 нажимать не надо, ибо коды, полученные в результате трансляции, в файле par_adc.hex представлены в формате ASC-II. Помимо них, файл par_adc.hex содержит дополнительную служебную информацию, говорить о которой я не буду, но содержание которой становится более-менее понятным при внимательном сравнении файлов par_adc.obj и par_adc.hex или при чтении документации на ассемблер TASM.

Зачем нужен этот формат? Хотя большинство программаторов понимает оба формата, встречаются тем не менее отдельные изделия, которым один из них недоступен. Если вы станете счастливым обладателем программатора, который понимает только hex-формат, вам придется транслировать программы таким образом, чтобы на выходе получать именно такие файлы (о том, как это делается — чуть ниже). В остальных случаях рекомендую транслировать файлы в obj-формат — с ним проще разбираться, если что-то не так.

Теперь перейдем к трансляции. Если вы этого еще не сделали, создайте на винчестере директорию, в которой должны будут располагаться файлы, с коими нам предстоит работать. Пусть она будет размещена на диске C и наречена ASM51 (C:\ASM51). В ней должны находиться файлы ассемблера tasm.exe, tasm51.tab, tasm51b.bat, tasm51h.bat, tasm_rus.doc (файл с документацией), симулятор emf52l.exe и файл с программой par_adc.a51.

Войдите в Norton Commander (можно в сеансе работы в Windows95/98). Далее войдите в упомянутую директорию C:\ASM51. После этого наберите в командной строке tasm51b.bat и через пробел после него имя транслируе-

мого файла (в нашем случае par_adc.a51). Нажмите <Enter>. Трансляция осуществляется быстро, за доли или единицы секунд, в зависимости от быстродействия вашего компьютера. В ходе ее на экране появятся сообщения:

```
tasm: pass 1 complete.
tasm: pass 2 complete.
tasm: Number of errors = 0
```

Последняя строчка очень важна — ассемблер сообщил, что он не обнаружил ошибок и создал obj- или hex-файл (в нашем случае — файл par_adc.obj; убедитесь в том, что он появился). Но бывает (и к сожалению, гораздо чаще, чем хотелось бы), что написанная программа содержит ошибки. Некоторые из них ассемблер находит и информирует нас строкой:

```
tasm: Number of errors = n
```

где *n* — число обнаруженных ошибок. Помимо этого, TASM дает некоторую дополнительную информацию относительно этих ошибок — тип ошибки и в какой строке она находится. Более подробно о сообщениях, выдаваемых им при обнаружении ошибок, мы поговорим в следующем разделе. Сейчас же вернемся к процессу трансляции.

Bat-файл tasm51b.bat, как некоторые из вас уже наверное догадались, осуществляет запуск ассемблера TASM с ключами (на нормальном языке — указаниями), рекомендующими транслировать исходный ассемблерный файл в объектный. Кроме того, ассемблеру предписывается также выполнить некоторые другие действия — сформировать файл листинга, включить в него таблицу меток и т. д. Подробнее о том, что он может сделать, и как заставить его выполнить те или иные функции, вы узнаете, прочитав содержимое файла документации (tasm_rus.doc), а так же проанализировав командный файл tasm51b.bat. Но на первых порах можете просто ограничиться использованием этого командного файла, подобно тому, как это было сделано тремя абзацами выше.

Если ваш программатор требует для программирования файлы в hex-формате, то последовательность действий следующая. Как и в предыдущем случае войдите в Norton Commander, затем в упомянутую директорию C:\ASM51 и наберите в командной строке tasm51h.bat, после чего через пробел — имя транслируемого файла (в нашем случае par_adc.a51). Как и ранее, нажмите <Enter>. Трансляция осуществляется так же, как и в предыдущем случае, с теми же сообщениями, только в результате транслирования сформируется файл в hex-формате, в чем я также рекомендую убедиться. К сожалению, в обоих случаях оттранслированный TASM'ом файл имеет расширение *.obj, хотя в первом случае это будет файл в объектном формате, а

во втором — в hex-формате. Чтобы не путаться, файл hex-формата, полученный при трансляции с помощью tasm51h.bat, переименуйте самостоятельно в par_adc.hex.

Теперь поговорим о файле листинга. Этот файл (par_adc.lst) формируется ассемблером в процессе трансляции и представляет из себя исходный ассемблерный файл (par_adc.a51), дополненный следующей информацией: перед каждой командой стоит номер ее строки в ассемблерном тексте, адрес ячейки памяти программ, в которой расположен код операции команды, а после этого адреса — один, два или три байта самой команды (откуда у команд берутся второй и третий байт, мы рассмотрим в главе, посвященной системе команд). Далее, в конце программы находится таблица имен и меток, а также содержимое полученного в результате трансляции obj- или hex-файла. И, наконец, файл листинга разбит на страницы. Ниже приведен пример листингового файла.

Листинговый файл par_adc.lst

```
TASM 8051 Assembler.      PAR_ADC.A51                    page 1
Speech Technology Incorporated.

0001    0000  ;           ПРОГРАММА ЧТЕНИЯ АЦП AD7880,
0002    0000  ;           РАБОТАЕМ С ПОРТАМИ P1 И P3, CS = P3.7,
0003    0000  ;           RD = P3.6 , CONVST = P3.5.
0004    0000  ;           *****
0005    0000  ;
0006    0000  R7          .EQU   7     ;АДРЕСА РЕГИСТРОВ R0-R7
0007    0000  R6          .EQU   6
0008    0000  R5          .EQU   5
0009    0000  R4          .EQU   4
0010    0000  R3          .EQU   3
0011    0000  R2          .EQU   2
0012    0000  R1          .EQU   1
0013    0000  R0          .EQU   0
0014    0000  ACC         .EQU   0E0H  ;АДРЕС АККУМУЛЯТОРА
0015    0000  B           .EQU   0F0H  ;АДРЕС РЕГИСТРА B
0016    0000  PSW         .EQU   0D0H  ;АДРЕС РЕГИСТРА (СЛОВА) СОСТОЯНИЯ
0017    0000  SP          .EQU   81H   ;АДРЕС УКАЗАТЕЛЯ СТЕКА
0018    0000  DPL         .EQU   82H   ;АДРЕС МЛАДШЕЙ ПОЛОВИНЫ DPTR
0019    0000  DPH         .EQU   83H   ;АДРЕС СТАРШЕЙ ПОЛОВИНЫ DPTR
0020    0000  P0          .EQU   80H   ;АДРЕС РЕГИСТРА ПОРТА P0
0021    0000  P1          .EQU   90H   ;АДРЕС РЕГИСТРА ПОРТА P1
0022    0000  P2          .EQU   0A0H  ;АДРЕС РЕГИСТРА ПОРТА P2
0023    0000  P3          .EQU   0B0H  ;АДРЕС РЕГИСТРА ПОРТА P3
0024    0000  B.0         .EQU   0F0H  ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА B
0025    0000  B.1         .EQU   0F1H
0026    0000  B.2         .EQU   0F2H
0027    0000  B.3         .EQU   0F3H
```

```
0028    0000   B.4         .EQU   0F4H
0029    0000   B.5         .EQU   0F5H
0030    0000   B.6         .EQU   0F6H
0031    0000   B.7         .EQU   0F7H
0032    0000   ACC.0       .EQU   0E0H  ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ АККУМУЛЯТОРА
0033    0000   ACC.1       .EQU   0E1H
0034    0000   ACC.2       .EQU   0E2H
0035    0000   ACC.3       .EQU   0E3H
0036    0000   ACC.4       .EQU   0E4H
0037    0000   ACC.5       .EQU   0E5H
0038    0000   ACC.6       .EQU   0E6H
0039    0000   ACC.7       .EQU   0E7H
0040    0000   PSW.0       .EQU   0D0H  ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА PSW
0041    0000   PSW.1       .EQU   0D1H
0042    0000   PSW.2       .EQU   0D2H
0043    0000   PSW.3       .EQU   0D3H
0044    0000   PSW.4       .EQU   0D4H
0045    0000   PSW.5       .EQU   0D5H
0046    0000   PSW.6       .EQU   0D6H
0047    0000   PSW.7       .EQU   0D7H
0048    0000   P0.0        .EQU   080H  ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P0
0049    0000   P0.1        .EQU   081H
0050    0000   P0.2        .EQU   082H
0051    0000   P0.3        .EQU   083H
0052    0000   P0.4        .EQU   084H
0053    0000   P0.5        .EQU   085H
0054    0000   P0.6        .EQU   086H
0055    0000   P0.7        .EQU   087H
0056    0000   P1.0        .EQU   090H  ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P1
0057    0000   P1.1        .EQU   091H
0058    0000   P1.2        .EQU   092H
0059    0000   P1.3        .EQU   093H

TASM 8051 Assembler.      PAR_ADC.A51                      page 2
Speech Technology Incorporated.

0060    0000   P1.4        .EQU   094H
0061    0000   P1.5        .EQU   095H
0062    0000   P1.6        .EQU   096H
0063    0000   P1.7        .EQU   097H
0064    0000   P2.0        .EQU   0A0H  ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P2
0065    0000   P2.1        .EQU   0A1H
0066    0000   P2.2        .EQU   0A2H
0067    0000   P2.3        .EQU   0A3H
0068    0000   P2.4        .EQU   0A4H
0069    0000   P2.5        .EQU   0A5H
0070    0000   P2.6        .EQU   0A6H
0071    0000   P2.7        .EQU   0A7H
0072    0000   P3.0        .EQU   0B0H  ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P3
0073    0000   P3.1        .EQU   0B1H
0074    0000   P3.2        .EQU   0B2H
```

```
0075    0000   P3.3        .EQU   0B3H
0076    0000   P3.4        .EQU   0B4H
0077    0000   P3.5        .EQU   0B5H
0078    0000   P3.6        .EQU   0B6H
0079    0000   P3.7        .EQU   0B7H
0080    0000   ;
0081    0000   CS          .EQU   P3.7
0082    0000   RD          .EQU   P3.6
0083    0000   CONVST      .EQU   P3.5
0084    0000   ;
0085    0000              .ORG   0      ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 0
0086    0000   ;
0087    0000   02 01 00    LJMP   START  ;НА КОМАНДУ ПОСЛЕ МЕТКИ START
0088    0003   ;
0089    0100              .ORG   100H   ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 100H
0090    0100   ;
0091    0100   START:
0092    0100   75 80 FF    MOV    P0,#11111111B       ;НАЧАЛЬНАЯ УСТАНОВКА
0093    0103   75 90 FF    MOV    P1,#11111111B
0094    0106   75 A0 FF    MOV    P2,#11111111B
0095    0109   75 B0 FF    MOV    P3,#11111111B
0096    010C   ;
0097    010C   L7880:                    ;СОБСТВЕННО ЧТЕНИЕ
0098    010C   ;
0099    010C   C2 B5       CLR   CONVST  ;ИМПУЛЬС СТАРТА ПРЕОБРАЗОВАНИЯ
0100    010E   D2 B5       SETB  CONVST
0101    0110   ;
0102    0110   00          NOP           ;ЗАДЕРЖКА НА ВРЕМЯ ПРЕОБРАЗОВАНИЯ
0103    0111   00          NOP
0104    0112   00          NOP
0105    0113   00          NOP
0106    0114   00          NOP
0107    0115   00          NOP
0108    0116   00          NOP
0109    0117   00          NOP
0110    0118   00          NOP
0111    0119   00          NOP
0112    011A   00          NOP
0113    011B   00          NOP
0114    011C   00          NOP
0115    011D   00          NOP
0116    011E   00          NOP
0117    011F   00          NOP
0118    0120   00          NOP

TASM 8051 Assembler.      PAR_ADC.A51                      page 3
Speech Technology Incorporated.

0119    0121   00          NOP
0120    0122   00          NOP
0121    0123   00          NOP
```

```
0122    0124   ;
0123    0124   C2 B7       CLR   CS          ;CS=0
0124    0126   C2 B6       CLR   RD          ;RD=0
0125    0128   ;
0126    0128   E5 90       MOV   A,P1   ;ЧИТАЕМ ИЗ ПОРТА P1 МЛ.И СР. ТЕТРАДЫ
0127    012A   FC          MOV   R4,A        ;СОХРАНЯЕМ ИХ В R4
0128    012B   ;
0129    012B   E5 B0       MOV   A,P3        ;ЧИТАЕМ ИЗ ПОРТА P3 СТ. ТЕТРАДУ
0130    012D   FD          MOV   R5,A        ;В R5R4 - РЕЗУЛЬТАТ
0131    012E   ;
0132    012E   D2 B6       SETB  RD          ;УСТАНОВКА RD В 1
0133    0130   D2 B7       SETB  CS          ;УСТАНОВКА CS В 1
0134    0132   ;
0135    0132   80 D8       SJMP  L7880       ;ЗАЦИКЛИВАНИЕ
0136    0134   ;
0137    0134               .END
```

Label	Value	Label	Value	Label	Value
ACC	00E0	ACC.0	00E0	ACC.1	00E1
ACC.2	00E2	ACC.3	00E3	ACC.4	00E4
ACC.5	00E5	ACC.6	00E6	ACC.7	00E7
B	00F0	B.0	00F0	B.1	00F1
B.2	00F2	B.3	00F3	B.4	00F4
B.5	00F5	B.6	00F6	B.7	00F7
CS	00B7	CONVST	00B5	DPL	0082
DPH	0083	L7880	010C	PSW	00D0
P0	0080	P1	0090	P2	00A0
P3	00B0	PSW.0	00D0	PSW.1	00D1
PSW.2	00D2	PSW.3	00D3	PSW.4	00D4
PSW.5	00D5	PSW.6	00D6	PSW.7	00D7
P0.0	0080	P0.1	0081	P0.2	0082
P0.3	0083	P0.4	0084	P0.5	0085
P0.6	0086	P0.7	0087	P1.0	0090
P1.1	0091	P1.2	0092	P1.3	0093
P1.4	0094	P1.5	0095	P1.6	0096
P1.7	0097	P2.0	00A0	P2.1	00A1
P2.2	00A2	P2.3	00A3	P2.4	00A4
P2.5	00A5	P2.6	00A6	P2.7	00A7
P3.0	00B0	P3.1	00B1	P3.2	00B2
P3.3	00B3	P3.4	00B4	P3.5	00B5
P3.6	00B6	P3.7	00B7	R7	0007
R6	0006	R5	0005	R4	0004
R3	0003	R2	0002	R1	0001
R0	0000	RD	00B6	SP	0081
START	0100				

```
ADDR  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
--------------------------------------------------
0000  02 01 00 FF FF FF FF FF FF FF FF FF FF FF FF FF
0010  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0020  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0030  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF

TASM 8051 Assembler.             PAR_ADC.A51                    page 4
Speech Technology Incorporated.

0040  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0050  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0060  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0070  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0080  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0090  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
00A0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
00B0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
00C0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
00D0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
00E0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
00F0  FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
0100  75 80 FF 75 90 FF 75 A0 FF 75 B0 FF C2 B5 D2 B5
0110  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0120  00 00 00 00 C2 B7 C2 B6 E5 90 FC E5 B0 FD D2 B6
0130  D2 B7 80 D8 FF FF FF FF FF FF FF FF FF FF FF FF


tasm: Number of errors = 0
```

Этот листинговый файл вы найдете в директории C:\ASM51 после того, как оттранслируете исходный файл par_adc.a51 при помощи tasm51b.bat или tasm51h.bat.

2.5. Ассемблер: ошибки трансляции

Теперь, как я и обещал, рассмотрим некоторые типичные ошибки, допускаемые программистами при написании программ на ассемблере, и ту информацию, которую выводит на экран TASM при их обнаружении.

Наиболее распространенные ошибки — неправильное написание команд и имен переменных, отсутствие метки, на которую должен быть осуществлен переход, дважды (или чаще) повторяющаяся метка с одним и тем же именем, неправильное написание директивы `.END`. Сейчас мы с вами умышленно сделаем эти ошибки в нашей программе par_adc.a51, оттранслируем ее с ними и посмотрим на реакцию ассемблера на них. Если вы не поленитесь и проделаете рекомендуемые действия, вы приобретете полез-

ный навык реальной трансляции программы и последовательности действий при обнаружении ошибок при трансляции.

1. Неправильное написание команд. Для примера предположим, что в строке `SETB CONVST` (сразу после метки `L7880:`) мы ошибочно поставили `SET CONVST` (пропустили букву «В» в конце команды). Внесите эту ошибку в текст программы и запустите ее на ассемблирование. Напомню, для этого наберите в командной строке tasm51b.bat, после чего через пробел — par_adc.a51, и нажмите <Enter>. В ходе трансляции на экране отобразится отчет о результате трансляции:

```
tasm: pass 1 complete.
tasm: unrecognized instruction.        Line 0100 in PAR_ADC.A51
tasm: pass 2 complete.
tasm: Number of errors = 1
```

TASM сообщил, что в строке №100 обнаружена неизвестная ему команда (инструкция). Если вы посмотрите файл листинга, то увидите сразу под сотой строкой, где обнаружена ошибка, то же самое сообщение, что и в вышеприведенном отчете (`tasm: unrecognized instruction. Line 0100 i PAR_ADC.A51`). Таким образом, если в результате трансляции ассемблер нашел ошибку, в данном случае — неверно записанную команду, то просмотрите листинговый файл, найдите в нем строку с сообщением об ошибке, и внимательно посмотрите на команду в вышестоящей строке — именно она содержит ошибку. Исправьте ее, снова запустите программу на ассемблирование и убедитесь, что все в порядке.

2. Ошибка в написании имени переменной. Для примера возьмем случай, когда в блоке присвоения именам переменных численных значений имя переменной `CS` (`CS .EQU P3.7`), а в тексте программы мы ее обозначили как `CSS`, причем сразу в двух местах:

```
(текст программы)
CLR     CSS          ;CS=0
(текст программы)
SETB    CSS          ;УСТАНОВКА CS В 1
(текст программы)
```

Внесите эти ошибки в текст программы и запустите ее на ассемблирование. В ходе трансляции на экране отобразится следующий отчет о результате трансляции:

```
tasm: pass 1 complete.
tasm: Label not found:       (CSS)    Line 0122
tasm: Label not found:       (CSS)    Line 0132
tasm: pass 2 complete.
tasm: Number of errors = 2
```

TASM сообщил, что в строках 122 и 132 обнаружена неизвестная ему метка (на самом деле имя). Если вы, как и в предыдущем случае, посмотрите файл листинга, то обнаружите вышеприведенные сообщения об ошибках прямо перед 122-й и 132-й строками, где эти ошибки и находятся. Вам остается только понять, почему TASM не понял этих имен, для чего нужно пролистать листинговый файл назад, к началу, найти то имя, которое вами было определено, и сравнить с тем, которое оказалось в 122-й и 132-й строках. После этого обычно сразу все становится ясно, остается лишь исправить ошибки и запустить исправленный файл на ассемблирование. Проделайте сказанное до получения строки `tasm: Number of errors = 0`.

3. Отсутствие метки, на которую должен быть осуществлен переход. Допустим, вы забыли поставить метку `START:` или пропустили в ней букву (написали `STRT:`, для ассемблера это то же самое, что забыть метку). В результате ассемблирования вы должны получить отчет следующего содержания:

```
tasm: pass 1 complete.
tasm: Label not found:       (START)    Line 0088
tasm: pass 2 complete.
tasm: Number of errors = 1
```

Здесь ссылка на то, что TASM не нашел метку, более правильна — это действительно так, метки нет. Соответственно, об этом он сообщает и в листинговом файле перед 88-й строкой, в которой размещена команда `LJMP START`. Рекомендую при появлении подобной информации в отчете внимательно посмотреть вначале на строку, где находится ссылка на несуществующую метку, а затем на то место, где она должна бы присутствовать. Внимательный анализ содержимого этих двух строк должен прояснить, в чем вы промахнулись.

4. Дважды (или чаще) повторяющаяся метка с одним и тем же именем. Опять экспериментируем с меткой `START:`, но в этот раз давайте не только не забудем поставить ее в нужном месте, но и воткнем еще в одном, где это не нужно, например перед первой командой `NOP`. Запустим программу на ассемблирование и получим:

```
tasm: pass 1 complete.
tasm: label value misaligned.          (START)   Line 0102 in PAR_ADC.A51
tasm: pass 2 complete.
tasm: Number of errors = 1
```

В файле листинга сообщение об этой ошибке будет стоять под 102-й строкой, где TASM нашел упомянутую метку во второй раз. В этом случае ищите выше по тексту место, где она встречается в первый раз, и удаляйте ту, которая лишняя.

5. Неправильное написание директивы .END. Предположим, в результате ассемблирования вы получили отчет следующего содержания:

```
tasm: pass 1 complete.
tasm: No END directive before EOF.    Line 0135 in PAR_ADC.A51
tasm: pass 2 complete.
tasm: Number of errors = 1
```

Для тех, кто не знает, EOF — это End Of File (конец файла), символ, стоящий в начале последней строки в файле. Если в этой строке стоит директива .END, то TASM сначала находит символ EOF, а затем — директиву .END. По правилам же должно быть наоборот — вначале .END, затем EOF. Ошибка заключается в том, что в файле par_adc.a51 после набора строчки с директивой .END вы не нажали клавишу «Enter». Нажмите ее, чтобы появилась следующая строка, и курсор переместился на нее. Если после этого осуществить ассемблирование, все встанет на свои места.

Конечно, в настоящем разделе перечислены не все возможные варианты ошибок и сообщения, выдаваемые TASM'ом при их обнаружении. Мы рассмотрели лишь основные из них. С остальными (точнее, с некоторыми из них) мы еще познакомимся в главе, посвященной системе команд микроконтроллера, ибо пока многие команды вам незнакомы, понять смысл некоторых сообщений будет вам крайне затруднительно.

Добавим, что мы рассмотрели сообщения об ошибках, выдаваемых ассемблером TASM. Другие ассемблеры могут выдавать при обнаружении этих ошибок информацию несколько отличающегося содержания. В целом, когда вы поймете команды микроконтроллера, вы без труда будете находить ошибки. Тем более, что все ассемблеры помечают в листинге строки с ошибками, и внимательно посмотрев содержание этих строк, вы быстро поймете, в чем дело.

2.6. Сопряжение с последовательным АЦП

Не знаю, утомились ли вы читать про ассемблер, или нет, а я писать про него слегка утомился. Поэтому предлагаю сделать некоторую передышку и на время вернуться к рассмотрению «железа». В этом разделе мы познакомимся с одним из типичнейших представителей микросхем, обменивающихся информацией в последовательном формате — АЦП ADS7816 фирмы Burr-Brown.

Микросхема выпускается в 8-выводном корпусе. Схема ее соединения с микроконтроллером, а также цоколевка приведена на **Рис. 2.3.**

Два вывода микросхемы являются аналоговыми входами (вход –IN рекомендуется соединить с общим проводом), на вход V_{ref} подается опорное

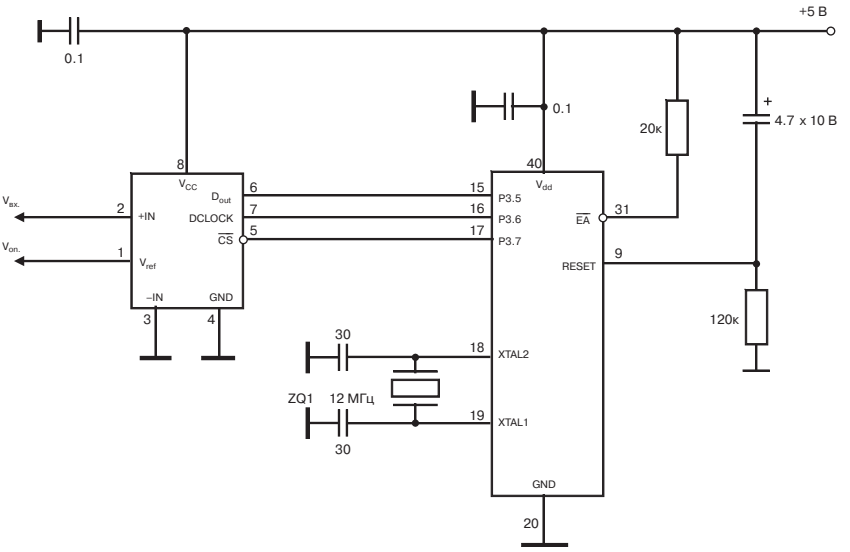


Рис. 2.3. Соединение последовательного АЦП ADS7816 и микроконтроллера

напряжение, V_{CC} и GND — соответственно питание и «земля». Для обмена с микроконтроллером используются 3 оставшихся вывода АЦП. На вход $\overline{CS}$ микроконтроллер подает сигнал старта преобразования, по входу DCLOCK он тактирует АЦП, а с выхода D_{out} — принимает результат преобразования, бит за битом. Временные диаграммы сигналов на выводах $\overline{CS}$, DCLOCK и D_{out} приведены на **Рис. 2.4.**

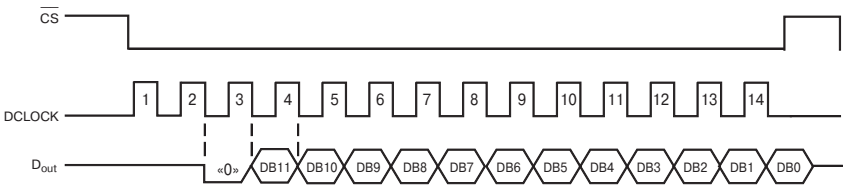


Рис. 2.4. Временные диаграммы для ADS7816

Как видите, алгоритм работы с ADS7816 несложен. При включении микроконтроллер должен установить на выводах $\overline{CS}$ и D_{out} единичные уровни сигналов, а на DCLOCK — нулевой. Запуск преобразования осу-

ществляется установкой нулевого уровня на выводе $\overline{CS}$. После этого микроконтроллер должен сформировать на DCLOCK три положительных импульса. По спаду последнего из них на выходе D_{out} появится старший бит результата преобразования (DB11). Считав его, микроконтроллер должен сформировать на DCLOCK следующий положительный импульс. По спаду его на выходе D_{out} появится следующий бит результата преобразования (DB10). Считав его, микроконтроллер снова формирует импульс на DCLOCK, по спаду которого на D_{out} появится бит DB9 и т. д., вплоть до DB0. Считав последний, микроконтроллер должен оставить DCLOCK нулевым, а $\overline{CS}$ вернуть в единицу. На этом цикл запуска преобразования и считывания информации завершается.

Программа, реализующая этот алгоритм, содержится в файле ser_adc.a51 и приведена ниже.

Программа работы микроконтроллера с последовательным АЦП ADS7816

```
; ПРОГРАММА ЧТЕНИЯ АЦП ADS7816,
; РАБОТАЕМ С ПОРТАМИ P1 И P3, CS = P3.7,
; DCLOCK = P3.6 , DOUT = P3.5.
;              *****
;
R7      .EQU   7           ;АДРЕСА РЕГИСТРОВ R0-R7
R6      .EQU   6
R5      .EQU   5
R4      .EQU   4
R3      .EQU   3
R2      .EQU   2
R1      .EQU   1
R0      .EQU   0
ACC     .EQU   0E0H        ;АДРЕС АККУМУЛЯТОРА
B       .EQU   0F0H        ;АДРЕС РЕГИСТРА B
PSW     .EQU   0D0H        ;АДРЕС РЕГИСТРА (СЛОВА) СОСТОЯНИЯ
SP      .EQU   81H         ;АДРЕС УКАЗАТЕЛЯ СТЕКА
DPL     .EQU   82H         ;АДРЕС МЛАДШЕЙ ПОЛОВИНЫ DPTR
DPH     .EQU   83H         ;АДРЕС СТАРШЕЙ ПОЛОВИНЫ DPTR
P0      .EQU   80H         ;АДРЕС РЕГИСТРА ПОРТА P0
P1      .EQU   90H         ;АДРЕС РЕГИСТРА ПОРТА P1
P2      .EQU   0A0H        ;АДРЕС РЕГИСТРА ПОРТА P2
P3      .EQU   0B0H        ;АДРЕС РЕГИСТРА ПОРТА P3
B.0     .EQU   0F0H        ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА B
B.1     .EQU   0F1H
B.2     .EQU   0F2H
B.3     .EQU   0F3H
B.4     .EQU   0F4H
B.5     .EQU   0F5H
B.6     .EQU   0F6H
B.7     .EQU   0F7H
```

```
ACC.0   .EQU   0E0H        ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ АККУМУЛЯТОРА
ACC.1   .EQU   0E1H
ACC.2   .EQU   0E2H
ACC.3   .EQU   0E3H
ACC.4   .EQU   0E4H
ACC.5   .EQU   0E5H
ACC.6   .EQU   0E6H
ACC.7   .EQU   0E7H
PSW.0   .EQU   0D0H        ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА PSW
PSW.1   .EQU   0D1H
PSW.2   .EQU   0D2H
PSW.3   .EQU   0D3H
PSW.4   .EQU   0D4H
PSW.5   .EQU   0D5H
PSW.6   .EQU   0D6H
PSW.7   .EQU   0D7H
P0.0    .EQU   080H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P0
P0.1    .EQU   081H
P0.2    .EQU   082H
P0.3    .EQU   083H
P0.4    .EQU   084H
P0.5    .EQU   085H
P0.6    .EQU   086H
P0.7    .EQU   087H
P1.0    .EQU   090H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P1
P1.1    .EQU   091H
P1.2    .EQU   092H
P1.3    .EQU   093H
P1.4    .EQU   094H
P1.5    .EQU   095H
P1.6    .EQU   096H
P1.7    .EQU   097H
P2.0    .EQU   0A0H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P2
P2.1    .EQU   0A1H
P2.2    .EQU   0A2H
P2.3    .EQU   0A3H
P2.4    .EQU   0A4H
P2.5    .EQU   0A5H
P2.6    .EQU   0A6H
P2.7    .EQU   0A7H
P3.0    .EQU   0B0H        ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P3
P3.1    .EQU   0B1H
P3.2    .EQU   0B2H
P3.3    .EQU   0B3H
P3.4    .EQU   0B4H
P3.5    .EQU   0B5H
P3.6    .EQU   0B6H
P3.7    .EQU   0B7H
;
CS      .EQU   P3.7
DCLOCK  .EQU   P3.6
```

```
DOUT    .EQU   P3.5
;
        .ORG   0            ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 0
;
        LJMP   START        ;НА КОМАНДУ ПОСЛЕ МЕТКИ START
;
        .ORG   100H         ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 100H
;
START:
        MOV    P0,#11111111B  ;НАЧАЛЬНАЯ УСТАНОВКА
        MOV    P1,#11111111B
        MOV    P2,#11111111B
        MOV    P3,#11111111B
        CLR    DCLOCK       ;УСТАНОВКА DCLOCK В 0
;
L7816:                      ;СОБСТВЕННО ЧТЕНИЕ
;
        CLR    CS           ;ИМПУЛЬС СТАРТА ПРЕОБРАЗОВАНИЯ
;
        SETB   DCLOCK       ;1-Й ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        SETB   DCLOCK       ;2-Й ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        SETB   DCLOCK       ;3-Й ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    B.3,C        ;DB11 В B.3
        SETB   DCLOCK       ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    B.2,C        ;DB10 В B.2
        SETB   DCLOCK       ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    B.1,C        ;DB9 В B.1
        SETB   DCLOCK       ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    B.0,C        ;DB8 В B.0
        SETB   DCLOCK       ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.7,C      ;DB7 В ACC.7
        SETB   DCLOCK       ;ТАКТОВЫЙ ИМПУЛЬС
```

```
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.6,C     ;DB6 В ACC.6
        SETB   DCLOCK      ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.5,C     ;DB5 В ACC.5
        SETB   DCLOCK      ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.4,C     ;DB4 В ACC.4
        SETB   DCLOCK      ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.3,C     ;DB3 В ACC.3
        SETB   DCLOCK      ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.2,C     ;DB2 В ACC.2
        SETB   DCLOCK      ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.1,C     ;DB1 В ACC.1
        SETB   DCLOCK      ;ТАКТОВЫЙ ИМПУЛЬС
        CLR    DCLOCK
;
        MOV    C,DOUT
        MOV    ACC.0,C     ;DB0 В ACC.0
;
        SETB   CS          ;ЗАВЕРШЕНИЕ СЧИТЫВАНИЯ
;
        MOV    R4,A        ;СОХРАНЯЕМ МЛ.И СР. ТЕТРАДЫ В R4
;
        MOV    A,B         ;ЧИТАЕМ ИЗ РЕГИСТРА В СТ. ТЕТРАДУ
        ANL    A,#00001111B ;ЗАНУЛЯЕМ СТАРШИЕ 4 БИТА
        MOV    R5,A        ;В R5R4 - РЕЗУЛЬТАТ
;
        SJMP   L7816       ;ЗАЦИКЛИВАНИЕ
;
        .END
```

Почти все, что написано в этой программе, вам уже знакомо — фрагмент присвоения директивой `.EQU` адресов основным регистрам микроконтроллера, их битам и линиям портов, метки, оканчивающиеся двоеточиями, директивы `.ORG`, команды пересылок, установки и сброса линий

портов и т. д. Новым для вас является регистр B, команды пересылки битов типа `MOV C,DOUT` и команда `ANL`. Рассмотрим их более подробно. Но перед этим замечу, что тем, кому не по душе наше крайне медленное знакомство с системой команд микроконтроллера, советую обратиться к литературе, в первую очередь рекомендую справочник Боборыкина А. В. и др. «Однокристальные микроЭВМ» (М.: МИКАП, 1994, — 400 с.: ил.)

Регистр B иначе называют расширителем аккумулятора. В основном его используют при выполнении команды умножения — перед операцией в нем хранится один из сомножителей (второй — в аккумуляторе), а после выполнения — старший байт произведения. Главное достоинство этого регистра в сравнении с регистрами R0...R7 заключается в том, что мы можем осуществлять обмен между каждым битом этого регистра и уже упоминавшимся битом переноса. Кстати, такое возможно также и с битами аккумулятора, поэтому в программе ser_adc.a51 первоначально результат измерения помещается именно в регистр `B` и в аккумулятор.

Теперь о командах `MOV C,DOUT`. При рассмотрении предыдущей программы мы уже знакомились с командами пересылки данных (помните, `MOV A,P1` и `MOV R4,A`). Но те команды пересылали содержимое того или иного порта или регистра целиком. А это содержимое представляло из себя 8-битные числа (от 00000000b до 11111111b). Одной из замечательных особенностей микроконтроллеров x51 является то, что они допускают также пересылку одного бита порта P0...P3 или регистра (`ACC, B`) в бит переноса и наоборот (бит переноса обозначают как CY, он является одним из битов регистра состояния PSW, о котором мы будем говорить при подробном знакомстве с системой команд). Так, уже упомянутая команда `MOV C,DOUT` перенесет нолик или единичку с линии P3.5 (помните, у нас `DOUT .EQU P3.5`) в бит переноса. Далее команда `MOV B.x, C` или `MOV A.x,C` перенесет этот ноль или единицу в бит B.x (x = 0...3) или A.x (x = 0...7) соответственно. Таким образом, в рассматриваемой выше программе в процессе чтения на входе D_{out} (P3.5) микроконтроллера последовательно возникают биты результата DB11, DB10, DB9, ..., DB0, а он считывает их и размещает соответственно в 3, 2, 1, 0-м битах регистра B, далее в 7-м, 6-м и т. д. битах аккумулятора. Как видите, все очень просто.

Замечу, что описанная реализация программы чтения ADS7816 крайне неизящна с точки зрения правил написания программ — она длинна, в ней много повторяющихся кусков и т. д. Но у нее есть два преимущества в сравнении с тем, что обычно считается изящно написанной программой. Во-первых, она доступна для понимания самых малоподготовленных пользователей, и пока вы не научитесь писать программы лучше, пишите их так, они тоже будут работать. А во-вторых, программы, написанные подобным образом, чаще всего работают быстрее других — микроконт-

роллер не тратит время на вызов подпрограмм и т. д. Но об этом — как-нибудь позже.

И последнее, с чем мы познакомимся в этом разделе — команда `ANL A,#00001111b`. Она осуществляет функцию логического И между битами аккумулятора и числа 00001111b. Для тех, кто не помнит — логическое И нуля с нулем или единицей дает ноль, и только логическое И двух единиц дает в результате единицу. Следовательно, в результате выполнения этой команды старшие 4 бита аккумулятора занулятся (какими бы они ни были, в результате операции логического И между ними и нулями, находящимися в старших четырех битах числа 00001111b получатся нули), а младшие — останутся без изменений. Подобное использование команды ANL для зануления тех или иных битов аккумулятора очень распространено, поскольку это гораздо проще, чем ставить несколько команд `CLR ACC.x` (x = 0...7). Отмечу, что этой командой мы занулили с 7-го по 4-й биты аккумулятора, 3, 2, 1 и 0-й биты которого в этот момент содержали старшую тетраду результата, считанного с АЦП. Сделано это для того, чтобы после завершения чтения АЦП в тех битах, хранящих результат считывания, которые по забывчивости можно было бы трактовать как 12, 13, 14 и 15-й, были бы нули.

Вот и все про работу с последовательным АЦП. В качестве упражнения рекомендую познакомиться с 14-битным последовательным АЦП AD7894 от Analog Devices и с 10-битным MAX1243 от Maxim и попытаться адаптировать рассмотренную программу под них — это вам вполне уже по силам.

2.7. Краткие выводы

Итак, уважаемые читатели, состоялось ваше первое знакомство с тем, как сопрягать микроконтроллеры с микросхемами, имеющими интерфейс, ориентированный на микроконтроллеры. Как видите, ничего сложного в этом нет. Вы соединяете выводы требуемого числа линий портов ввода/вывода микроконтроллера с управляющими и информационными входами и выходами этих микросхем и дальше пишите программу, которая реализует заданный в описании на микросхему алгоритм работы с ней. При этом на начальном этапе достаточно крайне ограниченного числа команд из тех 255-ти, которые понимает рассматриваемый нами микроконтроллер семейства x51. Установка в 0 или в 1 линий ввода/вывода осуществляется командами `CLR` и `SETB`. Эти же команды используются для формирования импульсов, отдельных фронтов и спадов, запускающих обмен и преобразование данных. Для пересылки же 8-битных данных между портами и регистрами применяют разновидности команды `MOV`. Используют эту команду и для пересылки отдельных битов (преимущественно между какой-либо линией ввода/вывода микроконтроллера и битом переноса

CY, а также между битом переноса и битом одного из регистров, в частности, аккумулятора или регистра B).

Мы рассмотрели варианты программ для работы с конкретными микросхемами — параллельным АЦП AD7880 и последовательным ADS7816. Они просты для понимания и абсолютно работоспособны, хотя не очень рациональны с точки зрения продвинутых программистов. Пока вы не наберетесь нужного опыта, программы ваши будут, с их точки зрения, довольно примитивными, но вас это пусть не смущает. На первом этапе важно, чтобы они работали. А изящество в написании программ со временем само придет, опыт — дело наживное.

Также состоялось более близкое знакомство с конкретной версией ассемблера. Теперь вы знаете, что ассемблер переводит (транслирует) текстовый ассемблерный файл, содержащий строки с командами микроконтроллера и директивами ассемблера, в obj- или hex-файлы. Файлы в этих форматах нужны для занесения написанной вами программы в микроконтроллер. Как правило, программаторы понимают оба этих формата, хотя бывают и исключения. Транслируйте вашу программу в тот формат, который доступен используемому вами программатору. Если же ему доступны оба формата, рекомендую работать с объектным (obj-).

Управление процессом трансляции осуществляется при помощи директив ассемблера и ключей (на нормальном языке — указаний), предписывающих ассемблеру при трансляции выполнять определенные действия — формировать файл листинга, включать в него таблицу меток и т. д. Подробнее о том, что он может сделать, и как заставить его выполнить те или иные функции, можно узнать, прочитав содержимое файла документации (в нашем случае tasm_rus.doc), а также проанализировав командные файлы tasm51b.bat и tasm51h.bat — ключи в них записаны правильным образом. На первых порах можете просто ограничиться использованием этих командных файлов, не вникая в их содержание.

Наиболее часто используемые директивы ассемблера — `.ORG`, `.EQU` и `.END`. Первая из них предписывает ассемблеру транслировать идущий вслед за ней фрагмент программы (или всю программу) с того адреса, который указан после слова `.ORG` (`.ORG 100h` — это значит, с адреса 100h или 256d).

Следующая директива — `.END`. Она информирует ассемблер о том, что команда, стоящая перед ней — последняя в этой программе, и что на этом месте нужно завершить трансляцию написанных команд в понятные микроконтроллеру коды. Перед `.END`, кстати, так же, как и перед `.ORG`, нужно поставить один или несколько пробелов (или нажать Tab).

Очень полезной является третья из рассматриваемых директив — `.EQU`. Она используется для сообщения ассемблеру, что то или иное имя переменной, та или иная константа при трансляции должны иметь значения,

записанные в строке с именем этой переменной или константы и директивой `.EQU` (например, `CONVST .EQU P3.5`). Отметим, что в этой директиве имя переменной (в данном случае `CONVST`) записывается с самого начала строки, без предшествующих ему пробелов.

При написании программ рекомендуется использовать подобные символические имена везде, где это возможно. Помимо легкости понимания, это дает нам ряд дополнительных удобств. В самом деле, если вы, к примеру, по причине упрощения разводки платы вынуждены будете соединить вход АЦП $\overline{\text{CONVST}}$ не с линией P3.5 микроконтроллера, а с P3.2, вам всего-навсего нужно будет заменить P3.5 на P3.2 лишь в строке `CONVST .EQU P3.5`.

При трансляции ассемблер формирует также листинговый файл. Он представляет из себя исходный ассемблерный файл, дополненный следующей информацией: перед каждой командой стоит номер ее строки в ассемблерном тексте, адрес ячейки памяти программ, в которой расположен код операции команды, а после этого адреса — один, два или три байта самой команды. При обнаружении ошибок ассемблер сообщает о них в листинговом файле, причем сообщение об ошибке находится перед или после строки, где эта ошибка была найдена. Далее, в конце программы находится таблица имен и меток, а также содержимое полученного в результате трансляции obj- или hex-файла.

Завершив трансляцию, ассемблер выводит на экран отчет о результате трансляции. Если в этом отчете присутствуют слова `tasm: Number of errors = 0` (или аналогичные, если вы используете другой ассемблер), то можете использовать полученный obj- или hex-файл для программирования микроконтроллера. В противном случае вам необходимо с помощью листингового файла найти ошибку (-и), исправить и произвести трансляцию по новой, до тех пор, пока ассемблер не сочтет, что все в порядке.

Глава 3. **РЕГИСТРЫ МИКРОКОНТРОЛЛЕРА**

Одними из основных элементов архитектуры микроконтроллера являются его регистры (напомню, что регистры — это ячейки памяти внутри микроконтроллера, обмен информацией между которыми осуществляется простыми и короткими командами). В предыдущих главах настоящего повествования я уже вскользь упоминал о них и даже воспользовался некоторыми из них при рассмотрении примеров сопряжения микроконтроллеров с параллельными и последовательными АЦП. Однако я пока еще не сказал ни сколько их в рассматриваемых нами микроконтроллерах, ни каковы их свойства, ни каковы команды работы с ними, а также не упомянул о тех регистрах, которые отвечают за работу находящихся внутри микроконтроллера таймеров, счетчиков, приемопередатчиков и т. д. Настоящая глава призвана частично восполнить этот пробел.

Обращаю ваше внимание на то, что хотя мы с вами еще не знакомились в полном объеме с системой команд микроконтроллера, мы регулярно анализировали и будем продолжать анализировать фрагменты программ, содержащие эти команды. Объясняется это тем, что внешние выводы и формируемые на них сигналы с одной стороны, внутренняя структура микроконтроллера (память, регистры и т. д.) с другой и система команд с третьей теснейшим образом переплетены между собой. Нельзя рассказывать о чем-то одном, не касаясь другого и третьего. Поэтому я вынужден был в первых главах лишь вскользь упомянуть о регистрах, не давая более развернутого их описания — если бы я попытался это сделать, материал стал бы намного более трудным для восприятия (а он и без того нелегок). Точно также я стараюсь лишь в минимальной степени знакомить вас с командами, говоря только то, что они предписывают микроконтроллеру, и опуская их формат, длину, время выполнения, разновидности и т. д. Все это у нас впереди. Но если кто-то из вас, дорогие читатели, готов воспринять эту усложняющую материал дополнительную информацию, и желает поскорее с ней ознакомиться — в вашем распоряжении глава 5, где описаны команды микроконтроллера семейства x51, обратитесь к ней. Ну, а мы с теми, кто не горит желанием забежать вперед, потихоньку двинем дальше.

3.1. Регистры общего назначения и слово состояния программы

Однако прежде, чем начать рассказ о регистрах микроконтроллеров семейства x51, я предлагаю совершить небольшой экскурс в историю микропроцессоров.

В середине 70-х годов прошлого века, когда микропроцессоры уже миновали этап становления, а микроконтроллеры еще не выделились из них в самостоятельный класс изделий, наиболее совершенными микросхемами были 8-битные процессоры второго поколения 8080, 8085 фирмы Intel, 6800 фирмы Motorola и Z80 фирмы Zilog. Технология, по которой они создавались, не позволяла разместить на кристалле более 5...7 тысяч транзисторов, поэтому при создании этих микросхем разработчикам приходилось до разумных пределов уменьшать количество входящих в них элементов и узлов.

По этой причине упомянутые микропроцессоры имели в своем составе всего по десятку регистров. Одним из них был счетчик команд PC (Programm Counter), содержимое которого увеличивалось после выполнения каждой команды. Его я поставил на первое место в списке регистров, поскольку нет такого микропроцессора или микроконтроллера, в котором бы он отсутствовал. Помимо него, все упомянутые микропроцессоры имели один или два регистра-аккумулятора, с содержимым которых можно было выполнять любые доступные микропроцессору действия по перемещению и обработке данных. Далее, 6800 имел 4 регистра, где программист мог размещать адреса ячеек памяти, с которыми ему предстояло работать. В противовес ему, остальные три микропроцессора имели по 6 так называемых регистров общего назначения (РОН), в которых могли храниться не только адреса ячеек памяти, но и константы, переменные, значения функций — словом, любые данные. И этим резервы всех микропроцессоров практически исчерпывались. Исключение составлял лишь Z80, который, в сравнении с 8080 и 8085, имел двойной комплект аккумуляторов и регистров общего назначения (правда, в каждый конкретный момент доступным был только один аккумулятор и один комплект РОН).

А зачем микропроцессору нужны были регистры типа РОН? Не проще ли было работать без них, храня все обрабатываемые данные в памяти? В общем, может в чем-то и проще. Но уж точно практически на порядок медленнее. С числами, расположенными в регистрах, микропроцессор выполнял те или иные действия гораздо быстрее, чем с теми же данными, расположенными во внешней оперативной памяти (внутренняя была еще не по плечу существовавшей тогда технологии). Так, сложение двух чисел, находящихся в РОН, занимало несколько тактов (единицы микросекунд).

Сложение двух чисел, расположенных в ОЗУ, требовало в 5...10 раз большего числа тактов и выполнялось за 20...50 мкс. Поэтому алгоритмы наиболее часто выполняемых фрагментов программы составлялись таким образом, чтобы уменьшить число требуемых для их выполнения констант и переменных до того количества, которое целиком (или почти целиком) разместилось бы в имеющихся регистрах. И, надо сказать, большинство программистов успешно справлялось с этой задачей.

В отличие от упомянутых микропроцессоров, появившиеся в конце 70-х первые микроконтроллеры уже содержали на кристалле несколько десятков ячеек оперативной памяти. Часть из них была отведена под регистры, которых стало возможным сделать больше, чем у 8085 или Z80. Таким образом, программы стало писать заметно легче — большое количество регистров и дополняющая их почти столь же быстрая внутренняя оперативная память практически сняли ограничения на число используемых программой переменных и констант. Так что вам, дорогие читатели, учиться писать программы будет гораздо проще, чем тем, кто учился этому на заре эпохи микропроцессоров.

При этом, однако, микроконтроллеры семейства x51, первоначально выпущенные Intel, имели (и имеют) довольно много общего с ею же разработанными микропроцессорами 8080 и 8085. Так, рассматриваемые нами микроконтроллеры содержат один аккумулятор и 8 регистров общего назначения, обозначаемых как R0, R1, R2, ..., R6, R7. Правда, если быть точным, этих регистров не 8, а 32 (4 так называемых банка по 8 в каждом), но, как и в Z80, доступным в каждый момент может быть лишь один банк. Поэтому лучше считать, что x51 имеют всего 8 РОН.

Где расположены эти регистры общего назначения? В первых 32 ячейках внутренней памяти данных микроконтроллера (**Рис. 3.1**). R0 первого банка располагается в ячейке с адресом 0, R1 — с адресом 1 и т. д., вплоть до адреса 7 (R7). В ячейках с адресами с 8-го по 15-й расположены РОН второго банка (R0 — в 8-й, R1 — в 9-й, ..., R7 — в 15-й). Третий банк, как вы уже, наверное, догадались, занимает ячейки с адресами с 16-го по 23-й (R0 — в 16-м, R7 — в 23-м). Ну, а четвертый — правильно, в 24-й, 25-й и т. д., вплоть до 31-й ячейки памяти.

Как отмечалось, в каждый конкретный момент доступны регистры общего назначения только одного какого-либо банка — первого, второго, третьего или четвертого. А какого? И как переключать банки РОН?

Для ответа на этот вопрос нам необходимо познакомиться с еще одним регистром — словом состояния программы PSW (Program Status Word). Его структура приведена на **Рис. 3.2.**

Сразу обращаю ваше внимание на биты RS1, RS0. Именно их состояние определяет, какой из банков РОН задействован в настоящий момент. Если RS1 = 0, RS0 = 0, то мы работаем с первым банком, расположенным

АДРЕСА	БИТ 7	БИТ 6	БИТ 5	БИТ 4	БИТ 3	БИТ 2	БИТ 1	БИТ 0
128(7FH) ⋮ 48(30H)	ОБЫЧНОЕ ОЗУ							
47(2FH)	7FH	7EH	7DH	7CH	7BH	7AH	79H	78H
46(2EH)	77H	76H	75H	74H	73H	72H	71H	70H
45(2DH)	6FH	6EH	6DH	6CH	6BH	6AH	69H	68H
44(2CH)	67H	66H	65H	64H	63H	62H	61H	60H
43(2BH)	5FH	5EH	5DH	5CH	5BH	5AH	59H	58H
42(2AH)	57H	ПОБИТНО АДРЕСУЕМАЯ ОБЛАСТЬ ОЗУ (АДРЕСА БИТОВ 00H...7FH)					51H	50H
41(29H)	4FH						49H	48H
40(28H)	47H						41H	40H
39(27H)	3FH						39H	38H
38(26H)	37H						31H	30H
37(25H)	2FH						29H	28H
36(24H)	27H	26H	25H	24H	23H	22H	21H	20H
35(23H)	1FH	1EH	1DH	1CH	1BH	1AH	19H	18H
34(22H)	17H	16H	15H	14H	13H	12H	11H	10H
33(21H)	0FH	0EH	0DH	0CH	0BH	0AH	09H	08H
32(20H)	07H	06H	05H	04H	03H	02H	01H	00H
31(1FH) ⋮ 24(18H)	БАНК 3							
23(17H) ⋮ 16(10H)	БАНК 2							
15(0FH) ⋮ 08(08H)	БАНК 1							
07(07H)	РЕГИСТР БАНКА R7 0							
06(06H)	РЕГИСТР БАНКА R6 0							
05(05H)	РЕГИСТР БАНКА R5 0							
04(04H)	РЕГИСТР БАНКА R4 0							
03(03H)	РЕГИСТР БАНКА R3 0							
02(02H)	РЕГИСТР БАНКА R2 0							
01(01H)	РЕГИСТР БАНКА R1 0							
00(00H)	РЕГИСТР БАНКА R0 0							

Рис. 3.1. Структура внутренней памяти данных микроконтроллеров x51

в ячейках памяти данных с адресами 0...7. Комбинация RS1 = 0, RS0 = 1 означает, что мы работаем со вторым банком (ячейки с адресами 8...15). RS1 = 1, RS0 = 0 означает, что мы работаем с третьим банком (ячейки с адресами 16...23), а RS1 = 1, RS0 = 1 — что с четвертым (ячейки 24...31).

При старте микроконтроллера RS1 = 0, RS0 = 0, т. е. вначале мы всегда работаем с первым банком РОН. А если нам захотелось переключиться на четвертый? Для этого в программу всего-навсего нужно вставить уже знакомые нам две команды, которые изменят состояние битов RS1, RS0:

```
;
  SETB PSW.4          ;УСТАНОВКА В 1 БИТА RS1
  SETB PSW.3          ;УСТАНОВКА В 1 БИТА RS0
```

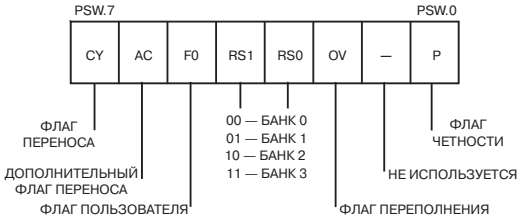


Рис. 3.2. Слово состояния программы микроконтроллеров x51

Естественно, сброс бита RS1 или RS0 в 0 осуществляется командой `CLR PSW.4` или `CLR PSW.3` соответственно. Таким образом, комбинируя четыре вышеупомянутые команды, мы можем заставить микроконтроллер работать с любым из банков РОН.

Какие еще биты содержатся в регистре PSW? Старший, седьмой бит — это уже не раз упоминавшийся бит переноса CY. Кстати, программисты иногда вместо термина «бит переноса» употребляют выражение «флаг переноса». Когда CY = 1, они говорят, что флаг переноса установлен, когда CY = 0 — что флаг сброшен.

Шестой бит (PSW.6) — это дополнительный флаг переноса AC. Он используется при выполнении арифметических операций, и мы еще упомянем о нем в разделе, посвященном системе команд микроконтроллера. Также при выполнении арифметических операций используется и бит переполнения (или флаг переполнения) OV (PSW.2). Бит (флаг) четности P (PSW.0) обычно используется при передаче информации от одного микроконтроллера к другому. Он передается вместе с передаваемым восьмибитным словом в качестве дополнительного девятого бита. Если в процессе передачи произошел сбой, то четность принятого слова может отличаться от четности переданного, и флаг четности при проверке слова принимающим микроконтроллером не совпадет с переданным в качестве девятого бита флагом четности передающего микроконтроллера. В этом случае необходимо повторить пересылку ошибочного слова. Все это звучит весьма мудрено, и не расстраивайтесь, если не все понятно — мы это еще обсудим при рассмотрении приемопередатчика микроконтроллера.

Отдельно скажу о флаге пользователя F0 (бит PSW.5). Командами `CLR PSW.5` или `SETB PSW.5` вы из своей программы можете сбрасывать или устанавливать его при выполнении тех или иных важных для вас условий — например, если сработал какой-то из опрашиваемых датчиков или нажата кнопка на клавиатуре. Далее, анализируя состояние этого флага,

ваша программа выполняет те или иные действия, в зависимости от того, установлен ли он или сброшен. При этом отмечу, что если вы установили этот флаг, например, в единицу, его состояние останется неизменным до тех пор, пока вы самостоятельно не сбросите его командой `CLR PSW.5`, и никакие действия микроконтроллера не приведут к его самостоятельному, без вашего вмешательства, сбросу. Именно поэтому F0 называется флагом пользователя — только пользователь может изменять состояние этого бита (флага).

Итак, мы ознакомились со всеми используемыми битами слова состояния PSW (бит PSW.1 не используется).

3.2. Аккумулятор, расширитель аккумулятора, указатель стека и подпрограммы

Перечисленные в этом подзаголовке 3 регистра, наряду с РОН, являются наиболее часто используемыми у микроконтроллеров семейства x51. Поэтому познакомимся с ними поближе.

Регистр-аккумулятор или просто аккумулятор мы упоминали уже не один раз. Это основной регистр микроконтроллеров x51. Что это означает? То, что у нашего микроконтроллера нет второго такого регистра, содержимое которого можно было бы сложить с содержимым ***любого другого*** регистра, переместить в него содержимое ***любого другого*** регистра или ***любой*** ячейки памяти. Содержимое аккумулятора можно сдвигать, побитно инвертировать (все нули заменить единицами и наоборот), анализировать, и в зависимости от его состояния выполнять те или иные фрагменты программы. Когда мы познакомимся с системой команд, вы убедитесь, что большинство из них так или иначе затрагивает аккумулятор.

Прежде, чем двинуться дальше, совершим еще одно небольшое лирическое отступление. Дело в том, что далеко не во всех микроконтроллерах существует один-единственный аккумулятор, играющий столь важную роль в его архитектуре. Есть микроконтроллеры с двумя равноправными аккумуляторами — все действия, которые можно сделать с одним из них, можно сделать и с другим. Команд обработки данных таким микроконтроллерам требуется поменьше, чем x51, да и сами команды покороче и выполняются побыстрее. Казалось бы, это весьма существенное преимущество, и благодаря ему двухаккумуляторные микроконтроллеры должны были бы вытеснить одноаккумуляторные. Ан нет. Прежде, чем выполнить те или иные действия над данными в обоих аккумуляторах, их нужно в тот и в другой занести. Кроме того, после выполнения операции нужно также переписать данные из обоих аккумуляторов обратно в память.

В одноаккумуляторной же структуре данные из памяти нужно переносить только в один аккумулятор, ибо в качестве источника второго числа (второго слагаемого или сомножителя, вычитаемого или делителя) мы предпишем контроллеру использовать ту ячейку памяти, где это число хранится, и его не потребуется куда-либо переносить. Так что одноаккумуляторная структура, хотя и уступает двухаккумуляторной в скорости обработки данных, но превосходит ее в скорости подготовки данных для обработки. Таким образом, обе они оказываются практически эквивалентными как с точки зрения быстродействия, так и по функциональным возможностям. Именно поэтому ни та, ни другая архитектура не смогла вытеснить конкурирующую. Так что имейте ввиду, что в иных, отличных от x51 микроконтроллерах, аккумуляторов может быть и 2, и даже 32, пусть это вас не смущает.

Как я уже сказал, в аккумулятор мы можем перенести данные из любого регистра общего назначения (командой `MOV A,Rn`, где n = 0...7) или из любой ячейки памяти. Последнее осуществляется командой `MOV A,addr`, где addr, принимающий значения от 0 до 255 — это адрес ячейки памяти. Мы можем также занести в аккумулятор любое целое число из диапазона 0...255 (командой `MOV A,#data`, где data = 0...255; не забудьте про знак #, без него микроконтроллер воспримет число, записанное после символа A, не как данные, а как адрес ячейки памяти). Естественно, из аккумулятора можно вернуть данные в любой регистр или в любую ячейку памяти (командами `MOV Rn, A` и `MOV addr, A` соответственно). К нему можно прибавить содержимое любого РОН, любого 8-битного числа или любой ячейки памяти (командами `ADD A,Rn; ADD A,#data` и `ADD A,addr` соответственно). С тем же успехом при помощи соответствующей команды `SUBB` из него можно вычесть содержимое любого РОН, любого 8-битного числа или любой ячейки памяти. И это далеко не полный перечень того, что можно делать с содержимым аккумулятора x51. Ни с каким другим регистром микроконтроллера мы не в состоянии проделать всего этого.

О регистре B мы также уже упоминали — он используется чаще всего при умножении и делении. Команда `MUL AB` осуществляет быстрое (за 4 мкс при тактовой частоте 12 МГц) перемножение чисел, хранящихся в аккумуляторе и в регистре B. После завершения умножения в B хранятся старшие 8 бит результата, а в аккумуляторе — младшие 8 бит. Командой `DIV AB` осуществляется деление содержимого аккумулятора на содержимое регистра `B`, результат деления — в аккумуляторе, остаток от деления — в B.

Отмечу, что последняя команда — довольно бестолковая, ибо делимое в ней должно быть не более 255. А как быть, если вам нужно разделить, например, результат измерения, полученный при помощи уже упоминавшихся 12-битных АЦП (он может быть в пределах 0...4095), например, на 10 или на 100? Командой `DIV AB` здесь не воспользуешься. Для таких слу-

чаев мы составим соответствующие подпрограммы, которые позволят выполнить подобные действия. Но об этом — чуть позже.

Еще один важный регистр — SP (Stack Pointer) или указатель стека. Термин этот для многих из вас нов, поэтому я постараюсь поподробнее объяснить, что такое стек, и какова функция регистра SP. Но прежде нам нужно понять, что такое подпрограммы, и зачем они нужны.

В предыдущей главе мы рассматривали сопряжение микроконтроллера с параллельными и последовательными АЦП и анализировали программы, которые обеспечивали считывание результата преобразования в регистры R4, R5. Но если вдуматься, ценность этих программ почти нулевая — ну считали, а дальше-то что? Нужно хотя бы отобразить считанное. Потом, сами по себе коды считанного результата часто неинформативны — нам нужно мерять ток, напряжение, температуру и т. д. А для получения этих величин результат считывания надо преобразовать — на что-то умножить, с чем-то сложить. Иногда даже провести предварительное измерение какого-то параметра, а последующее преобразование осуществлять с учетом этого результата. Например, пусть мы используем наш микроконтроллер в системе, измеряющей температуру чего-то с помощью термопары (две сваренных в одной точке проволочки из различных материалов; если кто не знает, что такое термопара, и как она работает — не беда, просто следите за ходом рассуждений). Прежде, чем с помощью термопары что-то измерить, нужно знать температуру тех концов входящих в нее проволочек, которые соединены с измерительным прибором (у нас — с АЦП). Следовательно, перед измерением с помощью термопары мы должны каким-то датчиком измерить температуру упомянутых концов, а затем по известным специалистам формулам внести соответствующую поправку в результат последующего измерения, выполняемого при помощи самой термопары.

Вот мы и добрались до главного. Нам для получения результата нужно делать не одно, а два измерения — сначала измерить температуру концов, а затем — сигнал с термопары. (Я сейчас не вдаюсь в подробности аппаратной реализации такой задачи — ясно, что в состав системы должен входить мультиплексор, переключающий АЦП с термопары на датчик и наоборот, какие-то усилители и т. д. — это в данный момент неважно.) Важно то, что в ходе выполнения неведомой нам пока программы термопарного измерения нам как минимум дважды нужно запустить АЦП на преобразование и считать его результат. То есть получается, что содержимое программы par_adc.a51 (или ser_adc.a51) в этой программе должно повторяться минимум дважды! А может и трижды или четырежды — зависит от того, что мы захотели от нашего микроконтроллера, и как мы это программно реализовали.

Давайте теперь вспомним, что память программ у микроконтроллеров не бездонная — всего несколько тысяч ячеек. И ее всегда не хватает. А тут получается, что нам приходится часто повторять в нашей программе одни и те же фрагменты. Возникает законный вопрос — нельзя ли как-то лишь однажды написать эти фрагменты, а затем просто обращаться к ним по мере необходимости?

Радуйтесь, можно. Вы пишете этот фрагмент (программисты его называют подпрограммой) один раз, присваиваете ему какое-то понятное вам имя и дальше вызываете его по мере необходимости командой `LCALL`. Например, программу (пардон, теперь уже подпрограмму) измерения при помощи параллельного АЦП вы назвали `IZMPAR`, и собираетесь дважды использовать в своем алгоритме. Тогда ваша программа будет выглядеть следующим образом.

Пример программы, дважды вызывающей подпрограмму IZMPAR

```
;       НАЧАЛО ПРОГРАММЫ ДО МОМЕНТА ПЕРВОГО
;       ВЫЗОВА ПОДПРОГРАММЫ ИЗМЕРЕНИЯ С
;       ПАРАЛЛЕЛЬНЫМ АЦП МЫ ЗДЕСЬ НЕ ПРИВОДИМ
;
  MOV   R6,A              ;ЭТО КОМАНДА ИЗ ПРЕДШЕСТВУЮЩЕГО
                          ;ФРАГМЕНТА,
  LCALL IZMPAR            ;А ВОТ ЭТО — ПЕРВЫЙ ВЫЗОВ
                          ; ПОДПРОГРАММЫ ИЗМЕРЕНИЯ
  MOV   A,R4              ;ЭТО КОМАНДА СЛЕДУЮЩЕГО
                          ;ФРАГМЕНТА — ОБРАБАТЫВАЕМ РЕЗУЛЬТАТ
;
;       СЛЕДУЮЩИЙ КУСОК ПРОГРАММЫ ДО МОМЕНТА
;       ВТОРОГО ВЫЗОВА ПОДПРОГРАММЫ ИЗМЕРЕНИЯ С
;       ПАРАЛЛЕЛЬНЫМ АЦП МЫ ОПЯТЬ ОПУСКАЕМ
;
  ADD   R2,#6             ;ЭТО КОМАНДА ИЗ ПРЕДШЕСТВУЮЩЕГО
                          ;ФРАГМЕНТА,
  LCALL IZMPAR            ;А ВОТ ЭТО — ВТОРОЙ ВЫЗОВ
                          ; ПОДПРОГРАММЫ ИЗМЕРЕНИЯ
  MOV   A,R5              ;ЭТО КОМАНДА СЛЕДУЮЩЕГО
                          ;ФРАГМЕНТА — ОПЯТЬ ОБРАБАТЫВАЕМ РЕЗ.
;
;       ИДУЩАЯ ДАЛЕЕ ЧАСТЬ ПРОГРАММЫ, СВЯЗАННАЯ
;       С ОБРАБОТКОЙ И ОТОБРАЖЕНИЕМ, МЫ ЕЕ ТАКЖЕ ОПУСКАЕМ
;
  LJMP  START             ;ЭТО КОМАНДА — ОКОНЧАНИЕ ОСНОВНОЙ
                          ;ЧАСТИ ПРОГРАММЫ И ВОЗВРАТ К НАЧАЛУ
;
;       А ВОТ ТЕПЕРЬ ПОШЛИ ПОДПРОГРАММЫ,
;       И ПЕРВАЯ ИЗ НИХ — IZMPAR
;
IZMPAR:                   ;ЕЕ ИМЯ С ОБЯЗАТЕЛЬНЫМ
```

```
                                ;ДВОЕТОЧИЕМ НА КОНЦЕ
  MOV P1,#11111111B             ;НАЧАЛЬНАЯ УСТАНОВКА
  MOV P3,#11111111B
;
L7880:                          ;СОБСТВЕННО ЧТЕНИЕ
;
  CLR CONVST                    ;ИМПУЛЬС СТАРТА ПРЕОБРАЗОВАНИЯ
  SETB CONVST
;
  NOP                           ;ЗАДЕРЖКА НА ВРЕМЯ ПРЕОБРАЗОВАНИЯ
  NOP
;
;       ИДУЩАЯ ДАЛЕЕ ЧАСТЬ ПОДПРОГРАММЫ
;       ПРОПУЩЕНА ДЛЯ ЭКОНОМИИ МЕСТА
;
  MOV R4,A                      ;СОХРАНЯЕМ ИХ В R4
;
  MOV A,P3                      ;ЧИТАЕМ ИЗ ПОРТА P3 СТ. ТЕТРАДУ
  MOV R5,A                      ;В R5R4 - РЕЗУЛЬТАТ
;
  SETB RD                       ;УСТАНОВКА RD В 1
  SETB CS                       ;УСТАНОВКА CS В 1
;
  RET                           ;КОМАНДА ЗАВЕРШЕНИЯ ПОДПРОГРАММЫ
;
;
;       ДАЛЕЕ ИДУТ ДРУГИЕ ПОДПРОГРАММЫ, СВЯЗАННЫЕ
;       С ОБРАБОТКОЙ И ОТОБРАЖЕНИЕМ
;
```

В приведенном фрагменте пропущены куски программы, содержание которых для нас в данный момент не важно. Важными являются лишь следующие аспекты:

1. В тех местах, где нам нужно вызвать подпрограмму (в данном случае `IZMPAR`), мы ставим команду `LCALL IZMPAR`.

2. Непосредственно перед первой командой, входящей в состав подпрограммы, мы ставим ее имя, оканчивающееся двоеточием. Как мы помним из предыдущего, двоеточие ставится в конце метки. Так что имя подпрограммы для нашей программы является меткой, куда нужно перейти для того, чтобы подпрограмму выполнить.

3. В конце подпрограммы стоит пока еще неизвестная нам команда RET. А теперь — внимание! Микроконтроллер, выполняя программу, дошел до того места, где нужно вызывать подпрограмму измерения `IZMPAR`. Здесь он встретил команду `LCALL IZMPAR`, параметром которой является метка, куда ему нужно перейти, чтобы найти коды этой так необходимой нам подпрограммы. Перейдя туда, он начал выполнять команды подпрограммы, вплоть до самого ее конца. А затем... Да, как вы думаете, что будет затем?

Если кто еще не догадался — подскажу. После того, как подпрограмма выполнена, микроконтроллер должен выполнять команду, которая стоит в программе следующей после команды вызова подпрограммы. Так, в приведенной выше программе после того, как первый вызов подпрограммы будет завершен, микроконтроллер должен будет выполнить команду `MOV A,R4`, а после второго — `MOV A,R5`. А как, завершив подпрограмму, микроконтроллер перейдет к выполнению этих команд, ведь перед ними нет никаких меток, которые подсказали бы ему, куда переходить?

Вот мы наконец и добрались до стека. Стек — это какое-то количество ячеек памяти, в которые микроконтроллер перед переходом на исполнение подпрограммы заносит адрес той самой следующей команды, которую ему предстоит выполнять после завершения подпрограммы. Как видите, и здесь все просто. Наткнувшись на команду вызова подпрограммы `LCALL`, микроконтроллер сохраняет в стеке адрес следующей за ней команды и отправляется выполнять подпрограмму. А в конце любой подпрограммы — запомните это! — должна стоять команда возврата `RET`. Как только микроконтроллер доберется до нее, для него это послужит сигналом, что подпрограмма завершена, и он, прочитав из стека сохраненный в нем адрес следующей команды, перейдет на ее выполнение. Таким образом, вы можете вызывать из вашей программы интересующую вас подпрограмму хоть сто раз, и это не приведет к безумному раздуванию объема программы за счет сотни повторяющихся одинаковых кусков. Подпрограмма будет написана вами всего однажды, а везде, где нужно ее выполнить, вы поставите всего-навсего одну команду — вызов этой подпрограммы.

Где располагается стек? В оперативной памяти микропроцессора или микроконтроллера. А на конкретную ячейку, где хранится адрес команды, той самой, следующей за вызовом подпрограммы, указывает именно регистр SP. Да, забыл сказать, адрес этой команды, следующей за вызовом подпрограммы, обычно называют адресом возврата.

Забавно — самому регистру SP посвящен всего один абзац. Но прежде, чем о нем упомянуть, понадобилось десять абзацев с описанием того, что такое подпрограммы, и каков механизм их вызова и возврата из них.

3.3. Работа МК с внешней памятью данных

До сих пор мы упоминали о наличии внутри микроконтроллера 128 ячеек памяти данных. При этом мы подчеркивали, что она — внутренняя, т. е. расположенная на том же кристалле, что и остальные элементы микроконтроллера. Но x51 может работать и с внешней памятью данных. Последняя представляет собой одну или несколько самостоятельных микросхем памяти. Чаще всего используют статическую память с байтовой организацией объемом 2K×8 или 8K×8 бит (1K = 1024). Такие микро-

схемы имеют 8 выводов данных (D0...D7), по которым осуществляется одновременная запись в микросхему всех 8 бит в выбранную ячейку памяти или чтение 8 бит из этой ячейки. Далее, в таких микросхемах есть 11 или 13 адресных входов (A0...A10 или A0...A12), комбинация сигналов на которых задает адрес ячейки, к которой мы обращаемся. Вход $\overline{WE}$ определяет характер обращения: если на нем установлена 1, то осуществляется чтение из выбранной ячейки; при $\overline{WE} = 0$ в ячейку будет записана информация. Вход $\overline{CE}$ активизирует микросхему памяти — когда на ее входе $\overline{CE}$ установлена 1, она выключена, при $\overline{CE} = 0$ она допускает запись в нее информации и чтение из нее записанных данных. Нулевой сигнал на входе $\overline{OE}$ включает выходные буферы микросхемы памяти на пропускание информации по линиям данных D0...D7, единичный сигнал переводит эти линии в серое состояние, т. е. отключает находящиеся внутри микросхемы ячейки памяти от ее выводов. Описываемые микросхемы изображены на **Рис. 3.3.**

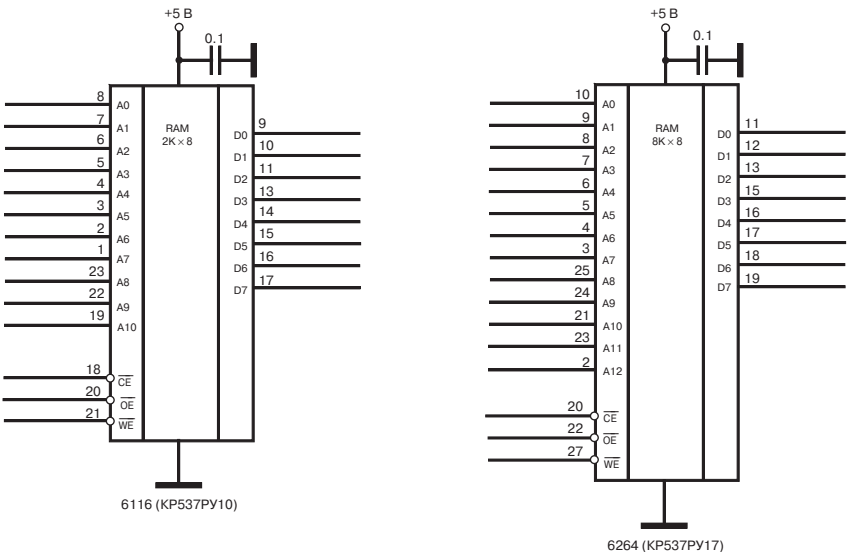


Рис. 3.3. Типичные микросхемы внешней оперативной памяти

Кстати, обратите внимание на то, что некоторые выводы микросхем обозначены на схеме кружками, а над их названиями стоят черточки ($\overline{CE}$, $\overline{OE}$, $\overline{WE}$). Так в микропроцессорной схемотехнике принято обозначать входы и выходы, активными сигналами для которых являются отрицательные импульсы.

Работа с подобными микросхемами должна осуществляться следующим образом. Положим, мы хотим записать число 145d = 10010001b в ячейку с адресом 84d = 54h = 01010100b. Для этого микроконтроллер должен установить записываемое число на линиях данных D0...D7 микросхемы (D0 = D4 = D7 = 1, D1 = D2 = D3 = D5 = D6 = 0), а адрес ячейки — на адресных линиях (A2 = A4 = A6 = 1, A0 = A1 = A3 = A5 = 0; A7, A8 и последующие старшие адреса вплоть до A10 для микросхем объемом 2K×8 или до A12 для микросхем 8K×8 также должны быть установлены в 0). Установив адресную информацию и данные, микроконтроллер одновременно с этим или чуть позже должен установить 0 на входе $\overline{WE}$ микросхемы (будет запись) и 0 на $\overline{CE}$ (знак того, что мы обращаемся именно к этой микросхеме). Как только после этого на входе $\overline{OE}$ микросхемы памяти микроконтроллер установит 0, осуществится запись числа 145 в ее 84-ю ячейку.

Соответственно, если мы хотим прочитать данные из все той же, к примеру, 84-й ячейки, мы должны, как и в предыдущем случае, установить адрес ячейки на адресных линиях, и одновременно с этим или чуть позже установить 1 на входе $\overline{WE}$ микросхемы (будет чтение) и 0 на $\overline{CE}$ (знак того, что мы обращаемся именно к этой микросхеме). Как только после этого на входе $\overline{OE}$ микросхемы памяти микроконтроллер установит 0, осуществится чтение числа из выбранной ячейки, и оно появится на линиях данных D0...D7 микросхемы памяти. Сказанное поясняется временными диаграммами, приведенными на **Рис. 3.4.**

Как аппаратно осуществляется в x51 реализация описанного алгоритма? Для того, чтобы понять это, нужно рассмотреть, как это было сделано в более простых с точки зрения шинной архитектуры микропроцессорах, например, в Z80.

У Z80 есть выводы трех так называемых шин — адреса, данных и управления. Под шинами обычно понимают некоторое количество линий (от 2...4 до 32...64), по которым передаются схожие сигналы и которые соединяются с соответствующими шинными выводами всех микросхем системы. Шестнадцать выводов Z80 (A0, A1, A2, ..., A14 и A15) называются адресными выходами, их соединяют с адресными входами микросхем памяти. Совокупность линий, осуществляющих эти соединения, и есть шина адреса. Соответственно, шина данных, состоящая из 8 проводников, соединяет выводы данных микросхем памяти и микропроцессора (у него, как и у микросхем памяти, их тоже 8, и обозначаются они также — D0, D1, ..., D7).

Из сигналов управления у Z80 нас интересуют только два — $\overline{RD}$ и $\overline{WR}$. С их помощью, как показано на **Рис. 3.5**, осуществляется управление памятью.

При обращении к памяти Z80 формирует на своих выводах A0...A15 адрес ячейки, к которой будет происходить обращение (запись или чтение). Как будет показано чуть ниже, соответствующая комбинация сигналов на

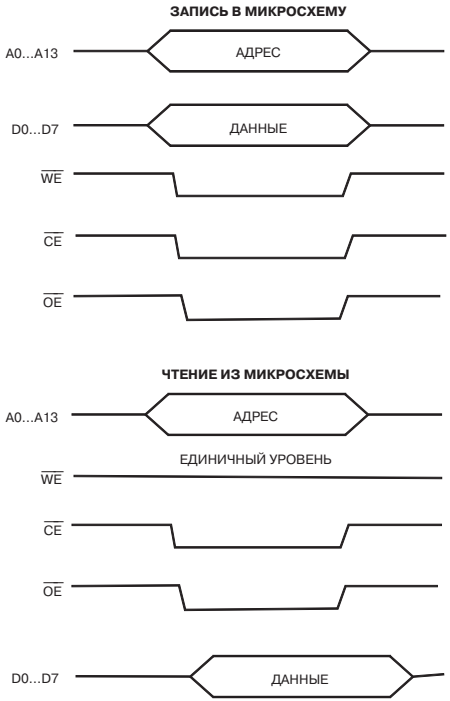


Рис. 3.4. Временные диаграммы работы с микросхемами памяти

A13...A15 сформирует при помощи дешифратора DD3 нулевой сигнал на входе $\overline{CE}$ той или иной микросхемы. При записи на выводах данных D0...D7 процессора появится байт, который предстоит записать в память, и чуть-чуть позже его появления — нулевой сигнал на выходе $\overline{WR}$ ($\overline{RD}$ при этом останется в 1). Как нетрудно догадаться, микросхема DD2 в любом цикле обращения (как чтения, когда $\overline{RD}$ = 0, так и записи, когда $\overline{WR}$ = 0) сформирует нуль на входах $\overline{OE}$ микросхем ОЗУ. Вот таким образом Z80 формирует на входах микросхем памяти требуемую для записи комбинацию сигналов.

Соответственно, при чтении выходы данных превращаются во входы, и на них процессор ничего не выводит, а после установки адреса он переводит в 0 сигнал $\overline{RD}$ ($\overline{WR}$ при этом остается в 1).

На **Рис. 3.5** помимо процессора и микросхем 555-й серии изображены две микросхемы памяти. Нетрудно сообразить, что в каждой из них есть своя нулевая, первая, вторая и т. д. ячейки памяти, вплоть до 2048-й. Когда

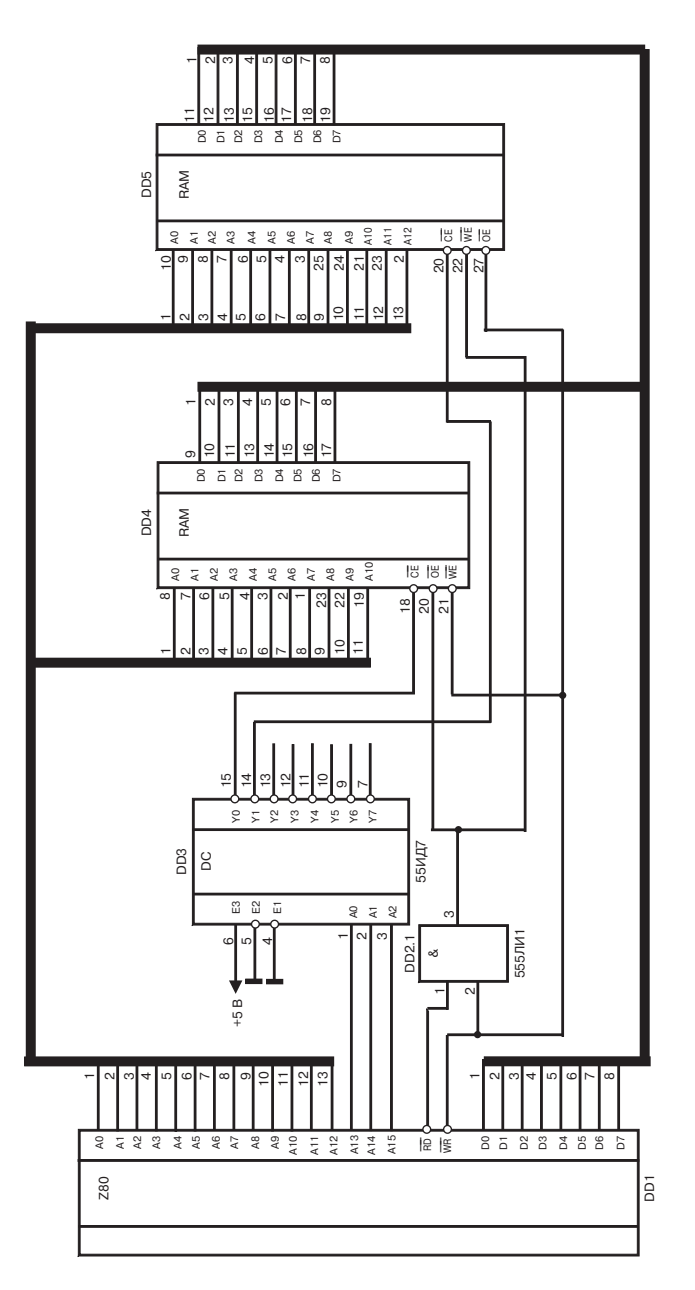


Рис. 3.5. Подключение микросхем памяти к микропроцессору Z80

я сказал чуть выше, что мы будем записывать информацию, например, в 84-ю ячейку, то как определить, в 84-ю ячейку какой из них микропроцессор занесет информацию?

Ответ прост. Обратите внимание, что входы $\overline{CE}$ микросхем соединены с выходами дешифратора DD3, в частности $\overline{CE}$ DD4 — с DD3.15, а $\overline{CE}$ DD5 — с DD3.14. На выходе DD3.15 нулевой сигнал возникнет в том случае, когда на своих трех старших адресных выходах микропроцессор установит следующую комбинацию сигналов: A15 = 0, A14 = 0, A13 = 0. Соответственно, нуль на DD3.14 появится, если A15 = 0, A14 = 0, A13 = 1. Следовательно, чтобы вести обмен с 84-й ячейкой микросхемы DD4, микропроцессор должен установить на адресных выходах комбинацию A0 = A1 = 0; A2 = 1; A3 = 0; A4 = 1; A5 = 0; A6 = 1; A7 = A8 = A9 = A10 = A13 = A14 = A15 = 0. Поскольку у DD4 отсутствуют входы A11 и A12, и эти адресные линии к ней не подводятся, то их состояние (нули или единицы) никак не сказывается на адресе выбираемой ячейки в этой микросхеме. Для определенности положим, что и они должны быть при обращении к DD4 нулевыми. Таким образом, в изображенной на **Рис. 3.5** системе к 84-й ячейке микросхемы DD4 микропроцессор обратится, установив на адресной шине 0000000001010100b = 0054h = 84. Другими словами, 84-я ячейка памяти нашей системы находится именно в DD4.

А что же 84-я ячейка DD5? Для обращения к ней процессор должен установить на адресной шине 0010000001010100b = 2054h = 8276. Иначе, 84-я ячейка памяти микросхемы DD5 является 8276-й ячейкой системы.

И последнее, на **Рис. 3.5** — обратите внимание на то, что на схеме шины адреса и данных обозначаются в виде линий, как бы расширившихся за счет вошедших в них линий адресов и данных. Благодаря этому схемы становятся более читаемыми, и линии шин перестают «съедать» львиную долю площади схемы.

После того, как мы завершили знакомство с тем, как работают с внешней памятью микропроцессоры второго поколения, вновь обратим свой взор на микроконтроллеры x51. И начнем с того, что задумаемся: могли ли его разработчики позволить себе такую роскошь, как использование, подобно Z80, 16-ти выводов микроконтроллера для линий адреса, 8-ми — для данных и двух — для управления? С учетом того, что еще 2 вывода нужны для питания и земли, 2 — для генератора, и 1 для сброса, на все остальные нужды остается... 9 выводов! Ясно, что столь нерационально использовать выводы микросхемы было нельзя. Также было нежелательно увеличивать число выводов микросхемы свыше 40. Как же быть?

Первое, на что пошли создатели x51 — на многофункциональное использование выводов микросхемы. Так, линии порта P2 при обращении к внешней памяти выводят адресные биты A8...A15. Далее, сигнал $\overline{RD}$ формируется на P3.7, а $\overline{WR}$ на P3.6.

В принципе, можно было бы по аналогии использовать линии порта P1 для адресных битов A0...A7, а линии P0 — для D0...D7. Но этот способ имеет один недостаток. У вас в распоряжении остается всего 6 линий (P3.0...P3.5), на которых информация может сохраняться неизменной независимо от того, есть ли обращение к внешней памяти или нет. На остальных же линиях она пропадает на время цикла обращения к внешней памяти.

Для решения проблемы (увеличения числа линий, информация на которых сохраняется независимо от того, работает ли микроконтроллер с внешним ОЗУ или нет) было сделано следующее. Линии порта P0 используются для ввода/вывода информации с шины данных. Но, кроме того, в первый момент обращения к памяти по этим же линиям выводятся адреса A0...A7, и одновременно с этим устанавливается в 1 сигнал на выходе ALE. Спустя 2 периода тактового генератора ALE сваливается в 0, и через несколько наносекунд после этого адресная информация пропадает с линий порта P0, давая возможность вести по ним обмен данными. А для того, чтобы использовать адреса A0...A7, применяют 8-битный регистр-защелку типа 555ИР22, информация в котором фиксируется по спаду сигнала на его входе STB. В качестве этого сигнала, как нетрудно догадаться, используется сигнал ALE (это основное его назначение). Сказанное иллюстрируется **Рис. 3.6.**

Таким образом, работа микроконтроллера с внешней памятью данных осуществляется следующим образом. Прочитав команду обращения к внешней памяти (о командах — чуть ниже), микроконтроллер выводит по линиям порта P2 старшие 8 бит адреса, а по линиям P0 — младшие. Одновременно с этим он устанавливает в 1 сигнал на выходе ALE. Когда эта 1 поступает на вход STB регистра-защелки DD3, информация с его входов D0...D7 напрямую поступает на выходы Q0...Q7, соединенные с младшими 8 линиями шины адреса. Спустя некоторое время ALE устанавливается в 0. Перепад из 1 в 0 на входе STB DD3 приводит к защелкиванию в регистрах выходов Q0...Q7 поступающей на них информации, и защелкнутые младшие 8 бит адреса остаются неизменными на выходах Q0...Q7 до следующего цикла обмена с внешней памятью. Старшие 8 бит, выводимые через P2, также не меняются до завершения текущего цикла работы с внешним ОЗУ.

Поскольку A0...A7 сохранены в регистре-защелке, они убираются с выводов порта P0, и последние превращаются в выводы шины данных. Если микроконтроллер записывает данные во внешнее ОЗУ, записываемый байт выводится им на P0.0...P0.7, и устанавливается в 0 сигнал на его выходе $\overline{WR}$ (P3.6). Появление же нулевых сигналов на входах $\overline{WE}$ и $\overline{OE}$, соответствующей микросхемы памяти приводит, как мы помним, к записи в нее байта, поданного на ее входы D0...D7. Нулевой сигнал на $\overline{OE}$, как вы должны были догадаться, формирует микросхема DD2. Сказанное иллюстрируется временными диаграммами на **Рис. 3.7.**

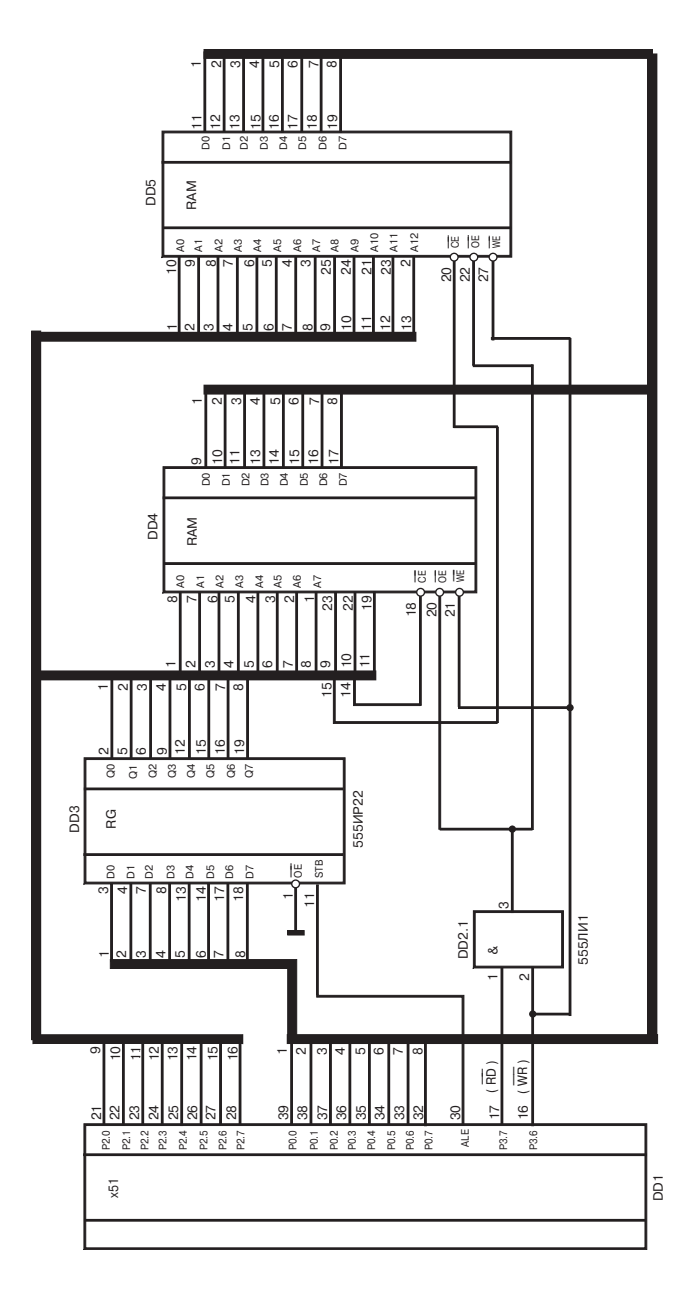


Рис. 3.6. Подключение микросхем памяти к микроконтроллеру семейства x51

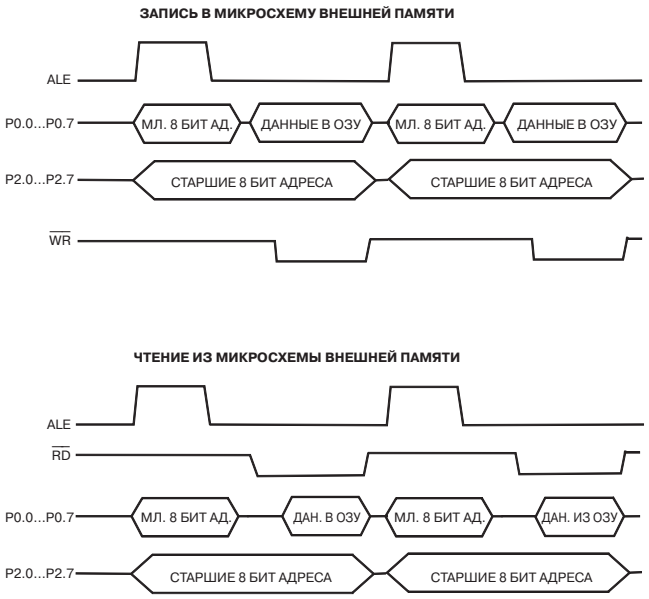


Рис. 3.7. Временные диаграммы циклов обмена микроконтроллера с внешней памятью данных

Вы уже, наверное, обратили внимание на то, что в отличие от схемы на **Рис. 3.5** в схеме на **Рис. 3.6** отсутствует дешифратор, управлявшийся адресами A13...A15. Вместо этого на входы $\overline{CE}$ микросхем памяти поданы сигналы с A13 и A14. Такое решение возможно, если вы точно знаете, что в вашей системе не будет использоваться более трех микросхем внешней памяти (дешифратор DD3 на **Рис. 3.5** позволял работать с 8 микросхемами). При обращении к DD4 микроконтроллер должен установить следующую комбинацию старших адресов: A13 = 0, A14 = A15 = 1. Соответственно для DD5 A13 = 1, A14 = 0, A15 = 1. А какая комбинация потребовалась бы для 3-й микросхемы, если ее вход $\overline{CE}$ соединить с A15? Правильно, A13 = 1, A14 = 1, A15 = 0.

Процесс чтения информации микроконтроллером аналогичен только что рассмотренному процессу записи с той лишь разницей, что микроконтроллер устанавливает в 0 не $\overline{WR}$, а $\overline{RD}$, и данные на шину данных поступают не с него, а с соответствующей микросхемы памяти.

Отмечу в заключение, что описанный способ выдачи адресной информации по выводам шины данных очень распространен (он применялся, например, в процессорах 8086/8088, на которых были выполнены первые IBM PC), и называется мультиплексированием шин адреса/данных.

3.4. Регистр-указатель данных

Последний из рассматриваемых в этой главе регистров — регистр-указатель данных DPTR. В отличие от предыдущих, 8-битных (однобайтных) регистров он — 16-битный (двухбайтный). Его основное назначение — хранение адреса ячейки внешней памяти данных при обращении микроконтроллера к этой памяти.

Вообще у x51 есть всего две нормальные команды для работы с внешней памятью данных — `MOVX A,@DPTR` и `MOVX @DPTR,A`. Первая из них осуществляет чтение данных в аккумулятор из ячейки внешней памяти, адрес которой помещен в DPTR. Вторая же, ясное дело, осуществляет обратную операцию — запись данных из аккумулятора в ячейку внешней памяти, адрес которой хранится в DPTR. Замечу, что буква X в конце команды `MOVX` подчеркивает, что команда предназначена для работы с внешней памятью данных (eXternal — внешний). Еще замечу, что знак @ означает, что данные пересылаются в ячейку, адрес которой хранится в регистре, записанном после этого знака (в данном случае DPTR). С подобным мы еще не раз встретимся при рассмотрении системы команд микроконтроллера — там вы, например, столкнетесь с командами `MOV A,@R0` или `MOV @R1,A`. Попробуйте самостоятельно догадаться, что микроконтроллеру предписывают выполнить эти команды.

Ну, а занести адрес интересующей нас ячейки в регистр DPTR можно командой `MOV DPTR,#data16`, где #data16 — 16-битное число, которое может принимать значение от 0 до 65535 (0FFFFh). Вспомним наш пример из предыдущего подраздела, когда мы говорили о том, что микроконтроллер записывает число 145 в 84-ю ячейку внешнего ОЗУ. Сделать это можно при помощи следующего фрагмента:

```
;
  MOV   A,#145            ;ЗАНОСИМ В АККУМУЛЯТОР ЧИСЛО 145
  MOV   DPTR,#84          ;ЗАНОСИМ В DPTR ЧИСЛО 84 (АДРЕС ЯЧЕЙКИ)
  MOVX  @DPTR,A           ;ЗАПИСЫВАЕМ 145 В 84-Ю ЯЧЕЙКУ
;
```

Попробуйте самостоятельно модифицировать этот фрагмент таким образом, чтобы он предписал микроконтроллеру прочитать в аккумулятор число из 84-й ячейки внешнего ОЗУ.

3.5. Пример: подпрограмма, использующая регистры МК

Как уже было замечено в предыдущей главе, нельзя научиться плавать в бассейне без воды. Точно также нельзя научиться работать с микроконтроллерами, не освоив основных навыков написания программ. Поэтому с целью закрепления материала, касающегося регистров, я предлагаю вам как пример одну довольно важную для дальнейших ваших изысканий подпрограмму, которая использует почти все рассмотренные в этой главе регистры. Заодно я покажу вам, как составляется блок-схема алгоритма. Замечу, что на самом деле составление блок-схем — это самое главное в программировании. Когда блок-схема составлена, написать текст программы — дело техники, причем почти неважно, на каком языке писать и для какого микроконтроллера.

Теперь о подпрограмме, которую нам предстоит рассмотреть. Это подпрограмма преобразования хранящегося в регистрах R3 и R2 микроконтроллера двухбайтного числа (т. е. лежащего в диапазоне от 0000h до 0FFFFh) в так называемое двоично-десятичное.

Что представляет из себя двоично-десятичное представление числа? Проще всего это объяснить на следующем примере. Воспользуйтесь, как было описано в главе 1, Windows-калькулятором и убедитесь, что 0FF00h = 1111111100000000b = 65280d. Это шестнадцатеричное, двоичное и обычное десятичное представление числа 65280. Двоично-десятичное его представление выглядит следующим образом: 0110 0101 0010 1000 0000. Если вы внимательно всмотритесь в него, то обнаружите, что старшие 4 бита (0110) кодируют в двоичном представлении цифру 6, идущие вслед за ними 4 бита (0101) — цифру 5, следующие 4 бита (0010) — цифру 2, далее восьмерку (1000) и 0 (0000). Таким образом, каждый десятичный разряд нашего числа кодируется четырехбитным двоичным числом. Напомню, каковы эти двоичные числа: 0000 — это 0, 0001 — это 1, 0010 — это 2, 0011 — это 3, 0100 — это 4, 0101 — это 5, 0110 — это 6, 0111 — это 7, 1000 — это 8 и 1001 — это 9. Если понятно, то какому числу соответствует следующее двоично-десятичное представление: 0010 0001 1001 0101 0111? Правильно, 21957. Кто этого не понял, еще раз внимательно прочитайте текущий абзац.

Для чего нужно двоично-десятичное представление чисел? Для подпрограмм, работающих с цифровыми знакосинтезирующими индикаторами. Прежде, чем отобразить тот или иной результат на семисегментном индикаторе, жидкокристаллическом или светодиодном, отображаемое число нужно преобразовать в двоично-десятичное представление. В этом мы убедимся в следующей главе, когда будем рассматривать вывод информации на один из подобных индикаторов.

Теперь более конкретно, что, откуда и куда мы будем преобразовывать. Исходное число будет храниться в регистрах R3 и R2: старшие 8 бит в R3,

младшие — в R2. Результат преобразования будет располагаться в регистрах R6, R5, и R4. При этом в младших 4 битах R6 будет располагаться старший десятичный разряд (десятки тысяч), в старших 4 битах R5 — разряд единиц тысяч, в младших 4 битах R5— разряд сотен, а в старших и младших 4 битах R4 — разряды десятков и единиц соответственно. Например, уже упомянутое 0FF00h должно оказаться в регистрах R6, R5, и R4 в следующем виде: R6 = 00000110b, R5 = 01010010b, R4 = 10000000b — сравните с тем, как 0FF00h было представлено двумя абзацами выше.

А теперь — самое главное, каким образом из числа 0FF00h (или любого другого) выделить, сколько в нем десятков тысяч, единиц тысяч, сотен, десятков и единиц? Как без помощи Windows-калькулятора определить, что 0FF00h состоит из 6 десятков тысяч, 5 тысяч, 2 сотен, 8 десятков и 0 единиц? На самом деле очень просто, хотя пока у вас нет никаких навыков, вы даже и не представляете, как это делается.

Ответьте пожалуйста, сколько раз подряд из числа, принадлежащего к диапазону от 60000 до 69999, можно вычитать 10000, чтобы при этом остаток был больше или равен нулю? Не правда ли, ровно 6 раз. Не 7, не 5, не 3, а именно 6. Когда вы попытаетесь это сделать в седьмой раз, у вас получится отрицательный результат. Соответственно, из числа, лежащего в диапазоне 20000...29999 можно до получения нулевого или положительного остатка вычесть 10000 не более 2 раз, для числа из диапазона 40000...49999 — не более 4 раз и т. д. Чувствуете, к чему я клоню?

Для тех, кто еще не догадался, формулирую. Вычитая раз за разом из преобразуемого числа 10000, определим, сколько раз эти 10000 содержатся в нем — это будет разряд десятков тысяч. Далее будем из остатка точно также вычитать 1000 и так определим, сколько раз в нем содержится 1000 — это будет разряд тысяч. И так далее.

Отсюда, алгоритм преобразования должен выглядеть следующим образом.

А) Вначале разряд десятков тысяч преобразуемого числа принимаем равным 0.

Б) После этого из преобразуемого числа нужно вычесть 10000 и проверить, остаток больше или равен 0, или нет. Если да, то разряд десятков тысяч увеличиваем на 1 (т. е. после первого удачного вычитания он будет равен 1) и снова повторяем действия, сформулированные в пункте (Б). После второго удачного вычитания разряд десятков тысяч будет равен 2, после третьего — трем и т. д.

Рано или поздно, но после какого-то вычитания остаток станет отрицательным. (Кстати, если преобразуемое число не более 9999, он отрицательным будет уже после первого вычитания.) В этом случае вступает в действие пункт (В).

В) Оставить неизменным разряд десятков тысяч, прибавить к полученному отрицательному остатку 10000 и запомнить его и найденный разряд десятков тысяч для дальнейшего преобразования. В рассматриваемом на-

ми примере с преобразованием числа 0FF00h перед седьмым вычитанием остаток будет равен 5280, а разряд десятков тысяч — 6. После 7-го вычитания в остатке остается (простите за повторение) минус 4720. Отрицательный остаток сигнализирует нам, что с вычитаниями мы слегка переборщили, и что надо действовать опять по пункту (В) — прибавить к этому остатку 10000, получив при этом исходные 5280, запомнить это число и то, что разряд десятков тысяч в 0FF00h равен 6. А далее, как нетрудно догадаться, перейти к нахождению разряда единиц тысяч.

Определять разряд тысяч будем по тому же алгоритму:

Г) Вначале разряд единиц тысяч преобразуемого числа принимаем равным 0.

Д) После этого из преобразуемого числа нужно вычесть 1000 и проверить, остаток больше или равен 0, или нет. Если да, то разряд единиц тысяч увеличиваем на 1 (т. е. после первого удачного вычитания он будет равен 1), и снова повторяем действия, сформулированные в пункте (Д). После второго удачного вычитания разряд единиц тысяч будет равен 2, после третьего — трем и т. д.

Когда после какого-то вычитания остаток станет отрицательным, то вступает в действие пункт (Е).

Е) Оставить неизменным разряд единиц тысяч, прибавить к остатку 1000 и запомнить его и полученный разряд единиц тысяч для дальнейшего преобразования. В рассматриваемом нами примере с 0FF00h мы начали вычитать тысячи из 5280, доставшегося нам в наследство от этапа определения разряда десятков тысяч. Перед шестым вычитанием остаток будет равен 280, а разряд единиц тысяч— 5. После 6-го вычитания в остатке будет минус 720. Отрицательный остаток опять сигнализирует нам, что с вычитаниями мы переборщили, и что надо действовать снова по пункту (Е) — прибавить к этому отрицательному остатку 1000, получив при этом исходные 280, запомнить это число и то, что разряд единиц тысяч в 0FF00h равен 5. А далее, как очевидно, перейти к нахождению разряда сотен.

Описывать, как мы должны из 280 раз за разом вычитать 100, я не буду — это должно быть уже очевидным для всех, кто читает настоящие строки. Если что-то непонятно — попробуйте еще раз внимательно прочесть содержимое последних восьми абзацев. Если же и это не поможет — не опускайте руки, у вас в распоряжении мой электронный адрес, попытайтесь сформулировать, что же вы не поняли, и киньте мне письмо с этой формулировкой.

После определения разряда сотен определим разряд десятков, а остаток в результате определения десятков и будет разрядом единиц. Вот, собственно, и весь алгоритм.

Сказанное иллюстрируется блок-схемой алгоритма, приведенной на **Рис. 3.8.**

Блок-схема представляет из себя последовательность идущих сверху вниз прямоугольников и ромбов, внутри которых кратко описаны действия, которым они соответствуют, а стрелки, их соединяющие, показыва-

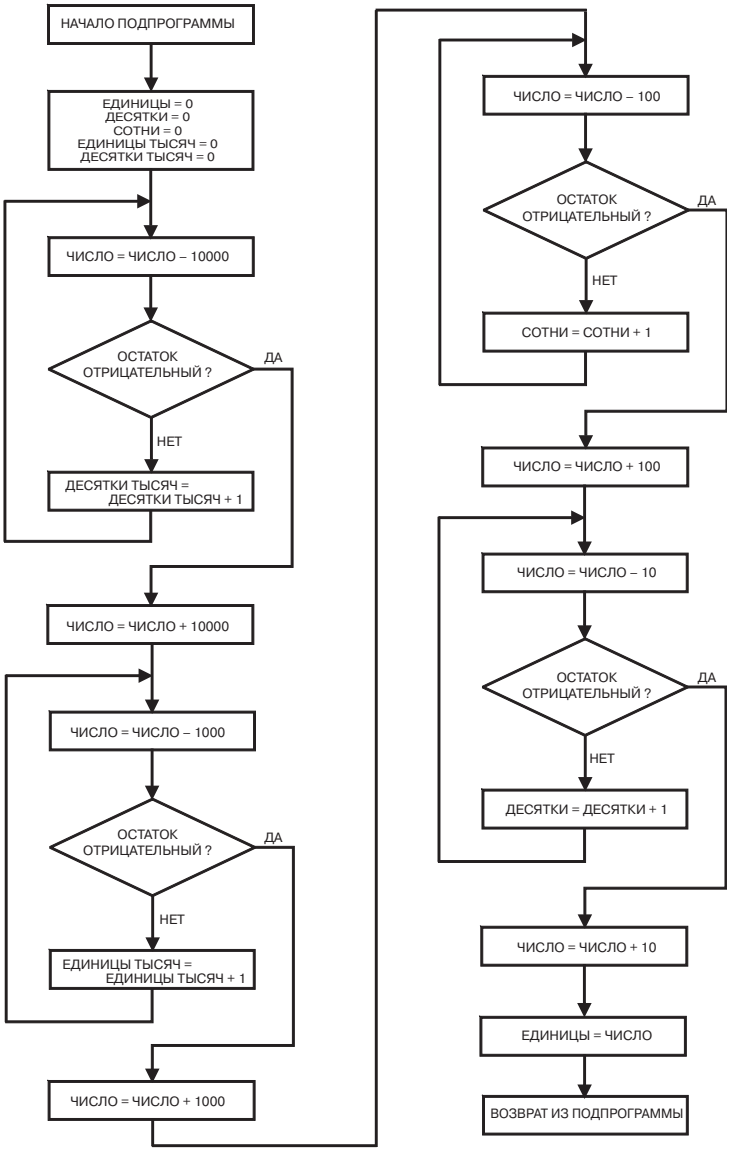


Рис. 3.8. Блок-схема алгоритма преобразования чисел из двоичного в двоично-десятичный формат

ют порядок выполнения этих действий. Действия, описанные в ромбах, являются проверкой тех или иных условий (в нашем случае проверка результата вычитания, отрицателен он или нет). Поскольку при невыполнении проверяемого условия мы должны совершить одну последовательность действий, а при выполнении — другую, то из каждого ромба выходит две стрелки (одна с надписью «ДА», т. е. соответствующая выполнению, другая — «НЕТ» — невыполнению). Собственно, больше и говорить нечего — сравните то, что говорилось в семи последних абзацах с тем, что изображено на **Рис. 3.8** и убедитесь, что алгоритм, изображенный графически, соответствует алгоритму, сформулированному словами. Но графическое представление проще и нагляднее.

Приведенная на **Рис. 3.8** блок-схема, если так можно выразиться, машинно-независима, т. е. никак не учитывает особенностей того или иного микроконтроллера (и, следовательно, применима к любому из них). Но нам нужно составить блок-схему, ориентированную на регистры и систему команд x51. Такая схема приведена на **Рис. 3.9.**

Как видите, структура алгоритма осталась неизменной. Просто действия описаны поконкретнее — вместо «вычесть из результата 10000» стоит «R3R2-10000», вместо «результат вычитания отрицательный?» стоит «CY = 1?» и т. д.

На что я хотел бы обратить выше внимание? Во-первых, в приведенном ниже тексте подпрограммы, соответствующей этому алгоритму, вы обнаружите, что вместо того, чтобы вычитать из преобразуемого числа вначале 10000, затем 1000, затем 100 и затем 10 (это 4 различающихся фрагмента программы) я вначале заношу 10000 (1000, 100 или 10) в регистр DPTR, а затем вычитаю его из R3R2. В этом случае все 4 вычитания выполняются идентично, поэтому они могут быть оформлены в виде подпрограммы (я назвал ее R32MNDPT), которая вызывается по мере необходимости. Аналогичная подпрограмма (R32PLDPT) осуществляет прибавление по мере надобности к числу в R3R2 10000, 1000, 100 и 10.

Во-вторых, разряды десятков тысяч, единиц тысяч, сотен, десятков и единиц я занулял не по мере необходимости, как в описанном словами алгоритме, а в самом начале программы (записью нулевого значения в регистры R6, R5 и R4, где будет храниться результат преобразования).

В-третьих, поскольку разряд единиц тысяч и разряд сотен должны в результате преобразования храниться в одном и том же регистре (R5), то единицы тысяч я подсчитываю в дополнительно используемом регистре R7, а сотни — в R5, и затем младшие 4 бита R7 я переношу в старшие, суммируя его с R5. В результате в старшей половине R5 окажутся единицы тысяч, в младшей — сотни, что нам и требовалось. Аналогично я действую и в отношении десятков и единиц, которые я собираю в регистре R4.

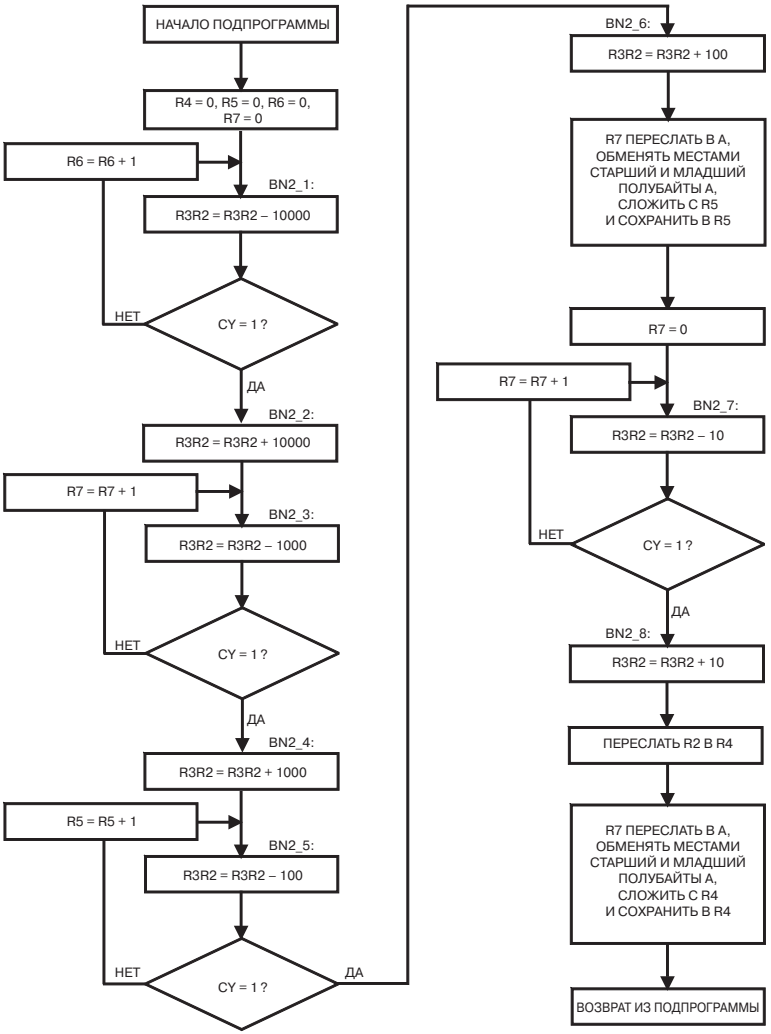


Рис. 3.9. Более подробная блок-схема алгоритма преобразования чисел из двоичного в двоично-десятичный формат

В-четвертых, когда стрелка, показывающая переход к следующей команде, указывает не на ближайший снизу ромб или прямоугольник блок-схемы, это означает, что следующей должна выполняться не нижеидущая за текущей команда, а другая, расположенная несколькими командами выше или ниже. Для правильного перехода на эту (расположенную выше или ниже) команду она должна быть снабжена меткой (до 8 букв и цифр с двоеточием на конце). Эти метки я также внес в блок-схему — по стандартным правилам делать это не обязательно, но практика показывает, что если не пренебрегать этим, то количество ошибок, возникающих при написании программы, заметно сокращается.

Ну, а теперь — текст самой программы с соответствующими комментариями.

Программа преобразования чисел из двоичного в двоично-десятичный формат

```
; ПРОГРАММА, ДЕМОНСТРИРУЮЩАЯ РАБОТУ
; ПОДПРОГРАММЫ ПРЕОБРАЗОВАНИЯ ДВУХБАЙТНЫХ
; ЧИСЕЛ ИЗ ШЕСТНАДЦАТИРИЧНОГО В ДВОИЧНО-ДЕСЯТИЧНЫЙ
; ФОРМАТ
;                        *****
;
R7      .EQU  7          ;АДРЕСА РЕГИСТРОВ R0-R7
R6      .EQU  6
R5      .EQU  5
R4      .EQU  4
R3      .EQU  3
R2      .EQU  2
R1      .EQU  1
R0      .EQU  0
ACC     .EQU  0E0H       ;АДРЕС АККУМУЛЯТОРА
B       .EQU  0F0H       ;АДРЕС РЕГИСТРА B
PSW     .EQU  0D0H       ;АДРЕС РЕГИСТРА (СЛОВА) СОСТОЯНИЯ
SP      .EQU  81H        ;АДРЕС УКАЗАТЕЛЯ СТЕКА
DPL     .EQU  82H        ;АДРЕС МЛАДШЕЙ ПОЛОВИНЫ DPTR
DPH     .EQU  83H        ;АДРЕС СТАРШЕЙ ПОЛОВИНЫ DPTR
P0      .EQU  80H        ;АДРЕС РЕГИСТРА ПОРТА P0
P1      .EQU  90H        ;АДРЕС РЕГИСТРА ПОРТА P1
P2      .EQU  0A0H       ;АДРЕС РЕГИСТРА ПОРТА P2
P3      .EQU  0B0H       ;АДРЕС РЕГИСТРА ПОРТА P3
B.0     .EQU  0F0H       ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА B
B.1     .EQU  0F1H
B.2     .EQU  0F2H
B.3     .EQU  0F3H
B.4     .EQU  0F4H
```

```
B.5    .EQU  0F5H
B.6    .EQU  0F6H
B.7    .EQU  0F7H
ACC.0  .EQU  0E0H       ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ АККУМУЛЯТОРА
ACC.1  .EQU  0E1H
ACC.2  .EQU  0E2H
ACC.3  .EQU  0E3H
ACC.4  .EQU  0E4H
ACC.5  .EQU  0E5H
ACC.6  .EQU  0E6H
ACC.7  .EQU  0E7H
PSW.0  .EQU  0D0H       ;АДРЕСА ОТДЕЛЬНЫХ БИТОВ РЕГИСТРА PSW
PSW.1  .EQU  0D1H
PSW.2  .EQU  0D2H
PSW.3  .EQU  0D3H
PSW.4  .EQU  0D4H
PSW.5  .EQU  0D5H
PSW.6  .EQU  0D6H
PSW.7  .EQU  0D7H
P0.0   .EQU  080H       ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P0
P0.1   .EQU  081H
P0.2   .EQU  082H
P0.3   .EQU  083H
P0.4   .EQU  084H
P0.5   .EQU  085H
P0.6   .EQU  086H
P0.7   .EQU  087H
P1.0   .EQU  090H       ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P1
P1.1   .EQU  091H
P1.2   .EQU  092H
P1.3   .EQU  093H
P1.4   .EQU  094H
P1.5   .EQU  095H
P1.6   .EQU  096H
P1.7   .EQU  097H
P2.0   .EQU  0A0H       ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P2
P2.1   .EQU  0A1H
P2.2   .EQU  0A2H
P2.3   .EQU  0A3H
P2.4   .EQU  0A4H
P2.5   .EQU  0A5H
P2.6   .EQU  0A6H
P2.7   .EQU  0A7H
P3.0   .EQU  0B0H       ;АДРЕСА ОТДЕЛЬНЫХ ЛИНИЙ ПОРТА P3
P3.1   .EQU  0B1H
P3.2   .EQU  0B2H
```

```
P3.3   .EQU  0B3H
P3.4   .EQU  0B4H
P3.5   .EQU  0B5H
P3.6   .EQU  0B6H
P3.7   .EQU  0B7H
;
;
       .ORG  0          ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 0
;
       LJMP  START      ;НА КОМАНДУ ПОСЛЕ МЕТКИ START
;
       .ORG  100H       ;НИЖЕСЛЕДУЮЩАЯ КОМАНДА С АДРЕСА 100H
;
START:                  ;ПРОГРАММА, ЗАГРУЖАЮЩАЯ ДВУХБАЙТНОЕ
       MOV   DPTR,#0FF00H ;ЧИСЛО В R3 И R2 И ВЫЗЫВАЮЩАЯ
       MOV   R2,DPL     ;ЗАТЕМ ПОДПРОГРАММУ ПРЕОБРАЗОВАНИЯ
       MOV   R3,DPH     ;ЭТОГО ЧИСЛА В ДВОИЧНО-ДЕСЯТИЧНЫЙ
       LCALL BN2BCD     ;ФОРМАТ
       SJMP  START
;
BN2BCD:                 ;ПРЕОБРАЗУЕМОЕ ЧИСЛО В R3R2
                        ;(МЛ.БАЙТ В R2)
       MOV   R4,#0      ;НАЧАЛЬНАЯ УСТАНОВКА
       MOV   R5,#0
       MOV   R6,#0
       MOV   R7,#0
;
       MOV   DPTR,#10000
BN2_1:
       LCALL R32MNDPT   ;R3R2 - 10000
       JC    BN2_2      ;ПЕР. НА BN2_2, ЕСЛИ CY=1
       INC   R6         ;R6=R6 + 1 - ЭТО ДЕЛАЕМ, ЕСЛИ CY=0
       SJMP  BN2_1      ;ПЕРЕХОД НА BN2_1
BN2_2:
       LCALL R32PLDPT   ;R3R2 + 10000
;
       MOV   DPTR,#1000
BN2_3:
       LCALL R32MNDPT   ;R3R2 - 1000
       JC    BN2_4      ;ПЕР. НА BN2_4, ЕСЛИ CY=1
       INC   R7         ;R7=R7 + 1 - ЭТО ДЕЛАЕМ, ЕСЛИ CY=0
       SJMP  BN2_3      ;ПЕРЕХОД НА BN2_3
BN2_4:
       LCALL R32PLDPT   ;R3R2 + 1000
;
       MOV   DPTR,#100
```

```
BN2_5:
       LCALL R32MNDPT   ;R3R2 - 100
       JC    BN2_6      ;ПЕР. НА BN2_6, ЕСЛИ CY=1
       INC   R5         ;R5=R5 + 1 - ЭТО ДЕЛАЕМ, ЕСЛИ CY=0
       SJMP  BN2_5      ;ПЕРЕХОД НА BN2_5
BN2_6:
       LCALL R32PLDPT
;
       MOV   A,R7       ;ПЕРЕСЛАЛИ ИЗ R7 В A
       SWAP  A          ;ОБМЕНЯЛИ МЕСТАМИ СТАРШИЙ И МЛАДШИЙ
                        ;ПОЛУБАЙТЫ A
       ADD   A,R5       ;ПРИБАВИЛИ К A R5
       MOV   R5,A       ;ПЕРЕСЛАЛИ СУММУ В R5
       MOV   R7,#0      ;R7 = 0
;
       MOV   DPTR,#10
BN2_7:
       LCALL R32MNDPT   ;R3R2 - 10
       JC    BN2_8      ;ПЕР. НА BN2_8, ЕСЛИ CY=1
       INC   R7         ;R7=R7 + 1 - ЭТО ДЕЛАЕМ, ЕСЛИ CY=0
       SJMP  BN2_7      ;ПЕРЕХОД НА BN2_7
BN2_8:
       LCALL R32PLDPT   ;R3R2 + 10
;
       MOV   A,R2
       MOV   R4,A       ;R2 ПЕРЕСЛАЛИ ЧЕРЕЗ АККУМУЛЯТОР В R4
;
       MOV   A,R7       ;ПЕРЕСЛАЛИ ИЗ R7 В A
       SWAP  A          ;ОБМЕНЯЛИ МЕСТАМИ СТАРШИЙ И МЛАДШИЙ
                        ;ПОЛУБАЙТЫ A
       ADD   A,R4       ;ПРИБАВИЛИ К A R4
       MOV   R4,A       ;ПЕРЕСЛАЛИ СУММУ В R4
       RET              ;ВОЗВРАТ ИЗ ПОДПРОГРАММЫ
;
;ПОДПРОГРАММЫ ДЛЯ BN2BCD
;
R32PLDPT:
       MOV   A,R2       ;R2 В АККУМУЛЯТОР
       ADD   A,DPL      ;СКЛАДЫВАЕМ ЕГО С МЛ.БАЙТОМ DPTR
       MOV   R2,A       ;ВОЗВРАЩАЕМ СУММУ В R2
       MOV   A,R3       ;R3 В АККУМУЛЯТОР
       ADDC  A,DPH      ;СКЛАДЫВАЕМ ЕГО С БИТОМ ПЕРЕНОСА
                        ;И С МЛ.БАЙТОМ DPTR
       MOV   R3,A       ;ВОЗВРАЩАЕМ СУММУ В R3
       RET
;
```

```
R32MNDPT:
        CLR   C          ;ОЧИЩАЕМ CY
        MOV   A,R2       ;R2 В АККУМУЛЯТОР
        SUBB  A,DPL      ;ВЫЧИТАЕМ ИЗ НЕГО МЛ.БАЙТ DPTR И CY
        MOV   R2,A       ;ВОЗВРАЩАЕМ РАЗНОСТЬ В R2
        MOV   A,R3       ;R3 В АККУМУЛЯТОР
        SUBB  A,DPH      ;ВЫЧИТАЕМ ИЗ НЕГО БИТ ПЕРЕНОСА
                         ;И СТ.БАЙТ DPTR
        MOV   R3,A       ;ВОЗВРАЩАЕМ РАЗНОСТЬ В R3
        RET
;
;
        .END
```

Долго комментировать его после обсуждения блок-схемы программы нет необходимости. Собственно программа представляет собой занесение в регистры R2 и R3 числа 65280d = 0FF00h и вызов подпрограммы преобразования BN2BCD. Сама же последняя написана в соответствии с блок-схемой на **Рис. 3.9** и комментариями к ней.

Теперь о новых для вас командах, с которыми вы столкнулись при знакомстве с текстом подпрограммы. Самая интересная из них — `JC` (`JC BN2_2`, `JC BN2_4`, `JC BN2_6` и `JC BN2_8`). Это так называемая команда условного перехода. В ходе ее выполнения микроконтроллер проверяет состояние флага переноса CY, и если он установлен (т. е. CY = 1), то переходит к выполнению команды, отмеченной меткой, имя которой идет вслед за JC (соответственно `BN2_2`, `BN2_4`, `BN2_6` или `BN2_8`). Естественно, если флаг переноса сброшен, то никакого перехода нет, и выполняется команда, идущая вслед за `JC xxx` (xxx— помещенная в скобки в предыдущем предложении метка адреса перехода).

Кстати, команд условного перехода довольно много — переход, если флаг CY установлен (`JC xxx`), если флаг CY сброшен (`JNC xxx`), если содержимое аккумулятора равно 0 (`JZ xxx`), если оно не равно 0 (`JNZ xxx`) и ряд других. Их разумное использование позволяет организовывать простые, но в то же время весьма эффективные алгоритмы.

`INC Rn` (n = 0...7) — это команда инкрементирования (т. е. увеличения на 1) содержимого соответствующего регистра. Проста и удобна, мы будем пользоваться ею довольно часто.

`SWAP A`. — команда обмена местами старших и младших 4 бит аккумулятора. В основном она используется для того, чтобы собрать в аккумуляторе содержимое двух регистров — одного в младшей половине аккумулятора, другого — в старшей. Для этого содержимое одного регистра пересылается в аккумулятор, старший и младший полубайты обмениваются местами, и затем к содержимому аккумулятора прибавляется содержимое второго регистра. Если старшие 4 бита обоих регистров изначально были

нулями, то после описанной операции в старших 4 битах аккумулятора будет храниться содержимое первого из упомянутых регистров, а в младших 4 битах — второго.

Следующие две команды часто употребимы — это сложение аккумулятора с одним из регистров и вычитание из аккумулятора содержимого регистра. Команда `ADD A,reg` (reg — любой регистр, в данном случае R5: `ADD A,R5`) вынуждает микроконтроллер сложить с аккумулятором содержимое R5 и результат оставить в аккумуляторе. Еще одна разновидность команды сложения — `ADDC A,reg` (reg — опять-таки любой регистр, в данном случае DPH: `ADDC A,DPH`) — складывает с аккумулятором содержимое старшей половины регистра DPTR — 8-битного регистра DPH — и вдобавок к нему прибавляет бит переноса. (Кстати, младшая половина DPTR, также 8-битная, называется DPL.)

Вышерассмотренные две команды (`ADD` и `ADDC`) складывают 8-битные числа (от 00000000b до 11111111b). А как быть, если нужно сложить, например, два шестнадцатибитных числа? С этим мы познакомимся на примере подпрограммы `R32PLDPT`. Она складывает два шестнадцатибитных числа, одно из которых хранится в регистрах R2 (младший бит) и R3 (старший), а другое — в DPTR. Вначале содержимое R2 пересылается в аккумулятор, складывается с DPL и возвращается обратно в R2. Затем то же самое совершается с регистрами R3 и DPH, но используемая команда `ADDC` складывает с содержимым регистров еще и бит переноса.

Для чего при втором сложении нужно прибавлять бит переноса? Вспомните, как нас учили в школе складывать «в столбик». Записываем одно число под другим и вначале складываем между собой две правые цифры (т. е. младшие биты, единицы). Если их сумма больше 10, то «единичка идет на ум», и при сложении следующих двух цифр (вторых справа, т. е. десятков) к ним нужно прибавить еще и эту единицу.

Собственно, команда `ADDC` именно это и делает — если при первом сложении сумма оказалась больше 16 (т. е. больше, чем 10h), то в аккумуляторе окажется число, на которое сумма превышает 16, и установится флаг переноса, та самая единичка. Поэтому, когда мы осуществим при помощи команды `ADDC` сложение следующих цифр, эта единичка учтется правильным образом. Кстати, если сумма младших двух цифр будет меньше 16, флаг переноса не установится, и при следующем сложении с помощью команды `ADDC` никакой единицы к ним не прибавится. Таким образом, команда `ADDC` специально предназначена именно для этого случая — сложения второй, третьей и т. д. пары байт в многобайтных числах.

Подведем небольшой итог. Когда мы складываем многобайтные числа (длиной два, три и более байт), то первое сложение нужно осуществлять при помощи команды `ADD`, которая не принимает во внимание никаких переносов, а все последующие — с помощью команды `ADDC`. Подпрограм-

ма `R32PLDPT` является примером такого сложения многобайтных (здесь двухбайтных) чисел.

Аналогична и подпрограмма вычитания одного двухбайтного числа из другого (`R32MNDPT`), но вместо команд сложения в нее, естественно, входят команды вычитания. Здесь также есть своя хитрость. Вспомним опять школьные годы, вычитание «в столбик». По-прежнему, первыми вычитаются самые правые, младшие биты. При этом, если нам нужно вычесть из меньшей цифры большую, мы «занимаем единичку» (осуществляем «заем») из более старшего бита уменьшаемого (т. е. из той цифры уменьшаемого, которая стоит левее используемой в текущий момент). А далее, когда мы будем осуществлять вычитание следующей цифры вычитаемого из той цифры уменьшаемого, где мы что-то заняли, последнюю сначала нужно уменьшить на 1, а затем что-то из нее вычитать.

Для осуществления описанных действий в микроконтроллерах x51 есть команда вычитания с заемом `SUBB`. Она вначале вычитает из уменьшаемого содержимое бита переноса (т. е. 1, если флаг CY установлен, и 0, если сброшен; кстати, обратите внимание на то, что в этом случае бит переноса фактически является битом заема), а затем — и само вычитаемое. Собственно, именно это нам и нужно для организации программ вычитания. Но здесь есть один нюанс: нужно помнить, что вычитание младших цифр — первое, ему не предшествовал никакой заем, и для получения правильного результата нужно либо использовать команду вычитания без заема (такие есть у многих микроконтроллеров, но не у x51), либо использовать команду вычитания с заемом `SUBB`, но перед этим принудительно установить в 0 флаг CY. Именно это и делается при помощи знакомой вам команды `CLR C`.

Вернемся теперь к самой программе. Я рекомендую загрузить ее в симулятор EMF52L (том 2, приложение 7) и осуществить ее выполнение по шагам с внимательным анализом чисел в регистрах и состояния флага CY. Если вы не поленитесь и благополучно справитесь с этим заданием, то заложите прочный фундамент в формирование у вас навыков написания программ. А для закрепления этого первого навыка я рекомендую вам написать по образу и подобию рассмотренной подпрограмму преобразования в двоично-десятичный формат трехбайтного числа (т. е. лежащего в диапазоне от 000000h до 0FFFFFFh; при этом преобразуемое число должно изначально находиться в R3R2R1, а результат после преобразования — в R7R6R5R4).

3.6. Краткие выводы

Итак, уважаемые читатели, мы познакомились с главными регистрами микроконтроллеров семейства x51. Параллельно с этим мы рассмотрели, как наши микроконтроллеры сопрягаются с внешней памятью данных, какие микросхемы памяти для этого наиболее применимы и какими ко-

мандами осуществляется обмен данными с ней. А для закрепления знакомства с регистрами мы с вами рассмотрели программу преобразования двухбайтного числа в двоично-десятичный формат — в ней использованы почти все регистры, рассмотренные в этой главе. Кстати, обратите внимание на то, что при написании этой программы я попытался все используемые ею данные размещать исключительно в регистрах, а не в памяти, что, как упоминалось выше, необходимо для быстрого ее выполнения микроконтроллером. В данном случае это, как видите, удалось.

Необходимо отметить, что мы рассмотрели далеко не все регистры нашего микроконтроллера. За пределами нашего рассмотрения остались регистры, управляющие дополнительными аппаратными узлами микроконтроллера — таймерами/счетчиками, приемопередатчиком, системой прерывания и т. д. Не пугайтесь этих слов — в главах 6 и 8 мы познакомимся с этими устройствами, и вы увидите, что не так уж они и сложны. Пока же просто запомните, что мы рассмотрели только часть регистров нашего микроконтроллера.

Также, как и в предыдущих главах, мы с вами рассмотрели фрагменты программ и познакомились с еще несколькими командами микроконтроллеров. Помимо этого, вы впервые столкнулись с блок-схемой программы— совершенно неотъемлемым элементом программирования. Если вы научитесь составлять подобные блок-схемы, писать программы для микроконтроллера для вас не составит труда. Правильно составленная блок-схема — это, по сути дела, 80...90% программы. Так что прошу вас очень серьезно отнестись к тем нескольким примерам, где я буду довольно подробно описывать составление блок-схем — их надо научиться составлять, и точка. Кстати, не так уж это и сложно — все, кто пишут программы на любом из языков, вполне справляются с этим, так что и вам это, наверняка, по силам — вопрос только в желании и во времени. Поэтому еще раз призываю тех, кто поленился разобраться с предложенным материалом — потратьте еще некоторое количество времени и разберитесь с ним, ибо если этого не сделать, вряд ли вам удастся освоить микроконтроллеры.

И в заключение — одно замечание. Обратите внимание на то, какого ограниченного набора команд хватило нам для написания подпрограммы преобразования числа из одного формата в другой. Наш микроконтроллер не умеет ни делить толком, ни умножать — лишь пересылает данные туда-сюда, что-то суммирует и вычитает, да выполняет условные и безусловные переходы. И этого оказывается достаточно для решения поставленной задачи! Ну не удивительно ли? Более того, далее мы с вами еще не однажды и не дважды убедимся в том, что в основе всех тех умных интеллектуальных машин и приборов, поражающих нас иной раз своими «мозгами», стоят простые и вполне заурядные команды. Ну и, конечно, мозги (уже без кавычек) программистов, придумавших эти поражающие нас алгоритмы и реализовавших их в своих программах.

Глава 4. СОПРЯЖЕНИЕ МК С ИНДИКАТОРАМИ РАЗЛИЧНЫХ ТИПОВ

Как уже упоминалось выше, система, в которой используется микроконтроллер, должна не только что-то измерять или чем-то управлять, но и, как правило, что-то отображать. В подавляющем большинстве случаев в качестве узла отображения используются знакосинтезирующие индикаторы. Последние могут быть жидкокристаллическими или светодиодными (остальные, например, люминесцентные или электроннолучевые, практически вышли из употребления). По способу формирования символов индикаторы могут быть семисегментными или матричными. Далее, многие из них, в особенности жидкокристаллические, снабжены самостоятельными микроконтроллерами, существенно упрощающими работу с ними. В этом случае в обязанности разработчика входит лишь организация передачи информации от основного микроконтроллера в контроллер индикатора, а все, что связано с отображением переданных символов, осуществляется последним. Светодиодные индикаторы снабжаются самостоятельными контроллерами гораздо реже, так как работа с ними намного проще. При этом для работы с ними используются системы динамической индикации. Последние, хотя и требуют дополнительных аппаратных ресурсов, но все равно используют меньшее количество микросхем, чем системы с динамической индикацией без микроконтроллера, на одной только дискретной логике.

Из всего перечисленного очевидно, что каких-либо стандартных правил сопряжения микроконтроллеров с индикаторами не существует, и в каждом конкретном случае оно выполняется по-своему. Поэтому в настоящей главе я решил предложить вашему вниманию схемы, алгоритмы и программы сопряжения микроконтроллеров с четырьмя наиболее типичными устройствами индикации. Разобравшись с ними, вы не только сможете любой из этих индикаторов использовать в дальнейшем в своих устройствах, но и, опираясь на эти наработки, самостоятельно сопрягать ваш микроконтроллер с любым другим устройством индикации.

По-прежнему напоминаю, что хотя мы с вами еще не знакомились в полном объеме с системой команд микроконтроллера, мы, тем не менее,

будем продолжать анализировать необходимые нам фрагменты программ. Систему команд мы рассмотрим в следующей главе. К этому моменту со многими инструкциями вы в той или иной степени уже столкнетесь, вследствие чего обстоятельное знакомство со всей системой команд и с особенностями каждой из них окажется для вас вполне посильной задачей. А ознакомившись с ними и с типичными примерами их использования в часто применяемых программах, вы будете уже в состоянии сделать первые шаги в самостоятельной разработке системы на основе микроконтроллера.

4.1. Сопряжение с ЖКИ на основе контроллера HT1611 фирмы HOLTEK

Мы начнем с этих индикаторов в силу того, что они наиболее просты из тех, с которыми вы познакомитесь в этой главе. Просты не в смысле своего внутреннего устройства (внутри них есть свой микроконтроллер!), а в смысле аппаратного и программного сопряжения с пользовательским микроконтроллером.

Внешний вид индикатора приведен на **Рис. 4.1.** Он представляет собой двустороннюю печатную плату, с одной стороны которой смонтирован 10-разрядный 7-сегментный ЖК-дисплей, а с другой — немногочисленные обслуживающие его электронные компоненты — контроллер, кварц, несколько емкостей. Размер индикатора — 67×36×6 мм. Отметим, что индикатор разработан для использования в телефонных аппаратах среднего класса, и именно там эти изделия встречаются наиболее часто. Упомянутое назначение определило их некоторые технические особенности — отсутствие десятичной точки, полуторавольтовое питание и крайне низкое энергопотребление. В силу этого подобные индикаторы хорошо подходят для систем с автономным питанием.

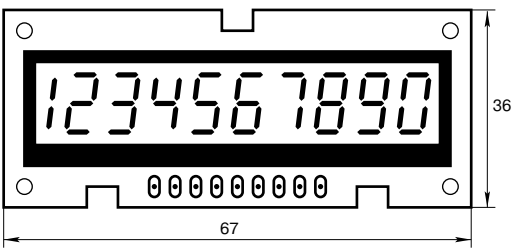
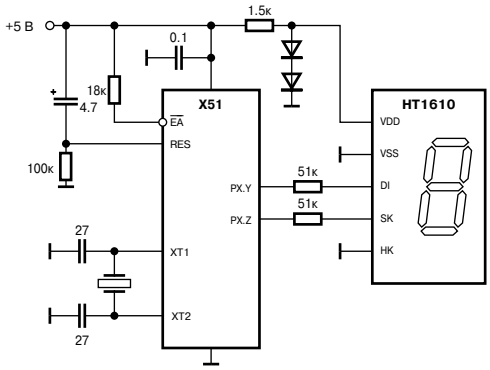


Рис. 4.1. Внешний вид HT1610

В нижней части индикатора расположены 8 контактных площадок, предназначенных для подачи питания и управляющих сигналов. Три из них не представляют для нас интереса, по крайней мере, на первых порах. А вот при помощи пяти оставшихся мы организуем работу индикатора.



Примечание. Отдельные экземпляры индикаторов HT1610 требуют, чтобы питающее напряжение было в пределах 1.5...1.55 В. В этом случае необходимо использовать простейший стабилизатор напряжения, например, на микросхеме LM317 (кроме 142EHIZ) в стандартной схеме включения.

Рис. 4.2. Схема соединения HT1610 с микроконтроллером

Схема соединения индикатора с микроконтроллером приведена на **Рис. 4.2.** Питающее напряжение (номинально 1.5 В) подается на вход V_{DD} индикатора, общий провод соединен с выводом V_{SS}. На вход HK также нужно подать нулевой потенциал, это переводит микроконтроллер индикатора в режим внешнего управления. Передача информации осуществляется по двум сигнальным линиям — DI и SK. Вы уже, наверное, догадались, что на вход DI наш микроконтроллер передает требуемую для отображения информацию, бит за битом, а на вход SK — тактовые импульсы, информирующие индикатор о готовности бита на DI. Сказанное иллюстрируется **Рис. 4.3.** Замечу, что с подобным способом обмена информацией мы уже встречались, когда рассматривали сопряжение микроконтроллера с последовательным АЦП. В последнее время такой последовательный обмен стал чрезвычайно популярным в силу крайне малых аппаратных и программных затрат, требующихся для его реализации (для соединения микроконтроллера и устройства необходимы всего два провода!). Кстати, в силу последнего обстоятельства такой интерфейс еще иногда называют двухпроводным (забывая, правда, об еще одной паре проводов, питания и земли).

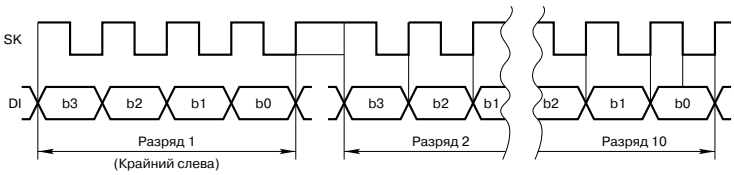


Рис. 4.3. Алгоритм обмена HT1610 и микроконтроллера

Как следует из **Рис. 4.3**, вначале наш микроконтроллер должен передать индикатору четырехбитный код цифры, которая должна быть отображена в крайнем слева разряде. За ним передается код цифры, отображаемой во втором слева разряде и т. д., вплоть до последнего, десятого. По завершении передачи SK нужно установить в1. Кстати, хотя DI и можно после окончания передачи оставить как в нулевом, так и в единичном состоянии, лучше в подобных безразличных случаях оставлять его в 1. Почему? Да потому, что может так случиться, что, например, вывод, по которому вы крайне нечасто передаете в индикатор данные, вам придется задействовать в качестве принимающего. Ох, и намучаетесь же вы, пытаясь понять, почему в одном случае этот прием осуществляется нормально, а в другом — кое-как. А причина окажется крайне проста — оставленный на выводе после завершения передачи нолик будет шунтировать принимаемую информацию. Так что лучше стелить коврик заранее...

Таблица соответствия передаваемых кодов отображаемым цифрам приведена в **Табл. 4.1**. Выбор, как видите, невелик — десяток цифр, пробел, знак минус, буквы F и P, да еще два ни с чем не ассоциирующиеся символа. Чтобы не расстраиваться по поводу этой скудности, будем считать, что это — плата за предельную простоту работы с описываемым индикатором.

Таблица 4.1. **Соответствие передаваемых кодов отображаемым цифрам**

Бит 3	Бит 2	Бит 1	Бит 0	Символ
0	0	0	0	Пробел
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	0
1	0	1	1	F
1	1	0	0	⌐
1	1	0	1	¬
1	1	1	0	P
1	1	1	1	–

Как уже говорилось выше, номинальное питающее напряжение на индикаторе — 1.5 В. Я формирую его на цепочке, состоящей из резистора и двух включенных в прямом направлении кремниевых диодов широкого применения. При токе через диоды в пределах 2...3 мА падение напряжения на них в сумме составляет около 1.4 В, что на практике вполне приемлемо.

Еще один распространенный вариант — использовать вместо двух обычных диодов включенный в прямом направлении красный светодиод. Почему красный? Потому что на всех выпускавшихся в недавнем прошлом красных светодиодах падение напряжения в прямом направлении составляло примерно 1.6 В, в то время как на зеленых и желтых оно было 1.9 В. Однако в последние год-два получили широкое распространение красные светодиоды, выполненные по новой технологии, более яркие, но с падением напряжения в прямом направлении на уровне 1.9...2 В, а это для рассматриваемого индикатора уже многовато. Поэтому, чтобы не заморачиваться с поисками устаревших красных светодиодов, я рекомендую для питания индикатора использовать цепочку с обычными диодами типа КД503, КД509, КД521 и т. д.

Еще один аспект: сигналы управления, формируемые нашим микроконтроллером — пятивольтовые. В то же время на наш полуторавольтовый индикатор они должны приходить с амплитудой не более 1.7 В. Согласование по уровням осуществляется при помощи резисторов сопротивлением 43...68 кОм, включенных между выходами микроконтроллера и входами индикатора. Поскольку входы последнего, как видно из **Рис. 4.4**, защищены диодами, уровень логической 1 на этих входах не может оказаться более, чем V_{DD} + 0.3 В. Собственно, что и требовалось...

Ну вот и все об аппаратном сопряжении индикатора с нашим микроконтроллером. Вроде бы и не много информации, но на десяток абзацев потянуло. Ох, что будет, когда мы доберемся до светодиодного матричного индикатора! Там ведь аппаратных особенностей на порядок больше. Ну да ладно, справимся.

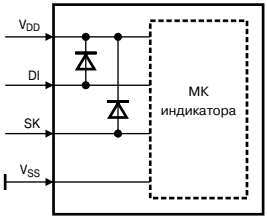


Рис. 4.4. Защитные диоды на входах индикатора

Ниже приведен фрагмент подпрограммы `INDVIV`, осуществляющей программное сопряжение индикатора HT1610 с нашим микроконтроллером. Для определенности предполагается, что на индикатор выводятся два четырехбитных числа, цифры которых (в двоично-десятичном представлении) хранятся во внутреннем ОЗУ в ячейках памяти с адресами от `AD00+3` (старший бит первого числа) до `AD00` (младший бит; в данном примере символическому адресу `AD00` я присвоил численное значе-

ние 30h, AD00+1 — соответственно 31h, и т. д.), и с адресами от AD00+7 (старший бит второго числа) до AD00+4 (младший бит; AD00+4 — это 34h, AD00+7 — соответственно 37h). Первое число я вывожу в 4 правых разряда индикатора, второе — в 4 левых, а в два средних (пятый и шестой) вывожу пробелы (т. е. там не отображаются никакие символы).

Таким образом, если ячейка с адресом 30h содержит число 07h, 31h — число 02h, 32h — число 05h, 33h — число 04h, а в следующих четырех ячейках (с адреса 34h по 37h) хранятся соответственно числа 08h, 00h, 01h и 02h, то при запуске подпрограммы INDVIV на индикаторе HT1610 вы увидите (слева направо) числа 2108 и 4527, разделенные двумя пробелами.

Как вы уже, наверное, догадались, AD00 может быть любым из диапазона от 20h до 78h. Обратите внимание, младшие биты отображаемых чисел я храню в ячейке с младшими адресами (AD00 и AD00+4), а старшие биты — в ячейках со старшими адресами (AD00+3 и AD00+7). Можно, конечно, и наоборот, но мне привычнее именно так.

Фрагмент подпрограммы INDVIV, осуществляющей программное сопряжение индикатора HT1610 с МК семейства x51

```
;
AD00    .EQU   30h
DATIND  .EQU   P1.0        ;к выводу DI
CLKIND  .EQU   P1.1        ;к выводу SK
;
INDVIV:
        MOV    A,AD00+7    ;старший бит
        ACALL  SIMBOL1     ;второго числа
        MOV    A,AD00+6
        ACALL  SIMBOL1
        MOV    A,AD00+5
        ACALL  SIMBOL1
        MOV    A,AD00+4    ;младший бит
        ACALL  SIMBOL1
        MOV    A,#0Fh      ;код пробела
        ACALL  SIMBOL1
        MOV    A,#0Fh      ;код пробела
        ACALL  SIMBOL1
        MOV    A,AD00+3    ;старший бит
        ACALL  SIMBOL1     ;первого числа
        MOV    A,AD00+2
        ACALL  SIMBOL1
        MOV    A,AD00+1
        ACALL  SIMBOL1
        MOV    A,AD00+0    ;младший бит
```

```
        ACALL   SIMBOL1
        SETB    DATIND
        RET
  ;
SIMBOL1:
        ANL     A,#00001111b
        CJNE    A,#0,SIMB11
        MOV     A,#10
        SJMP    SIMB12
SIMB11:
        CJNE    A,#0Fh,SIMB12
        MOV     A,#0
SIMB12:
        CLR     CLKIND
        SWAP    A
        ACALL   BIT1
        ACALL   BIT1
        ACALL   BIT1
        ACALL   BIT1
        SETB    CLKIND
        RET
  ;
BIT1RLC A
        MOV     DATIND,C    ;выв. данных
                            ;в ЖК-дисплей
        SETB    CLKIND      ;импульс
        CLR     CLKIND      ;защелкивания
        RET
  ;
```

Теперь по поводу самой подпрограммы INDVIV. Она организована следующим образом. Командами `MOV A,AD00+n` мы заносим в аккумулятор цифры, которые необходимо переслать в индикатор, а затем командами `ACALL SIMBOL1` вызываем подпрограмму (ее блок-схема приведена на **Рис. 4.5**), организующую пересылку. Обратите внимание, вызов подпрограммы осуществляется командой `ACALL`, а не `LCALL`. Собственно, вызвать требуемую подпрограмму можно было бы и знакомой нам командой `LCALL`, но я специально для знакомства поставил `ACALL`. Последняя более короткая, выполняется чуть быстрее, но применима только в тех случаях, когда вызываемая подпрограмма находится в том же двухкилобайтном куске памяти программ, что и команда, которая ее вызывает. До тех пор, пока вся написанная вами программа будет менее 2 КБ, можете не задумываясь вызывать подпрограммы командой `ACALL`, а не `LCALL`.

Теперь относительно того, как числа выводятся командой `ACALL` из аккумулятора в индикатор. Вспомните **Рис. 4.3**. Мы должны вывести старший бит отображаемого числа на вход DI индикатора и перепадом из 1 в 0

на входе SK защелкнуть его в HT1610. Затем нужно вывести на DI следующий бит и «взвести» (установить из 0 в 1) сигнал на SK. Сбросив затем SK в 0, мы защелкнем в индикаторе второй бит отображаемого числа, и далее по тому же алгоритму — третий и четвертый.

Как это реализовано в подпрограмме `SIMBOL1`? Обратите внимание на ту ее часть, которая идет после метки `SIMB12`. Уже знакомая вам команда `SWAP A` обменивает местами старшие и младшие 4 бита аккумулятора, т. е. переносит выводимое число в старшую его тетраду. Далее вызывается подпрограмма `BIT (ACALL BIT)`. Первая ее команда (`RLC A`) осуществляет так называемый циклический сдвиг аккумулятора влево через перенос. Звучит страшно, но не пугайтесь, сейчас я объясню, что это такое. При этом сдвиге самый старший, седьмой бит аккумулятора, пересылается в бит переноса CY, шестой — в седьмой, пятый — в шестой и т. д., вплоть до нулевого, который после этого сдвига окажется в первом бите. Ну, а на место нулевого будет переслан бит, хранившийся до сдвига в CY. Таким образом, можно представить себе, что бит переноса в этой команде выступает в роли «совсем старшего», 9-го бита эдакого расширенного до 9 бит аккумулятора, и в этой сложной конструкции осуществлен сдвиг содержимого каждого бита на одну позицию влево — из нулевой в первую, из первой — во вторую, ..., из шестой — в седьмую, из седьмой — в восьмую. Ну а что содержимое восьмого бита? Естественно, на освободившееся место в бите 0. Сказанное наглядно иллюстрируется **Рис. 4.5.**

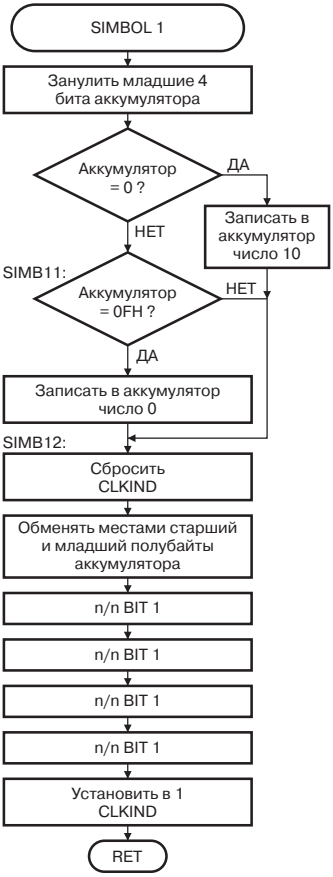


Рис. 4.5. Блок-схема подпрограмма SIMBOL1

Надеюсь, суть команды `RLC A` понятна. Тогда вернемся к нашим баранам. Напомню, что командой `SWAP A` мы обменяли местами старшие и младшие 4 бита аккумулятора, разместив выводимое в индикатор число соответственно в 7, 6, 5 и 4-й его биты. Далее, команда `RLC A` в подпро-

грамме `ACALL BIT` перенесла старший бит отображаемого числа из седьмого бита аккумулятора в бит переноса. А что делает идущая за `RLC A` команда `MOV DATIND,C`? Правильно, переносит бит из `CY` на линию 0 порта P1 или, что то же самое, на вход DI индикатора (напомню: `DATIND .EQU P1.0`). А далее команда `SETB CLKIND` «взводит» сигнал на входе SK индикатора, после чего команда `CLR CLKIND` «защелкивает» выведенный бит в индикаторе. Все, подпрограмма `ACALL BIT` перенесла из аккумулятора в индикатор старший бит отображаемого числа.

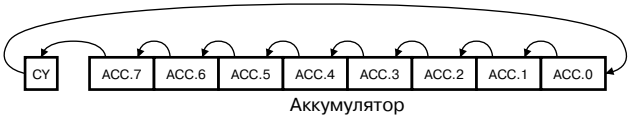


Рис. 4.6. Команда сдвига влево через перенос

А как выводится второй бит числа? Напомню, что команда `RLC A` в подпрограмме `ACALL BIT` не только перенесла старший бит отображаемого числа из седьмого бита аккумулятора в бит переноса, но и также перекинула на место седьмого бита следовавший за ним шестой, т.е. именно тот, который нам предстоит теперь вывести в индикатор. Поэтому нам просто нужно еще раз вызвать подпрограмму `ACALL BIT`, и она выведет в индикатор второй бит числа, хранившийся после выполнения команды `SWAP A` в шестом бите аккумулятора.

После второго вызова подпрограммы `ACALL BIT` в седьмом бите аккумулятора окажется третий бит отображаемого числа, хранившийся после выполнения команды `SWAP A` в пятом бите аккумулятора. Как нетрудно догадаться, вызвав в третий раз `ACALL BIT`, мы и его благополучно выведем в индикатор. Ну а четвертый вызов `ACALL BIT` завершит пересылку всей цифры. На этом подпрограмма `ACALL SIMBOL1` завершается, о чем свидетельствует команда `RET`, идущая после четвертого вызова `ACALL BIT`.

Итак, мы уже поняли, что подпрограмма `INDVIV` командами `MOV A,AD00+n` заносит в аккумулятор цифры, которые необходимо переслать в индикатор, а затем командами `ACALL SIMBOL1` осуществляет требуемую пересылку. Последнее реализуется той частью подпрограммы `SIMBOL1`, которая идет после метки `SIMB12`. А что делает та ее часть, которая размещается между этой меткой и началом подпрограммы?

Вспомним, что в двоично-десятичном представлении 0 — это 0000b, 1 — это 0001b, 2 — это 0010b и т. д. В то же время из таблицы отображаемых символов индикатора HT1610 (**Табл. 4.1**) следует, что у него 0000b — это пробел, а для того, чтобы на нем отобразился 0, в него нужно переслать 1010b. Таким

образом, нуль перед отображением нужно перекодировать. Иными словами, если подпрограмма перед отображением встретит 0000b, она должна заменить его на 1010b, и только после этого осуществить пересылку символа в индикатор. Посмотрим, как это реализовано в подпрограмме SIMBOL1.

```
SIMBOL1:
        ANL     A,#00001111b
        CJNE    A,#0,SIMB11
```

Первой командой я зануляю находящиеся в аккумуляторе старшие 4 бита отображаемого числа — цифра в двоично-десятичном представлении должна быть четырехбитной, от 0000b до 1001b, следовательно, биты с пятого по восьмой должны быть незначащими нулями. С этой командой мы уже знакомы. А вот следующая для нас еще нова. CJNE — это аббревиатура выражения Compare and Jump No Equal, что в переводе означает «сравнить и перейти, если не равно». В ходе выполнения этой команды микроконтроллер сравнивает содержимое аккумулятора с нулем (именно с цифрой 0, о чем нам напоминает символ # перед ней), и если A не равно 0, то переходит на выполнение той части программы, которая идет после метки SIMB11. О том, что там — чуть позже. Пока же отметим, что переход туда осуществляется в том случае, если выводимое из аккумулятора в индикатор число — не нуль. Ну а если в нем нуль (который нужно перекодировать)? Тогда микроконтроллер выполняет команды, идущие непосредственно за CJNE A,#0,SIMB11:

```
        MOV     A,#10
        SJMP    SIMB12
```

Команда MOV A,#10 заносит в аккумулятор десятку, т. е. именно ту цифру, которая, будучи пересланной в HT1610, отобразит в соответствующем его разряде ноль. Далее стоит команда SJMP SIMB12, предписывающая микроконтроллеру перейти на выполнение программы с метки SIMB12 (а там, как нетрудно увидеть, рассмотренная выше часть подпрограммы, непосредственно осуществляющая вывод цифры из аккумулятора в индикатор). Т. е., как и предлагалось, при обнаружении перед выводом в HT1610 нуля микроконтроллер заменяет его на 1010b, и только потом осуществляет пересылку. Ну а если в аккумуляторе был не нуль?

```
SIMB11: CJNE    A,#0Fh,SIMB12
        MOV     A,#0
SIMB12:
```

Как видите, еще одно сравнение, на этот раз — с цифрой 0Fh. Если содержимое аккумулятора не равно 15 (0Fh = 15), то микроконтроллер перейдет на выполнение программы с метки SIMB12, т. е. на вывод цифры из

аккумулятора в индикатор. А если в аккумуляторе было число 0Fh, то мы заменим его на 0, соответствующий пробелу. Зачем это?

Конечно, этого можно было бы и не делать. Но во многих дешифраторах, используемых, в частности, со светодиодными семисегментными индикаторами (например, серии 514), подача на вход числа 0Fh приводит к погасанию всех сегментов, т. е. к отображению пробела. Если нам при работе с таким индикатором нужно погасить то или иное его знакоместо, достаточно в соответствующую ему ячейку памяти занести 0Fh, и при выводе этого числа в индикатор нужный разряд погаснет. Но это при использовании индикатора с дешифратором типа 514ИД1(2). А если нам захотелось в новой разработке заменить светодиодный индикатор на HT1610? Тогда нам нужно либо искать в нашей программе все те места, где для погашения разряда индикатора мы заносим в соответствующую ячейку памяти число 0Fh, и заменять 0Fh на 0, либо сделать лишь одну перекодировку — в подпрограмме вывода числа в индикатор. Второй случай, естественно, предпочтительнее — основная программа везде, где надо, гасит разряды индикатора записью в ОЗУ числа 0Fh, и не задумывается, какой индикатор будет подцеплен к микроконтроллеру, а все нюансы индикатора учитывает обслуживающая его программа. Захотели сменить индикатор — перепишите лишь подпрограмму работы с ним, при этом вы наверняка сделаете меньше ошибок, чем перелопачивая под новый индикатор всю свою программу. Так что привыкайте к подобной унификации, это сэкономит вам массу времени и нервов.

Итак, мы познакомились со всем, что необходимо знать для разработки аппаратного и программного сопряжения нашего микроконтроллера семейства x51 с ЖК-индикатором на основе контроллера HT1611 фирмы Holtek. Как я уже говорил, это самый простой индикатор с точки зрения его сопряжения с микроконтроллером. Следующим пунктом нашей программы будет сопряжение микроконтроллера с многоразрядным семисегментным светодиодным индикатором, т. е. с самым простым представителем многоразрядных знакосинтезирующих светодиодов.

4.2. Сопряжение со светодиодными индикаторами типа АЛС318

АЛС318 — наиболее удобный для рассмотрения многоразрядный семисегментный светодиодный индикатор. Разобравшись с тем, как использовать его, вы без особых усилий адаптируете рассматриваемые аппаратные и программные средства под любой семисегментный индикатор, будь то панель, набранная из десятка одиночных индикаторов с большими цифрами или малогабаритный (в 14-выводном DIP'е) пятиразрядный АЛС328.

Напомню, что семисегментные светодиодные индикаторы выпускаются либо с объединенными анодами, либо с объединенными катодами. АЛС318 принадлежит к последним. Его анодами обычно управляет дешифратор типа КР514ИД1. Управление катодами можно организовать поразному — с использованием второго дешифратора или напрямую от микроконтроллера. Мы рассмотрим первый вариант — он требует использования меньшего числа выводов микроконтроллера.

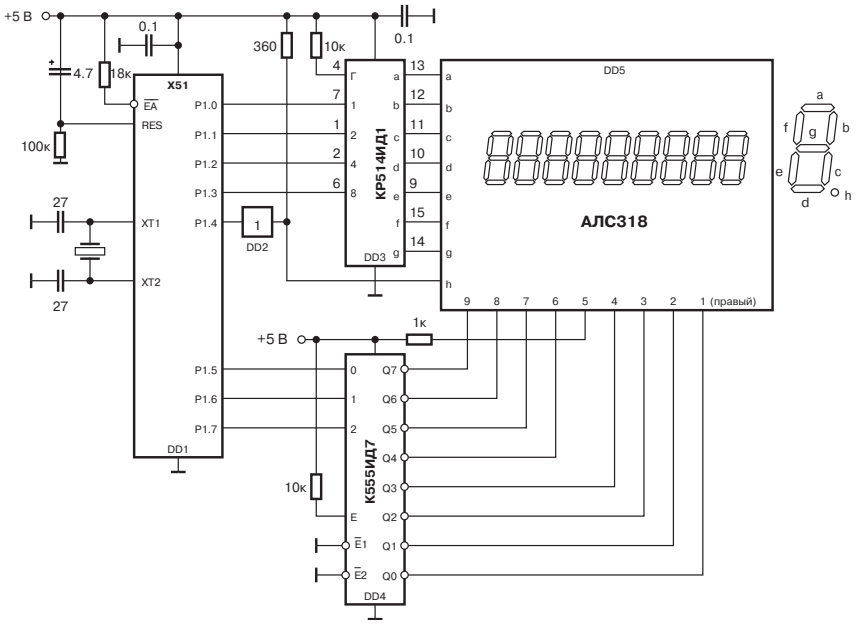


Рис. 4.7. Схема сопряжения микроконтроллера с АЛС318

Схема сопряжения нашего микроконтроллера с индикатором АЛС318 приведена на **Рис. 4.7.** Она включает в себя два дешифратора DD3 (КР514ИД1) и DD4 (К555ИД7) и половинку микросхемы с открытым коллектором DD2 (КР155ЛЛ2). Последняя, как будет показано ниже, управляет десятичной точкой. Естественно, вполне возможно вместо всех этих трех микросхем использовать запрограммированную соответствующим образом ПЛИС (программируемые логические интегральные схемы), которая «вберет» их в себя и будет выполнять все свои функции совершенно идентично тому, как это сделали бы три перечисленные микросхемы. Но при этом потеряется ясность, почему мы именно так, а не

иначе построили нашу программу связи микроконтроллера с индикатором, и как изменить программу, если что-то изменено в схеме сопряжения. Поэтому я и рассматриваю схему на дискретных элементах, пусть даже несколько архаичную, но наиболее удобную для первоначального знакомства. А разобравшись с ней, вы будете делать то, что для вас легче, проще, элегантнее — когда знаешь, что и как сделать, придумать десяток вариантов на любой вкус несложно.

Итак, рассмотрим схему на **Рис. 4.7**. Для работы с индикатором используем порт P1. Четыре его младшие линии (P1.0...P1.3) выводят на дешифратор DD3 код отображаемой цифры: 0000b — 0; 0001b — 1; ... ; 1001b — 9. Выходы дешифратора DD3 соединены с одноименными входами индикатора (соответствующими анодами сегментов). Катоды сегментов каждого разряда, как упоминалось, объединены внутри индикатора и управляются выходами второго дешифратора (DD4). На информационные входы последнего поступают сигналы с трех старших линий порта P1 (P1.5...P1.7). Они позволяют управлять индикатором, имеющим до 8 индицируемых разрядов. Оставшаяся линия (P1.4) используется для управления десятичной точкой — разрядом h индикатора. Установка этой линии в 1 зажигает точку в том разряде, катодный вывод которого установлен в 0 соответствующим выходом дешифратора DD4.

Как видите, для управления 8-разрядным семисегментным светодиодным индикатором нам понадобилось 8 линий вывода — весь порт P1. В предыдущем случае, с HT1610, линии ввода/вывода использовались более экономно. Но ничего не поделаешь, это плата за отсутствие внутри индикатора АЛС318 дополнительного микроконтроллера.

Наверное, вы уже догадались, что если вам нужно управлять не восьми-, а 16-разрядным индикатором, вам необходимо в качестве DD4 использовать дешифратор «4 в 16». Соответственно, для управления им понадобятся не 3, а 4 линии порта. Логичнее всего использовать для этого P1.4...P1.7 (внеся соответствующие изменения в приведенную ниже программу). Ну, а управление десятичной точкой, если она вам необходима, придется осуществить по какой-либо линии другого порта, например, по P3.0.

Будем считать, что перед нами стоит та же задача, что и в предыдущем случае — отобразить подпрограммой `IZOBR` на индикаторе два четырехбитных числа, хранящихся в двоично-десятичном представлении во внутреннем ОЗУ микроконтроллера в ячейках памяти с адресами от `AD00+3` (старший бит первого числа) до `AD00` (младший бит; в данном примере, как и ранее, символическому адресу `AD00` я присвоил численное значение 30h, `AD00+1` — соответственно 31h и т. д.), и с адресами от `AD00+7` (старший бит второго числа) до `AD00+4` (младший бит; `AD00+4` — это 34h, `AD00+7` — соответственно 37h). Первое число я вывожу в 4 правых разряда индикатора, второе — в 4 левых, а в средний (индикатор-то 9-разрядный) вывожу пробел.

Таким образом, если ячейка с адресом 30h содержит число 07h, 31h — число 02h, 32h — число 05h, 33h — число 04h, а в следующих четырех ячейках (с адреса 34h по 37h) хранятся соответственно числа 08h, 00h, 01h и 02h, то при запуске подпрограммы `IZOBR` на индикаторе АЛС318 вы увидите (слева направо) числа 2108 и 4527, разделенные пробелом.

А как быть, если нам нужно отобразить не 2108 и 4527, а 2.108 и 45.27? Очень просто. Вспомним, что отображаемые цифры, хранящиеся в `AD00+0`—`AD00+7`, четырехбитные (от 0000b до 1111b), а старшие 4 бита каждой из этих цифр хранят незначащие нули. Давайте договоримся, что пятый бит в каждой из ячеек памяти `AD00+0`...`AD00+7` отвечает за десятичную точку, идущую непосредственно за соответствующей отображаемой цифрой. В нашем примере в числе 2.108 точка должна отображаться вместе с двойкой, т. е. в ячейке с адресом 37h должно храниться не 02h, а 12h. Соответственно, в числе 45.27 десятичная точка идет после пятерки, т. е. она хранится в виде единички в пятом по счету бите числа в ячейке 32h — там вместо 05h должно находиться 15h. В остальных же ячейках, хранящих цифры 1, 0 и 8 числа 2.108 и 4, 2, 7 числа 45.27, все 4 старших бита по-прежнему должны хранить незначащие нули.

При отображении на индикаторе той или иной цифры мы в соответствующей подпрограмме должны не только ее перенести из четырех младших битов ячейки `AD00+n` в 4 младших бита порта P1 (т. е. P1.0...P1.3), но и перенести также пятый бит ячейки `AD00+n` в пятый бит P1 (в P1.4). Вот тогда мы сможем на индикаторе увидеть не только 2108 и 4527, но и 2.108 и 45.27.

И еще, на что я хочу обратить ваше внимание. Как мы договорились, дешифратор DD4 может управлять восемью разрядами индикатора. В то же время АЛС318 — 9-разрядный, и мы хотим, чтобы средний (пятый с любого края) разряд был всегда погашен. Как погасить разряд, на управление которым у нас уже нет свободного выхода дешифратора? Правильно, подать на его катод единичный уровень. А если бы мы захотели погасить самый старший (левый) разряд индикатора, а в младших 8 разрядах отображать число от 00000000 до 99999999? Естественно, нужно было бы подать единичку на катод старшего разряда, а с выходами дешифратора соединить оставшиеся 8 катодов. Естественно, не забыв сделать соответствующие изменения в приводимой ниже программе...

Ну а теперь перейдем непосредственно к программе IZOBR.

Фрагмент подпрограммы IZOBR, осуществляющей вывод информации на АЛС318

```
;
AD00    .EQU   30H
CNSKIND .EQU   40H          ;счетчик 0-255
;
```

```
;ПОДПРОГРАММА IZOBR ОБЕСПЕЧИВАЕТ ВЫВОД ИНФОРМАЦИИ
;НА ЭКРАН 8-РАЗРЯДНОГО ДИСПЛЕЯ АЛС318
;
IZOBR:
        MOV    CNSKIND,#0
IZOBR1:
        MOV    A,CNSKIND
        INC    A
        JZ     GASH
        MOV    CNSKIND,A
        ANL    A,#00000111B
        MOV    R0,A
        LCALL  DISPLEY
        SJMP   IZOBR1
GASH:   MOV    P1,#11101111B
        RET
;
;ПРИ ВХОДЕ В ЭТУ П/П R0=0...7 (СКАНИ-
;РОВАНИЕ).НАДО ПОСЧИТАТЬ AD00+R0,
;ПРОЧИТАТЬ ПО ЭТОМУ АДРЕСУ ЧИСЛО,ПОМЕС-
;ТИТЬ ЕГО В МЛ. 5 БИТ P1, А СОДЕРЖИМОЕ
;R0 - В СТАРШИЕ 3 БИТА
DISPLEY:
        MOV    A,#AD00
        ADD    A,R0
        MOV    R1,A        ;R1=AD00+R0
        MOV    A,@R1
        ANL    A,#00011111B
        MOV    R1,A
        MOV    A,R0
        RR     A
        RR     A
        RR     A
        ADD    A,R1
        MOV    P1,A
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        RET
```

Она реализует отображение методом динамической индикации. Напомню, что при этом одноименные (в нашем случае анодные) сегменты разных разрядов индикатора объединены между собой, и вследствие этого

код отображаемого символа одновременно подается на аноды всех разрядов. Отображается же этот символ в том разряде, катоды которого находятся под нулевым потенциалом, при этом все остальные разряды индикатора оказываются погашены. Следовательно, если мы на дешифратор DD3 подадим код символа, который должен быть отображен в первом разряде, то для того, чтобы он высветился именно в нужном месте, на входы второго дешифратора (DD4) нужно подать такой код, чтобы на его выходе, соединенном с катодом первого разряда, появился логический 0. Дав небольшую задержку, далее на DD3 подадим код символа, который должен отобразиться во втором разряде, а на входы DD4 — такой код, чтобы логический ноль появился на его выходе, соединенном с катодом второго разряда. Процедуру повторим до тех пор, пока не переберем все разряды индикатора и не высветим в каждом из них соответствующую цифру, затем еще раз, еще, еще и т. д.

Обратимся еще раз к схеме на **Рис. 4.7**. Заметьте, что крайний правый разряд я соединил с выходом 0 дешифратора DD4, второй справа — с выходом 1, третий — с выходом 2 и т. д. Это означает, что при отображении цифры, стоящей в самом правом разряде, на входах DD4 должен присутствовать код 000b, при отображении цифры во втором справа разряде — код 001b, в третьем справа — код 010b, в четвертом — 011b, в шестом (пятый отключен подачей на его катоды единицы) — 100b, в седьмом — 101b, в восьмом — 110b и в девятом, крайнем слева — 111b. Вспомним также, что цифра, которая должна отобразиться в крайнем справа разряде, хранится в ячейке с адресом `AD00+0`, следующая — с адресом `AD00+1` и т. д. Таким образом, для реализации отображения методом динамической индикации мы, во-первых, должны где-то в микроконтроллере иметь счетчик, последовательно перебирающий значения 0, 1, 2, 3, 4, 5, 6, 7, 0, 1, ... Во-вторых, когда значение этого счетчика (обзовем его счетчиком сканирования) равно 0, то нужно отображать цифру из ячейки `AD00+0`, когда 1 — из ячейки `AD00+1` и т. д. Иными словами, если значение счетчика равно n, то отображаемая цифра должна быть извлечена из ячейки `AD00+n`. И в-третьих, вместе с отображаемой цифрой мы должны на входы дешифратора DD4 (т. е. на линии P1.5...P1.7) вывести текущее значение упомянутого счетчика. Вот, собственно, и все, что должна сделать наша программа.

Как это реализовано в `IZOBR`? В ячейке памяти `CNSKIND` организован счетчик, значение которого в начальный момент устанавливается в 0 командой `MOV CNSKIND,#0`. В ходе выполнения программы оно непрерывно увеличивается — команда `MOV A,CNSKIND` переносит его содержимое в аккумулятор, затем команда `INC A` увеличивает его на 1, а `MOV CNSKIND,A` возвращает инкрементированное значение обратно в ячейку `CNSKIND`. Идущая затем команда `ANL A,#00000111b` зануляет пять старших битов этого счетчика, точнее, оставшейся в аккумуляторе его

копии. Зачем? Если вы еще не догадались, поясняю. После выполнения этой команды в аккумуляторе остаются либо 0, либо 1, либо 2, либо 3, либо 4, либо 5, либо 6, либо 7. Никакой другой цифры там остаться не может. Чтобы осознать это, представьте, что до выполнения команды `ANL A,#00000111b` в нем была восьмерка. Вспомним, что 8 — это 00001000b. Занулите ее 5 старших бит, и получите 0. От девятки (00001001b) после зануления 5 старших бит останется 1, от десятки (00001010b) — двойка и т. д. до 15 (00001111b), которые при этом оставят семерку. А вот от идущего после 15 числа 16 (00010000b) — обратите на это внимание — зануление 5 старших бит оставит снова 0. От 17 останется единица, от 18 — двойка и т. д.

Таким образом, в результате выполнения вышеупомянутых команд в аккумуляторе последовательно, друг за другом, оказываются 0, 1, 2, 3, 4, 5, 6, 7. Затем эти цифры снова повторяются в той же последовательности, затем еще и еще, пока не завершится выполнение программы. А ведь именно это нам и нужно! Далее мы должны это хранящееся в аккумуляторе число перенести в три старших бита порта P1, и помимо этого прибавить его к `AD00`. Найденную сумму мы возьмем в качестве адреса ячейки, откуда извлечем отображаемую цифру (она находится в младших четырех битах этой ячейки) и выведем ее в четыре младших бита P1. Вспомним еще, что в пятом бите упомянутой ячейки, как мы договаривались, хранится бит десятичной точки. Его нужно вывести на ту линию порта, которая управляет сегментом h индикатора, т. е. в рассматриваемом случае — в P1.4. Если все это понятно, то смотрите, как это реализовано дальше в программе.

Полученное после выполнение команды `ANL A,#00000111b` значение реализованного программным путем счетчика мы сохраняем в регистре R0 (команда `MOV R0,A`), затем вызываем подпрограмму `DISPLEY`. Последняя загружает в аккумулятор число `#AD00` и командой `ADD A,R0` суммирует его со значением счетчика, хранимого в R0. Полученная сумма сохраняется в регистре R1 `(MOV R1,A)`.

Далее идет пока еще не очень нам знакомая команда `MOV A,@R1`. Что она делает?

Напомню, что в предыдущей главе, рассматривая регистр DPTR, мы узнали, что команда `MOVX A,@DPTR` предписывает микроконтроллеру прочитать в аккумулятор данные из ячейки внешней памяти, адрес которой хранится в DPTR. Вспомнив это, вы догадаетесь, что `MOV A,@R1` заставит наш микроконтроллер прочитать в аккумулятор данные из ячейки памяти, адрес которой хранится... где? Правильно, в R1. Вспомните также, что при рассмотрении команды `MOVX A,@DPTR` я отметил, что `X` на конце команды `MOVX` говорит о том, что чтение должно осуществляться из ***внешней памяти данных***. В команде же `MOV A,@R1` этого `X` нет, что говорит о том, что чтение должно быть осуществлено из **внутренней** памяти данных. Следователь-

но, `MOV A,@R1` предписывает микроконтроллеру найти, какой адрес хранится в регистре R1, после чего перенести в аккумулятор данные из той ячейки внутреннего ОЗУ, адрес которой найден в R1. Попутно отмечу, что команда `MOV @R1,A` вынуждает микроконтроллер совершить обратное действие — перенести данные из аккумулятора в ячейку внутренней памяти, адрес которой хранится все в том же R1. Кстати, подобный метод адресации, когда адрес ячейки памяти, участвующей в обмене данными, находится в каком-либо регистре, носит название косвенной адресации (адрес мы находим косвенно, при помощи R1 или DPTR), в отличие от прямой адресации (например, `MOV A,R1`), где адрес, регистр R1 прямо, т. е. в явном виде указан в команде.

Теперь вернемся к нашей программе, от которой мы слегка отвлеклись. Чуть раньше мы нашли адрес ячейки памяти, где хранится цифра, соответствующая значению счетчика сканирования (т. е. счетчика, последовательно перебирающего значения 0, 1, 2, 3, 4, 5, 6, 7, 0, 1, ...). Этот адрес, равный `AD00+n`, мы сохранили в R1. Затем мы вызвали команду `MOV A,@R1`. Думаю, что всем читающим эти строки, очевидно, что после выполнения последней в аккумуляторе будет находиться та самая цифра, которую нам надо отобразить. Но цифра эта хранится в четырех младших битах аккумулятора, а в пятом хранятся 1 или 0, зажигающие или гасящие десятичную точку после отображаемой цифры. Три же старших бита пока не несут полезной информации. Занулим их для пущей надежности командой `ANL A,#00011111b`, после чего разместим в этих старших трех битах значение счетчика сканирования (напомню, что оно хранится в R0, и в нашем случае равно 000, 001, 010, ..., 110 или 111).

Операция занесения счетчика сканирования в старшие биты аккумулятора также довольно проста. Сначала мы сохраняем значение аккумулятора все в том же регистре R1 (командой `MOV R1,A`). Взамен этого числа мы переносим в аккумулятор счетчик сканирования из R0 (`MOV A,R0`). Зачем? Да потому что значение счетчика сканирования хранится в трех младших битах R0, а нам нужно перенести его в три старших бита. А подобное преобразование мы можем сделать только в аккумуляторе, в связи с чем нам и пришлось его вначале освободить, а затем перенести в него значение счетчика сканирования.

Перенос из младших битов в старшие мы осуществим при помощи трех команд циклического сдвига аккумулятора вправо (`RR A`). При выполнении этой команды старший, седьмой бит аккумулятора, переместится в шестой, шестой переедет в пятый и т. д., вплоть до первого. Первый же переместится на место нулевого бита, а как бы «вытолкнутый» из аккумулятора нулевой бит пересылается на место старшего, седьмого. Сказанное поясняется **Рис. 4.8**, показывающим состояние аккумулятора до выполнения команды `RR A`, после выполнения первой из них, а затем второй и третьей.

a7	a6	a5	a4	a3	a2	a1	a0

Аккумулятор перед выполнением команд сдвига

a0	a7	a6	a5	a4	a3	a2	a1

Аккумулятор после 1-го выполнения команды `RR A`

a1	a0	a7	a6	a5	a4	a3	a2

Аккумулятор после 2-го выполнения команды `RR A`

a2	a1	a0	a7	a6	a5	a4	a3

Аккумулятор после 3-го выполнения команды `RR A`

Рис. 4.8. Последовательность выполнения трех команд `RR A`

Нетрудно убедиться, что применив ее три раза, мы добьемся желаемого эффекта — счетчик сканирования из трех младших бит переместится в три старших. При этом в пяти младших битах будут находиться нули.

Аккумулятор		a2	a1	a0	0	0	0	0	0
Регистр R1	+	0	0	0	b4	b3	b2	b1	b0
Аккумулятор		a2	a1	a0	b4	b3	b2	b1	b0

Рис. 4.9. Сложение аккумулятора и регистра R1

Теперь нам осталось только сложить содержимое аккумулятора и регистра R1. Делается это при помощи уже знакомой нам команды `ADD A,R1`. Нули в трех старших битах R1 не исказят при сложении с аккумулятором хранящееся в его старших трех битах значение счетчика сканирования. Пять же младших битов аккумулятора, хранивших нули, при сложении с пятью битами R1, содержащими отображаемую цифру, в результате суммирования будут именно ее и хранить (**Рис. 4.9**). После этого нам ничего не остается, как вывести это число в порт P1 (командой `MOV P1,A`), и требуемая цифра загорится в нужном разряде АЛС318. Дав ей погореть какое-то время (пока микроконтроллер будет выполнять поставленные для реализации задержки команды `NOP`), контроллер осуществит возврат из подпрограммы `DISPLEY`, инкрементирует счетчик сканирования и приступит к отображению следующей цифры.

А как быть, если мы решили использовать индикаторы не с общими катодами, а с общими анодами? Схема сопряжения с таким индикатором приведена на **Рис. 4.10.** Нам нужно лишь заменить дешифратор 514ИД1 на 514ИД2, да управлять общими анодами не напрямую с дешифратора DD4, а через буферные транзисторы. Поскольку информация для дешифратора DD3 по-прежнему выводится через младшие четыре линии порта P1, а для DD4 — через его старшие три линии, т. е. как и в схеме, рассмотренной на **Рис. 4.7**, то для схемы на **Рис. 4.10** годится та же программа, что и для **Рис. 4.7** без каких-либо изменений.

Последний рассматриваемый в качестве примера случай — нам нужно сделать 16-разрядное табло из одиночных индикаторов, например, с об-

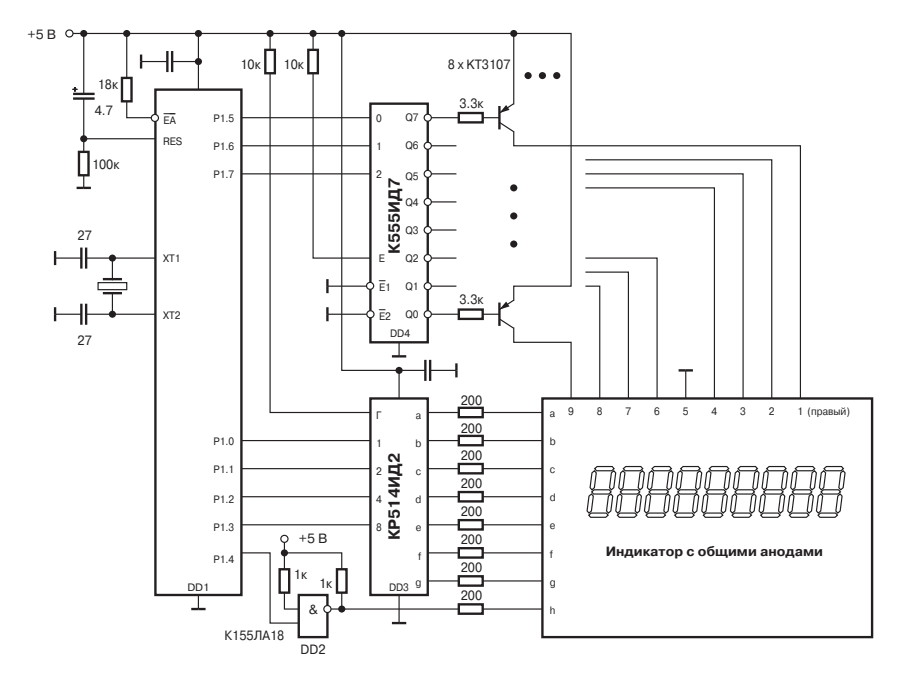


Рис. 4.10. Схема сопряжения с индикатором с общими анодами

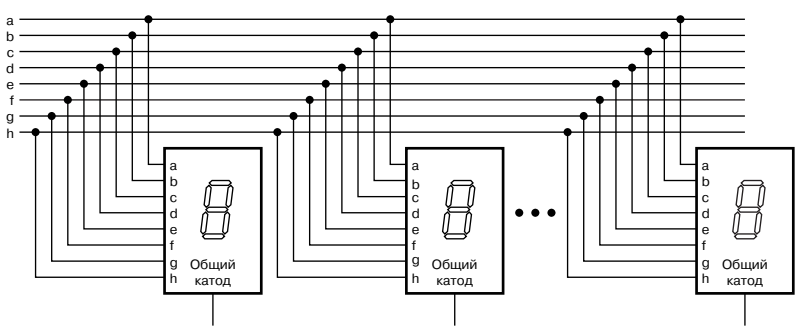


Рис. 4.11. Многоразрядный индикатор из отдельных одноразрядных

щими катодами. Фактически эта задача почти полностью сводится к случаю, рассмотренному на **Рис. 4.7**. Объединим, как показано на **Рис. 4.11**, у всех одиночных индикаторов одноименные аноды a, b, c, d, e, f, g. Для управления (необъединяемыми!) 16-ю катодами используем дешифратор 155ИД3 (4 в 16). Как говорилось выше, для управления последним дешифратором используем линии P1.4...P1.7 порта P1, а для десятичной точки — линию P3.0. В остальном же схема остается без изменений.

Ниже приведена программа, управляющая этим 16-разрядным табло. Она аналогична той, что рассмотрена нами чуть выше, но учитывает отличия в аппаратной части — использование 16-разрядного индикатора и управление десятичной точкой по другой линии порта. Комментировать эту программу я не буду — вам должно быть вполне по силам самостоятельно найти отличия в программах и разобраться, чем вызваны эти отличия.

Программа, управляющая 16-разрядным табло, приведенным на Рис. 4.11

```
;
AD00    .EQU  30h
CNSKIND .EQU  40h         ;счетчик 0-255
;
;ПОДПРОГРАММА IZOBR ОБЕСПЕЧИВАЕТ ВЫВОД 16 ЦИФР ИЗ AD00+0…AD00+15 НА ЭКРАН
;16-РАЗРЯДНОГО ДИСПЛЕЯ
;
IZOBR:
  MOV   CNSKIND,#0
IZOBR1:
  MOV   A,CNSKIND
  INC   A
  JZ    GASH
  MOV   CNSKIND,A
  ANL   A,#00001111b
  MOV   R0,A
  LCALL DISPLEY
  SJMP  IZOBR1
GASH:
  MOV   P1,#11111111b
  CLR   P3.0
  RET
;
;ПРИ ВХОДЕ В ЭТУ П/П R0=0...15 (СКАНИ-
;РОВАНИЕ).НАДО ПОСЧИТАТЬ AD00+R0,
;ПРОЧИТАТЬ ПО ЭТОМУ АДРЕСУ ЧИСЛО,ПОМЕС-
;ТИТЬ ЕГО В МЛ. 4 БИТА P1, А СОДЕРЖИМОЕ
;R0 - В СТАРШИЕ 4 БИТА. ТОЧКУ В P3.0
;
DISPLEY:
  MOV   A,#AD00
  ADD   A,R0
```

```
MOV   R1,A   ;R1=AD00+R0
MOV   A,@R1
MOV   C,ACC.4
MOV   P3.0,C
ANL   A,#00001111b
MOV   R1,A
MOV   A,R0
RR    A
RR    A
RR    A
RR    A
ADD   A,R1
MOV   P1,A
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
RET
```

4.3. Сопряжение с ЖКИ на основе контроллера HD44780 фирмы HITACHI

Эти индикаторы также принадлежат к числу наиболее распространенных — подобные устройства производят более полудюжины фирм, такие, как Data Vision, Bolymin, SII, Optrex, Batron, Picvue, EDT, Varitronix или Povertip. Индикаторы бывают одно-, двух- и четырехстрочными. Каждая строка отображает 8, 12, 16, 20, 24 или 40 символов, каждый из которых формируется матрицей формата 5×7 или 5×10 точек.

В состав индикатора входит микроконтроллер HD44780 или совместимый с ним, производимый фирмами Hitachi, Epson, Toshiba, Philips, Samsung, Sanyo. Он управляет точками ЖК-дисплея и интерфейсной частью индикатора. Как и HT1610, этот индикатор представляет собой печатную плату (ее размеры для однострочного 16-символьного индикатора — 80×36×10 мм), на которой смонтирован ЖК-дисплей, контроллер и необходимые дополнительные электронные компоненты. Внешний вид одного из таких индикаторов, DV-16100 от Data Vision, приведен на **Рис. 4.12.**

На этом общность DV-16100 с рассмотренным выше HT1610 заканчивается, и начинаются различия.

Интерфейс DV-16100 — параллельный. Для соединения индикатора с микроконтроллером нужно использовать 11 линий — 8 для передачи дан-

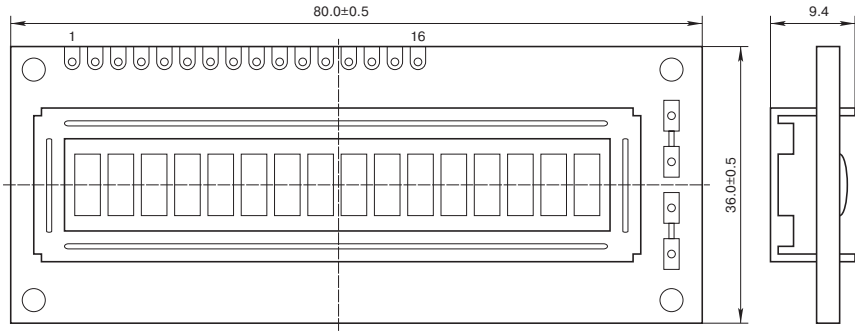


Рис. 4.12. Внешний вид индикатора DV-16100

ных (DB0...DB7), одну R/$\overline{W}$ для информирования индикатора о направлении обмена (R/$\overline{W}$ = 1 — чтение, R/$\overline{W}$ = 0 — запись), одну— для информирования о типе передаваемых данных (RS; RS = 1— данные, RS = 0 — команда), и одну в качестве строб-сигнала, по перепаду которого из 1 в 0 осуществляется запись данных в индикатор или чтение из него. Естественно, на индикатор нужно подать также питающее напряжение (+5 В), соединить его с общим проводом, а также подать на соответствующий вывод некий потенциал (от 0 до 5 В), который регулирует контраст формируемого индикатором изображения. Схема соединения индикатора DV-16100 с микроконтроллером приведена на **Рис. 4.13.**

Отмечу, что DV-16100 допускает возможность работы с использованием не только 8, но и 4 линий данных (DB4...DB7). Однако здесь мы этот режим рассматривать не будем— желающие, опираясь на приведенную ниже информацию, могут попробовать реализовать этот вариант самостоятельно, так сказать, в качестве домашнего задания.

Рассматриваемые индикаторы на основе HD44780 — чрезвычайно гибкие изделия, позволяющие использовать различные режимы ввода в них информации и ее просмотра. Они формируют изображения не только цифр, но и букв латинского (а многие — еще и русского) алфавита, а также псевдографических символов. Однако подобная гибкость имеет и свою оборотную сторону — более сложные (в сравнении с HT1610) алгоритмы инициализации и ввода в них информации. В связи с этим вам придется затратить определенные усилия, чтобы понять, что нужно сделать для реализации тех или иных режимов ввода информации и ее отображения. Минимальные начальные сведения об этом, без которых нельзя написать простейшую программу работы с таким индикатором, вы получите из материала, содержащегося в настоящей главе. Всю же информацию по тому или иному конкретному индикатору вы найдете в фирменном описании

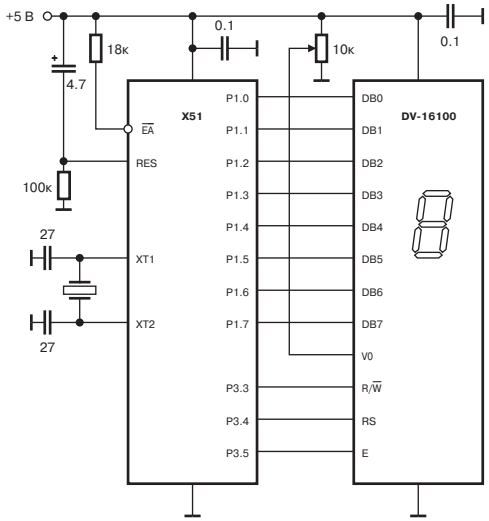


Рис. 4.13. Схема сопряжения индикатора DV-16100 с микроконтроллером

на него. А поскольку даже одинаково организованные индикаторы от различных производителей иногда в деталях отличаются друг от друга, я настоятельно рекомендую перед использованием того или иного индикатора найти описание на него и ознакомиться с ним — это поможет свести к минимуму проблемы с отладкой собранного изделия.

Теперь давайте вернемся к конкретному примеру и рассмотрим более подробно уже упоминавшийся DV-16100. Как уже говорилось, это однострочный индикатор, отображающий в строке 16 символов. Для того, чтобы понять, что же он отображает и как его программировать, полезно представить себе его внутреннюю структуру. Очень упрощенно она выглядит следующим образом (**Рис. 4.14**).

Внутри индикатора есть 80 ячеек памяти, называемых обычно видеопамятью. При помощи соответствующих команд, о которых я скажу чуть ниже, мы можем в любую из этих ячеек занести любое 8-битное число (т. е. от 0 до 0FFh). Каждому из этих чисел взаимно однозначно соответствует при отображении определенный символ — например, числу 32h — цифра 2, числу 47h — заглавная латинская буква G и т. д. Таблица соответствия, иногда называемая таблицей кодов символов, таблицей знакогенератора или таблицей фонтов (кто во что горазд!) приводится в описании на каждый конкретный индикатор. Для DV-16100 она приведена в **Табл. 4.2**.

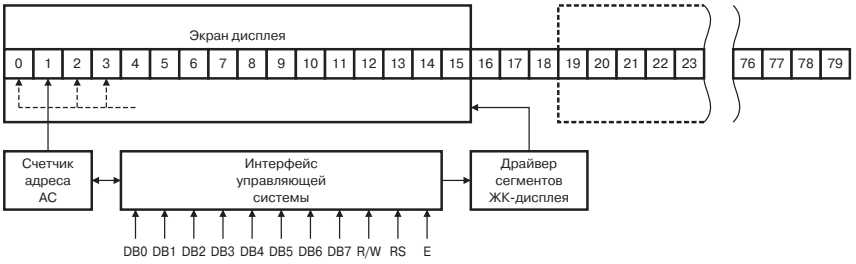


Рис. 4.14. Очень упрощенная внутренняя структура индикатора DV-16100

Не вдаваясь в подробности, отмечу, что наибольшие различия между одинаковыми индикаторами различных производителей лежат именно в таблицах кодов символов, точнее, в правых их половинах, соответствующих числам от 0A0h до 0FFh. В так называемых русифицированных индикаторах среди символов, соответствующих этим числам, есть символы нашей родной кириллицы. Поэтому, если вы собираетесь на индикаторе отображать русские буквы, лучше приобретать индикаторы именно с такой таблицей кодов символов (подобные индикаторы, как я уже упомянул, еще называют русифицированными). Если же вы планируете обойтись только цифрами и латинскими буквами, то вам подойдет индикатор с любой таблицей кодов символов.

Итак, повторю — внутри индикатора есть 80 ячеек видеопамяти, в которые для отображения мы должны занести 8-битные коды, соответствующие тем символам, которые нам нужно отобразить. Но ведь индикатор-то 16-разрядный! Естественно, возникает вопрос — какие 16 из этих 80 символов будут отображены на нем?

Ответ прост. Любые 16 идущих последовательно символов. А вот какие конкретно — зависит от нас. Очень часто для определенности первой командой перед занесением в индикатор информации ставят команду сброса дисплея. В ходе ее выполнения контроллер индикатора заносит во все ячейки видеопамяти код пробела (20h) и настраивает дисплей так, чтобы в крайнем левом разряде отобразился символ, код которого находится в ячейке видеопамяти с адресом 0, во втором слева разряде — символ, код которого в ячейке с адресом 1 и т. д. Таким образом, сразу после выполнения команды сброса дисплея отображаются символы, коды которых хранятся в первых 16 ячейках видеопамяти.

Правда, как я только что сказал, эта команда заносит во все ячейки видеопамяти пробелы, поэтому для того, чтобы увидеть на дисплее что-то

Таблица 4.2. **Коды символов для индикатора DV-16100**

Младшие 4 бита (D0...D3) кода символа в HEX-представлении	Старшие 4 бита (D4...D7) кода символа в HEX-представлении																
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	CG RAM (1)															
1	CG RAM (2)															
2	CG RAM (3)															
3	CG RAM (4)															
4	CG RAM (5)															
5	CG RAM (6)															
6	CG RAM (7)															
7	CG RAM (8)															
8	CG RAM (1)															
9	CG RAM (2)															
A	CG RAM (3)															
B	CG RAM (4)															
C	CG RAM (5)															
D	CG RAM (6)															
E	CG RAM (7)															
F	CG RAM (8)															

помимо пустой строки, нам, после сброса дисплея, нужно занести в эти 16 ячеек видеопамяти коды символов, которые нам хотелось бы отобразить. Для этого надо установить в 0 сигнал на входе R/$\overline{W}$ индикатора (напомню, что R/$\overline{W}$ = 1 означает, что должна быть операция чтения, а R/$\overline{W}$ = 0 — записи). Далее, надо установить в 1 сигнал на входе RS дисплея — как уже говорилось, при RS = 1 HD44780 воспринимает передаваемую информацию как данные, а при RS = 0 — как команду. На DB7...DB0 нужно вывести передаваемые в индикатор 8 бит, и после того, как все сказанное будет сделано, подать на вход E индикатора строб-сигнал — положительный импульс длительностью не менее 500 нс. По перепаду строб-сигнала из 1 в 0 осуществится запись данных в индикатор. Временные диаграммы операций чтения и записи приведены на **Рис. 4.15.**

У самых внимательных из читателей может уже возникнуть следующий вопрос: а в какую именно ячейку видеопамяти мы таким образом занесем информацию? Ответ таков — в ту, на которую указывает так называемый счетчик адреса памяти индикатора AC. После команды сброса дисплея AC устанавливается в 0. Следовательно, первые после этого сброса записываемые данные попадут в нулевую ячейку видеопамяти.

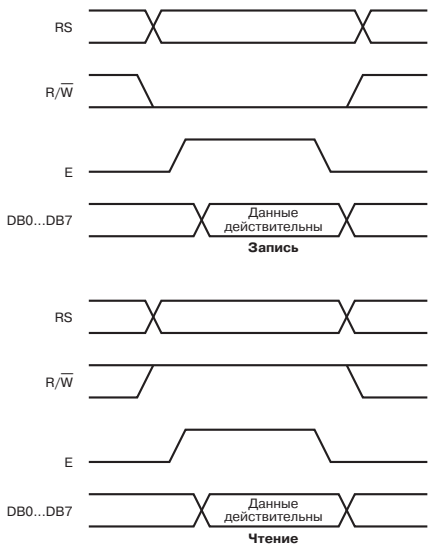


Рис. 4.15. Временные диаграммы операций чтения и записи для индикатора DV-16100

А как записать данные во вторую, третью и т. д. ячейки? В нашем распоряжении есть две возможности. Первая — перед каждой записью данных заставить индикатор выполнить команду установки AC в требуемое положение (от 0 до 79) и передавать данные после такой предустановки AC. Это универсальный способ, с помощью которого можно записать данные в любую ячейку видеопамяти в любой последовательности. Но при этом запись данных в 16 ячеек видеопамяти требует 32 циклов записи — перед каждой передачей данных в индикатор должна идти команда установки AC.

Вторая возможность — настроить соответствующей командой HD44780 таким образом, чтобы после каждой записи в видеопамять AC увеличивался бы на 1. Если мы сделаем это, то, как уже говорилось, первая после сброса дисплея запись данных осуществится в

ячейку видеопамяти с адресом 0. При этом в момент завершения записи АС увеличится на 1, т. е. будет содержать уже не 0, а 1. Следовательно, у нас нет необходимости заставлять индикатор выполнять команду установки АС в 1, и мы сразу вслед за упомянутой пересылкой кода символа в нулевую ячейку можем записать в индикатор код символа, который должен оказаться в ячейке 1. Когда мы осуществим эту запись, АС снова увеличится на 1 и станет равным 2. Значит, следующие записываемые данные попадут во вторую ячейку. А как АС? Правильно, увеличится до 3. Еще раз запишем в индикатор данные (теперь уже в третью ячейку), и снова АС возрастет на 1, указывая на следующую (теперь уже 4-ю) ячейку видеопамяти. И так далее...

Надеюсь, то, что я рассказал в предыдущем абзаце, понятно всем. Если нет, еще раз внимательно, не торопясь, прочитайте его — именно таким образом мы будем в рассматриваемой программе сопряжения микроконтроллера с DV-16100 передавать данные в индикатор.

Мы рассмотрели самый простой способ вывода данных на индикатор типа DV-16100 — предварительно настраиваем его на работу в режиме увеличения АС на 1 после каждой записи данных в видеопамять, даем команду сброса дисплея и после этого записываем в индикатор один за другим коды всех 16 символов, которые мы хотим отобразить. Кстати замечу, что такую же последовательность действий для отображения данных на этом индикаторе должен выполнить любой контроллер — будь то x51, AVR, или PIC-контроллер. Так что если вы разберетесь с описанным алгоритмом, реализуете его на x51, а спустя некоторое время захотите перейти на работу с другими контроллерами, вам понадобится лишь переписать вашу программу на языке этого другого контроллера. А это не так уж сложно, гораздо проще, чем разбираться с работой неизвестного вам доселе устройства.

Еще одно замечание. Вернемся к **Рис. 4.14**. На нем, помимо ячеек видеопамяти, вы видите прямоугольное окошко длиной в 16 ячеек. Как вы, наверное, догадались, прямоугольник этот — не что иное как экран дисплея. На рисунке он расположен таким образом, что вбирает в себя (т. е. отображает) первые 16 ячеек видеопамяти. Это его положение после выполнения команд сброса дисплея или возврата (последняя отличается от первой тем, что не меняет содержимого ячеек видеопамяти). Но в системе команд HD44780 есть и такие команды, которые сдвигают это окошко влево или вправо. В результате можно отобразить на дисплее не только с 0-й по 15-й ячейки видеопамяти, но и, к примеру, с 8-й по 23-ю, или с 19-й по 34-ю (положение окошка, соответствующее последнему случаю, отмечено пунктиром). Вы можете занести информацию, к примеру, во все 80 ячеек видеопамяти, а затем отображать то одно, то другое, то третье, переводя это окошко туда, где оно выхватит те 16 ячеек, которые вам нужно отобразить в текущий момент. Я плохо представляю себе, зачем нужен подобный

режим, но наверняка для кого-то это покажется весьма удобно, и он с удовольствием его реализует. Правда, этому кому-то придется реализовывать такой режим работы индикатора самостоятельно, без моих подсказок — я не планирую рассмотрение реализации этого режима.

Но мы опять уклонились от основной темы. Несколькими абзацами выше я сказал, что индикатор нужно настроить таким образом, чтобы счетчик АС увеличивался на 1 после каждой записи данных в видеопамять. А какие еще настройки нужно осуществить?

Как уже упоминалось, индикатор может вести обмен информацией с микроконтроллером как по 4-битной, так и по 8-битной шине данных. Следовательно, мы должны указать ему, по какой же шине мы планируем передавать ему информацию. Далее, индикатор может быть как однострочным, так и многострочным, и мы должны сообщить ему, с каким количеством строк он работает. Мы должны также выбрать размер матрицы, формирующей символы на индикаторе — 5×7 или 5×9 точек. Также подлежит уточнению наличие и форма отображения курсора — в виде линии подчеркивания или в виде мигающего символа. Все эти (и некоторые другие) настройки осуществляются соответствующими командами, посылаемыми индикатору нашим микроконтроллером.

Перечень этих команд приведен в **Табл. 4.3.** Передача каждой из них осуществляется записью в индикатор (при RS = 0) соответствующего ей байта, в котором в зависимости от требуемых нам настроек установлены в нули или в единицы биты I/D, S, D, C, B, S/C, R/L, DL, N, F. Что они означают?

Таблица 4.3. **Команды индикатора DV-16100**

R/$\overline{W}$	RS	D7	D6	D5	D4	D3	D2	D1	D0	Название
0	0	0	0	0	0	0	0	0	0	Сброс дисплея
0	0	0	0	0	0	0	0	1	*	Возврат дисплея
0	0	0	0	0	0	0	1	I/D	S	Автоинкремент/сдвиг
0	0	0	0	0	0	1	DL	C	B	Управление включением/выключением
0	0	0	0	0	1	S/C	R/L	*	*	Сдвиг курсора/экрана
0	0	0	0	1	D/L	N	F	*	*	Параметры развертки и шины данных
0	0	0	1	AG	AG	AG	AG	AG	AG	Установка АС в ОЗУ знакогенератора
0	0	1	AD	AD	AD	AD	AD	AD	AD	Установка АС в ОЗУ видеопамяти

* — безразлично.

I/D = 1 как раз и означает, что после записи в видеопамять каждого байта данных АС должен увеличиваться на 1. S = 1 разрешает упомянутый

выше сдвиг окошка дисплея, S = 0 — запрещает сдвиг. D = 1 включает дисплей, C = 1 включает курсор в виде символа подчеркивания, B = 1 — в виде мигающего символа. Установка D, C и B в 0 отключает соответственно дисплей и отображение всех видов курсора.

DL = 1 устанавливает режим работы с 8-битной шиной данных, N = 0 предписывает дисплею работать с одной строкой, F = 1 — с матрицей 5×9 точек, F = 0 — 5×7 точек. Биты S/C, R/L относятся к одновременным сдвигам курсора и дисплея, и мы здесь рассматривать их не будем.

AD — это адрес ячейки видеопамяти. В последней команде вместо 7 битов AD вы должны разместить адрес ячейки видеопамяти, на которую будет указывать счетчик AC. Этот адрес должен быть в пределах от 0 до 79 (от 0000000b до 1001111b).

AG — это адрес ячейки памяти в так называемом знакогенераторе. Здесь мы его не рассматриваем, поэтому я воздержусь от объяснений, что это такое.

Обычно команды настройки посылают индикатору сразу после подачи на него напряжения питания. Мы пошлем ему следующие команды: команду установки функций, команду управления включением/выключением дисплея и команду автоинкремента. При этом мы зададим 8-битную шину данных (DL = 1), работу индикатора с одной строкой (N = 0) с матрицей 5×7 точек (F = 0), с отключенным изображением курсора (C = 0, B = 0). Также установим I/D = 1 (после записи в видеопамять каждого байта данных AC должен увеличиваться на 1) и S = 0 — запретим сдвиг окошка дисплея. Все это выполнит подпрограмма `INICDV16`, которую при использовании DV-16100 нужно вызвать в самом начале вашей программы, до того, как микроконтроллер должен будет начать отображать результаты выполнения каких-то действий, вычислений или измерений.

Подпрограмма инициализации индикатора DV-16100

```
;
;
INICDV16:
;
  MOV   P1,#11111111B      ; УСТАНОВИЛИ ПОРТ P1
  MOV   P3,#11000111B      ; УСТАНОВИЛИ ПОРТ P3
;
;КОМАНДА УСТАНОВКИ ФУНКЦИЙ С DL=1, N=0, F=0
;
  MOV   P3,#11000111B      ;RS=0, E=0, R/W=0
  MOV   P1,#00110000B      ;DL=1, N=0, F=0
  LCALL IMPULS1            ;ИМПУЛЬС E
;
;КОМАНДА УПРАВЛЕНИЯ ВКЛЮЧЕНИЕМ/ВЫКЛЮЧЕНИЕМ ДИСПЛЕЯ
;C D=1, C=0, B=0
```

```
;
  MOV   P3,#11000111B      ;RS=0, E=0, R/W=0
  MOV   P1,#00001100B      ;D=1, C=0, B=0
  LCALL IMPULS1            ;ИМПУЛЬС E
;
;КОМАНДА АВТОИНКРЕМЕНТА С I/D=1, S=0
;
  MOV   P3,#11000111B      ;RS=0, E=0, R/W=0
  MOV   P1,#00000110B      ;I/D=1, S=0
  LCALL IMPULS1            ;ИМПУЛЬС E
;
  LCALL DEL16MS            ;ЗАДЕРЖКА 16 МС
;
  RET
;
;ПОДПРОГРАММА ЗАДЕРЖКИ 64 МС
;
DEL64MS:LCALL  DEL16MS     ;ЗАДЕРЖКА 64 МС
        LCALL  DEL16MS
        LCALL  DEL16MS
        LCALL  DEL16MS
        RET
;
;ПОДПРОГРАММА ЗАДЕРЖКИ 16 МС
;
DEL16MS:LCALL  DEL4MS      ;ЗАДЕРЖКА 16 МС
        LCALL  DEL4MS
        LCALL  DEL4MS
        LCALL  DEL4MS
        RET
;
;ПОДПРОГРАММА ЗАДЕРЖКИ 4 МС
;
DEL4MS: LCALL  DEL1MS      ;ЗАДЕРЖКА 4 МС
        LCALL  DEL1MS
        LCALL  DEL1MS
        LCALL  DEL1MS
        RET
;
;ПОДПРОГРАММА ЗАДЕРЖКИ 1 МС
;
DEL1MS:                    ;ЗАДЕРЖКА 1 МС
        MOV    R1,#25      ;ПОВТОРЯЕМ 25 РАЗ
;
LREX:   MOV    R2,#18
LRIN:   DJNZ   R2,LRIN     ;36+1 МКС НА 12 МГЦ
;
        DJNZ   R1,LREX
        RET
;
;ПОДПРОГРАММА, ФОРМИРУЮЩАЯ ИМПУЛЬС НА ВХОДЕ E
```

```
;
IMPULS: SETB    P3.5                ;ИМПУЛЬС E
        NOP
        NOP
        NOP
        NOP
        CLR     P3.5
        RET
;
;ПОДПРОГРАММА ИМПУЛЬСА НА E С
;ПОСЛЕДУЮЩЕЙ ЗАДЕРЖКОЙ 1 МС
;
IMPULS1:ACALL   IMPULS              ;ИМПУЛЬС E
        ACALL   DEL1MS              ;ПОСЛЕД.ЗАДЕРЖКА 1 МС
        RET
;
```

Обращаю ваше внимание на следующие моменты. Как следует из **Рис. 4.13**, входы E, RS и R/$\overline{W}$ индикатора соединены соответственно с линиями P3.5, P3.4 и P3.3 нашего микроконтроллера, поэтому перед тем, как подать на индикатор строб-сигнал записи (положительный импульс на вход E), мы устанавливаем эти линии в нули (P3.5 = 0 — чтобы можно было сформировать положительный импульс, P3.4 = 0 — мы передаем в индикатор команду, P3.3 = 0 — будет операция записи). Остальные линии порта P3 в данном применении не используются, в связи с чем мы устанавливаем на них единицы.

Далее, для упрощения работы я не проверяю, завершил ли индикатор выполнение той или иной операции, а ставлю после этих операций задержки, длительность которых гарантированно больше, чем время выполнения предшествующих им операций. Это 16-миллисекундная задержка после завершения подпрограммы `INICDV16` (`LCALL DEL16MS`), и миллисекундная после окончания строб-сигнала записи (`ACALL DEL1MS` в составе подпрограммы `IMPULS1`). Более подробно о том, как организованы эти подпрограммы задержек — чуть ниже. Пока же вернемся к подпрограмме инициализации индикатора.

Первыми командами (`MOV P1,#11111111b` и `MOV P3,#11000111b`) мы устанавливаем в 1 все линии портов P1 и P3, кроме упомянутых P3.5, P3.4 и P3.3. Далее этими же командами выводим на линии порта P3 нули на все тех же P3.5, P3.4 и P3.3, а на P1 — коды, соответствующие командам установки функций, управления включением/выключением дисплея и автоинкремента с выбранными значениями DL, N, F, D, C, B, I/D, S. После вывода на линии P1 кода каждой из упомянутых команд вызываем подпрограмму `IMPULS1`, формирующую на входе E индикатора положительный импульс длительностью несколько микросекунд с последующей миллисекундной задержкой. Собственно импульс формирует подпро-

грамма IMPULS, а далее стоящая команда (ACALL DEL1MS) вызывает формирующую миллисекундную задержку подпрограмму DEL1MS.

Теперь непосредственно о том, как устроена подпрограмма DEL1MS. До сих пор мы формировали задержки, вставляя в нужные места программы команды NOP, время выполнения каждой из которых при тактовой частоте 12 МГц составляет ровно 1 мкс. Пока речь шла о задержках длительностью не более 10...20 мкс, это было вполне приемлемо. Но сейчас нам нужна тысячемикросекундная задержка (1 мс = 1000 мкс). Не писать же подпрограмму, состоящую из 1000 команд NOP. Как быть?

Чаще всего подобную задачу решают следующим образом. Рассмотрим фрагмент подпрограммы задержки.

Фрагмент подпрограммы задержки

```
;
LREX:   MOV     R2,#18
LRIN:   DJNZ    R2,LRIN ;36+1 МКС НА 12 МГЦ
;
```

Первая команда, идущая после метки LREX, очевидна — занесение в регистр R2 числа 18. Вторая — DJNZ — нам не знакома. Познакомимся же с ней.

Аббревиатура этой команды состоит из первых букв словосочетания Decrement and Jump Not Zero, что в переводе означает декрементировать (т. е. уменьшить на 1) и перейти, если то, что мы декрементировали, не равно 0. Как теперь нетрудно догадаться, команда DJNZ R2,LRIN означает буквально следующее: уменьшить R2 на 1, и если содержимое этого регистра после уменьшения не равно 0, то перейти на выполнение команды, идущей после метки, имя которой (LRIN) стоит через запятую после наименования нашего регистра (R2).

Но ведь после этой метки в нашей подпрограмме стоит именно сама команда DJNZ R2,LRIN! Следовательно, уменьшив R2 на 1 и обнаружив, что регистр содержит все еще ненулевой результат, микроконтроллер снова вернется к выполнению этой же самой команды. Выполнив ее еще раз и вновь получив ненулевой R2, микроконтроллер вынужден будет повторять команду до тех пор, пока R2 не станет равным 0. Сколько же раз он ее повторит? Естественно, 18 — ведь именно это число мы поместили в R2 перед тем, как заставили контроллер выполнять команду DJNZ R2,LRIN.

Теперь я забегу немного вперед и скажу, что у микроконтроллеров x51 при тактовой частоте 12 МГц команда занесения числа в какой-либо из регистров осуществляется за 1 мкс, а команда DJNZ R2,LRIN — за 2 мкс. Однократное выполнение первой из них и последующее 18-кратное выполнение второй осуществится за 36 + 1 = 37 мкс, ни больше, ни меньше. Запомним это и двинемся дальше.

Перед вами команды подпрограммы миллисекундной задержки:

```
        MOV   R1,#25  ;ПОВТОРЯЕМ 25 РАЗ
LREX:   MOV   R2,#18
LRIN:   DJNZ  R2,LRIN ;36+1 МКС НА 12 МГЦ
        DJNZ  R1,LREX
        RET
```

Со второй и третьей командами мы уже разобрались. Первая, как вы понимаете, заносит в регистр R1 число 25. А вот четвертая заставляет микроконтроллер именно эти 25 раз выполнить фрагмент из второй и третьей команды.

Как это происходит? Наверное, многие уже догадались, но я все же прокомментирую. Вначале мы заряжаем регистр R1 числом 25. Далее выполняем вторую и третью команды — как мы уже несколько раз говорили, их выполнение длится до тех пор, пока R2 не станет равным 0, и будет это продолжаться 37 мкс. После того, как R2 достигнет нуля, наступит черед выполнения последней команды. Она уменьшит R1 на 1 (т. е. до 24), и поскольку 24 не равно 0, то отправит микроконтроллер на выполнение команды, идущей после метки `LREX:`. А там... да, да, мы опять заносим в R2 число 18 и в течение 37 мкс командой `DJNZ R2,LRIN` уменьшаем его до 0. Затем снова наступит черед выполнения последней команды. Она опять уменьшит R1 на 1 (теперь уже до 23), и опять отправит микроконтроллер на выполнение команды, идущей после метки `LREX:`. И так будет продолжаться, пока R1 не достигнет 0, т.е. 25 раз.

Обратите внимание, каждое выполнение фрагмента, состоящего из второй и третьей команды, занимает 37 мкс. Далее, каждый раз после этого фрагмента микроконтроллер выполняет четвертую команду, длительность которой, как вы догадываетесь, 2 мкс. Итого, 25-кратное выполнение блока из второй, третьей и четвертой команд отнимет у микроконтроллера $25 \times 37 + 25 \times 2 = 975$ мкс. Приплюсуем сюда 1 мкс на выполнение первой команды, 2 мкс на выполнение команды `RET` и 2 мкс, которые микроконтроллер тратит на выполнение команды вызова подпрограммы миллисекундной задержки (`ACALL DEL1MS`). В сумме получаем 980 мкс. Это чуть меньше обещанной миллисекунды, но в настоящем примере 980 микросекунд вполне достаточно, чтобы индикатор гарантированно завершил процесс записи. По этой причине я не стал добиваться того, чтобы выполнение подпрограммы `DEL1MS` занимало ровно миллисекунду, ни микросекундой больше или меньше. Но если такая задача возникла бы, то решить ее не составило бы труда — нужно было бы всего-навсего перед командой `RET` поставить 20 команд `NOP`.

Кажется, с подпрограммой миллисекундной задержки все. Если кому что осталось не ясно — еще раз попытайтесь внимательно перечитать последние 8 абзацев. Там достаточно информации, чтобы понять и идею пост-

роения подпрограммы, и реализацию этой идеи. Перейдем теперь к подпрограммам 4-, 16- и 64-миллисекундной задержек.

Подпрограмма 4-миллисекундной задержки (DEL4MS) устроена предельно просто — она всего-навсего четырежды вызывает только что рассмотренную DEL1MS и все! Не сложнее ее подпрограммы 16-миллисекундной (DEL16MS) и 64-миллисекундной (DEL64MS) задержек — они, в свою очередь, четырежды вызывают подпрограммы соответственно 4-миллисекундной и 16-миллисекундной задержек. В общем, теперь в ваших руках есть средства для построения задержек любой длительности — от единиц микросекунд до десятка секунд. Напомню только, что в приводимом примере упомянутые задержки чуть-чуть меньше 1, 4, 16 и 64 миллисекунд. Если же вам понадобятся задержки, точно равные 4 или, к примеру, 10 мс, вам придется дополнять рассмотренные фрагменты необходимым числом команд NOP.

Итак, мы научились инициализировать индикатор DV-16100 и ему аналогичные, а заодно познакомились с методом формирования задержек средней и большой длительности. Теперь надо бы разобраться с главным — как же выводить требуемую нам информацию на экран индикатора.

Давайте представим себе, что сразу после включения питания нашего устройства, содержащего рассматриваемый индикатор, мы хотим вывести на его экран надпись, демонстрирующую название прибора и номер версии работающей в микроконтроллере программы. Кстати, настоятельно рекомендую придерживаться по возможности такой практики. Дело в том, что написанные нами программы практически всегда содержат некоторое количество ошибок, выявляемых не сразу, а с течением времени. Также иногда по тем или иным причинам в изделии приходится делать некоторые аппаратные доработки, и как следствие этого, так или иначе корректировать программу. По этим причинам с течением времени у вас будет накапливаться несколько слегка различающихся версий одной и той же программы. И когда вы столкнетесь с необходимостью доработки или ремонта изделия, вам будет необходимо точно знать, какая же из них зашита в памяти программ стоящего в нем микроконтроллера. Так что если при включении он сам об этом вам скажет, выведя на экран название прибора и номер версии работающей в нем программы, вам это не покажется ненужным излишеством.

Будем считать, что я убедил вас в необходимости вывода на экран индикатора такой информации. Положим далее, что изделие наше является оптимизатором какого-либо процесса, и номер версии стоящей в нем программы — 11. Поскольку рассматриваемый индикатор — однострочный 16-разрядный, выводимая информация не должна содержать более 16 символов. Этому условию удовлетворяет, например, такая надпись: OPTIMIZER VER.11. Надеюсь, не надо объяснять, что с тем же успехом

это могла быть и другая надпись — AREOMETER VER.33 или VOLTMETER VER.07, и для их формирования нам нужно было бы попросту занести в индикатор коды не тех букв и цифр, которые сформируют надпись OPTIMIZER VER.11, а несколько иных.

Для того, чтобы решить поставленную задачу, программист должен сделать следующее. После обычно идущей в самом начале любой программы настройки линий портов и занесения требуемой информации в регистры он должен поставить вызов рассмотренной выше подпрограммы инициализации индикатора, а вслед за ним — вызов приведенной ниже подпрограммы отображения на экране индикатора надписи OPTIMIZER VER.11.

Подпрограмма отображения на экране индикатора надписи OPTIMIZER VER.11

```
;ИНДИКАТОР НАСТРОЕН. ТЕПЕРЬ ВЫВОД НА ЭКРАН
;НАДПИСИ OPTIMIZER VER.11
;
OTB:    MOV    P3,#11000111B   ;Clear display
        MOV    P1,#00000001B
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01001111B   ;«O»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01010000B   ;«P»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01010100B   ;«T»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01001001B   ;«I»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
```

```
        MOV    P3,#11010111B
        MOV    P1,#01001101B   ;«M»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01001001B   ;«I»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01011010B   ;«Z»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01000101B   ;«E»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01010010B   ;«R»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#00100000B   ;« »
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01010110B   ;«V»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01000101B   ;«E»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#01010010B   ;«R»
        LCALL  IMPULS1
```

```
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#00101110B   ;«.»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#00110001B   ;«1»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    P1,#00110001B   ;«1»
        LCALL  IMPULS1
;
        LCALL  DEL16MS
;
        RET
;
```

Как нетрудно заметить, по структуре она очень похожа на приведенную на стр. 123 подпрограмму инициализации индикатора. Мы настраиваем линии порта P3, управляющие сигналами на входах E, RS и R/$\overline{W}$ индикатора (соответственно P3.5, P3.4 и P3.3), затем выводим на линии порта P1 код команды или отображаемого символа, вызываем формирующую строб-импульс записи подпрограмму `IMPULS1` и 16-миллисекундную задержку. И так пока не выведем всю необходимую информацию.

Но есть и отличия. Вначале подпрограммы идет команда сброса дисплея, устанавливающая счетчик адреса AC в 0 и проецирующая окошко дисплея на первые 16 ячеек видеопамяти. При этом на линии порта P1 выводится код команды сброса дисплея (00000001b), а на линии порта P3 — код 11000111b, устанавливающий нулевые сигналы на входах E, RS и R/$\overline{W}$ индикатора. Далее мы последовательно начинаем пересылать в индикатор коды отображаемых символов. При этом — обратите внимание! — на линии порта P3 мы выводим не 11000111b, а 11010111b, т. е. бит, управляющий входом RS, мы устанавливаем в 1. Зачем? Затем, что если сигнал на входе RS индикатора нулевой, то последний воспринимает передаваемую ему по входам DB7...DB0 (т. е. по линиям P1) информацию как код команды, а если RS = 1, то как код отображаемого символа. Поэтому при пересылке кодов отображаемых символов мы на линии порта P3 выводим 11010111b.

Остальное предельно просто. 01001111b — это, как следует из приведенной в **Табл. 4.2** таблицы кодов символов, код буквы O, 01010000b — буквы P, 01010100b — буквы T и т. д. Поняв это, вы теперь сможете сфор-

мировать на нашем индикаторе любую надпись. Как видите, все действительно элементарно — нужно лишь придумать, какой текст вы хотите вывести на экран, выписать без ошибок коды составляющих его символов да подставить их в приведенную на стр. 129 подпрограмму ОТВ. И любоваться творением рук своих...

В общем, вы узнали об индикаторах на основе HD44780 почти все, что нужно для того, чтобы успешно начать работать с ними. Осталась малость — научиться выводить на экран цифровую информацию, получаемую микроконтроллером в процессе измерений или вычислений. Я думаю, что многие из вас уже и самостоятельно смогли бы справиться с этой задачей. Но тем не менее я приведу еще один пример — подпрограмму отображения на экране индикатора цифр, хранящихся в ОЗУ микроконтроллера в ячейках с адресами от `AD00+1` до `AD00+11`. Этот пример интересен тем, что цифровая информация перед выводом в индикатор требует определенной перекодировки, т. к. коды цифр от 0 до 9 (см. **Табл. 4.2**) отличаются от двоичного или двоично-десятичного представления, в котором они, как правило, хранятся в ОЗУ микроконтроллера.

О том, в какой из разрядов индикатора выводится содержимое той или иной ячейки, ясно из комментариев. Также отмечу, что в 4, 7, 14, 15 и 16-й разряды индикатора эта подпрограмма выводит пробелы (`SPACE`). Как следует из **Табл. 4.2**, код пробела — 20h. Но в теле подпрограммы, как несложно заметить, я ставлю не этот код, а его символическое имя (т. е. не `MOV P1,#020h`, а `MOV P1,#SPACE`). Поэтому в начале вашей основной программы, где идет описание адресов регистров и символических имен, не забудьте поставить строчку `SPACE .EQU 20h`. Если этого не сделать, ассемблер не поймет, что же вы от него хотите, и в тех строках, где он найдет это имя, он даст сообщения об ошибках (см. главу 2)

Подпрограмма вывода на индикатор цифровой информации из ячеек AD00+1...AD00+11

```
;
;
;
IZOBR1:
        MOV    P3,#11000111B     ;СБРОС ДИСПЛЕЯ
        MOV    P1,#00000001B
        LCALL  IMPULS1
        LCALL  DEL16MS
;
        MOV    P3,#11010111B
        MOV    A,AD00+1
        CJNE   A,#0FH,IZR1
        MOV    A,#SPACE
        SJMP   IZR2
```

```
IZR1:   ORL    A,#00110000B
IZR2:   MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+1 В
                           ;1-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+2
        CJNE   A,#0FH,IZR3
        MOV    A,#SPACE
        SJMP   IZR4
IZR3:   ORL    A,#00110000B
IZR4:   MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+2 ВО
                           ;2-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+3
        CJNE   A,#0FH,IZR5
        MOV    A,#SPACE
        SJMP   IZR6
IZR5:   ORL    A,#00110000B
IZR6:   MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+3 В
                           ;3-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    P1,#SPACE
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ SPACE В
                           ;4-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+4
        CJNE   A,#0FH,IZR7
        MOV    A,#SPACE
        SJMP   IZR8
IZR7:   ORL    A,#00110000B
IZR8:   MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+4 В
                           ;5-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+5
        CJNE   A,#0FH,IZR9
        MOV    A,#SPACE
        SJMP   IZR10
IZR9:   ORL    A,#00110000B
IZR10:  MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+5 В
                           ;6-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
```

```
        MOV    P1,#SPACE
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ SPACE В
                           ;7-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+6
        CJNE   A,#0FH,IZR11
        MOV    A,#SPACE
        SJMP   IZR12
IZR11:  ORL    A,#00110000B
IZR12:  MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+6 В
                           ;8-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+7
        CJNE   A,#0FH,IZR13
        MOV    A,#SPACE
        SJMP   IZR14
IZR13:  ORL    A,#00110000B
IZR14:  MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+7 В
                           ;9-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+8
        CJNE   A,#0FH,IZR15
        MOV    A,#SPACE
        SJMP   IZR16
IZR15:  ORL    A,#00110000B
IZR16:  MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+8 В
                           ;10-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+9
        CJNE   A,#0FH,IZR17
        MOV    A,#SPACE
        SJMP   IZR18
IZR17:  ORL    A,#00110000B
IZR18:  MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+9 В
                           ;11-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+10
        CJNE   A,#0FH,IZR19
        MOV    A,#SPACE
        SJMP   IZR20
IZR19:  ORL    A,#00110000B
IZR20:  MOV    P1,A
```

```
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+10 В
                           ;12-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    A,AD00+11
        CJNE   A,#0FH,IZR21
        MOV    A,#SPACE
        SJMP   IZR22
IZR21:  ORL    A,#00110000B
IZR22:  MOV    P1,A
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ AD00+11 В
                           ;13-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    P1,#SPACE
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ SPACE В
                           ;14-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    P1,#SPACE
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ SPACE В
                           ;15-Й РАЗРЯД ИНДИКАТОРА
;
        MOV    P3,#11010111B
        MOV    P1,#SPACE
        LCALL  IMPULS1     ;ОТОБРАЗИЛИ SPACE В
                           ;16-Й РАЗРЯД ИНДИКАТОРА
;
        RET
;
;
```

Как и для ранее рассмотренных индикаторов, мы предполагаем, что цифры в ячейках `AD00+1`…`AD00+11` хранятся в обычном двоичном представлении: 00000000b — это 0, 00000001b — это 1, ..., 00001001b — это 9. Также аналогично предыдущим случаям 00001111b в `AD00+1`…`AD00+11` — это погашенный символ, т. е. пробел. Как уже упоминалось, при выводе информации на индикатор и цифры, и пробел требуют перекодировки. Осуществляется это следующим образом. Вначале каждый символ помещается в аккумулятор, и командой `CJNE A,#0Fh,IZRxx` сравнивается с 0Fh. В случае равенства микроконтроллер понимает, что выводимый в индикатор символ — пробел, и в аккумулятор помещается для дальнейшего вывода в порт P1 код пробела. В противном случае, перекодировка осуществляется с использованием команды `ORL A,#00110000b`.

Что делает эта команда? Она осуществляет операцию побитового логического ИЛИ содержимого аккумулятора и числа, в данном случае 00110000b. В результате этой операции 7, 6, 3, 2, 1 и 0-й биты аккумулятора останутся неизменными, т. к. соответствующие биты числа 00110000b со-

держат нули, а 5-й и 4-й биты аккумулятора установятся в 1, независимо от того, какими они были до выполнения рассматриваемой команды.

Зачем это нам нужно? Обратимся еще раз к таблице символов (**Табл. 4.2**). Заметьте, что код нуля — 30h или 00110000b, код единицы — 31h или 00110001b, код двойки — 32h или 00110010b и т. д., вплоть до девятки (39h или 00111001b). Иными словами, чтобы вывести на индикатор DV-16100 хранящийся в `AD00+n` нуль (00000000b), нужно переслать в него код нуля, или 00110000b, для отображения единицы (00000001b) — код единицы (00110001b) и т. д. А поскольку код любой и цифр от 0 до 9 отличается от хранящейся в `AD00+n` в двоичном представлении самой цифры именно содержимым 5-го и 4-го бита, то преобразование цифры в соответствующий ей код осуществляется простой установкой этих самых битов в 1. Что, собственно, и делает нам команда `ORL A,#00110000b`.

На этом мы закончим знакомство с индикаторами на основе контроллеров типа HD44780 — теперь работа с ними вам вполне по плечу. Следующий объект нашего знакомства — четырехразрядный матричный (5×7) светодиодный индикатор HCMS-2xxx от Hewlett-Packard.

4.4. Сопряжение с 4-разрядными светодиодными матричными индикаторами

Индикаторы HCMS-2xxx (отечественные аналоги — КИПВ70А-4/5×7К, КИПВ71А-4/5×7К) являются малогабаритными 4-разрядными светодиодными матричными индикаторами. Они позволяют отображать практически любые символы и требуют для сопряжения относительно небольших аппаратных ресурсов микроконтроллера, даже меньших, чем рассмотренный ранее АЛС318. Они хороши для встраивания в переносную аппаратуру, работающую в условиях недостаточного освещения и/или низкой температуры, где применение жидкокристаллических индикаторов сопряжено с некоторыми проблемами.

Внешний вид и габаритные размеры индикаторов приведены на **Рис. 4.16.**

Далее по тексту я эти индикаторы буду называть HCMS-2xxx, но все сказанное будет справедливо и для упомянутых выше отечественных аналогов.

Индикаторы совместимы с TTL-уровнями, имеют 4 знаковых разряда и представляют собой гибридную сборку в керамическом корпусе из четырех матриц по 7×5 элементов (140 светоизлучающих диодов) и регистра сдвига с формирователями (драйверами) постоянного втекающего тока. Каждая матрица светоизлучающих диодов имеет 7 строк (у диодов в строке соединяются катоды) и 5 столбцов (у диодов в столбце соединяются ано-

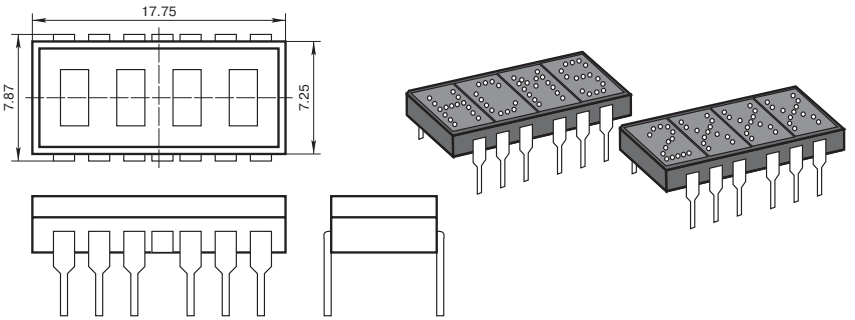


Рис. 4.16. Внешний вид и габаритные размеры индикаторов HCMS-2xxx

ды). Соответствующие выводы столбцов всех матриц соединены между собой и с 1...5 выводами индикатора (**Рис. 4.17**). Строки матриц наружу не выводятся, и внутри индикатора соединены каждая со своим драйвером втекающего тока (на **Рис. 4.17** эти драйверы символически изображены в виде транзисторов обратной проводимости).

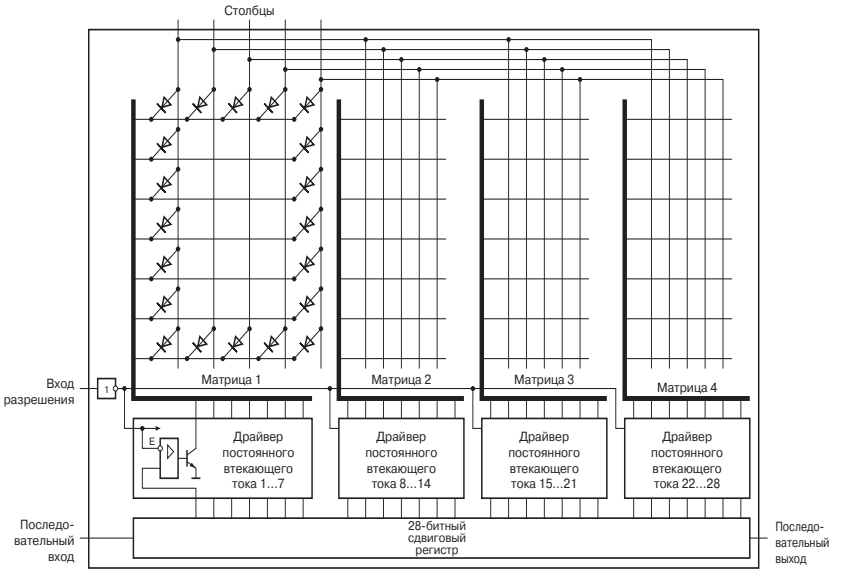


Рис. 4.17. Внутренняя структура индикаторов HCMS-2xxx

Схема управления содержит 28-битный внутренний сдвиговый регистр с последовательным входом и последовательным и параллельными выходами, причем последние, как упоминалось, управляют драйверами втекающего постоянного тока. Драйверы, в свою очередь, подключены к общим катодным выводам строк светоизлучающих матриц. Последовательный выход сдвигового регистра связан с последним его битом и может быть подключен ко входу следующего такого же индикатора, что позволяет создавать 8-, 12-, 16- и т. д. разрядные индикаторы.

Сдвиговый регистр снабжен также входом разрешения, нулевой сигнал на котором отключает отображение независимо от состояния сигналов на входах столбцов индикатора и содержимого регистра.

Схема сопряжения HCMS-2xxx с микроконтроллером приведена на **Рис. 4.18.**

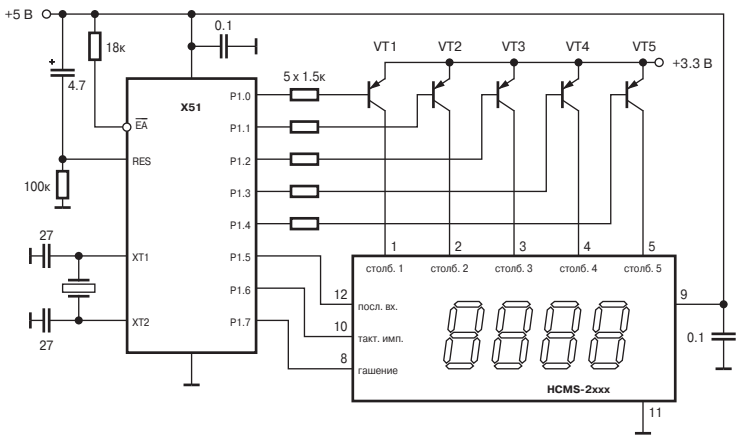


Рис. 4.18. Схема сопряжения HCMS-2xxx с микроконтроллером

Как видите, она довольно проста. Пять линий порта P1 (с P1.0 по P1.4) управляют транзисторами прямой проводимости, формирующими токи через столбцы матриц светоизлучающих диодов. Шестая линия, P1.5, передает информацию в сдвиговый регистр индикатора. Запись информации в сдвиговый регистр осуществляется тактовыми импульсами, формируемыми на P1.6. Для отключения индикатора можно использовать еще одну линию порта (P1.7), сигнал на которой при разрешении отображения должен быть единичным, а при запрещении — нулевым. Если мы не собираемся гасить отображение нулевым сигналом на входе га-

шения индикатора, его нужно оторвать от P1.7 и подать на него уровень логической единицы.

Работает наш индикатор в режиме динамической индикации. Вначале мы должны загрузить в него 28 бит, которые определят состояние (включен или выключен) каждого из светодиодов выбранного столбца у всех четырех матриц. Пусть, для определенности, вначале мы зажжем светодиоды первых (крайних слева) столбцов индикаторных матриц, открыв для этого транзистор VT1. Первый из пересланных в сдвиговый регистр бит определит состояние нижнего светодиода левого столбца крайней справа индикаторной матрицы, второй бит — второго снизу светодиода этого же столбца этой же матрицы и т. д. до 7-го бита, задающего состояние верхнего светодиода левого столбца правой матрицы (см. **Рис. 4.19**). Кстати, если занесенный в сдвиговый регистр соответствующий светодиоду бит равен 1, то светодиод при активировании столбца будет гореть, если равен 0 — соответственно будет погашен.

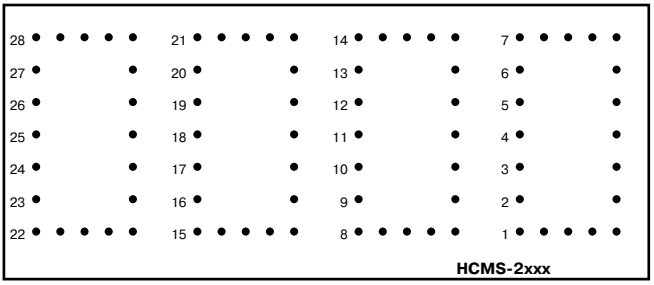


Рис. 4.19. Соответствие между битами сдвигового регистра и светодиодами выбранного (первого) столбца

Далее, 8-й из переданных в сдвиговый регистр бит определит состояние нижнего светодиода левого столбца второй справа индикаторной матрицы, 9-й бит — второго снизу светодиода этого же столбца этой же матрицы и т. д. до 14-го бита, задающего состояние верхнего светодиода левого столбца второй справа матрицы. Аналогичным образом биты с 15-го по 21-й зададут состояние светодиодов левого столбца третьей справа матрицы, а с 22-го по 28-й — четвертой матрицы, крайней слева. Последний, 28-й бит будет управлять ее левым верхним светодиодом.

Итак, мы разобрались, какой из битов сдвигового регистра отвечает за какой светодиод. В соответствии с тем, какие из них для отображения требуемой информации должны быть зажжены, а какие погашены, мы заносим информацию в упомянутый регистр и открываем VT1. При этом те

светодиоды левого столбца всех четырех матриц, которым соответствуют единичные биты, загораются, а все остальные светодиоды этих матриц оказываются погашенными.

Дав им погореть несколько миллисекунд, мы должны вывести в сдвиговый регистр информацию для светодиодов второго слева столбца. Так же, как и в предыдущем случае, первый из пересланных в сдвиговый регистр бит определит состояние нижнего светодиода второго слева столбца крайней правой из индикаторных матриц, второй бит — второго снизу светодиода этого же столбца этой же матрицы и т. д. до 7-го бита, задающего состояние верхнего светодиода второго слева столбца правой матрицы. Точно так же, как и в предыдущем случае, биты с 8-го по 14-й отвечают за светодиоды второй справа матрицы, с 15-го по 21-й — третьей справа, и с 22-го по 28-й— левой матрицы. Но еще раз подчеркну, поскольку мы собираемся пропускать ток через второй слева столбец, названные биты отвечают теперь за светодиоды именно этого, второго столбца.

Занеся информацию в упомянутый регистр, мы на несколько миллисекунд открываем VT2. При этом, как и в предыдущем случае, те светодиоды второго слева столбца всех четырех матриц, которым соответствуют единичные биты, загораются, а все остальные светодиоды этих матриц оказываются погашенными.

Далее по описанному алгоритму мы выводим в сдвиговый регистр информацию для третьего слева столбца и на несколько миллисекунд открываем VT3. После этого, как нетрудно догадаться, нужно вывести в него информацию для четвертого слева столбца и открыть VT4, а затем — и для пятого, сопровождая это открыванием VT5. По завершении отображения пятого столбца цикл необходимо повторить, и повторять его до того момента, пока мы не прервем процесс отображения.

Для тех, кто только делает первые шаги в микроконтроллерной технике, это выглядит крайне непросто. Откуда взять информацию, какие светодиоды надо зажигать в первом столбце, какие во втором и т. д.? Как выводить информацию в сдвиговый регистр, управлять транзисторами, формировать десятичную точку? На фоне этих вопросов работа рассмотренных выше индикаторов, особенно HT1610, кажется вам, наверное, ох какой простой!

Но не отчаивайтесь, с теми знаниями, которые у вас уже имеются, вам и этот индикатор вполне по зубам. Вам это может показаться удивительным, но программа, которая оживит этот индикатор, содержит лишь те команды, которые нам с вами уже хорошо знакомы. Вся сложность работы заключается в том, чтобы разбить поставленную задачу на ряд более простых, и написать подпрограммы для этих более простых задач. Пока у вас еще просто очень мало навыков, и описанные в этой главе примеры, равно как и те, которые вы встретите в последующих главах, направлены в пер-

вую очередь на то, чтобы эти навыки у вас сформировать, по крайней мере некоторое минимальное их количество, которое позволит вам, опираясь на них, двигаться дальше самостоятельно.

Мы чуть-чуть отвлеклись, но сделал это я умышленно. Я хочу, чтобы вы обратили особое внимание на то, как мы разобьем задачу на части, решим каждую из этих малых задач, причем даже не сразу, а, как говорится, в два-три приема. Я специально покажу вам, что получается при первой попытке решения задачи, и как это решение затем оптимизируется. Те, кто имеет большой опыт, проводят эти предварительные стадии разработки в уме. Но у нас с вами такого опыта нет, поэтому мы будем решать нашу задачу постепенно, не всегда сразу оптимально, но в конце получим как некоторые новые навыки, так и правильно работающую подпрограмму, а именно это и есть наша цель.

Итак, поехали. По аналогии с рассмотренными выше индикаторами сформулируем задачу следующим образом. Пусть в ячейках микроконтроллера `AD00+0...AD00+3` хранятся в обычном двоичном представлении четыре цифры, которые нам нужно отобразить на экране индикатора следующим образом: цифру из `AD00+0` — в крайнем справа разряде индикатора, из `AD00+1` — во втором справа разряде, из `AD00+2` — в третьем справа и из `AD00+3` — в крайнем слева. Вот и вся задачка. Естественно, с учетом правил работы с HCMS-2xxx.

Первый шаг, который мы сделаем — введем некоторое упрощение в упомянутые только что правила работы с индикатором. Как вы помните, мы говорили о том, что мы должны вывести в сдвиговый регистр индикатора вначале информацию, касающуюся первых столбцов *всех четырех отображаемых символов* и открыть VT1, затем — касающуюся вторых столбцов *всех четырех отображаемых символов* и открыть VT2 и т. д. Это означает, что перед выводом в сдвиговый регистр информации мы должны так или иначе обрабатывать все 4 отображаемых символа одновременно. Для начинающих это — достаточно сложная задача. Чтобы ее упростить, давайте поступим следующим образом. Мы будем отображать символы по одному. Как? Сначала занесем в сдвиговый регистр в биты 1...7 информацию, касающуюся первого столбца того символа, который должен отобразиться в крайнем справа разряде индикатора, а в биты 8...28, ответственные за соответствующие столбцы остальных символов, занесем нули. Откроем VT1 и отобразим первый столбец этого правого символа. Затем, попрежнему занулив биты 8...28, в биты 1...7 занесем информацию относительно второго столбца этого правого символа, выведем полученные 28 бит в сдвиговый регистр и, открыв VT2, отобразим второй его столбец. Далее, занеся опять-таки в 1...7 биты информацию о третьей строке отображаемого символа, а в 8...28 биты —опять-таки нули, выведем их в регистр, откроем VT3 и отобразим третий столбец (а затем аналогично ему

четвертый и пятый). Таким образом мы отобразим весь символ, при этом остальные три символа отображены не будут, т. к. в биты, их формирующие, мы все время заносили нули.

Думаю, что вы уже догадались, что для отображения второго символа соответствующую его столбцам информацию мы будем заносить в 8...14 биты, а в 1...7 и в 15...28 биты опять будем помещать нули. Занося в 8...14 биты информацию, управляющую светодиодами первой, второй и т. д., вплоть до пятой колонки и открывая последовательно все те же VT1, VT2, ..., VT5, мы отобразим второй символ при погашенных первом, третьем и четвертом. Естественно, для отображения третьего символа соответствующую его столбцам информацию мы будем заносить в 15...21-й биты, а помещать нули будем в 1...14-й и в 22...28 биты, а для отображения четвертого символа занулять будем 1...21 биты, а информацию, соответствующую его столбцам, станем выводить в 22...28биты.

Итак, задача слегка упростилась — мы собираемся выводить в индикатор (в соответствии с описанным в двух предыдущих абзацах упрощенным алгоритмом) символы последовательно — сначала из `AD00+0` в крайний справа знаковый разряд, затем из `AD00+1` во второй справа и т. д. Что нам для этого нужно?

Нам нужно написать подпрограмму, которая отображала бы цифру, помещенную в один из регистров микроконтроллера (пусть для определенности в R2) в разряд индикатора, номер которого размещен в другом регистре (к примеру, в R3). Также для определенности будем считать, что номер крайнего справа отображаемого индикатором символа — 0, второго справа — 1, третьего справа — 2, и четвертого справа, т. е. левого — 3. Лучше эти номера выбрать именно таким образом — тогда из `AD00+0` символ выводится в 0-й разряд индикатора, из `AD00+1`— в 1-й разряд, из `AD00+2` — во 2-й разряд, а из `AD00+3` — в 3-й разряд.

Еще раз повторюсь — нам нужна подпрограмма (назовем ее `VIVHCMS`), выводящая символ, размещенный в регистре R2, в разряд индикатора, номер которого помещен в регистр R3. Если она будет в нашем распоряжении, то поставленная задача (вывод информации в индикатор из ячеек микроконтроллера с адресами `AD00+0`...`AD00+3`) легко решится при помощи следующей программки.

Фрагмент подпрограммы отображения

```
;
OTBHCMS:
;
        MOV   R1,#255
;
OTBHC1:
        MOV   R2,AD00+0
```

```
        MOV   R3,#0
        LCALL VIVHCMS
;
        MOV   R2,AD00+1
        MOV   R3,#1
        LCALL VIVHCMS
;
        MOV   R2,AD00+2
        MOV   R3,#2
        LCALL VIVHCMS
;
        MOV   R2,AD00+3
        MOV   R3,#3
        LCALL VIVHCMS
;
        DJNZ  R1,OTBHC1
        RET
```

Перед вами простенькая подпрограмма, которая 255 раз выполняет следующие действия: вначале помещает в R2 данные из ячейки с адресом `AD00+0`, в R3 — ноль, т. е. адрес ячейки индикатора, где нужно отобразить эти данные, и вызывает задуманную, но еще не написанную `VIVHCMS`, выводящую символ, размещенный в регистре R2, в разряд индикатора, номер которого помещен в регистр R3. Затем помещает в R2 данные из ячейки с адресом `AD00+1`, в R3— единицу, т. е. адрес ячейки индикатора, где нужно отобразить эти данные, и вновь вызывает `VIVHCMS`. То же самое повторяется с данными из `AD00+2` и `AD00+3`, которые попадают соответственно во второй и третий разряды индикатора. И все! Как только мы сформулировали требования к тому, что же должна делать `VIVHCMS`, написать подпрограмму, решающую основную задачу, оказалось довольно просто. Все сложности теперь именно в подпрограмме `VIVHCMS`, но нас уже больше ничего не отвлекает, мы можем сконцентрировать свои усилия только на ней, и скоро вы убедитесь, что она не столь уж сложна.

А теперь — еще одно небольшое отвлечение. Почему я решил выделить в отдельную подпрограмму эту `VIVHCMS`, которая должна выводить символ, размещенный в регистре R2, в разряд индикатора, номер которого помещен в регистр R3? Как определить, когда нужно пытаться те или иные действия вынести в определенную подпрограмму, и как сформулировать, что же эта подпрограмма должна делать?

Ответ простой — любые действия, которые вам придется делать в вашей программе более, чем один или два раза, полезно оформить в виде подпрограммы со сразу понятным для вас именем. Например, вы не раз будете в своих программах осуществлять операции умножения одного 16-битного числа на другое или делить 32-битное число на 16-битное, следовательно, эти действия нужно оформить в виде соответствующих подпро-

грамм (в одной из следующих глав мы тщательно их разберем). По той же причине мы выделили в самостоятельную подпрограмму процесс преобразования двоичного числа в двоично-десятичное (вспомните главу 3) — эта подпрограмма всегда нужна, если вы собираетесь отобразить на знакосинтезирующем индикаторе результаты тех или иных измерений или вычислений.

Да и зачем далеко ходить? Вспомните, как мы организовали вывод информации на экран HT1610. Мы занесли в аккумулятор число из `AD00+7` и вызвали подпрограмму `SIMBOL1`, затем — число из `AD00+6` и снова вызвали `SIMBOL1` и т. д. Вам это ничего не напоминает? А подпрограмма `DISPLEY` из раздела, посвященного АЛС318? При ее вызове в R0 мы помещали число от 0 до 7, и она далее извлекала цифру из ячейки памяти микроконтроллера с адресом `AD00+n` (n = 0...7), и выводила его в порт P1 для отображения в *n*-й разряд АЛС318. Как видите, по крайней мере в том случае, когда вам нужно вывести на тот или иной дисплей цифровую информацию (а это всегда не менее 3...4 цифр), очень полезно выделить в самостоятельную подпрограмму все действия по пересылке символа на дисплей, при этом сам пересылаемый символ (а если надо, то и его место отображения на индикаторе), должен находиться в каком-нибудь регистре (регистрах) нашего микроконтроллера. Если вы привыкнете это делать, то написанные вами программы будут проще, стройнее и безошибочнее, чем если вы будете писать их без подобной привычки.

Кстати, при написании подпрограммы вывода цифр в DV-16100, приведенной на стр. 132, я поленился выделить совокупность команд, организующих вывод цифры в индикатор, в отдельную подпрограммку. Результат не замедлил сказаться — посмотрите, сколь большой и сложной оказалась приведенная подпрограмма `IZOBR1`! А если бы не поленился, то получилось бы нечто похожее на подпрограмму `INDVIV` на стр. 99. Кстати, можете в качестве примера попробовать совершить такую оптимизацию подпрограммы `IZOBR1` самостоятельно, это будет полезной практикой.

Но вернемся к нашему матричному индикатору. Подпрограмма вывода информации на индикатор HCMS-2xxx нами уже написана (стр. 142), и теперь нужно только составить отдельную подпрограмму `VIVHCMS`, которая должна выводить символ, размещенный в регистре R2, в разряд индикатора, номер которого помещен в регистр R3. Займемся ею.

Вспомним, что при отображении того или иного символа мы должны в семь выбранных ячеек сдвигового регистра HCMS-2xxx вывести вначале информацию о том, какие из светодиодов первого столбца формирующей изображение матрицы должны быть зажжены, а какие погашены, затем туда же вывести аналогичную информацию для второго, третьего, четвертого и пятого столбцов. Вот вам и вопрос — откуда взять эту информацию?

Обратимся к **Рис. 4.20.** На нем схематически изображены несколько вариантов нашей знакосинтезирующей матрицы формата 7×5 со сформированными изображениями символов «0», «3» и «3.» (в последнем случае — цифра 3 с идущей сразу после нее десятичной точкой). Для того, чтобы эти цифры были сформированы реальным индикатором, нужно, чтобы те светодиоды матрицы, которые соответствуют квадратикам, содержащим цифру 1, были зажжены, а те, которые соответствуют пустым квадратикам — погашены. Это, думаю, понятно всем, кто читает эти строки.

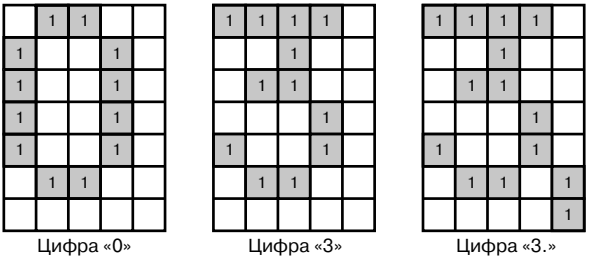


Рис. 4.20. К принципу формирования символов в матрице 7×5

Далее, вспомним, что при отображении первого столбца (открывании VT1) первый пересланный в сдвиговый регистр бит отвечает за состояние левого нижнего светодиода знакосинтезирующей матрицы (бит 1 = 1 — светодиод загорится, бит 1 = 0 — погаснет). Второй бит, как тоже нетрудно вспомнить, отвечает за второй снизу светодиод этого же столбца и т. д., вплоть до седьмого, который отвечает за седьмой снизу, т. е. за верхний светодиод все того же первого столбца. А теперь— внимание! С учетом сказанного, для того, чтобы отобразить первую колонку цифры 0, биты с 1 по 7 в сдвиговом регистре должны иметь следующие значения: бит 1 = 0, бит 2 = 0, бит 3 = 1, бит 4 = 1, бит 5 = 1, бит 6 = 1, бит 7 = 0. Для отображения второй и третьей колонок цифры 0, биты с 1 по 7 в сдвиговом регистре должны иметь следующие значения: бит 1 = 0, бит 2 = 1, бит 3 = 0, бит 4 = 0, бит 5 = 0, бит 6 = 0, бит 7 = 1. Чтобы отобразить четвертую колонку цифры 0, биты с 1 по 7 в сдвиговом регистре должны иметь следующие значения: бит 1 = 0, бит 2 = 0, бит 3 = 1, бит 4 = 1, бит 5 = 1, бит 6 = 1, бит 7 = 0, а для пятой колонки все семь бит, пересылаемые в сдвиговый регистр, должны быть нулевыми.

Я специально так подробно расписал состояние всех семи бит для каждой из пяти колонок — в описываемом здесь принципе формирования цифр на экране матричного индикатора обязательно нужно разобраться так, чтобы была исключена любая неясность или двусмысленность, в противном случае вы очень скоро потеряете нить рассуждений. Если вам что-

то оказалось непонятным — еще раз внимательно прочитайте содержимое последних двух абзацев и вникните в приводимую в них информацию — ее вполне достаточно для понимания.

Идем далее. Давайте разберемся с битами, формирующими цифру 3. Для того, чтобы отобразить первую колонку цифры 3, биты с 1 по 7 в сдвиговом регистре должны иметь следующие значения: бит 1 = 0, бит 2 = 0, бит 3 = 1, бит 4 = 0, бит 5 = 0, бит 6 = 0, бит 7 = 1. Для отображения второй колонки цифры 3, биты с 1 по 7 в сдвиговом регистре должны иметь следующие значения: бит 1 = 0, бит 2 = 1, бит 3 = 0, бит 4 = 0, бит 5 = 1, бит 6 = 0, бит 7 = 1. Третья колонка предполагает, что бит 1 = 0, бит 2 = 1, бит 3 = 0, бит 4 = 0, бит 5 = 1, бит 6 = 1, бит 7 = 1. Чтобы отобразить четвертую колонку тройки, биты с 1 по 7 в сдвиговом регистре должны иметь следующие значения: бит 1 = 0, бит 2 = 0, бит 3 = 1, бит 4 = 1, бит 5 = 0, бит 6 = 0, бит 7 = 1, а для пятой колонки, как и в предыдущем случае, все семь бит, пересылаемые в сдвиговый регистр, должны быть нулевыми.

Третий пример — тройка со стоящей после нее десятичной точкой. Формирующие изображение этого символа (точнее, двух символов, ведь десятичная точка — самостоятельный символ) биты для колонок с первой по четвертую совершенно идентичны битам для этих же колонок, формирующих цифру 3. В этом нет ничего удивительного — и там, и там формируется цифра 3. А вот пятая колонка для цифры со стоящей после нее десятичной точкой (причем любой, не только тройки) должна выглядеть следующим образом: бит 1 = 1, бит 2 = 1, бит 3 = 0, бит 4 = 0, бит 5 = 0, бит 6 = 0, бит 7 = 0.

Я надеюсь, рассмотренные три примера дают ясное, четкое и однозначное объяснение принципа формирования цифр на экране матричного индикатора. Кстати, совершенно аналогично они формируются и в индикаторах типа DV-16100, но входящий в состав этих индикаторов контроллер выполняет такое формирование самостоятельно, благодаря чему процесс вывода в него информации намного проще, чем аналогичный в HCMS-2xxx.

Пойдем дальше. Как мы только что убедились, каждому из отображаемых на HCMS-2xxx символу должны быть поставлены в соответствие пять 7-битных чисел: первое — с информацией для первой колонки, второе — для второй колонки и т. д. Вы, наверное, не забыли, что ячейки как памяти программ нашего микроконтроллера, так и памяти данных, 8-битные. Следовательно, информацию о каждом символе нам придется хранить в 5 ячейках памяти программ. Если нам предстоит отображать 32 символа, информация о них займет 32 × 5 = 160 байт памяти программ, если 128 символов — то 128 × 5 = 640 байт. Это нужно учитывать при анализе того, хватит ли у применяемого микроконтроллера объема ПЗУ и для про-

граммы, и для этих данных, либо придется применять микроконтроллер с бо́льшим объемом ПЗУ.

Согласитесь, что естественно то, что первыми в этом массиве информации, необходимой для формирования изображения цифр (этот массив часто называют таблицей знакогенератора) должны быть 5 байт, требуемые для формирования цифры 0, за ними — 5 байт с информацией для цифры 1 и т. д., по крайней мере до цифры 9 включительно. Логично также и то, что внутри каждых пяти байт, несущих информацию о той или иной цифре, первый байт содержит информацию, необходимую для формирования первой колонки этой цифры, второй байт — второй колонки, третий байт — третьей колонки, четвертый — четвертой, и пятый — пятой. По крайней мере, если соблюсти именно такой порядок, интересующая нас подпрограмма `VIVHCMS` напишется относительно просто. Любой же другой порядок следования информации в таблице знакогенератора заметно усложнит нашу подпрограмму.

Кстати, а как вы думаете, почему таблица знакогенератора должна храниться именно в памяти программ? Не догадались? Ну а где же еще? Ведь и память данных, и регистры теряют свое содержимое при выключении питания...

У вас может возникнуть еще вопрос — информация для вывода в сдвиговый регистр 7-битная, а ячейки памяти — 8-битные. Как быть?

Ну, хранить 7 бит информации в 8-битной ячейке можно (наоборот нельзя). Давайте договоримся, что эта 7-битная информация будет у нас храниться в младших семи битах отведенного под нее байта. В старшем, восьмом его бите, который мы никак не будем использовать, может с равным успехом быть и 0, и 1. Пусть для определенности это всегда будет 1. В общем, это абсолютно несущественно, важно то, что мы должны исключить любые неопределенности, это правило, которого всегда полезно придерживаться.

Итак, мы договорились, что в памяти программ начиная с некоторого определенного места (пометим его какой-либо меткой, например `TABZNAK:`) хранятся 5 байт с информацией для отображения цифры 0, за ними — 5 байт с информацией для отображения цифры 1, затем по пять байт с информацией для двойки, тройки, четверки, ..., восьмерки, девятки. Далее, после 9 в шестнадцатеричной системе идут еще цифры A, B, C, D, E и F (надеюсь, не забыли). Отведем еще по 5 байт под каждую из них. А какие цифры идут в этой HEX-системе дальше, после F? Правильно, 10, 11, 12, ..., 19, 1A, 1B, ... 1F . Если вы помните, при анализе АЛС318 мы договаривались, что хранящиеся в `AD00+n` для отображения числа 00000011b (03h) или 00000111b (07h) должны отображаться в виде упомянутых в скобках тройки или семерки, а вот 00010011b (13h) или 00010111b (17h) — в виде тройки с десятичной точкой после нее или семерки с той же точкой. Да-

вайте сохраним эту договоренность. При этом в позициях с 10h по 19h мы разместим 5-байтные фрагменты, формирующие изображения символов «0.», «1.», ..., «9.». В итоге мы должны получить таблицу знакогенератора. Не пугайтесь, встретив в ней новую для вас директиву ассемблера `.DB` — без нее никак не сформировать эту таблицу, т. к. именно она предписывает ему (ассемблеру) разместить байт за байтом в памяти программ те байты, которые записаны друг за другом через запятую после этой директивы.

Таблица знакогенератора на 32 символа для HCMS-2xxx

```
;
;ЗНАКОГЕНЕРАТОР НА 32 СИМВОЛА
;
TABZNAK:
;
;ЦИФРА 0 [00H]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА 1 [01H]
.DB     10000000B,10000000B,10000010B,10111111B,10000000B
;ЦИФРА 2 [02H]
.DB     10100010B,10110001B,10101001B,10100110B,10000000B
;ЦИФРА 3 [03H]
.DB     10010001B,10100101B,10100111B,10011001B,10000000B
;ЦИФРА 4 [04H]
.DB     10001111B,10001000B,10001000B,10111111B,10000000B
;ЦИФРА 5 [05H]
.DB     10010111B,10100101B,10100101B,10011001B,10000000B
;ЦИФРА 6 [06H]
.DB     10011110B,10100101B,10100101B,10011001B,10000000B
;ЦИФРА 7 [07H]
.DB     10100001B,10010001B,10001001B,10000111B,10000000B
;ЦИФРА 8 [08H]
.DB     10011010B,10100101B,10100101B,10011010B,10000000B
;ЦИФРА 9 [09H]
.DB     10100110B,10101001B,10101001B,10011110B,10000000B
;ЦИФРА A [0AH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА B [0BH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА C [0CH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА D [0DH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА E [0EH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА F [0FH]
.DB     10000000B,10000000B,10000000B,10000000B,10000000B
;
;ЦИФРА 0,[10H]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
```

```
;ЦИФРА 1,[11H]
.DB     10000000B,10000000B,10000010B,10111111B,11100000B
;ЦИФРА 2,[12H]
.DB     10100010B,10110001B,10101001B,10100110B,11100000B
;ЦИФРА 3,[13H]
.DB     10010001B,10100101B,10100111B,10011001B,11100000B
;ЦИФРА 4,[14H]
.DB     10001111B,10001000B,10001000B,10111111B,11100000B
;ЦИФРА 5,[15H]
.DB     10010111B,10100101B,10100101B,10011001B,11100000B
;ЦИФРА 6,[16H]
.DB     10011110B,10100101B,10100101B,10011001B,11100000B
;ЦИФРА 7,[17H]
.DB     10100001B,10010001B,10001001B,10000111B,11100000B
;ЦИФРА 8,[18H]
.DB     10011010B,10100101B,10100101B,10011010B,11100000B
;ЦИФРА 9,[19H]
.DB     10100110B,10101001B,10101001B,10011110B,11100000B
;ЦИФРА A,[1AH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА B,[1BH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА C,[1CH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА D,[1DH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА E,[1EH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА F,[1FH]
.DB     10000000B,10000000B,10000000B,10000000B,10000000B
;
```

Теперь можно сказать, что мы преодолели самую большую трудность на пути написания подпрограммы `VIVHCMS` — сформировали массив данных, где по известному нам принципу размещена вся информация, необходимая для отображения на HCMS-2xxx любой из цифр.

Далее, задачу написания подпрограммы `VIVHCMS` можно условно разбить на две: первая — сформировать массив данных, который надо выводить в сдвиговый регистр индикатора, второй — вывести в индикатор этот массив. Начнем с первой. Но перед тем, как это сделать, нужно еще отвести в ОЗУ микроконтроллера область из 4 байт, куда мы будем заносить информацию для вышеупомянутого формируемого массива данных. Пусть это будут ячейки с символическими именами `EKRAN`, `EKRAN+1`, `EKRAN+2` и `EKRAN+3`. В младших 7 битах ячейки `EKRAN` будут располагаться первые 7 выводимых в сдвиговый регистр бит, в младших 7 битах `EKRAN+1` — вторые 7 бит и т. д. Следовательно, первая из поставленных задач сводится к занесению информации в ячейки `EKRAN`...`EKRAN+3`, а вторая — вывод ее из них в регистр индикатора.

Ниже приведен фрагмент подпрограммы, осуществляющий заполнение ячеек EKRAN...EKRAN+3.

Фрагмент подпрограммы, осуществляющий заполнение ячеек EKRAN...EKRAN+3

```
;
;
  MOV   A,#0
  MOV   COUNSK,A          ;COUNSK М.Б. РАВЕН 0,1,2,3,4,
                          ;ЭТО НОМЕР ВЫВОДИМОГО СТОЛБЦА
  MOV   EKRAN,A
  MOV   EKRAN+1,A
  MOV   EKRAN+2,A
  MOV   EKRAN+3,A         ;ПРЕДВАРИТЕЛЬНО ЗАНУЛИЛИ
;
  MOV   A,R3
  ANL   A,#00000011B      ;R3 НЕ М.Б. БОЛЕЕ 3
  MOV   R3,A
  MOV   A,#EKRAN
  ADD   A,R3
  MOV   R0,A              ;В R0 АДРЕС ТОЙ ИЗ
                          ;EKRAN...EKRAN+3,
                          ;КУДА ВЫВОД
  MOV   DPTR,#TABZNAK
;
  MOV   A,R2
  RL    A
  RL    A
  ADD   A,R2              ;A = 5*R2
  ADD   A,COUNSK          ;A = 5*R2+COUNSK
  MOVC  A,@A+DPTR
  MOV   @R0,A             ;В EKRAN...EKRAN+3
                          ;ВЫВЕЛИ БАЙТ, СООТВ.
                          ;ПЕРВОЙ КОЛОНКЕ
                          ;ОТОБРАЖАЕМОГО СИМВОЛА
;
```

Теперь давайте разберемся с тем, что в нем делается. Вначале мы занулим некую переменную COUNSK (в ней должен храниться номер отображаемого столбца, т. е. она может принимать значения лишь 0, 1, 2, 3, 4) и ячейки EKRAN... EKRAN+3. Как вы должны помнить, только в одну из этих ячеек мы выведем информацию для отображения: в EKRAN, если будем отображать символ в правом разряде индикатора, в EKRAN+1 — если во второй справа и т. д. Остальные же, чтобы светодиоды неотображаемых символов не горели, должны содержать нули. Далее, перенесем из R3 в аккумулятор номер разряда индикатора, в который наша подпрограмма должна вывести информацию, и в целях страховки ограничиваем его чис-

лом 3 (при помощи команды `ANL A,#00000011b`). Затем сложим это число с адресом ячейки, которой мы дали символическое имя `EKRAN` (именно с адресом, а не с содержимым, на что указывает знак # перед именем `EKRAN`). Что при этом получилось? Если в R3 перед сложением был 0, т. е. предполагался вывод в крайний справа разряд индикатора, то после сложения в аккумуляторе будет храниться не изменившийся адрес ячейки с именем `EKRAN`. Если в R3 перед сложением была 1, т. е. предполагался вывод во второй справа разряд индикатора, то после сложения в аккумуляторе будет храниться адрес следующей ячейки (`EKRAN+1`). Двойка в R3 даст в аккумуляторе после сложения `EKRAN+2`, тройка, как нетрудно догадаться — `EKRAN+3`. А ведь именно это нам и было нужно!

Далее мы сохраним этот найденный адрес в R0, а в регистр DPTR занесем адрес начальной ячейки таблицы знакогенератора (`MOV DPTR,#TABZNAK`). После этого в аккумулятор из R2 поместим число, которое нам предстоит отобразить. Идущая затем команда `RL A` осуществит, как видно из **Рис. 4.21**, сдвиг каждого бита нашего двоичного числа на одну позицию влево, что эквивалентно удвоению числа (на **Рис. 4.21** в аккумуляторе до сдвига было число 00000111b = 7d, а после сдвига — 00001110b = 14d).

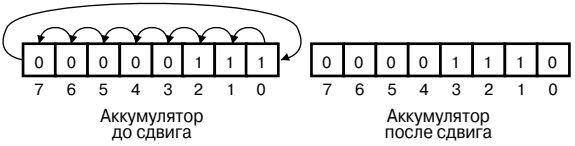


Рис. 4.21. Результат выполнения команды `RL A`

Зачем это нужно? Вот зачем. Положим, отображаемая цифра — двойка. Где в таблице знакогенератора мы должны найти информацию для формирования ее первого, второго, ..., пятого столбцов? Вспомним, в первых пяти байтах таблицы (`TABZNAK+0`… `TABZNAK+4`) хранятся байты для формирования цифры 0, в следующих пяти (`TABZNAK+5`… `TABZNAK+9`) — цифры 1. Информация для первого столбца двойки хранится в ячейке `TABZNAK+10`, для второго столбца — в `TABZNAK+11`, ..., для пятого столбца — в `TABZNAK+14`. В общем виде, при помощи формулы, можно записать, что информация для *n*-го столбца цифры 2 хранится в ячейке памяти программ с адресом `TABZNAK + 2*5 + n`, где для первого столбца n = 0, для второго n = 1, ..., для пятого n = 4. Если мы хотим отобразить цифру 3, то информация для ее *n*-го столбца хранится в ячейке памяти программ с адресом `TABZNAK + 3*5 + n`, для цифры 7 `TABZNAK + 7*5` + n и т. д.

Теперь вам понятно, что для нахождения байта с соответствующей нужному столбцу отображаемой цифры информацией мы должны взять адрес

начала таблицы знакогенератора (#TABZNAK), прибавить к нему умноженную на пять отображаемую цифру, и к сумме прибавить номер n отображаемого столбца (учитывая, что для первого столбца n = 0, для второго n = 1, ..., для пятого n = 4). Теперь вернемся чуть назад, к команде RL A, которая, как мы говорили, увеличила вдвое выводимое на индикатор число. Еще одно выполнение этой команды учетверило это число, а прибавление к нему содержимого регистра R2, где по-прежнему хранится предназначенная для отображения цифра, привело к тому, что в аккумуляторе у нас оказалось число, ровно в пять раз большее, чем отображаемая цифра из R2. Собственно, ведь именно это нам и нужно! Теперь осталось лишь прибавить к аккумулятору командой ADD A,COUNSK содержимое ячейки COUNSK (в ней должен храниться номер отображаемого столбца, причем именно в нужном нам виде — 0, 1, 2, 3, 4), и мы располагаем всем, что надо для нахождения информации о первом столбце отображаемой цифры.

Стоящая далее команда MOVC A,@A+DPTR выполняет следующее действие: она складывает содержимое аккумулятора и DPTR, находит ячейку памяти программ с адресом, равным полученной сумме, извлекает из нее информацию и помещает извлеченное в аккумулятор. Но вспомним, в DPTR у нас хранился адрес начала таблицы знакогенератора (#TABZNAK), а в аккумуляторе — умноженная на пять отображаемая цифра с прибавленным к ней номером n отображаемого столбца! Следовательно, после выполнения команды MOVC A,@A+DPTR мы получим в аккумуляторе байт с информацией о первом столбце отображаемой цифры. И дальше командой MOV @R0,A перешлем его в одну из ячеек EKRAN...EKRAN+3, ту, адрес которой мы сохранили в R0, и которая соответствует содержавшемуся в R3 номеру разряда индикатора, где мы хотели отобразить наш символ.

Поздравим себя с успехом — мы сформировали-таки массив данных, который нужно вывести в сдвиговый регистр индикатора, чтобы отобразить первую колонку этого символа. Осталась малость — вывести информацию в сдвиговый регистр. По сравнению с тем, что мы делали для формирования этого массива, вывод его в индикатор достаточно прост, как видно из следующих программных фрагментов:

Фрагмент подпрограммы, осуществляющий вывод в индикатор информации из ячеек EKRAN...EKRAN+3

```
;
  MOV         A,EKRAN+0
  ACALL       IND7BIT
;
  MOV         A,EKRAN+1
  ACALL       IND7BIT
;
```

```
  MOV         A,EKRAN+2
  ACALL       IND7BIT
;
  MOV         A,EKRAN+3
  ACALL       IND7BIT
;
;
  CLR         COLUMN0     ;ЗАСВЕТКА КОЛОНКИ
  MOV         A,COUNSK
  INC         A
  MOV         COUNSK,A
  LCALL       DEL1MS
  SETB        COLUMN0     ;ГАШЕНИЕ КОЛОНКИ
;
```

Подпрограмма, осуществляющая вывод в сдвиговый регистр 7 бит из выбранной ячейки

```
;
IND7BIT:
  RLC         A
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 7
  SETB        INDCLK
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 6
  SETB        INDCLK
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 5
  SETB        INDCLK
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 4
  SETB        INDCLK
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 3
  SETB        INDCLK
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 2
  SETB        INDCLK
```

```
;
  RLC         A
  MOV         INDLINE,C
  CLR         INDCLK      ;БИТ 1
  SETB        INDCLK
;
  RET
;
```

Непосредственно вывод осуществляется при помощи подпрограммы IND7BIT. Вначале мы помещаем в аккумулятор байт из ячейки EKRAN и вызываем эту подпрограмму, затем — из ячейки EKRAN+1 с повторным ее вызовом, а затем и из ячеек EKRAN+2 и EKRAN+3. После этого командой CLR COLUMN0 открываем транзистор VT1, увеличиваем на 1 счетчик столбцов (теперь COUNSK будет содержать не 0, а 1), вызываем подпрограмму миллисекундной задержки (LCALL DEL1MS) и командой SETB COLUMN0 закрываем VT1. Все, мы отобразили первую колонку выводимой на индикатор цифры, увеличили COUNSK на 1 и теперь должны повторить все описанные действия для второй, третьей, четвертой и пятой колонок.

Подпрограмма вывода информации в HCMS-2xxx в завершенном виде

```
;
AD00    .EQU   30H
COUNSK  .EQU   7BH
INDLINE .EQU   P1.5
INDCLK  .EQU   P1.6
EKRAN   .EQU   7CH
COLUMN1 .EQU   P1.0
COLUMN2 .EQU   P1.1
COLUMN3 .EQU   P1.2
COLUMN4 .EQU   P1.3
COLUMN5 .EQU   P1.4
;
;СОБСТВЕННО ПОДПРОГРАММА ВЫВОДА ИНФОРМАЦИИ
;НА HCMS-2XXX ИЗ ЯЧЕЕК С АДРЕСАМИ ОТ AD00
;ДО AD00+3
;
OTBHCMS:
;
        MOV    R7,#255
;
OTBHC1:
        MOV    R2,AD00+0
        MOV    R3,#0
        LCALL  VIVHCMS
;
```

```
        MOV    R2,AD00+1
        MOV    R3,#1
        LCALL  VIVHCMS
;
        MOV    R2,AD00+2
        MOV    R3,#2
        LCALL  VIVHCMS
;
        MOV    R2,AD00+3
        MOV    R3,#3
        LCALL  VIVHCMS
;
        DJNZ   R7,OTBHC1
        RET
;
;КОД СИМВОЛА В РЕГИСТРЕ R2, НОМЕР
;ЗНАКОМЕСТА ВЫВОДИМОГО СИМВОЛА В
;R3 -- ОН МОЖЕТ БЫТЬ РАВЕН ЛИШЬ
;00, 01,02,03 И НИКАКОМУ ДРУГОМУ
;
VIVHCMS:
        MOV    A,#0
        MOV    COUNSK,A
        MOV    EKRAN,A
        MOV    EKRAN+1,A
        MOV    EKRAN+2,A
        MOV    EKRAN+3,A
;
        MOV    A,R3
        ANL    A,#00000011B
        MOV    R3,A
        MOV    A,#EKRAN
        ADD    A,R3
        MOV    R0,A        ;В R0 АДРЕС ТОЙ
                           ;ИЗ EKRAN...EKRAN+3,
                           ;КУДА ВЫВОД
        MOV    DPTR,#TABZNAK
        MOV    A,R2
        RL     A
        RL     A
        ADD    A,R2        ;A = 5*R2
        ADD    A,COUNSK    ;A = 5*R2+COUNSK
        MOVC   A,@A+DPTR
        MOV    @R0,A       ;В EKRAN...EKRAN+3
                           ;ВЫВЕЛИ БАЙТ, СООТВ.
                           ;ПЕРВОЙ КОЛОНКЕ
;
        MOV    A,EKRAN+0
        ACALL  IND7BIT
        MOV    A,EKRAN+1
        ACALL  IND7BIT
```

```
        MOV    A,EKRAN+2
        ACALL  IND7BIT
        MOV    A,EKRAN+3
        ACALL  IND7BIT
;
        CLR    COLUMN1     ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN1     ;ГАШЕНИЕ КОЛОНКИ
;
;
        MOV    A,R3
        ANL    A,#00000011B
        MOV    R3,A
        MOV    A,#EKRAN
        ADD    A,R3
        MOV    R0,A        ;В R0 АДРЕС ТОЙ
                           ;ИЗ EKRAN...EKRAN+3,
                           ;КУДА ВЫВОД
        MOV    DPTR,#TABZNAK
        MOV    A,R2
        RL     A
        RL     A
        ADD    A,R2        ;A = 5*R2
        ADD    A,COUNSK    ;A = 5*R2+COUNSK
        MOVC   A,@A+DPTR
        MOV    @R0,A       ;В EKRAN...EKRAN+3
                           ;ВЫВЕЛИ БАЙТ, СООТВ.
                           ;ВТОРОЙ КОЛОНКЕ
;
        MOV    A,EKRAN+0
        ACALL  IND7BIT
        MOV    A,EKRAN+1
        ACALL  IND7BIT
        MOV    A,EKRAN+2
        ACALL  IND7BIT
        MOV    A,EKRAN+3
        ACALL  IND7BIT
;
        CLR    COLUMN2     ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN2     ;ГАШЕНИЕ КОЛОНКИ
;
;
        MOV    A,R3
        ANL    A,#00000011B
```

```
        MOV    R3,A
        MOV    A,#EKRAN
        ADD    A,R3
        MOV    R0,A         ;В R0 АДРЕС ТОЙ
                            ;ИЗ EKRAN...EKRAN+3,
                            ;КУДА ВЫВОД
        MOV    DPTR,#TABZNAK
        MOV    A,R2
        RL     A
        RL     A
        ADD    A,R2         ;A = 5*R2
        ADD    A,COUNSK     ;A = 5*R2+COUNSK
        MOVC   A,@A+DPTR
        MOV    @R0,A        ;В EKRAN...EKRAN+3
                            ;ВЫВЕЛИ БАЙТ, СООТВ.
                            ;ТРЕТЬЕЙ КОЛОНКЕ
;
        MOV    A,EKRAN+0
        ACALL  IND7BIT
        MOV    A,EKRAN+1
        ACALL  IND7BIT
        MOV    A,EKRAN+2
        ACALL  IND7BIT
        MOV    A,EKRAN+3
        ACALL  IND7BIT
;
;
        CLR    COLUMN3      ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN3      ;ГАШЕНИЕ КОЛОНКИ
;
;
        MOV    A,R3
        ANL    A,#00000011B
        MOV    R3,A
        MOV    A,#EKRAN
        ADD    A,R3
        MOV    R0,A         ;В R0 АДРЕС ТОЙ
                            ;ИЗ EKRAN...EKRAN+3,
                            ;КУДА ВЫВОД
        MOV    DPTR,#TABZNAK
        MOV    A,R2
        RL     A
        RL     A
        ADD    A,R2         ;A = 5*R2
        ADD    A,COUNSK     ;A = 5*R2+COUNSK
        MOVC   A,@A+DPTR
        MOV    @R0,A        ;В EKRAN...EKRAN+3
```

```
                                ;ВЫВЕЛИ БАЙТ, СООТВ.
                                ;ЧЕТВЕРТОЙ КОЛОНКЕ
;
        MOV     A,EKRAN+0
        ACALL   IND7BIT
        MOV     A,EKRAN+1
        ACALL   IND7BIT
        MOV     A,EKRAN+2
        ACALL   IND7BIT
        MOV     A,EKRAN+3
        ACALL   IND7BIT
;
        CLR     COLUMN4         ;ЗАСВЕТКА КОЛОНКИ
        MOV     A,COUNSK
        INC     A
        MOV     COUNSK,A
        LCALL   DEL1MS
        SETB    COLUMN4         ;ГАШЕНИЕ КОЛОНКИ
;
;
        MOV     A,R3
        ANL     A,#00000011B
        MOV     R3,A
        MOV     A,#EKRAN
        ADD     A,R3
        MOV     R0,A            ;В R0 АДРЕС ТОЙ
                                ;ИЗ EKRAN...EKRAN+3,
                                ;КУДА ВЫВОД
        MOV     DPTR,#TABZNAK
        MOV     A,R2
        RL      A
        RL      A
        ADD     A,R2            ;A = 5*R2
        ADD     A,COUNSK        ;A = 5*R2+COUNSK
        MOVC    A,@A+DPTR
        MO      @R0,A           ;В EKRAN...EKRAN+3
                                ;ВЫВЕЛИ БАЙТ, СООТВ.
                                ;ПЯТОЙ КОЛОНКЕ
;
        MOV     A,EKRAN+0
        ACALL   IND7BIT
        MOV     A,EKRAN+1
        MOV     A,EKRAN+2
        ACALL   IND7BIT
        MOV     A,EKRAN+3
        ACALL   IND7BIT
;
        CLR     COLUMN5         ;ЗАСВЕТКА КОЛОНКИ
        MOV     A,COUNSK
        INC     A
        MOV     COUNSK,A
```

```
        LCALL  DEL1MS
        SETB   COLUMN5      ;ГАШЕНИЕ КОЛОНКИ
;
;
        RET
;
;ПОДПРОГРАММА ВЫВОДА НА ДИСПЛЕЙ
;ОДНОГО СТОЛБЦА
;
IND7BIT:
        RLC    A
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 7
        SETB   INDCLK
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 6
        SETB   INDCLK
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 5
        SETB   INDCLK
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 4
        SETB   INDCLK
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 3
        SETB   INDCLK
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 2
        SETB   INDCLK
        RLC    A
        MOV    INDLINE,C
        CLR    INDCLK         ;БИТ 1
        SETB   INDCLK
        RET
;
;ЗНАКОГЕНЕРАТОР НА 32 СИМВОЛА
;
TABZNAK:
;
;ЦИФРА 0 [00H]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА 1 [01H]
.DB     10000000B,10000000B,10000010B,10111111B,10000000B
;ЦИФРА 2 [02H]
.DB     10100010B,10110001B,10101001B,10100110B,10000000B
```

```
;ЦИФРА 3 [03H]
.DB     10010001B,10100101B,10100111B,10011001B,10000000B
;ЦИФРА 4 [04H]
.DB     10001111B,10001000B,10001000B,10111111B,10000000B
;ЦИФРА 5 [05H]
.DB     10010111B,10100101B,10100101B,10011001B,10000000B
;ЦИФРА 6 [06H]
.DB     10011110B,10100101B,10100101B,10011001B,10000000B
;ЦИФРА 7 [07H]
.DB     10100001B,10010001B,10001001B,10000111B,10000000B
;ЦИФРА 8 [08H]
.DB     10011010B,10100101B,10100101B,10011010B,10000000B
;ЦИФРА 9 [09H]
.DB     10100110B,10101001B,10101001B,10011110B,10000000B
;ЦИФРА A [0AH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА B [0BH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА C [0CH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА D [0DH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА E [0EH]
.DB     10011110B,10100001B,10100001B,10011110B,10000000B
;ЦИФРА F [0FH]
.DB     10000000B,10000000B,10000000B,10000000B,10000000B
;
;ЦИФРА 0,[10H]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА 1,[11H]
.DB     10000000B,10000000B,10000010B,10111111B,11100000B
;ЦИФРА 2,[12H]
.DB     10100010B,10110001B,10101001B,10100110B,11100000B
;ЦИФРА 3,[13H]
.DB     10010001B,10100101B,10100111B,10011001B,11100000B
;ЦИФРА 4,[14H]
.DB     10001111B,10001000B,10001000B,10111111B,11100000B
;ЦИФРА 5,[15H]
.DB     10010111B,10100101B,10100101B,10011001B,11100000B
;ЦИФРА 6,[16H]
.DB     10011110B,10100101B,10100101B,10011001B,11100000B
;ЦИФРА 7,[17H]
.DB     10100001B,10010001B,10001001B,10000111B,11100000B
;ЦИФРА 8,[18H]
.DB     10011010B,10100101B,10100101B,10011010B,11100000B
;ЦИФРА 9,[19H]
.DB     10100110B,10101001B,10101001B,10011110B,11100000B
;ЦИФРА A,[1AH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА B,[1BH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
```

```
;ЦИФРА C,[1CH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА D,[1DH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА E,[1EH]
.DB     10011110B,10100001B,10100001B,10011110B,11100000B
;ЦИФРА F,[1FH]
.DB     10000000B,10000000B,10000000B,10000000B,10000000B
;
;
DEL1MS:         MOV         R7,#07FH
LREX:           MOV         R6,#001H
LRIN:           DJNZ        R6,LRIN
                DJNZ        R7,LREX
                RET
;
```

В итоге, у нас получилась подпрограмма вывода информации в HCMS-2xxx (вместе с необходимыми для ее функционирования дополнительными подпрограммками). Она вполне работоспособна, и на этом можно было бы остановиться. Но...

Посмотрите еще раз внимательно на подпрограмму `VIVHCMS`. Обратите внимание на то, что в ней есть пять больших повторяющихся фрагментов, начинающихся с команды `MOV A,R3`, а заканчивающихся командой `ACALL IND7BIT`, идущей после `MOV A,EKRAN+3`. Очевидно, если мы выделим этот повторяющийся фрагмент в подпрограмму (назовем ее, к примеру, `VIVHCM1`), то главное наше детище, подпрограмма `VIVHCMS` станет гораздо короче и изящнее. Давайте попробуем это сделать.

Оптимизированный вариант подпрограммы VIVHCMS

```
;
VIVHCMS:
        MOV    A,#0
        MOV    COUNSK,A
        MOV    EKRAN,A
        MOV    EKRAN+1,A
        MOV    EKRAN+2,A
        MOV    EKRAN+3,A
;
;
        LCALL  VIVHCM1
;
        CLR    COLUMN1      ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
```

```
        SETB   COLUMN1     ;ГАШЕНИЕ КОЛОНКИ
;
;
        LCALL  VIVHCM1
;
        CLR    COLUMN2     ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN2     ;ГАШЕНИЕ КОЛОНКИ
;
;
        LCALL  VIVHCM1
;
        CLR    COLUMN3     ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN3     ;ГАШЕНИЕ КОЛОНКИ
;
;
        LCALL  VIVHCM1
;
        CLR    COLUMN4     ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN4     ;ГАШЕНИЕ КОЛОНКИ
;
;
        LCALL  VIVHCM1
;
        CLR    COLUMN5     ;ЗАСВЕТКА КОЛОНКИ
        MOV    A,COUNSK
        INC    A
        MOV    COUNSK,A
        LCALL  DEL1MS
        SETB   COLUMN5     ;ГАШЕНИЕ КОЛОНКИ
;
        RET
;
VIVHCM1:
        ANL    A,#00000011B
        MOV    R3,A
        MOV    A,#EKRAN
        ADD    A,R3
        MOV    R0,A        ;В R0 АДРЕС ТОЙ
                           ;ИЗ EKRAN...EKRAN+3,
```

```
                        ;КУДА ВЫВОД
        MOV    DPTR,#TABZNAK
        MOV    A,R2
        RL     A
        RL     A
        ADD    A,R2         ;A = 5*R2
        ADD    A,COUNSK     ;A = 5*R2+COUNSK
        MOVC   A,@A+DPTR
        MOV    @R0,A        ;B EKRAN...EKRAN+3
                            ;ВЫВЕЛИ БАЙТ, СООТВ.
                            ;КОЛОНКЕ
;
        MOV    A,EKRAN+0
        ACALL  IND7BIT
        MOV    A,EKRAN+1
        ACALL  IND7BIT
        MOV    A,EKRAN+2
        ACALL  IND7BIT
        MOV    A,EKRAN+3
        ACALL  IND7BIT
;
         RET
;
```

Как видите, полученный сокращенный вариант подпрограммы VIVHCMS гораздо короче, проще и нагляднее. Надо сказать, что обычно программы даже у опытных программистов никогда сразу не получаются оптимальными — поначалу не до оптимизации, хотя бы заставить их просто работать без ошибок. Но очень часто, к сожалению, после того, как все заработает нужным образом, нам лень «навести блеск» — на это не остается ни сил, ни времени. Однако все же к этому нужно стремиться — тогда, по опыту знаю, впоследствии такую программу гораздо проще дорабатывать. А заниматься доработками приходится гораздо чаще, чем это можно было бы предположить. Так что не стремитесь наступать на грабли обязательно самостоятельно, учитесь на чужих ошибках...

4.5. Краткие выводы

Мы познакомились с тем, как сопрягать микроконтроллер с различными типами знакосинтезирующих индикаторов. Как уже упоминалось выше, последние могут быть жидкокристаллическими или светодиодными (остальные, например люминесцентные или электроннолучевые, практически вышли из употребления). По способу формирования символов индикаторы могут быть семисегментными или матричными. Далее, как мы

имели возможность убедиться, многие из них, в особенности жидкокристаллические, снабжены самостоятельными микроконтроллерами, заметно упрощающими работу с ними. Именно такими были оба рассмотренных в настоящей главе жидкокристаллических индикатора. В обоих случаях нам пришлось лишь организовать передачу информации от основного микроконтроллера в контроллер индикатора, а все, что связано с отображением переданных символов, осуществлялось уже без нашего вмешательства. Светодиодные индикаторы, которые мы рассмотрели, не имели встроенных контроллеров. При этом для работы с АЛС318 нам пришлось использовать в цепях сопряжения несколько дополнительных микросхем, а для HCMS-2xxx потребовались 5 транзисторов. Программы, написанные для работы со светодиодными индикаторами, оказались заметно сложнее, чем аналогичные для ЖК-индикаторов.

Теперь уже не только мне, но и вам должно быть очевидно, что каких-либо стандартных правил сопряжения микроконтроллеров с индикаторами не существует, и в каждом конкретном случае оно выполняется по-своему. Поэтому мы и рассмотрели столь отличающиеся друг от друга варианты индикаторов, каждый со своей схемой включения, алгоритмом и программой сопряжения. После знакомства с предложенным материалом, я надеюсь, вы не только сможете любой из этих индикаторов использовать в дальнейшем в своих устройствах, но и, опираясь на эти наработки, самостоятельно сопряжете ваш микроконтроллер с любым другим устройством индикации.

Кроме того, мы попытались попрактиковаться в разбиении поставленной задачи на ряд более простых, и в написании подпрограмм для этих более простых задач. Пока у вас еще просто очень мало подобных навыков, и описанные в этой главе примеры, равно как и те, которые вы встретите в последующих главах, направлены в первую очередь на то, чтобы эти навыки у вас сформировать. Я попытался на практике показать вам, как разбивать задачу на части, как решать каждую из этих малых задач, причем даже не сразу, а, как говорится, в два-три приема. Я специально показал вам, что получается при первой попытке решения задачи, и как оно дальше оптимизируется. Те, кто имеет большой опыт, проводит эти предварительные стадии разработки в уме. Но у нас с вами такого опыта пока еще нет, поэтому мы решали нашу задачу постепенно, получив в конце концов как некоторые новые навыки, так и правильно работающую подпрограмму.

Еще раз обращаю ваше внимание на то, что любые действия, которые вам придется делать в вашей программе более, чем один или два раза, полезно оформить в виде подпрограммы с понятным для вас именем. Именно так мы и поступали в рассмотренных примерах. При организации вывода информации на экран HT1610 мы создали подпрограмму `SIMBOL1`, после чего заносили в аккумулятор числа из `AD00+7`, `AD00+6` и т. д., вы-

зывая эту подпрограмму `SIMBOL1` после каждого занесения. В случае с АЛС318 мы создали подпрограмму `DISPLEY`, при вызове которой в R0 мы помещали число от 0 до 7, и она далее извлекала цифру из ячейки памяти микроконтроллера с адресом `AD00+n` (n = 0...7), и выводила его в порт P1 для отображения в *n*-й разряд АЛС318. Мы написали подпрограмму `VIVHCMS` для вывода на HCMS-2xxx символа, помещенного в регистр R2 в разряд индикатора, номер которого занесен в регистр R3. С ее помощью выводить символы в HCMS-2xxx стало легко и просто.

Как видите, по крайней мере в тех случаях, когда вам нужно вывести на тот или иной дисплей цифровую информацию (а это всегда не менее 3...4 цифр), очень полезно выделить в самостоятельную подпрограмму все действия по пересылке символа на дисплей, при этом сам пересылаемый символ (а если надо, то и его место отображения на индикаторе) должен находиться в каком-нибудь регистре (регистрах) вашего микроконтроллера. Если вы привыкнете это делать, то написанные вами программы будут проще, стройнее и безошибочнее, чем если вы будете писать их без подобной привычки.

Глава 5. СИСТЕМА КОМАНД МИКРОКОНТРОЛЛЕРОВ X51

Итак, позади уже 4 главы, а мы лишь собираемся начать знакомство с системой команд микроконтроллеров. Увы, если бы я начал систематически излагать ее раньше, я бы здорово перегрузил вас незнакомой информацией, и не уверен, что все продрались бы через этот частокол новых понятий. Но по ходу изложения предыдущего материала я знакомил вас с командами, входящими в программы, оживляющими то или иное железо. Так что нельзя сказать, что вы понятия не имеете о командах микроконтроллера, и что материал этой главы будет вам в новинку. На самом деле вам уже в той или иной степени знакомы более 50% команд микроконтроллеров семейства x51. И в настоящей главе я собираюсь лишь сгруппировать все команды в сводные таблицы, ознакомить вас с общепринятыми правилами краткой записи в этих таблицах содержания команд, коротко рассказать о тех командах, которые вам еще не знакомы, и дать о них некоторую дополнительную информацию (о количестве байт в той или иной команде и о времени их выполнения). Я не планирую давать исчерпывающую информацию по командам, как это сделано в книге, написанной А. В. Боборыкиным, Г. П. Липовецким и еще семерыми авторами (Однокристальные микроЭВМ. — М.: МИКАП, 1994.— 400 с.: ил.— ISBN 5-85959-030-X), и рекомендую при необходимости обращаться к ней.

Что еще можно сказать о командах микроконтроллера, прежде чем мы начнем более детальное знакомство с ними? Эти команды составляют, как вы уже знаете, язык ассемблера микроконтроллера. Ассемблер же относится к так называемым машинно-зависимым языкам — он свой для каждого семейства микроконтроллеров и отличен от ассемблера любого другого семейства. В этом одна из основных его слабостей — переходя на новый микроконтроллер, вы вынуждены учить новый язык.

Второй недостаток ассемблера — образно говоря, малая плотность действий микроконтроллера на страницу текста, написанного на этом языке. Например, в языке Паскаль вывод символа или строки символов

на экран компьютера организовывается при помощи одной команды — writeln. Если вы попробуете написать то же самое на ассемблере стоящего в компьютере процессора семейства x86, программа потребует примерно такого же количества строк, как и наши программы для работы с ЖК-индикаторами, рассмотренные в предыдущей главе. А когда дело коснется арифметических вычислений, длина программы на языке высокого уровня, к которым относятся и Паскаль, и Си, становится просто несопоставимо короче, чем та же программа на языке низкого уровня (т. е. ассемблере). Так что писать на языке высокого уровня оказывается гораздо выгоднее — программы оказываются короче, понятнее, да и переносить их с одного процессора или контроллера на другой оказывается очень легко.

Так почему же мы с вами знакомимся с ассемблером, а не с тем или иным языком высокого уровня, версии которых существуют практически для любого контроллера, а уж тем более для такого распространенного, как x51? Вместо ответа приведу такой пример. Когда я приобрел на митинском рынке аналогичный 10-разрядному HT1610 16-разрядный индикатор HT1616, у меня возникли трудности в аппаратном сопряжении его с микроконтроллером. Для отладки продавец индикатора дал мне коды написанной им на Си программы, осуществлявшей вывод на его экран трех идущих с интервалом в пару секунд текстовых строк (эта программа нормально работала на демонстрационном образце). Так вот, объем кода составлял примерно 1.6 Кбайта. Когда я разобрался с сопряжением, я написал аналог этой программы на ассемблере. Она заняла 82 байта памяти программ плюс число байт, равное количеству выводимых в строках символов (что-то около 40, точно не помню). Другими словами, программа из 15 строк на Си после трансляции потребовала 1600 ячеек памяти программ, а ее ассемблерный аналог — на порядок меньше, байт 120. Впечатляет? Добавлю к этому, что и работают программы, написанные на ассемблере, в несколько раз быстрее аналогов, писанных на языках высокого уровня. Так что по части компактности кода и быстродействия с программой на ассемблере не может сравниться никакой Паскаль или Си.

Думаю, что теперь вам понятно, почему ассемблер, несмотря на все его неудобства, так и не вытеснен языками высокого уровня. И именно по причинам, упомянутым в предыдущем абзаце, программисты еще долго будут им пользоваться. Не будучи большими оригиналами, мы не составим исключения из этого правила, и поэтому приступим к обещанному знакомству с системой команд ассемблера микроконтроллеров x51.

5.1. Общие сведения о системе команд

Система команд x51 содержит 111 базовых команд, которые удобно разделить по функциональному признаку на пять групп: команды передачи данных, арифметических операций, логических операций, передачи управления и операций с битами.

Большинство команд (94) имеет формат один или два байта и выполняются за один или два машинных цикла. При тактовой частоте 12 МГц длительность машинного цикла, как вы должны помнить, составляет 1 мкс.

При анализе команд в программировании принято разделять их на две части — код операции (КОП) и операнды. Код операции — это байт, соответствующий команде, а операнд — это тот объект (бит, байт и т. д.), над которым должно совершиться определенное КОП'ом действие. Например, рассмотрим хорошо известную вам команду занесения константы в регистр R1: `MOV R1,#7`. В памяти программ после трансляции этой команде будут соответствовать два байта — 79h и 07h. Первый байт — 79h — это КОП, соответствующий команде занесения константы в регистр R1, а второй байт — это операнд, т. е. то число, которое нужно в этот регистр занести. Как нетрудно догадаться, КОП для другой команды будет отличным от 79h. Например, команде занесения константы в регистр R2 соответствует КОП 7Ah, команде сложения аккумулятора с регистром R7— соответственно 6Fh и т. д.

С кодом операции, надеюсь, все ясно. Ниже, по мере знакомства с командами, для всех их я приведу их КОП'ы, и у вас будет возможность увидеть, какой команде соответствует какой код операции. Здесь никакой сложности нет. А вот с операндами будет чуть сложнее, чтобы понять, что это такое, вам придется чуть-чуть поднапрячься.

С какими объектами работают команды микроконтроллера? Вы уже видели, что с битами, 8-битными и 16-битными числами, а также с 8-битными или 16-битными адресами ячеек памяти, где эти числа хранятся. Давайте вспомним те команды, с которыми мы ранее уже знакомились. Мы неоднократно заносили 8-битные константы в аккумулятор или в регистры, используя команды типа упомянутой `MOV R1,#7`. В главе 3 в регистр DPTR командой `MOV DPTR,#0FF25h` мы занесли 16-битную константу (в данном примере — 0FF25h, что равно 65317d). Мы переносили в аккумулятор числа из ячеек памяти данных (например, командами типа `MOV A,AD00`), а также осуществляли пересылку данных в обратном направлении (в данном случае командой `MOV AD00,A`). Мы пересылали биты из `CY` в регистры-защелки портов, в биты некоторых регистров, и наоборот (например, командами типа `MOV P3.1,C` или `MOV C,ACC.5`). И,

наконец, мы вызывали подпрограммы командой типа `LCALL DEL4MS` (в последнем случае имя DEL4MS после трансляции превратится в двухбайтный, т. е. 16-битный адрес первой команды этой подпрограммы).

Теперь посмотрим, что является операндом в каждом из рассмотренных выше случаев. В первом случае (`MOV R1,#7`) операндом является помещаемая в регистр 8-битная константа — в данном случае 7, но вместо семерки, как вы понимаете, после знака # могло стоять любое число от 0 до 255. Во втором случае (`MOV DPTR,#0FF25h`) операндом являлась 16-битная или двухбайтная константа. В третьем случае (`MOV A,AD00`) операнд — это содержимое ячейки, адрес которой определен символическим именем `AD00`. Почему именно содержимое ячейки, а не записанный через запятую после аккумулятора ее адрес — ведь он также 8-битный (от 0 до 255; в нашем конкретном случае директивой `AD00 .EQU 30h` мы присваивали ему численное значение 30h)? Думаю, долго объяснять это «почему?» не надо — команда пересылает в аккумулятор именно содержимое ячейки, а не ее адрес. Так что — подчеркну еще раз — не адрес в данном случае является операндом, а содержимое ячейки памяти с этим адресом. Как нетрудно догадаться, тот же самый операнд и у команды `MOV AD00,A`. В командах `MOV P3.1,C` или `MOV C,ACC.5` операндом является переносимый бит, хотя адреса этих операндов — битов P3.1 и ACC.5 — также 8-битные числа. Как будет показано ниже, адрес бита P3.1, например, — 0B1h, а 5-го бита аккумулятора ACC.5 — 0E5h. А вот в команде `LCALL DEL4MS` операндом является 16-битный адрес первой команды вызываемой подпрограммы, ибо команда `LCALL` работает не с данными, а с адресами.

Да, забыл, мы ведь уже знакомились с командой `SWAP A`. Она обменивает местами старшую и младшую тетрады аккумулятора (тетрада — это 4-битное число). Эта (и еще одна) команда работают, таким образом, с 4-битными операндами.

Итак, я перечислил все возможные для x51 операнды — это единичные биты, 4-битные, 8-битные и 16-битные числа, причем последние в некоторых случаях могут быть как данными, так и адресами. Отмечу еще, что есть команды, оперирующие с двумя операндами, например, `ANL addr,#data`. Эта команда осуществляет логическое И содержимого байта, размещенного в ячейке с адресом `addr`, и числа, записанного после знака #, т. е. оперирует с двумя байтами. Операнды здесь, как вы догадались — содержимое упомянутой ячейки и число `#data`.

После столь пространного объяснения, я полагаю, вы теперь будете легко ориентироваться в терминологии КОП'ов, операндов и их адресов. Для завершения терминологического ликбеза в области системы команд вспомним еще кое-что о методах адресации, без пространного объяснения

которых вы не найдете ни одного описания языка ассемблера, неважно, для x51, AVR или PIC-контроллеров.

Микроконтроллеры семейства x51 допускают 4 способа адресации к операндам. Иными словами, сказать микроконтроллеру, где искать тот или иной операнд можно четырьмя различными способами. Они носят названия прямая адресация, непосредственная адресация, косвенная адресация и неявная адресация.

Прямая адресация предполагает, что адрес операнда прямо указан в самой команде. Например, `MOV A,AD00` — адрес операнда (`AD00`) записан прямо в теле команды. Если быть точным — то главное, что адрес явно присутствует не столько в приведенной парой строк выше мнемонике команды, сколько в виде второго байта в двух байтах, полученных после ее трансляции.

Еще пример — `SETB Px.y`. Здесь адрес операнда (`Px.y`) также прямо записан в теле команды (адрес, соответствующий `Px.y`, будет вторым байтом команды после трансляции). Дальше вы убедитесь, что прямая адресация — самый распространенный способ адресации.

Непосредственная адресация предполагает, что в теле команды записан сам операнд — байт или двухбайтное число. Примеры — `MOV R1,#7` или `MOV DPTR,#0FF25h`. Обратите внимание — в ранее рассмотренном случае (при прямой адресации) в теле команды в явном виде записан адрес операнда, а во втором (при непосредственной адресации) в теле команды в явном виде записан сам операнд. Чувствуете разницу?

Третий вид адресации — косвенная — несколько непривычен для начинающих. Но мы уже сталкивались с ним. Вспомним подпрограмму `DISPLEY`, приведенную в предыдущей главе на стр. 114:

Фрагмент подпрограммы DISPLEY

```
DISPLEY:
        MOV    A,#AD00
        ADD    A,R0
        MOV    R1,A        ;R1=AD00+R0
        MOV    A,@R1
        и т. д.
```

Обратите внимание на команду `MOV A,@R1`. Она помещает в аккумулятор число из ячейки внутреннего ОЗУ, адрес которой находится в регистре R1. Зачем такие сложности? Бывают случаи, когда мы заранее не знаем, из какой ячейки нужно будет взять операнд для той или иной операции. Как, например, в подпрограмме `DISPLEY`: нам для отображения нужно взять данные из ячейки `AD00+n`, где n = 0...7. Точнее, в одном слу-

чае данные нужно брать из ячейки с адресом 31h, в другом — 32h, в третьем — 33h и т. д. Конкретное же значение этого адреса зависит от числа, находящегося в момент вызова подпрограммы в регистре R0, и является суммой этого числа и числа `#AD00`. Другими словами, значение адреса заранее неизвестно и является результатом вычисления. Именно для подобных случаев и придумана косвенная адресация. Адрес операнда вычисляется по заданному вами алгоритму и помещается в определенный регистр, в данном случае в R1. А далее вы используете команду, которая предписывает микроконтроллеру найти, какой адрес хранится в этом регистре, после чего перенести в аккумулятор данные из той ячейки внутреннего ОЗУ, адрес которой был найден в нем (т. е. R1). Как уже отмечалось в предыдущей главе, подобный метод адресации, когда адрес ячейки памяти, содержащей операнд, находится в каком-либо регистре, носит название косвенной адресации (адрес мы находим косвенно, при помощи R0, R1 или DPTR), в отличие от рассмотренной выше прямой адресации, где адрес прямо, т. е. в явном виде, указан в самой команде.

Признаком косвенной адресации является знак @ перед соответствующим регистром (R0, R1 или DPTR). В системе команд x51 адреса для режима косвенной адресации могут быть взяты только из этих трех регистров. Отмечу, что именно использованием в командах с косвенной адресацией регистры R0 и R1 отличаются от остальных регистров общего назначения (R2...R7), в остальном возможности всех этих регистров одинаковы.

С косвенной адресацией, надеюсь, все ясно. Приступим теперь к знакомству с неявной адресацией. Вспомним для этого команду `SWAP A`. Она, как вы уже знаете, обменивает местами старшую и младшую тетрады аккумулятора. И хотя в мнемонике команды есть упоминание об аккумуляторе (отдельно стоящая после слова `SWAP` буква `A`), в коде, полученном после трансляции, этой команде соответствует лишь код операции (0C4h) без каких-либо байтов, прямо или косвенно указывающих на аккумулятор. Другими словами, в коде, соответствующем команде `SWAP A`, нет явных ссылок на аккумулятор. Есть только их (кода и аккумулятора) неявная связь, вытекающая из описания этой команды. Отсюда и название — неявная адресация. Отметим, что этот тип адресации, как вы дальше увидите, характерен для большинства команд, работающих с аккумулятором и с регистрами общего назначения.

Ну вот и все о способах (или методах) адресации микроконтроллеров x51. Отмечу, что у микроконтроллеров некоторых других семейств встречается гораздо большее их количество. Но, разобравшись с вышеописанными основными способами, вы вполне в состоянии постичь и все остальные. По крайней мере, если они внятно описаны в книжке, по которой вы будете с ними знакомиться...

5.2. Группа команд передачи данных

Команды, входящие в эту группу, сведены в **Табл. 5.1**

Таблица 5.1. **Команды передачи данных**

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Пересылка в аккумулятор из регистра (n = 0...7)	`MOV A,Rn`	11101rrr	1	1	(A) ← (Rn)
Пересылка в аккумулятор прямоадресуемого байта	`MOV A,ad`	11100101	2	1	(A) ← (ad)
Пересылка в аккумулятор байта из внутреннего ОЗУ (i = 0, 1)	`MOV A,@Ri`	1110011i	1	1	(A) ← ((Ri))
Загрузка в аккумулятор константы	`MOV A,#data8`	01110100	2	1	(A) ← #data8
Пересылка в регистр из аккумулятора	`MOV Rn,A`	11111rrr	1	1	(Rn) ← (A)
Пересылка в регистр прямоадресуемого байта	`MOV Rn,ad`	10101rrr	2	2	(Rn) ← (ad)
Загрузка в регистр константы	`MOV Rn,#data8`	01111rrr	2	1	(Rn) ← #data8
Пересылка по прямому адресу аккумулятора	`MOV ad,A`	11110101	2	1	(ad) ← (A)
Пересылка по прямому адресу регистра	`MOV ad,Rn`	10001rrr	2	2	(ad) ← (Rn)
Пересылка прямоадресуемого байта по прямому адресу	`MOV add,ads`	10000101	3	2	(add) ← (ads)
Пересылка байта из внутреннего ОЗУ по прямому адресу	`MOV ad,@Ri`	1000011i	2	2	(ad) ← ((Ri))
Пересылка по прямому адресу константы	`MOV ad,#data8`	01110101	3	2	(ad) ← #data8
Пересылка во внутреннее ОЗУ из аккумулятора	`MOV @Ri,A`	1111011i	1	1	((Ri)) ← (A)
Пересылка во внутреннее ОЗУ прямоадресуемого байта	`MOV @Ri,ad`	0110011i	2	2	((Ri)) ← (ad)
Пересылка во внутреннее ОЗУ константы	`MOV @Ri,#data8`	0111011i	2	1	((Ri)) ← #data8

Продолжение

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Загрузка указателя данных	`MOV DPTR, #data16`	10010000	3	2	DPTR ← #data16
Пересылка в аккумулятор байта из памяти программ	`MOVC A,@A+DPTR`	10010011	1	2	(A) ← ((A) + (DPTR))
Пересылка в аккумулятор байта из памяти программ	`MOVC A,@A+PC`	10000011	1	2	(PC) ← (PC) + 1; (A) ← ((A) + (PC))
Пересылка в аккумулятор байта из внешнего ОЗУ	`MOVX A,@Ri`	1110001i	1	2	(A) ← ((Ri))
Пересылка в аккумулятор байта из расширенного внешнего ОЗУ	`MOVX A,@DPTR`	11100000	1	2	(A) ← ((DPTR))
Пересылка во внешнее ОЗУ из аккумулятора	`MOVX @Ri,A`	1111001i	1	2	((Ri)) ← (A)
Пересылка в расширенное внешнее ОЗУ из аккумулятора	`MOVX @DPTR,A`	11110000	1	2	((DPTR)) ← (A)
Загрузка в стек	`PUSH ad`	11000000	2	2	(SP) ← (SP) + 1; ((SP)) ← (ad)
Извлечение из стека	`POP ad`	11010000	2	2	(ad) ← ((SP)); (SP) ← (SP) – 1
Обмен аккумулятора с регистром	`XCH A,Rn`	11001rrr	1	1	(A) ↔ (Rn)
Обмен аккумулятора с прямоадресуемым байтом	`XCH A,ad`	11000101	2	1	(A) ↔ (ad)
Обмен аккумулятора с байтом из внутреннего ОЗУ	`XCH A,@Ri`	1100011i	1	1	(A) ↔ ((Ri))
Обмен младшей тетрады аккумулятора с младшей тетрадой байта внутреннего ОЗУ	`XCHD A,@Ri`	1101011i	1	1	(A0...3) ↔ ((Ri0...3))

Рассмотрим их поподробнее. Первой идет уже не раз встречавшаяся вам команда пересылки данных в аккумулятор из регистра общего назначения. Обращаю ваше внимание на то, что за этой одной строкой в таблице стоят на самом деле целых 8 команд: `MOV A,R0;` `MOV A,R1;` `MOV A,R2;` `MOV A,R3;` `MOV A,R4;` `MOV A,R5;` `MOV A,R6;` `MOV A, R7`. Дабы не перегружать таблицу перечислением мнемоник всех 8 команд этой подгруппы (давайте упомянутые 8 команд назовем подгруппой, кто нам мешает?), мы записали в ней одну обобщенную мнемонику

`MOV A,Rn`, уточнив при этом, что n может принимать значения от 0 до 7. Сообщу также, что и при дальнейшем рассмотрении команд мы будем пользоваться подобным способом сведения нескольких (обычно 8 или 2) идентичных команд в единую строку в таблице. Так что, когда в пятой строке рассматриваемой таблицы вы увидите мнемонику `MOV Rn,A`, для вас уже должно быть очевидно, что за ней стоят команды: `MOV R0,A; MOV R1,A; MOV R2,A; MOV R3,A` и т. д., вплоть до `MOV R7,A`.

После сказанного, наверное, будет уже излишним уточнять, что когда в остальных группах команд вы встретите мнемоники типа `ADD A,Rn` или `ANL A,Rn`, вы догадаетесь, какие восемь команд описаны каждой из них.

Думаю, что с названием первой рассмотренной команды и с ее обобщенной мнемоникой все ясно. Приступим к анализу ее кода операции. В таблице в рассматриваемой строке стоит следующий КОП: 11101rrr. Как понимать эти три идущие подряд латинские буквы r? Давайте еще раз вспомним, что в двоичном представлении цифре 0 соответствует число 000b, цифре 1 — 001b, цифре 2 — 010b, ..., цифре 7 — 111b. Ну что, догадались? Правильно, представленный в двоичном коде КОП команды `MOV A,R0` — 11101000 (rrr = 000), КОП команды `MOV A,R1` — 11101001 (rrr = 001), КОП команды `MOV A,R2` — 11101010 (rrr = 010), ..., КОП команды `MOV A,R7` — 11101111 (rrr = 111). Так что запись 11101rrr также является обобщенной, вместо трех букв r мы должны вставить в эту мнемонику трехбитный двоичный номер участвующего в этой операции регистра общего назначения.

Кстати, а почему rrr, а не, к примеру, jjj? Думаю, и это очевидно — и в русском, и в английском слово регистр начинается с р (r), так что ее использование выглядит гораздо более естественным, чем любой другой буквы.

И последнее в первой строчке — символическое описание выполняемой операции. Заключенная в скобки буква А означает *содержимое* аккумулятора, а Rn в скобках — *содержимое* соответствующего регистра общего назначения. Стрелка показывает направление пересылки. (A) ← (Rn) — это значит переслать в аккумулятор содержимое *n*-го регистра. Сравните это с названием команды, фигурирующим в начале рассматриваемой строки. Сходится?

Кажется, с первой командой все. А впереди еще 110! Ладно, справимся, не боги горшки обжигают. Обожжем что-нибудь и мы...

Вторая команда называется «Пересылка в аккумулятор прямоадресуемого байта». То есть в аккумулятор нужно переслать байт, адрес которого указан в мнемонике `MOV A,ad`. Идущие в ней после запятой символы `ad` и означают адрес той самой ячейки-источника, записанный или в явном виде (номер от 0 до 7Fh), или в виде символического имени (`AD00` или что-то в этом роде).

С кодом операции, как мне кажется, все очевидно — 11100101b. Обратите внимание, что в отличие от предыдущей команды, рассматриваемая является двухбайтной. При ее трансляции первым байтом будет упомянутый код операции, а вторым — как вы думаете, какой? Правильно, адрес этого самого прямоадресуемого байта.

Ну, и в завершение рассмотрения этой команды — краткое описание совершаемого ею действия: (A) ← (ad) , что значит переслать в аккумулятор содержимое ячейки с адресом ad. На всякий случай еще раз повторю, что заключенная в скобки буква A означает содержимое аккумулятора, а ad в скобках — содержимое ячейки памяти с этим адресом; стрелка показывает направление пересылки.

Третья команда — «Пересылка в аккумулятор байта из внутреннего ОЗУ». Другими словами, в аккумулятор нужно переслать из ОЗУ байт, адрес которого хранится в регистре R0 или R1. Обратите внимание, пересылаем байт все из того же внутреннего ОЗУ, из которого пересылались данные и предыдущей командой. Но, заметьте, хотя и источник пересылки, и приемник в обеих командах одинаковый, способы адресации в них различны. В предыдущей команде адрес прямо указывался в теле команды, а в текущей — извлекается из соответствующего регистра, т. е. указывается не прямо, а косвенно. Я намеренно уже в третий раз подробно объясняю, что такое способ косвенной адресации, т. к. это одно из самых сложных, на мой взгляд, понятий в системе команд.

Мнемоника команды — `MOV A,@Ri`, где i может принимать значение 0 или 1, а ее КОП — 1110011i; думаю, нет смысла объяснять, что команде `MOV A,@R0` соответствует КОП 11100110b, а команде `MOV A,@R1` — КОП 11100111b. Команда однобайтная, выполняется, как и предыдущие две, за 1 машинный цикл. Краткое описание совершаемого ей действия: (A) ← ((Ri)). Обратите внимание, Ri заключено в двойные скобки. Это не опечатка. Двойные скобки означают, что речь идет о содержимом ячейки, адрес которой ***и сам в свою очередь является содержимым*** того или иного регистра. Неясно? Попробую чуть поподробнее.

В самом деле, как мы договорились ранее, (Ri) — это ***содержимое*** регистра Ri; но ведь это содержимое — не что иное, как адрес ячейки, принимающей участие в пересылке! Далее вспомним, что содержимое той или иной ячейки обозначается скобками, в которые заключен ее адрес. В нашем случае адрес — это (Ri), т. е. запись, уже имеющая один комплект скобок. Заключив ее во вторые скобки, как раз и получим пересылаемый в аккумулятор операнд — содержимое ячейки, адрес которой находится в регистре Ri (т. е. является содержимым этого регистра). Думаю, что теперь уже причина появления в описании команды двойных скобок стала понятной всем.

И последняя из команд пересылки данных, в которых приемником байта является аккумулятор — это приведенная в 4-й строке команда загрузки в аккумулятор константы (`MOV A,#data8`). В тело команды включена константа, которая и переносится в аккумулятор. В мнемонике команды эта константа размещается после символа, называемого программистами «решетка» (#), а в кодах программы, полученных после трансляции, вы обнаружите ее после кода операции (01110100b). Думаю, ясно, что `#data8` — это сокращенное обозначение 8-битного числа (0...255), которое в рассматриваемой команде и является этой самой константой. Обратите внимание, насколько эта команда схожа с командой пересылки в аккумулятор прямоадресуемого байта — обе они двухбайтные, обе осуществляют пересылку данных в аккумулятор, обе в теле содержат 8-битную константу. Но в команде `MOV A,ad` эта константа трактуется как адрес ячейки-источника, а в `MOV A,#data8` — именно как помещаемая в аккумулятор константа. Характер трактовки определяется, повторюсь еще раз, наличием или отсутствием знака «решетки» — если ее нет, то константа — это 8-битный адрес, если есть — байт данных.

Также, думаю, очевидно и краткое описание совершаемого ею действия: (A) ← #data8. Обратите внимание, #data8 не заключено ни в какие скобки — это сами данные, а не адрес, где они находятся.

Мы очень подробно рассмотрели все команды пересылки данных, в которых приемником байта является аккумулятор. Их, как видите, всего 4, и в **Табл. 5.1** они расположены в первых 4-х строках. Следующие три строки — это команды пересылки данных, в которых приемником байта является регистр общего назначения (Rn, n = 1...7). Думаю, всем уже очевидно, что каждая из этих строк вобрала в себя по 8 родственных команд, различающихся лишь номером регистра. Также, думаю, нет необходимости объяснять, что означают три латинские r в обобщенной мнемонике каждой из этих обобщенных команд — все совершенно идентично тому, что говорилось при анализе команды пересылки в аккумулятор из регистра.

Как видите, в регистр общего назначения можно переслать данные или из аккумулятора (`MOV Rn,A`), или из прямоадресуемого байта (`MOV Rn,ad`), или непосредственно загрузить в него константу (`MOV Rn,#data8`). Первая команда — однобайтная, две другие — двухбайтные, причем вторая, в отличие от первой и третьей (а также и четырех, рассмотренных ранее), выполняется не за 1, а за 2 машинных цикла. Собственно, говорить об этих командах больше нечего — все, необходимое для их понимания, рассказано при анализе команд пересылки данных в аккумулятор. Так что двинемся дальше.

Следующие пять строк — это команды пересылки данных, в которых приемником байта является прямоадресуемая ячейка внутреннего ОЗУ с адресом ad (ad — это адрес той самой ячейки-приемника, записанный или

в явном виде — номер от 0 до 7Fh — или в виде символического имени — что-то типа AD00). Первые две из этих команд должны уже быть вам очевидны — пересылка в ячейку ОЗУ байта из аккумулятора (MOV ad,A) или из одного из РОН (MOV ad,Rn, $n = 1...7$). Не столь очевидна третья команда, мудрено именуемая «Пересылка прямоадресуемого байта по прямому адресу». Правда, она не сложнее ранее рассмотренных. При ее выполнении микроконтроллер извлекает байт данных из прямоадресуемой ячейки памяти (т. е. из ячейки с указанным в теле команды адресом ads) и пересылает его в другую прямоадресуемую ячейку. Адрес второй — add — также прямо указан в теле команды, причем, что важно, указан в ней первым (вспомните, в одной из предыдущих глав я уже говорил, что в ассемблере x51 первым всегда записывается адрес приемника информации, а вторым — источника). Команда MOV add,ads — трехбайтная, выполняется за 2 машинных цикла. Пожалуй, больше говорить о ней нечего. Также, как и о двух оставшихся командах пересылки данных, в которых приемником байта является прямоадресуемая ячейка внутреннего ОЗУ — MOV ad,@Ri и MOV ad,#data8. Первая предписывает микроконтроллеру переслать в ячейку с адресом ad байт из ячейки, адрес которой хранится в регистре R0 или R1, а вторая — загрузить в ячейку с адресом ad восьмибитную константу.

Следующие три строки — это команды пересылки данных, в которых приемником байта является косвенно адресуемая ячейка внутреннего ОЗУ (т. е. адрес ячейки хранится или в R0, или в R1). Как видите, в косвенно адресуемую ячейку внутреннего ОЗУ можно переслать данные или из аккумулятора (MOV @Ri,A), или из прямоадресуемого байта (MOV @Ri,ad), или непосредственно загрузить в нее константу (MOV @Ri,#data8). Все, что касается этих команд, вы вполне в состоянии самостоятельно понять из информации, приведенной в **Табл. 5.1**. Кстати, также как и то, что касается команды MOV DPTR,#data16 — мы подробно рассмотрели ее в главе, посвященной регистрам микроконтроллера.

Поздравим себя — мы осилили 16 команд. На первый взгляд, не так уж это и много, но лиха беда начало! Чем дальше мы будем продвигаться в их изучении, тем скорее это продвижение будет происходить, ибо все больше и больше материала будет вам уже знакомо. Надеюсь, я вас обнадежил, но даже если и нет, все равно двинемся дальше.

Следующие две команды позволяют загрузить в аккумулятор байт данных из ***памяти программ***. Первая — «Пересылка с использованием указателя данных» (MOVC A, @A+DPTR) нам уже знакома: помните, в предыдущей главе мы с ее помощью отыскивали байты, выводимые в 28-битный регистр сдвига матричного светодиодного индикатора HCMS-2xxx? Она выполняет следующее действие: складывает содержимое аккумулятора и DPTR, находит ячейку памяти программ с адресом, равным полученной сумме, извлекает из нее информацию и помещает извлеченное в аккуму-

лятор. С названием, мнемоникой, количеством байт и временем выполнения команды вам все должно быть очевидно. Обратите внимание на краткое описание совершаемого ей действия: (A) ← ((A) + (DPTR)). Оно также должно быть вполне доступно для вашего понимания. (A) + (DPTR) означает взять содержимое аккумулятора и регистра DPTR и сложить их. А далее нужно взять содержимое ячейки с адресом, равным полученной сумме (это обозначается заключением суммы (A) + (DPTR) в скобки), и переслать его в аккумулятор, о чем свидетельствует направленная к (A) стрелка. Как видите, разобраться с правилами записи кратких описаний выполняемых командами действий весьма полезно — по ним можно безошибочно определить, что эти команды делают.

Вторая команда — «Пересылка с использованием программного счетчика PC» (`MOVC A, @A+PC`). Пока она нам ни о чем не говорит. Но посмотрите краткое ее описание, и сразу все станет ясно: встретив ее, микроконтроллер увеличивает на 1 значение PC, затем складывает это значение с содержимым аккумулятора, находит ячейку памяти программ с адресом, равным полученной сумме, извлекает из нее информацию и помещает извлеченное в аккумулятор. Правда, хоть убейте, плохо представляю, чем эта команда может быть полезна. Но не исключаю, что кто-нибудь из читателей придумает интересный вариант ее применения — дай бог!

Еще хочу обратить ваше внимание — на букву C в конце мнемоники самой операции (`MOVC`). Эта буква показывает, что пересылка осуществляется из памяти программ (C — первая буква слова Code, обычно ассоциируемого с кодами программы, т. е. с памятью программ). Нечто подобное мы уже видели, когда столкнулись с командой, извлекающей данные из внешней памяти данных (`MOVX`) — там на конце мнемоники самой операции стояла буква *X*, что связано было со словом eXtended — расширенная, а в текущем контексте — внешняя. Кстати, именно к рассмотрению таких команд мы сейчас и перейдем.

Первая из них — пересылка в аккумулятор байта из внешнего ОЗУ. Ее мнемоника — `MOVX A,@Ri`. Сравните ее с рассмотренной выше `MOV A,@Ri`. Они почти одинаковы, разница лишь в том, что первая работает с внешним ОЗУ, а вторая — с внутренним. В обычной практике команда `MOVX A,@Ri` почти не используется, она полезна лишь при работе микроконтроллера с микросхемами типа 8155, 8156, 1821РУ10, применяемыми сейчас крайне редко. Обратные действия совершаются командой `MOVX @Ri,A`.

С командами `MOVX A,@DPTR` и `MOVX @DPTR,A` мы уже познакомились в третьей главе, и особо добавлять к сказанному там нечего. Поэтому перейдем к командам работы со стеком.

Первая команда (`PUSH ad`) называется загрузкой в стек. Мы уже говорили о том, как микроконтроллер использует стек для хранения адреса воз-

врата из подпрограмм. Но, помимо этого, в стеке можно также хранить и данные. Упомянутая команда переносит содержимое ячейки с адресом ad в стек, после чего эту ячейку можно использовать в любых других целях. По завершении операции, потребовавшей освобождения этой ячейки от хранившихся в ней данных, их можно вернуть из стека в ячейку обратно командой `POP ad`.

Для чего это нужно? Очень часто оказывается, что при выполнении тех или иных действий нам не хватает регистров общего назначения. Тогда можно попытаться сохранить в стеке содержимое одного или нескольких регистров, которое в текущей операции не используется, и воспользоваться ими, восстановив их содержимое после завершения операции. Но здесь есть одна проблема. В командах `PUSH ad` и `POP ad` нужно прямо указывать адрес ячейки. Если у аккумулятора и у регистра B эти адреса неизменны, то у регистров общего назначения они зависят от номера используемого банка РОН. Поэтому сохранение в стеке содержимого регистров общего назначения может оказаться проблематичным. В свете сказанного у микроконтроллеров x51 в стеке обычно сохраняют при необходимости лишь содержимое аккумулятора и регистра B.

Обращаю ваше внимание на то, что если вы используете стек для хранения содержимого ячейки памяти, перенеся его туда командой `PUSH ad`, восстановить содержимое нужно до того, как вы вызовете какую-либо подпрограмму, т. к. вызов изменяет содержимое указателя стека, и при восстановлении содержимого в регистр попадут не те данные. Точно также, если сохранение совершается внутри какой-либо подпрограммы, то восстановить содержимое регистра (регистров) нужно до того, как наступит черед выполнения команды `RET`.

И еще по поводу работы со стеком. Легче всего представить себе принцип его работы следующим образом. Вспомните магазин автомата Калашникова или подобного оружия. Когда вы набиваете его патронами, каждый вставляемый в него патрон после вставки оказывается верхним, а тот, который был верхним до этого, опускается вниз. Предположим, вы набили его несколькими патронами — тремя, пятью, десятью, число в данном случае неважно. Вопрос: каким по счету при разряжении выйдет из магазина последний вставленный в него патрон? Ответ очевиден — первым. А предпоследний? Ясное дело, вторым. А самый первый? Конечно же, последним.

Точно также происходит процесс загрузки данных в стек и извлечения их оттуда. Последнее занесенное в него число должно извлекаться из него первым по счету, предпоследнее — вторым, предпредпоследнее — третьим и т. д. Поэтому, если вы вначале загрузите в стек, например, содержимое аккумулятора, а затем — регистра B, то извлекать их оттуда нужно в обратной последовательности: вначале восстановить содержимое B, а за ним —

аккумулятора. Если не соблюсти это правило, то после сохранения данных в стеке содержимое регистров перепутается со всеми вытекающими отсюда последствиями.

Ну вот и все про стек. Перейдем к последним четырем командам группы команд передачи данных. Это команды обмена содержимого аккумулятора соответственно с содержимым регистра общего назначения (`XCH A,Rn`), прямоадресуемого байта (`XCH A,ad`) и с косвенно адресуемым через R0 или R1 байтом из внутреннего ОЗУ (`XCH A,@Ri`). Объяснять здесь особо нечего, все, что нужно для понимания размещенной в **Табл. 5.1** информации, касающейся этих команд, выше уже подробно объяснено. А последняя команда рассматриваемой группы — `XCHD A,@Ri`. Она похожа на команду `XCH A,@Ri`, но, в отличие от последней, в `XCHD A,@Ri` обмениваются лишь младшие 4 бита аккумулятора с младшими четырьмя битами соответствующей ячейки памяти, в то время как старшие 4 бита обоих операндов остаются неизменными.

Итак, мы закончили рассмотрение группы команд передачи данных. Кое-кому из вас после знакомства с первой из них наверняка показалось, что знакомство это будет очень долгим и трудным. Но, как видите, чем дальше мы продвигались вглубь системы команд, тем легче это происходило. Так что наберитесь терпения, дальше это знакомство будет идти все легче и легче, и вы заметить не успеете, как я оповещу вас, что оно завершено, и что вы знакомы со всеми командами микроконтроллеров семейства x51. И независимо от того, убедил ли я вас в этом, или нет, перейдем к рассмотрению следующей группы команд.

5.3. Группа команд арифметических операций

Команды, входящие в эту группу, сведены в **Табл. 5.2.**

Рассмотрим их поподробнее. В эту группу входят 4 команды сложения (`ADD`), 4 команды сложения с учетом переноса (`ADDC`), 4 команды вычитания с учетом заема (`SUBB`), 5 команд инкремента (`INC`), 4 команды декремента (`DEC`) и по одной команде умножения, деления и десятичной коррекции аккумулятора.

Почти все эти команды в той или иной степени вам уже знакомы. Команды сложения складывают содержимое аккумулятора соответственно с содержимым регистра общего назначения (`ADD A,Rn`), прямоадресуемого байта (`ADD A,ad`), косвенно адресуемого байта из внутреннего ОЗУ (`ADD A,@Ri`) и с константой (`ADD A,#data8`). Полученная сумма сохраняется в аккумуляторе. Мнемоники, входящие в каждую из групп команды, количество байт в них, время выполнения и краткие описание выполняемых ими действий я не комментирую — вам все уже должно быть очевидно.

Этим командам практически идентичны команды сложения с учетом переноса — их столько же, они работают с теми же регистрами и ячейками ОЗУ, содержат столько же байт и выполняются за то же количество машинных циклов, что и соответствующие им команды обычного сложения. Но, в отличие от последних, результаты выполнения команд сложения с переносом будут на единичку больше, если CY = 1.

Таблица 5.2. **Команды арифметических операций**

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Сложение аккумулятора с регистром (n = 0...7)	`ADD A,Rn`	00101rrr	1	1	(A) ← (A) + (Rn)
Сложение аккумулятора с прямоадресуемым байтом	`ADD A,ad`	00100101	2	1	(A) ← (A) + (ad)
Сложение аккумулятора с байтом из внутреннего ОЗУ (i = 0, 1)	`ADD A,@Ri`	0010011i	1	1	(A) ← (A) + ((Ri))
Сложение аккумулятора с константой	`ADD A,#data8`	00100100	2	1	(A) ← (A) + #data8
Сложение аккумулятора с регистром и переносом	`ADDC A,Rn`	00111rrr	1	1	(A) ← (A) + (Rn) + (C)
Сложение аккумулятора с прямоадресуемым байтом и переносом	`ADDC A,ad`	00110101	2	1	(A) ← (A) + (ad) + (C)
Сложение аккумулятора с байтом из внутреннего ОЗУ и переносом	`ADDC A,@Ri`	0011011i	1	1	(A) ← (A) + ((Ri)) + (C)
Сложение аккумулятора с константой и переносом	`ADDC A,#data8`	00110100	2	1	(A) ← (A) + #data8 + (C)
Десятичная коррекция аккумулятора	`DA A`	11010100	1	1	
Вычитание из аккумулятора регистра и заема	`SUBB A,Rn`	10011rrr	1	1	(A) ← (A) – (Rn) – (C)
Вычитание из аккумулятора прямоадресуемого байта и заема	`SUBB A,ad`	10010101	2	1	(A) ← (A) – (ad) – (C)
Вычитание из аккумулятора байта из внутреннего ОЗУ и заема	`SUBB A,@Ri`	1001011i	1	1	(A) ← (A) – ((Ri)) – (C)
Вычитание из аккумулятора константы и заема	`SUBB A,#data8`	10010100	2	1	(A) ← (A) – #data8 – (C)
Инкремент аккумулятора	`INC A`	00000100	1	1	(A) ← (A) + 1
Инкремент регистра	`INC Rn`	00001rrr	1	1	(Rn) ← (Rn) + 1

Продолжение

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Инкремент прямоадресуемого байта	`INC ad`	00000101	2	1	(ad) ← (ad) + 1
Инкремент байта из внутреннего ОЗУ	`INC @Ri`	0000011i	1	1	((Ri)) ← ((Ri)) + 1
Инкремент указателя данных	`INC DPTR`	10100011	1	2	(DPTR) ← (DPTR) + 1
Декремент аккумулятора	`DEC A`	00010100	1	1	(A) ← (A) – 1
Декремент регистра	`DEC Rn`	00011rrr	1	1	(Rn) ← (Rn) – 1
Декремент прямоадресуемого байта	`DEC ad`	00010101	2	1	(ad) ← (ad) 1
Декремент байта из внутреннего ОЗУ	`DEC @Ri`	0001011i	1	1	((Ri)) ← ((Ri)) – 1
Умножение аккумулятора и регистра B	`MUL AB`	10100100	1	4	(B)(A) ← (A) × (B)
Деление аккумулятора на регистр B	`DIV AB`	10000100	1	4	(A),(B) ← (A) / (B)

С теми же операндами работают и 4 команды вычитания. Но здесь, как я уже говорил в главе, посвященной регистрам, есть своя хитрость. Вспомним школьные годы, вычитание «в столбик». Наверное, все помнят о том, что первыми вычитаются самые правые, младшие биты. При этом, если нам нужно вычесть из меньшего числа большее, мы «занимаем единичку» (осуществляем «заем») из более старшего бита уменьшаемого (т. е. из той цифры уменьшаемого, которая стоит левее используемой в текущий момент). А далее, когда мы будем осуществлять вычитание следующей цифры вычитаемого из той цифры уменьшаемого, где мы что-то заняли, последнюю сначала нужно уменьшить на 1, а затем что-то из нее вычитать.

Для осуществления описанных действий в микроконтроллерах x51 есть команда вычитания с заемом `SUBB`. Она вначале вычитает из уменьшаемого содержимое бита переноса (т. е. 1, если флаг CY установлен, и 0 — если сброшен; кстати, обратите внимание на то, что в этом случае бит переноса фактически является битом заема), а затем — и само вычитаемое. Как вы уже имели возможность убедиться, именно это и нужно для организации программ вычитания.

Напомню об еще одном нюансе — всегда нужно помнить, что когда мы производим вычитание младшей цифры вычитаемого из младшей цифры уменьшаемого, эта процедура по счету — первая, и ей не предшествовал никакой заем. И хотя для получения результата мы вынуждены использовать команду вычитания с заемом `SUBB`, правильный результат до-

стигается тогда, когда мы перед этим первым вычитанием принудительно установим в 0 флаг CY (это делается при помощи знакомой вам команды `CLR C`). Увы, мы не имеем возможности использовать команду вычитания без заема (хотя такие есть у многих микроконтроллеров, в системе команд микроконтроллеров семейства x51 они отсутствуют).

Команды инкремента и декремента вам также уже встречались. Приводимая в **Табл. 5.2** касающаяся их информация очевидна и не требует дополнительных объяснений. Отмечу разве что тот факт, что в системе команд x51 есть команда инкрементирования регистра DPTR, но нет команды его декрементирования. Последнее крайне неудобно. Чуть скрасить это неудобство можно, организовав небольшую подпрограммку для декрементирования DPTR (назовем ее `DECDPTR`):

Подпрограмма для декрементирования регистра DPTR

```
DECDPTR:
        MOV   A,DPL
        ADD   A,#0FFh
        MOV   DPL,A
        MOV   A,DPH
        ADDC  A,#0FFh
        MOV   DPH,A
        CLR   C
        RET
```

Также вам уже знакомы команды умножения и деления, так что и на них я не стану задерживаться. В итоге, из всех команд арифметических операций незнакомой для вас сейчас является лишь одна — команда десятичной коррекции (`DA A`). Рассмотрим ее поближе.

Постарайтесь запомнить следующее.

Команда эта применима лишь в том случае, если вы осуществляете сложение хранящегося в аккумуляторе ***в двоично-десятичном представлении*** *числа в диапазоне от 0 до 99 с представленным* ***в том же формате*** *числом из другого регистра или ячейки памяти. Если вы складываете числа в обычном двоичном (или, что то же самое, в HEX-представлении), использовать команду `DA A` крайне не рекомендуется, ибо это не принесет ничего, кроме ошибок.*

Команда корректирует операцию сложения, т. е. ***перед ней обязательно должна стоять команда сложения***. Если применить команду десятичной коррекции аккумулятора без идущей прямо перед ней команды сложения, никакой коррекции не произойдет, и содержимое аккумулятора не изменится.

Теперь о самой команде. Суть ее состоит в следующем. Как вы должны помнить, в двоично-десятичном представлении числа каждая его цифра

кодируется 4-мя битами. Следовательно, в любом 8-битном регистре хранятся две двоично-десятичные цифры. Например, если в аккумуляторе находится число 01100100b = 64h, то старшие 4 бита (0110) соответствуют цифре 6 в двоично-десятичном представлении, а младшие (0100) — цифре 4.

Положим, в регистре R7 хранится при этом число 10010111b = 97h, и мы хотим сложить эти два двоично-десятичных числа. Если их просто сложить при помощи команды `ADD`, как обычные двоичные числа, то вы получите 11111011b = 0Fbh, что вообще не соответствует никакому двоично-десятичному числу. Но если вслед за `ADD` поставить команду `DA A`, то вы получите в сумме 61h, и CY = 1, что в итоге соответствует десятичному числу 161 (сравните: 64d + 97d = 161d). Вот так-то!

Как же работает `DA A`? Она вначале проверяет сумму младших битов слагаемых (в нашем случае 4 и 7). Если эта сумма больше 9 (в нашем случае так и есть, 4 + 7 = 11d = Bh) или если она меньше 9, но при сложении этих цифр образовался перенос (так бы было, если бы вместо 4 у нас было бы 9; 9 + 7 = 10h = 0h с переносом), то сумма младших битов слагаемых увеличивается на 6. В нашем случае 4 + 7 = Bh; Bh + 6 = 1h, и при этом возникает перенос в сумму старших битов слагаемых.

Далее `DA A` точно также проверяет сумму старших битов слагаемых (в нашем случае 6 и 9), предварительно прибавив к ним упомянутый перенос (при его наличии). Если полученная сумма больше 9 или если она меньше 9, но при сложении этих цифр образовался перенос (в нашем случае так и есть, 6 + 9 + 1 = 16d = 10h = 0h с переносом), то сумма теперь уже и старших битов слагаемых увеличивается на 6. В нашем случае 6 + 9 + 1 = 0h с переносом; 0h + 6 = 6h, и при этом возникает перенос в `CY`. Итак, если бы мы сложили взятые для примера числа без учета переноса, то мы получили бы 0Fbh, что вообще не соответствует никакому двоично-десятичному числу. Но если вслед за `ADD` вы поставите команду `DA A`, то вы получите в сумме 61h и CY = 1, что в итоге соответствует правильному результату — десятичному числу 161.

Еще раз обращаю ваше внимание — команда `DA A` используется только при сложении чисел в двоично-десятичном представлении, и только в том случае, если перед ней стояла команда сложения (`ADD` или `ADDC`). Несоблюдение любого из этих двух условий неминуемо приводит к ошибке.

На этом мы заканчиваем знакомство с группой команд арифметических операций. Обращаю ваше внимание на то, что мы «прошли», как говорят школьники и студенты, 52 из 111 команд, т. е. почти половину доступных микроконтроллеру инструкций! Как видите, знакомство с системой команд для вас сейчас является хотя слегка и нудной, но уже вполне посильной задачей, и свет в конце этого тоннеля, образно говоря, виден невооруженным глазом.

5.4. Группа команд логических операций

Команды, входящие в эту группу, сведены в **Табл. 5.3.**

Таблица 5.3. **Команды логических операций**

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Логическое И аккумулятора и регистра (n = 0...7)	`ANL A,Rn`	01011rrr	1	1	(A) ← (A) И лог (Rn)
Логическое И аккумулятора и прямоадресуемого байта	`ANL A,ad`	1010101	2	1	(A) ← (A) И лог (ad)
Логическое И аккумулятора с байтом из внутреннего ОЗУ (i = 0, 1)	`ANL A,@Ri`	0101011i	1	1	(A) ← (A) И лог ((Ri))
Логическое И аккумулятора и константы	`ANL A,#data8`	1010100	2	1	(A) ← (A) И лог #data8
Логическое И прямоадресуемого байта и аккумулятора	`ANL ad,A`	1010010	2	1	(ad) ← (ad) И лог (A)
Логическое И прямоадресуемого байта и константы	`ANL ad,#data8`	1010011	3	2	(ad) ← (ad) И лог #data8
Логическое ИЛИ аккумулятора и регистра (n = 0...7)	`ORL A,Rn`	01001rrr	1	1	(A) ← (A) ИЛИ лог (Rn)
Логическое ИЛИ аккумулятора и прямоадресуемого байта	`ORL A,ad`	1000101	2	1	(A) ← (A) ИЛИ лог (ad)
Логическое ИЛИ аккумулятора с байтом из внутреннего ОЗУ (i = 0, 1)	`ORL A,@Ri`	0100011i	1	1	(A) ← (A) ИЛИ лог ((Ri))
Логическое ИЛИ аккумулятора и константы	`ORL A,#data8`	1000100	2	1	(A) ← (A) ИЛИ лог #data8
Логическое ИЛИ прямоадресуемого байта и аккумулятора	`ORL ad,A`	1000010	2	1	(ad) ← (ad) ИЛИ лог (A)

Продолжение

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Логическое ИЛИ прямоадресуемого байта и константы	`ORL ad,#data8`	1000011	3	2	(ad) ← (ad) ИЛИ лог #data8
Исключающее ИЛИ аккумулятора и регистра (n = 0...7)	`XRL A,Rn`	01101rrr	1	1	(A) ← (A) ИЛИ искл (Rn)
Исключающее ИЛИ аккумулятора и прямоадресуемого байта	`XRL A,ad`	1100101	2	1	(A) ← (A) ИЛИ искл (ad)
Исключающее ИЛИ аккумулятора с байтом из внутреннего ОЗУ (i = 0, 1)	`XRL A,@Ri`	0110011i	1	1	(A) ← (A) ИЛИ искл ((Ri))
Исключающее ИЛИ аккумулятора и константы	`XRL A,#data8`	1100100	2	1	(A) ← (A) ИЛИ искл #data8
Исключающее ИЛИ прямоадресуемого байта и аккумулятора	`XRL ad,A`	1100010	2	1	(ad) ← (ad) ИЛИ искл (A)
Исключающее ИЛИ прямоадресуемого байта и константы	`XRL ad,#data8`	1100011	3	2	(ad) ← (ad) ИЛИ искл #data8
Сброс аккумулятора	`CLR A`	11100100	1	1	(A) ← 0
Инверсия аккумулятора	`CPL A`	11110100	1	1	(A) ← /(A)
Сдвиг аккумулятора влево циклический	`RL A`	100011	1	1	(A.n+1) ← (A.n), n = 0...6; (A.0) ← (A.7)
Сдвиг аккумулятора влево через перенос	`RLC A`	110011	1	1	(A.n+1) ← (A.n), n = 0...6; (A.0) ← (C); (C) ← (A.7)
Сдвиг аккумулятора вправо циклический	`RL A`	11	1	1	(A.n) ← (A.n+1), n = 0...6; (A.7) ← (A.0)
Сдвиг аккумулятора вправо через перенос	`RLC A`	10011	1	1	(A.n) ← (A.n+1), n = 0...6; (A.7) ← (C); (C) ← (A.0)
Обмен местами тетрад в аккумуляторе	`SWAP A`	11000100	1	1	(A.0, A.1, A.2, A.3) ↔ (A.4, A.5, A.6, A.7)

Рассмотрим их поподробнее. Все они вам уже знакомы. Команды логического И (`ANL`) предписывают микроконтроллеру осуществить побитовое логическое И между содержимым аккумулятора и регистра общего назначения (`ANL A,Rn`), аккумулятора и прямоадресуемого байта (`ANL A,ad`) и т. д. Всего в эту подгруппу входит 6 команд. Следующие 6 инструкций предписывают микроконтроллеру осуществить побитовое логическое ИЛИ (`ORL`) меж-

ду содержимым тех же операндов, и еще 6 команд — исключающее ИЛИ (XRL) с теми же операндами. Команды CLR A и CPL A соответственно зануляют и инвертируют содержимое аккумулятора. Далее, 4 инструкции— это арифметические и циклические сдвиги содержимого аккумулятора (влево, вправо, через перенос и минуя его — RLC A, RRC A, RL A, RR A). Их мы подробно рассматривали в предыдущей главе. И последняя команда (SWAP A) — обмен местами старшей и младшей тетрад аккумулятора.

Обратите внимание на то, как записано символическое описание операции инверсии содержимого аккумулятора: (A) ← /(A). Обычно инверсия обозначается черточкой над инвертируемым операндом. Но в текстовых редакторах черточка над символом, как правило, не предусмотрена. Поэтому давайте договоримся, что здесь инверсия будет обозначаться не черточкой над операндом, а обратным слэшем перед этим операндом.

Ну вот и все о группе команд логических операций. Перейдем к рассмотрению следующей группы команд.

5.5. Группа команд операций с битами

Команды, входящие в эту группу, сведены в **Табл. 5.4**.

Таблица 5.4. **Команды операций с битами**

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Сброс переноса	CLR C	11000011	1	1	(C) ← 0
Сброс бита	CLR bit	11000010	2	1	(bit) ← 0
Установка переноса	SETB C	11010011	1	1	(C) ← 1
Установка бита	SETB bit	11010010	2	1	(bit) ← 1
Инверсия переноса	CPL C	10110011	1	1	(C) ← /(C)
Инверсия бита	CPL bit	10110010	2	1	(bit) ← /(bit)
Логическое И бита и переноса	ANL C,bit	10000010	2	2	(C) ← (C) И лог (bit)
Логическое И инверсии бита и переноса	ANL C,/bit	10110000	2	2	(C) ← (C) И лог /(bit)
Логическое ИЛИ бита и переноса	ORL C,bit	1110010	2	2	(C) ← (C) ИЛИ лог (bit)
Логическое ИЛИ инверсии бита и переноса	ORL C,/bit	10100000	2	2	(C) ← (C) ИЛИ лог /(bit)
Пересылка бита в перенос	MOV C,bit	10100010	2	1	(C) ← (bit)
Пересылка переноса в бит	MOV bit,c	10010010	2	2	(bit) ← (C)

Рассмотрим их. Здесь тоже немало знакомых вам команд. Но прежде, чем мы поговорим о них, нам необходимо более подробно ознакомиться с теми битами, которые являются операндами в этой группе.

Эти биты принадлежат к двум подгруппам. Первую составляют биты, входящие в байты внутреннего ОЗУ с адресами от 20h до 2Fh, см. **Рис. 3.1**. Младший бит байта 20h имеет самостоятельный (битовый) адрес 00h, второй бит этого байта — 01h, третий — 02h и т. д., вплоть до 7Fh (это — адрес старшего, седьмого бита байта с адресом 2Fh).

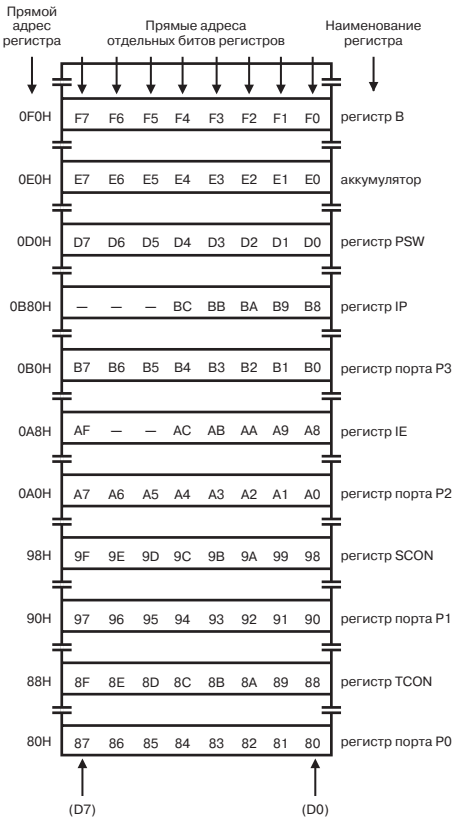


Рис. 5.1. Битовые адреса в регистрах

Вторая подгруппа — это биты аккумулятора, регистров B, PSW, IP, IE, SCON, TCON, а также портов P0...P3. Не все регистры вам пока еще знакомы, но пусть вас это не смущает, чуть позже мы до всех их доберемся. Пока же ознакомьтесь с битовыми адресами каждого из битов, входящих в эти регистры (**Рис. 5.1)**. Они, т. е. эти биты, составляют старшую половину битового адресного пространства.

Я сказал «битовое пространство» — пусть и это вас не смущает, ни в какое отдельное пространство из упомянутых регистров и ячеек памяти эти биты не вынесены и всегда находятся именно там, где мы их обнаружили в двух предыдущих абзацах. Просто программисты, как и все другие математики, любят использовать подобные абстрактные термины — если какие-то регистры, биты, байты, команды или массивы имеют какую-либо отличительную черту, выделяющую их среди других подобных объектов, то всегда находится кто-то, кто скажет, что они образуют то или иное пространство (регистровое, адресное, битовое и т. п.).

Еще раз обращаю ваше внимание — биты 16 байтов внутреннего ОЗУ и 11-ти регистров имеют собственные битовые адреса, при помощи которых мы можем прямо обращаться к любому из них при выполнении команд операций с битами. Таким образом, мы можем устанавливать, сбрасывать, инвертировать, пересылать в `CY` и обратно более 200 бит, входящих в состав упомянутых 27 байтов.

Если это понятно, пойдем дальше. Вспомните, когда мы, например, устанавливали в 1 или сбрасывали в 0 линии тех или иных портов, мы делали это командами `SETB Px.y` или `CLR Px.y`. Далее давайте вспомним еще и то, что в самом начале наших программ мы в разделе объявления имен и констант поместили строки, сообщающие ассемблеру, что адрес бита P1.0 — 90h (`P1.0 .EQU 90h`), адрес бита P1.1 — 91h (`P1.1 .EQU 91h`) и т. д. К чему я клоню? Да к тому, что установка в 1 или сброс в 0 тех или иных линий порта осуществляется при помощи все тех же рассматриваемых нами в текущем разделе команд операций с битами. Для многих это поначалу оказывается неочевидно, т. к. ряд ассемблеров по умолчанию знает, что битовый адрес линии P0.0 — это 80h, P0.3 — это 83h, P1.0 — это 90h и т.д. Поэтому начинающим и кажется, что команды, работающие с этими линиями — это нечто иное, чем команды, сбрасывающие и устанавливающие остальные биты, адреса которых нам приходится писать в явном виде. Но, как видите, это не так, и линиями порта управляют те же команды, которые управляют битами вышеупомянутых регистров и ячеек памяти.

Вот, собственно, и все об этой группе команд. Нам осталось лишь ознакомиться с группой команд передачи управления. Но прежде, чем мы это сделаем, давайте чуть более подробно познакомимся с флагами результата и с тем, какие команды изменяют состояние тех или иных флагов.

5.6. Флаги результата

Мы уже начали знакомство с ними, когда в главе 3 рассматривали регистр PSW. Он, как отмечалось, содержит 4 флага: CY — переноса, AC — вспомогательного переноса, OV — переполнения, и P — паритета.

Таблица 5.5. **Команды, модифицирующие флаги результата**

Команды	Модифицируемые флаги
ADD	CY, OV, AC
ADDC	CY, OV, AC
SUBB	CY, OV, AC
MUL	CY = 0, OV
DIV	CY = 0, OV
DA A	CY
RRC	CY
RLC	CY
SETB C	CY = 1
CLR C	CY = 0
CPL C	CY = /CY
ANL C,bit	CY
ANL C,/bit	CY
ORL C,bit	CY
ORL C,/bit	CY
MOV C,bit	CY
CJNE	CY

Флаг паритета напрямую зависит от текущего значения аккумулятора. Если число единичных бит аккумулятора нечетное, то флаг P устанавливается, если четное — сбрасывается. Все попытки изменить флаг P, присваивая ему новое значение, будут безуспешными, если содержимое аккумулятора при этом останется неизменным.

Флаг AC устанавливается в случае, если при выполнении операции сложения (или вычитания) между тетрадами байта возник перенос/заем. Флаг CY устанавливается, если перенос или заем возникает в старшем бите результата. При выполнении операций умножения и деления флаг CY сбрасывается.

Флаг OV устанавливается, если результат операции сложения или вычитания не укладывается в семи младших битах аккумулятора, и содержимое старшего бита претерпевает изменение в сравнении с исходным (т. е. до сложения/вычитания). Он устанавливается также, если результат умножения больше 255 или в случае деления на 0. Если делитель не равен 0, то независимо от состояния делимого после выполнения деления флаг OV сбрасывается.

В **Табл. 5.5** перечисляются команды, при выполнении которых происходит модификация перечисленных флагов. В таблице отсутствует флаг паритета, т. к. его модифицируют все команды, изменяющие состояние аккумулятора. Кроме команд, приведенных в таблице, флаги модифицируются

командами, в которых местом назначения результата является регистр PSW или его отдельные биты, а также командами операций над битами.

В последней строке таблицы стоит пока неизвестная вам команда `CJNE`. Вы познакомитесь с ней в следующем разделе настоящей главы. А пока просто запомните, что эта команда модифицирует флаг переноса.

5.7. Группа команд передачи управления

Команды, входящие в эту группу, сведены в **Табл. 5.6**.

Таблица 5.6. **Команды передачи управления (файл Команды передачи управления.xls)**

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Длинный абсолютный переход в полном объеме памяти программ	`LJMP ad16`	00000010	3	2	(PC) ← ad16
Абсолютный переход внутри двухкилобайтной страницы	`AJMP ad11`	a10a9a800001	2	2	(PC) ← (PC) + 2, затем (PC 0...10) ← ad11
Короткий относительный переход внутри страницы 256 байт	`SJMP rel`	10000000	2	2	(PC) ← (PC) + 2, затем (PC) ← (PC) + rel
Косвенный относительный переход	`JMP @A+DPTR`	01110011	1	2	(PC) ← (A) + (DPTR)
Переход, если аккумулятор равен 0	`JZ rel`	01100000	2	2	(PC) ← (PC) + 2, затем если (A) = 0, то (PC) ← (PC)+rel
Переход, если аккумулятор не равен 0	`JNZ rel`	01110000	2	2	(PC) ← (PC) + 2, затем если (A)<>0, то (PC) ← (PC)+rel
Переход, если перенос равен единице	`JC rel`	01000000	2	2	(PC) ← (PC) + 2, затем если (C) = 1, то (PC) ← (PC)+rel
Переход, если перенос равен нулю	`JNC rel`	01010000	2	2	(PC) ← (PC) + 2, затем если (C) = 0, то (PC) ← (PC)+rel
Переход, если бит равен единице	`JB bit,rel`	00100000	3	2	(PC) ← (PC) + 3, затем если (bit) = 1, то (PC) ← (PC)+rel
Переход, если бит равен нулю	`JNB bit,rel`	00110000	3	2	(PC) ← (PC) + 3, затем если (bit) = 0, то (PC) ← (PC)+rel

Продолжение

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Переход, если бит установлен, последующим сбросом бита	`JBC bit,rel`	00010000	3	2	(PC) ← (PC) + 3, затем если (bit) = 1, то (PC) ← (PC)+rel, (bit) ← 0
Декремент регистра и переход, если не нуль	`DJNZ Rn,rel`	11011rrr	2	2	(PC) ← (PC) + 2, (Rn) ← (Rn) – 1, затем если (Rn)<>0, то (PC) ← (PC)+rel
Декремент прямоадресуемого байта и переход, если не нуль	`DJNZ ad,rel`	11010101	3	2	(PC) ← (PC) + 2, (ad) ← (ad) – 1, затем если (ad)<>0, то (PC) ← (PC) + rel
Сравнение аккумулятора с прямоадресуемым байтом и переход, если не равно	`CJNE A,ad,rel`	10110101	3	2	(PC) ← (PC) + 3, если (A)<> (ad), то (PC) ← (PC)+rel, при этом если (A)<(ad), то (C) ← 1, иначе (C) ← 0
Сравнение аккумулятора с константой и переход, если не равно	`CJNE A, #data8,rel`	10110100	3	2	(PC) ← (PC) + 3, если (A) <> #data8, то (PC) ← (PC)+rel, при этом если (A)<#data8, то (C) ← 1, иначе (C) ← 0
Сравнение регистра с константой и переход, если не равно	`CJNE Rn, #data8,rel`	10111rrr	3	2	(PC) ← (PC) + 3, если (Rn)<>#data8, то (PC) ← (PC)+rel, при этом если (Rn) <#data8, то (C) ← 1, иначе (C) ← 0
Сравнение байта во внутреннем ОЗУ и переход, если не равно	`CJNE @Ri, #data8,rel`	1011011i	3	2	(PC) ← (PC) + 3, если ((Ri))<>#data8, то (PC) ← (PC)+rel, при этом если ((Ri))<#data8, то (C) ← 1, иначе (C) ← 0
Длинный абсолютный вызов подпрограммы	`LCALL ad16`	00010010	3	2	(PC) ← (PC) + 3; (SP) ← (SP)+1 и ((SP)) ← (PC 0...7); затем (SP) ← (SP)+1 и ((SP)) ← (PC 8...15); затем (PC) ← ad16

Продолжение

Название команды	Мнемоника	КОП	Байт	Циклов	Операция
Абсолютный вызов подпрограммы внутри двухкилобайтной страницы	ACALL ad11	a10a9a810001	2	2	(PC) ← (PC) + 3; (SP) ← (SP)+1 и ((SP)) ← (PC 0...7); затем (SP) ← (SP)+1 и ((SP)) ← (PC 8...15); затем (PC 0...10) ← ad11
Возврат из подпрограммы	RET	00100010	1	2	(PC 8...15) ← ((SP)); (SP) ← (SP) – 1; затем (PC 0...7) ← ((SP)); (SP) ← (SP) – 1
Возврат из подпрограммы обработки прерывания	RETI	00110010	1	2	(PC 8...15) ← ((SP)); (SP) ← (SP) – 1; затем (PC 0...7) ← ((SP)); (SP) ← (SP) – 1
Холостая команда	NOP	00000000	1	1	(PC) ← (PC) + 1

Это последняя группа команд из рассматриваемых нами. В нее входят команды переходов, команды вызова подпрограмм, команды организации циклов, команды сравнения с последующим переходом в зависимости от результата сравнения, команды возврата из подпрограмм и команда NOP. Последняя включена в эту группу по той простой причине, что она не вписывается ни в одну из групп, а поскольку ее все же нужно куда-то определить, то ее включили именно в группу команд передачи управления.

Теперь чуть более подробно об этих командах. Уже знакомая вам команда длинного перехода (LJMP ad16) заставляет микроконтроллер перейти на выполнение фрагмента программы, расположенного в любой части адресного пространства микроконтроллера, от 0000h до 0FFFFh. Обычно в тексте реальной программы вместо символического обозначения операнда (ad16) стоит имя той или иной метки (например, LJMP START). Ассемблер же в процессе трансляции программы вместо имени метки (в данном случае START) ставит ее 16-битный адрес. Как я уже сказал, в этой команде адрес может быть любым, от 0000h до 0FFFFh. Следовательно, команда длинного перехода позволяет перейти к любой метке, сколь бы далеко она не находилась от этой команды, вызвавшей переход.

В отличие от рассмотренной, команда абсолютного перехода внутри двухкилобайтной страницы (AJMP ad11) дает возможность перейти на выполнение фрагмента программы, расположенного не где угодно, а лишь *в любой части текущей двухкилобайтной страницы*. Напомню, что первая такая страница содержит адреса памяти программ от 0000h до 07FFh, вторая — от 0800h до 0FFFh, третья — от 1000h до 17FFh и т. д. Таким образом, если часть программы, содержащая команду AJMP ad11, расположена в

адресах от 0000h до 07FFh (точнее, от 0000h до 07FDh), то переход возможен лишь на фрагмент программы в адресах между 0000h и 07FFh, если она расположена в адресах от 07FEh до 0FFDh, то лишь на фрагмент в адресах между 0800h и 0FFFh и т. д. Многие ассемблеры предоставляют пользователю возможность ставить вместо команд `LJMP` и `AJMP` (а также и `SJMP`, которой мы коснемся чуть ниже) обобщенную команду перехода `JMP`, после чего в процессе трансляции сами определяют, какую из трех команд можно подставить в каждом конкретном случае. `TASM`, увы, к ним не относится, поэтому при работе с ним нам самостоятельно приходится осуществлять выбор между командами перехода. Мне всегда лень разбираться, можно ли ставить команду `AJMP` или нет, поэтому я ею обычно не пользуюсь, ставя везде, где нужно осуществить довольно длинный переход, команду `LJMP`.

Команда короткого относительного перехода `SJMP` позволяет перейти на выполнение фрагмента, отстоящего от места вызова перехода не более чем на 127 байт вперед и на 128 байт назад. Как правило, это переходы на метку, отстоящую от места вызова не дальше, чем на 40—50 строк. Если метка находится дальше, то нужно ставить команду длинного перехода `LJMP`.

Кстати, почему одни команды называются командами абсолютного перехода, а другие — относительного? Вот почему. После трансляции в командах абсолютного перехода TASM оставляет полный 16-битный (`LJMP ad16`) адрес метки, куда нужно перейти или младшие 11 бит этого адреса (`AJMP ad11`). В командах же относительного перехода после трансляции содержится не адрес метки в явном виде, а число, на которое нужно увеличить или уменьшить счетчик программы PC, чтобы вычислить требуемый адрес. Иными словами, абсолютный переход — это когда ассемблер сам вычисляет адрес, куда надо перейти, а относительный — когда он перекладывает работу по вычислению этого адреса на микроконтроллер. Пользователю чаще всего до лампочки, какой из переходов (абсолютный или относительный) будет реализован, главное, чтобы он осуществился на нужную метку. Так что пока просто запомните разницу между абсолютным и относительным переходом, дабы не путаться в терминах, а на практике ставьте команду `SJMP`, если метка, куда нужно перейти, находится где-то недалеко, и `LJMP` в остальных случаях.

Аналогично командам перехода `LJMP` и `AJMP`, команды вызова подпрограмм `LCALL` и `ACALL` осуществляют вызов подпрограмм, расположенных соответственно в любой точке адресного пространства микроконтроллера (`LCALL ad16`) или на той же двухкилобайтной странице (`ACALL ad11`), где расположена и сама команда вызова. Опять-таки на практике в исходном ассемблерном тексте вместо `ad16` и `ad11` стоят име-

на соответствующих меток, адреса которых TASM самостоятельно определяет в процессе трансляции.

Отмечу также, что среди команд вызова подпрограмм нет аналога команды SJMP.

Еще раз обращаю ваше внимание на то, что ad16 и ad11 — это на самом деле имена меток, которыми изобилуют наши ассемблерные программы. В 16-битные адреса их преобразует TASM в процессе трансляции или сам микроконтроллер при помощи входящего в его состав так называемого устройства выборки команд. Рассмотрим, как он это делает на примере команд ACALL ad11 и AJMP ad11. Хотя каждая из этих команд хранит в себе информацию только об 11 младших адресах соответствующей метки, микроконтроллер в процессе обработки команды надставляет эти 11 младших бит пятью старшими, являющимися порядковым номером двухкилобайтной страницы, на которой находится исполняемая ACALL ad11 или AJMP ad11. Так микроконтроллер определяет полный 16-битный адрес перехода или вызываемой подпрограммы для команд AJMP и ACALL.

Но вернемся к командам переходов. Команда JMP @A+DPTR осуществляет переход по адресу, являющемуся суммой содержимого аккумулятора и регистра DPTR. Зачем нужна такая команда, вы поймете, когда мы будем знакомиться с организацией клавиатурного интерфейса.

Все рассмотренные выше команды переходов (LJMP, AJMP, SJMP и JMP) относятся к так называемым безусловным переходам. Иными словами, встретив подобную команду, микроконтроллер *всегда* осуществляет соответствующий переход, независимо от состояния его флагов, содержания регистров и т. д. Но есть в системе команд и условные переходы. Условными их называют потому, что в процессе выполнения каждой из этих команд контроллер проверяет некоторое условие и осуществляет переход на соответствующую метку лишь тогда, когда условие выполняется. При невыполнении же условия микроконтроллер переходит к команде, следующей за командой условного перехода.

Например, встретив команду JZ LOOP1, микроконтроллер проверяет на ноль содержимое аккумулятора, и если аккумулятор равен 0, то микроконтроллер переходит на выполнение фрагмента программы с метки LOOP1. Если же в аккумуляторе не нуль, то вместо перехода будет выполняться команда, следующая за JZ LOOP1. Кстати, JZ — это аббревиатура словосочетания Jump if Zero, т. е. перейти, если ноль.

А если вы хотите, чтобы переход на метку LOOP1 осуществлялся в том случае, когда аккумулятор не равен 0? Тогда используйте команду JNZ LOOP1. Буква N внутри мнемоники как раз и говорит о том, что переход должен осуществиться, если содержимое аккумулятора не равно нулю — JNZ является аббревиатурой словосочетания Jump if No Zero, т. е. перейти, если не нуль.

Мы рассмотрели две команды условного перехода — переход, если содержимое аккумулятора равно 0 (`JZ LOOP1`), и переход, если оно не равно 0 (`JNZ LOOP1`). А сколько еще подобных команд есть у микроконтроллеров семейства x51 и какие они?

Их пять. Две из них связаны с состоянием бита переноса — `JC LOOP11` и `JNC LOOP21` (имена меток, как вы должны были догадаться, могут быть любыми, почему я и ставлю где-то `LOOP1`, где-то `LOOP21` и т. д.). Рассмотрим первую из них (`JC LOOP11`). Переход на метку `LOOP11` в ней осуществляется тогда, когда бит переноса равен 1 (т. е., как говорят программисты, флаг переноса установлен). Как и ранее, если условие не выполнено, т. е. флаг переноса сброшен, никакого перехода не произойдет, и вместо него будет выполняться команда, следующая за `JC LOOP11`. Кстати, `JC` — это аббревиатура словосочетания Jump if Carry, т. е. перейти, если перенос установлен.

Во втором случае (`JNC LOOP21`) переход на метку `LOOP21` осуществляется тогда, когда бит переноса равен 0 (т. е. флаг переноса сброшен). Естественно, если условие не выполнено, т. е. флаг переноса установлен, никакого перехода не произойдет, и вместо него будет выполняться команда, следующая за `JNC LOOP21` (`JNC` — это аббревиатура словосочетания Jump if No Carry, т. е. перейти, если нет переноса).

Следующие две команды условных переходов аналогичны только что рассмотренным командам `JC LOOP11` и `JNC LOOP21`, но вместо бита переноса в них проверяется состояние выбранного бита из битного адресного пространства микроконтроллера (см. подраздел «Группа команд операций с битами» настоящей главы). Например, встретив команду `JB ACC.5,LOOP32`, микроконтроллер проверяет, равен ли единице пятый бит аккумулятора, и если равен, то микроконтроллер переходит на выполнение фрагмента программы с метки `LOOP32`. Если же пятый бит аккумулятора равен нулю, то вместо перехода будет выполняться команда, следующая за `JB ACC.5,LOOP32`. Кстати, `JB` — это аббревиатура словосочетания Jump if bit, т. е. перейти, если бит установлен.

Думаю, что нет смысла подробно комментировать команду `JNB ACC.5,LOOP33` — переход будет осуществлен, если пятый бит равен 0, и не будет, если бит равен 1. А вот последнюю команду, `JBC ACC.5,LOOP44`, прокомментировать надо. Она практически идентична команде `JB ACC.5,LOOP32`, за исключением одного — если анализируемый бит равен 1, и микроконтроллер совершит переход на фрагмент программы, идущий с метки `LOOP44`, в процессе перехода он сбросит этот бит в 0.

Следующие две команды — `DJNZ` — нам уже знакомы. Поэтому долго о них говорить не будем.

Как уже отмечалось в предыдущей главе, аббревиатура этой команды состоит из первых букв словосочетания Decrement and Jump No Zero, что в переводе означает декрементировать (т. е. уменьшить на 1) и перейти, если то, что мы декрементировали, не равно 0. Как теперь нетрудно догадаться, команда `DJNZ Rn,LOOP51` означает буквально следующее: уменьшить Rn на 1, и если содержимое этого регистра после уменьшения не равно 0, то перейти на выполнение команды, идущей после метки, имя которой (`LOOP51`) стоит через запятую после наименования нашего регистра (Rn). Соответственно, для команды `DJNZ ad,LOOP52` все то же самое, за исключением того, что декрементироваться будет не регистр общего назначения Rn, а прямоадресуемая ячейка памяти.

Следующие 4 команды также по сути дела являются командами условного перехода. Это команды `CJNE A,ad,LOOP61; CJNE A,#data8, LOOP62; CJNE Rn,#data8,LOOP63` и `CJNE @Ri,#data8,LOOP64`.

В ходе их выполнения сравниваются два операнда — аккумулятор и прямоадресуемая ячейка памяти, аккумулятор и 8-битное число, регистр и 8-битное число, косвенно адресуемая ячейка памяти и 8-битное число — и если операнды не равны друг другу, то микроконтроллер совершит переход на фрагмент программы с метки `LOOP61, LOOP62, LOOP63` и `LOOP64` соответственно. Ну а при равенстве операндов, как нетрудно догадаться, будет выполнена команда, идущая непосредственно за рассматриваемой.

Из трех оставшихся команд две — `NOP` и `RET` — вам хорошо знакомы, и задерживаться на них нет необходимости. Последняя — `RETI` — это модификация команды `RET`, отличающаяся от обычной команды возврата лишь тем, что она ставится в конце подпрограмм обработки прерываний и разрешает прерывания с приоритетом, равным обслуженному. Пока вы ничего не знаете ни о прерываниях, ни об их приоритетах и подпрограммах их обработки, поэтому все сказанное в предыдущем предложении звучит для многих из вас полной тарабарщиной. Но не расстраивайтесь, чуть позже мы познакомимся с системой прерываний микроконтроллеров, и все сказанное станет для вас очевидным. Так что рассматривайте краткое описание команды `RETI` как заготовку на будущее, не пытаясь понять ее сейчас — всему свое время, и смысл этой команды от нас никуда не уйдет, когда мы вернемся к ней в шестой главе.

А в заключение этой главы я приведу сводную табличку со всеми командами стандартного микроконтроллера семейства x51, с которыми мы познакомились в текущей главе (**Табл. 5.7**).

Таблица 5.7. **Сводная таблица команд микроконтроллера семейства x51**

	0	1	2	3	4	5	6	7
0	NOP	AJMP 0XXh	LJMP ad16	RR A	INC A	INC ad	INC @R0	INC @R1
1	JBC bit,rel	ACALL 0XXh	LCALL ad16	RRC A	DEC A	DEC ad	DEC @R0	DEC @R1
2	JB bit,rel	AJMP 1XXh	RET	RL A	ADD A,#d8	ADD A,ad	ADD A,@R0	ADD A,@R1
3	JNB bit,rel	ACALL 1XXh	RETI	RLC A	ADDC A,#d8	ADDC A,ad	ADDC A,@R0	ADDC A,@R1
4	JC rel	AJMP 2XXh	ORL ad,A	ORL ad,#d8	ORL A,#d8	ORL A,ad	ORL A,@R0	ORL A,@R1
5	JNC rel	ACALL 2XXh	ANL ad,A	ANL ad,#d8	ANL A,#d8	ANL A,ad	ANL A,@R0	ANL A,@R1
6	JZ rel	AJMP 3XXh	XRL ad,A	XRL ad,#d8	XRL A,#d8	XRL A,ad	XRL A,@R0	XRL A,@R1
7	JNZ rel	ACALL 3XXh	ORL C,bit	JMP @A+DPTR	MOV A,#d8	MOV ad,#d8	MOV @R0,#d8	MOV @R1,#d8
8	SJMP rel	AJMP 4XXh	ANL C,bit	MOVC A,@A+PC	DIV AB	MOV add,ads	MOV ad,@R0	MOV ad,@R1
9	MOV DPTR,#d16	ACALL 4XXh	MOV bit,C	MOVC A,@A+DPTR	SUBB A,#d8	SUBB A,ad	SUBB A,@R0	SUBB A,@R1
A	ORL C,/ bit	AJMP 5XXh	MOV C,bit	INC DPTR	MUL AB		MOV @R0,ad	MOV @R1,ad
B	ANL C,/ bit	ACALL 5XXh	CPL bit	CPL C	CJNE A,#d8,rel	CJNE A,ad,rel	CJNE @R0,#d8,rel	CJNE @R1,#d8,rel
C	PUSH ad	AJMP 6XXh	CLR bit	CLR C	SWAP A	XCH A,ad	XCH A,@R0	XCH A,@R1
D	POP ad	ACALL 6XXh	SETB bit	SETB C	DA A	DJNZ ad,rel	XCHD A,@R0	XCHD A,@R1
E	MOVX A,@DPTR	AJMP 7XXh	MOVX A,@R0	MOVX A,@R1	CLR A	MOV A,ad	MOV A,@R0	MOV A,@R1
F	MOVX @DPTR,A	ACALL 7XXh	MOVX @R0,A	MOVX A,@R0	CPL A	MOV ad,A	MOV @R0,A	MOV @R1,A

Продолжение

	8	9	A	B	C	D	E	F
0	INC R0	INC R1	INC R2	INC R3	INC R4	INC R5	INC R6	INC R7
1	DEC R0	DEC R1	DEC R2	DEC R3	DEC R4	DEC R5	DEC R6	DEC R7
2	ADD A,R0	ADD A,R1	ADD A,R2	ADD A,R3	ADD A,R4	ADD A,R5	ADD A,R6	ADD A,R7
3	ADDC A,R0	ADDC A,R1	ADDC A,R2	ADDC A,R3	ADDC A,R4	ADDC A,R5	ADDC A,R6	ADDC A,R7
4	ORL A,R0	ORL A,R1	ORL A,R2	ORL A,R3	ORL A,R4	ORL A,R5	ORL A,R6	ORL A,R7
5	ANL A,R0	ANL A,R1	ANL A,R2	ANL A,R3	ANL A,R4	ANL A,R5	ANL A,R6	ANL A,R7
6	XRL A,R0	XRL A,R1	XRL A,R2	XRL A,R3	XRL A,R4	XRL A,R5	XRL A,R6	XRL A,R7
7	MOV R0,#d8	MOV R1,#d8	MOV R2,#d8	MOV R3,#d8	MOV R4,#d8	MOV R5,#d8	MOV R6,#d8	MOV R7,#d8
8	MOV ad,R0	MOV ad,R1	MOV ad,R2	MOV ad,R3	MOV ad,R4	MOV ad,R5	MOV ad,R6	MOV ad,R7
9	SUBB A,R0	SUBB A,R1	SUBB A,R2	SUBB A,R3	SUBB A,R4	SUBB A,R5	SUBB A,R6	SUBB A,R7
A	MOV R0,ad	MOV R1,ad	MOV R2,ad	MOV R3,ad	MOV R4,ad	MOV R5,ad	MOV R6,ad	MOV R7,ad
B	CJNE R0,#d8,rel	CJNE R1,#d8,rel	CJNE R2,#d8,rel	CJNE R3,#d8,rel	CJNE R4,#d8,rel	CJNE R5,#d8,rel	CJNE R6,#d8,rel	CJNE R7,#d8,rel
C	XCH A,R0	XCH A,R1	XCH A,R2	XCH A,R3	XCH A,R4	XCH A,R5	XCH A,R6	XCH A,R7
D	DJNZ R0,rel	DJNZ R1,rel	DJNZ R2,rel	DJNZ R3,rel	DJNZ R4,rel	DJNZ R5,rel	DJNZ R6,rel	DJNZ R7,rel
E	MOV A,R0	MOV A,R1	MOV A,R2	MOV A,R3	MOV A,R4	MOV A,R5	MOV A,R6	MOV A,R7
F	MOV R0,A	MOV R1,A	MOV R2,A	MOV R3,A	MOV R4,A	MOV R5,A	MOV R6,A	MOV R7,A

5.8. Краткие выводы

Итак, мы завершили краткое знакомство с системой команд микроконтроллеров x51. Более чем с половиной из них вы предварительно ознакомились в ходе рассмотрения материалов предыдущих глав, т.к. нельзя объяснить, как микроконтроллер считывает данные из АЦП или выводит их в индикатор, не прибегая к анализу фрагментов программ с теми или иными командами. Поэтому нам пришлось так нестандартно знакомиться с ними — по мере появления их в рассматриваемых программах. Но зато знакомство с материалом этой главы вряд ли должно было вызвать затруднения у кого-либо из тех, кто разобрался с предыдущими главами.

Система команд x51 содержит 111 базовых команд, которые удобно разделить по функциональному признаку на пять групп: команды передачи данных, арифметических операций, логических операций, передачи управления и операций с битами.

Большинство команд (94) имеет формат один или два байта и выполняются за один или два машинных цикла. При тактовой частоте 12 МГц длительность машинного цикла, как вы должны помнить, составляет 1 мкс.

Микроконтроллеры семейства x51 допускают 4 способа адресации к операндам. Иными словами, сказать микроконтроллеру, где искать тот или иной операнд можно четырьмя различными способами. Они носят названия прямая адресация, непосредственная адресация, косвенная адресация и неявная адресация.

Прямая адресация предполагает, что адрес операнда прямо указан в самой команде. Например, `MOV A,AD00` — адрес операнда (`AD00`) записан прямо в теле команды. Еще пример — `SETB Px.y`. Здесь адрес операнда (`Px.y`) также прямо записан в теле команды (адрес, соответствующий `Px.y`, будет вторым байтом команды после трансляции). Как вы могли убедиться, прямая адресация — самый распространенный способ адресации.

Непосредственная адресация предполагает, что в теле команды записан сам операнд — байт или двухбайтное число. Примеры — `MOV R1,#7` или `MOV DPTR,#0FF25h` . Еще раз обращаю ваше внимание — в ранее рассмотренном случае (при прямой адресации) в теле команды в явном виде записан адрес операнда, а во втором (при непосредственной адресации) в теле команды в явном виде записан сам операнд.

Третий вид адресации — косвенная. В этом случае значение адреса заранее неизвестно и является результатом вычисления. Например, адрес операнда вычисляется по заданному пользователем алгоритму и помещается в определенный регистр, например в R0 или в R1, после чего он используется соответствующей командой. Как отмечалось, подобный метод

адресации, когда адрес ячейки памяти, содержащей операнд, находится в каком-либо регистре, носит название косвенной адресации (адрес мы находим косвенно, при помощи R0, R1 или DPTR), в отличие от рассмотренной выше прямой адресации, где адрес прямо, т. е. в явном виде, указан в самой команде.

Признаком косвенной адресации является знак @ перед соответствующим регистром (R0, R1 или DPTR). В системе команд x51 адреса для режима косвенной адресации могут быть взяты только из этих трех регистров. Напомню, что именно использованием в командах с косвенной адресацией регистры R0 и R1 отличаются от остальных регистров общего назначения (R2...R7), в остальном возможности всех этих регистров одинаковы.

Последний тип адресации — неявная. Как вы уже убедились в ходе знакомства с системой команд микроконтроллера x51, существует много команд, в мнемониках которых явно не прописан аккумулятор или бит переноса, но они, тем не менее, являются операндами в этих командах. Такой способ адресации, без явного указания одного из операндов, носит название неявной адресации. Этот тип адресации характерен для большинства команд, работающих с аккумулятором и с регистрами общего назначения.

У микроконтроллеров некоторых других семейств встречается гораздо большее количество способов адресации. Но и перечисленных четырех вполне достаточно для того, чтобы заставить наш микроконтроллер выполнять любые нужные нам действия (в пределах его быстродействия и допустимых электрических параметров). В этом мы в очередной раз убедимся в следующей главе.

Глава 6. ТАЙМЕРЫ-СЧЕТЧИКИ И СИСТЕМА ПРЕРЫВАНИЙ МК X51

Я как-то уже говорил, что знакомство с любым микроконтроллером предполагает изучение четырех связанных с ним аспектов знаний. К ним относятся типовые схемы его включения, организация его памяти и регистров, организация внутренних периферийных устройств микроконтроллера с системой их обслуживания и система команд микроконтроллера. Деление довольно условное, но оно имеет право на существование. Так вот, мы с вами подробно ознакомились с первым, вторым и четвертым из этих аспектов, и этого уже достаточно, чтобы самостоятельно начать писать программы. Скажу даже больше, к моменту, когда я впервые начал использовать в своих программах внутренние периферийные устройства, я написал уже довольно много различных программ, вполне обходясь без таймеров-счетчиков, системы прерываний или последовательного порта. Так что не без оснований утверждаю, что полученных вами на настоящий момент знаний достаточно, чтобы приступить к самостоятельному конструированию относительно простых систем на основе микроконтроллеров семейства x51 или, вооружившись этими знаниями, начать знакомиться с какими-нибудь другими микроконтроллерами по рассчитанной на подготовленного читателя книжке.

Но не стоит торопить события и ставить точку в процессе нашего с вами знакомства с микроконтроллерами. В состав микроконтроллеров входят обычно еще некоторые дополнительные устройства, которые при правильном их использовании не только облегчают решение поставленных задач, но иногда даже помогают справиться с задачами, которые без их применения попросту неразрешимы. Поэтому, прежде чем обратиться к описанию нескольких законченных примеров практических разработок, я считаю нужным рассказать вам еще кое-что про внутреннее устройство наших микроконтроллеров. И начну с таймеров-счетчиков.

6.1. Таймеры-счетчики микроконтроллеров семейства x51

Надеюсь, вы не забыли, как в главе, посвященной использованию индикаторов, я показал вам подпрограммы, формирующие 1-, 4-, 16- и 64-миллисекундные задержки. Эти подпрограммы предписывают микроконтрол-

леру выполнить точно заданное количество специально подобранных команд, и время выполнения этого набора инструкций составляет требуемое количество миллисекунд. Вызывая по нескольку раз эти подпрограммы и комбинируя их в различном количестве, мы можем формировать временные интервалы практически любой длительности.

Однако не все так просто. Задумайтесь над тем, что, реализуя подобные задержки, микроконтроллер не в состоянии в этот момент делать ничего, кроме выполнения команд, заложенных в эти подпрограммы. Если подобные задержки требуются программе относительно редко, и время их выполнения составляет доли или единицы процентов от времени выполнения всей программы, то такой подход вполне допустим. В некоторых подпрограммах работы с индикаторами задержки могут «отъесть» до 80% всего времени работы подпрограммы. Но сама подпрограмма работы с индикатором в большинстве случаев составляет всего лишь несколько процентов работы всей программы, так что в ней подобный способ формирования временных (ударение на последнем слоге) промежутков вполне приемлем.

А если нам нужно, например, выдавать какому-либо исполнительному устройству последовательно идущие друг за другом импульсы с интервалом 1 мс, а в промежутках между ними что-то измерять и отображать? Предположим, что микроконтроллер будет самостоятельно непрерывно друг за другом формировать эти задержки описанным в четвертой главе программным путем. Тогда у него ни на что другое попросту не хватит времени — ни на измерение, ни на отображение. Так как же быть?

Для решения этой проблемы разработчики микроконтроллера предусмотрели в нем такую полезную вещь, как таймер-счетчик. Что это такое? Представьте себе, что внутри микроконтроллера есть 16-битный счетчик, на вход которого может подаваться сигнал с частотой, в 12 раз меньшей, чем частота внутреннего тактового генератора микроконтроллера. Причем счетчик этот не простой, а с возможностью при помощи записи информации в некоторые внутренние регистры запускать его на счет, останавливать (на время или совсем), осуществлять предварительную запись в него какого-нибудь выбранного числа, управлять им при помощи внешнего сигнала и т. д. А главное, с возможностью как-то сообщать микроконтроллеру о том, что он досчитал до своего максимального значения (т. е. до 0FFFFh, коль скоро он 16-битный).

Что при этом получается? Пусть требуется, чтобы, как было сказано выше, наш микроконтроллер формировал упомянутые короткие импульсы с интервалом в 1 мс. При этом нам также нужно, чтобы в промежутках между формированиями он занимался измерением чего-то и отображением этого чего-то измеренного на экране индикатора. Первый путь, состоящий в чисто программном формировании этих задержек при помощи упомянутых подпрограмм, отпадает. Но есть и второй путь — задействовать

таймер-счетчик. Если микроконтроллер работает на частоте 12 МГц, то на вход таймера-счетчика приходит мегагерцовый сигнал. Следовательно, когда счетчик включен, каждую микросекунду его значение увеличивается на 1. Достаточно лишь предварительно занести в него число 0FC18h (или, что то же самое, десятичное 64536), запустить его на счет, и через 999 мкс он досчитает до 0FFFFh (65535), а еще через 1 микросекунду его значение обнулится, и он сообщит об этом факте микроконтроллеру.

Я сейчас не рассматриваю то, как счетчик сообщает микроконтроллеру о произошедшем факте своего обнуления — об этом мы поговорим чуть позже. Пока просто запомним, что как-то сообщает. Сейчас для нас важнее другое. Если микроконтроллер вначале предустановит таймер-счетчик на 0FC18h, а затем запустит его на счет и займется выполнением какой-либо иной части своей программы, то он целую миллисекунду может спокойно ею заниматься, ибо лишь спустя 1 мс таймер-счетчик сообщит ему, что пришел момент сформировать требуемый импульс. Сформировав импульс, а затем снова предустановив таймер-счетчик на 0FC18h и запустив его на счет, микроконтроллер опять может в течение миллисекунды ни на что не отвлекаться и заниматься заказанными измерением и отображением. И так до бесконечности...

Не правда ли, неплохое решение? Хотя, надо сказать, идея его стара не то, чтобы как мир, но уж как мир микропроцессоров — это точно. Первый подобный программируемый таймер-счетчик, выполненный в виде самостоятельной микросхемы, появился еще в середине семидесятых годов прошлого столетия — это незабвенная 8253 от Intel (у нас она называлась КР580ВИ53, и к моменту написания этих строк упомянутые микросхемы все еще не вышли из употребления). Разработчики микроконтроллеров семейства x51, зная, какой популярностью пользовалась 8253, просто интегрировали ее в микроконтроллер, слегка видоизменив ее возможности. И с тех пор таймеры-счетчики превратились в практически обязательный атрибут микроконтроллера.

Ну вот я, кажется, обосновал перед вами если не необходимость, то желательность наличия таймера-счетчика внутри микроконтроллера и показал в общих чертах один из вариантов его применения. Надеюсь, все сказанное оказалось понятным. Если же это не так, не поленитесь, пробегите еще раз глазами последние 8 абзацев — с пониманием сказанного идти дальше будет намного легче.

Теперь настала пора рассказать про таймеры-счетчики поподробнее. Начну с того, что в любом микроконтроллере семейства x51 их как минимум по 2 (а в некоторых и по 3, но в рамках настоящей главы речь будет идти только об «обязательной» паре таймеров-счетчиков). Счетчики в этой «обязательной» паре почти идентичны, за исключением некоторых небольших отличий, о которых я ниже кратко упомяну. Каждый из счетчиков может работать

в 4-х различных режимах, а также считать как мегагерцовые импульсы (с выхода делителя тактовой частоты на 12), так и импульсы, поступающие на соответствующий вывод микроконтроллера. Кстати, отсюда и его название таймер-счетчик: когда на его вход поступают сигналы извне, то это — счетчик внешних импульсов, а когда с тактового генератора — то таймер, формирующий, как было описано, те или иные задержки. В любой из таймеров-счетчиков можно перед началом работы записать число от 0 до 0FFFFh, а также любой из них можно программным путем остановить или запустить.

Упрощенная структура таймера-счетчика C/$\overline{T0}$ (второй носит название C/$\overline{T1}$) приведена на **Рис. 6.1**.

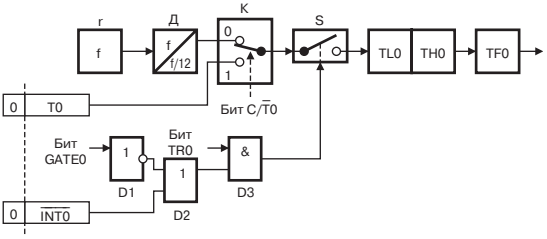


Рис. 6.1. Упрощенная структура таймера-счетчика C/$\overline{T0}$

В его состав входят два регистра, TL0 (младший) и TH0 (старший); упомянутый предделитель Д тактовой частоты на 12; коммутатор K, подключающий вход таймера-счетчика либо к выходу предделителя, либо ко входу T0 микроконтроллера; выключатель S, управляемый от логического узла, содержащего три простых двухвходовых логических элемента D1...D3; флаг переполнения счетчика TF0.

Те, кто внимательно следит за моим повествованием, могут спросить: а что это за вход такой T0? Мы ведь до сих пор ни словом о нем не обмолвились, хотя описали все 40 выводов микроконтроллера. Отвечаю. Да, действительно, описывая выводы микроконтроллеров, мы говорили о 32 линиях портов P0...P3, двух выводах для подключения кварцевого резонатора XTAL1 и XTAL2, входах сброса RESET и разрешения доступа к внешней памяти программ $\overline{EA}$, выходах ALE и $\overline{PSEN}$, а также о выводах, соединяемых с землей и с питанием. Я только что еще раз перечислил все 40 выводов, так и не упомянув при этом ни T0, ни T1. Так где же они?

Когда разработчики микроконтроллера x51 создавали свое детище, перед ними стоял вопрос — уместить микросхему в 40-выводном корпусе, от чего-то отказавшись или использовать корпус с 48 (или 64!) выводами. Тогда, на заре эпохи микроконтроллеров, 48- и 64-выводные корпуса были безумно дороги, гораздо дороже 40-выводных, и разработчикам приходи-

лось буквально впихивать все задуманное в «сороконожку». А для этого нужно было делать выводы микросхемы многофункциональными. Вспомните, по линиям портов P0 и P2 при работе с внешней памятью программ и данных выдается адресная информация. К тому же линии порта P0 еще играют роль шины данных (т. е. выполняют даже не две, а три различные функции!). Так что думаю, что вас не очень удивит, если я скажу вам, что функции входов T0 и T1 микроконтроллера «по совместительству» выполняют линии P3.4 и P3.5 порта P3. Как принято писать в литературе, посвященной микроконтроллерам семейства x51, линии порта P3 могут выполнять альтернативные функции: P3.4 и P3.5 — входы T0 и T1; P3.6 и P3.7 — соответственно выходы $\overline{WR}$ и $\overline{RD}$ при работе с внешней памятью данных; P3.0 и P3.1 — линии RxD и TxD последовательного порта (о нем речь еще впереди) и P3.2, P3.3 — входы внешних прерываний INT0 и INT1 (об этом также чуть позже). Если вы хотите, чтобы вывод выполнял соответствующую ему альтернативную функцию, запишите в регистр-защелку соответствующей линии порта единицу и пользуйтесь альтернативной функцией себе на здоровье, пока не надоест.

Ну вот мы с вами попутно и осилили альтернативные функции порта P3. Пойдем дальше. Регистры TL0 и TH0 (TL1 и TH1) — это регистры самого таймера-счетчика. В них мы заносим число, предустанавливающее таймер-счетчик, из них мы считываем его текущее значение. Адреса этих регистров — 8Ah и 8Ch для TL0 и TH0 соответственно, 8Bh и 8Dh для TL1 и TH1.

Управление же таймерами-счетчиками осуществляется при помощи битов, входящих в состав регистров TMOD (его адрес — 89h) и TCON (адрес — 88h). Назначение битов первого из них приведено в **Табл. 6.1.**, второго — в **Табл. 6.3**. Величины работы обоих таймеров-счетчиков определяются битами M1 и H0 этих регистров (**Табл. 6.2**), но об этом чуть позже.

Таблица 6.1. **Структура регистра TMOD**

Символ	Позиция	Имя и назначение
GATE	TMOD.7 для T/C1 и TMOD.3 для T/C0	Управление блокировкой. Если бит GATE установлен, то таймер-счетчик «x» (x = 0, 1) разрешен до тех пор, пока на входе $\overline{INT}$x высокий уровень, и бит управления TRx установлен. Если бит GATE сброшен, то $\overline{T}$/C разрешается, как только бит TRx устанавливается
C/T	TMOD.6 для T/C1 и TMOD.2 для T/C0	Бит выбора режима таймера или счетчика событий. Если бит C/$\overline{T}$ сброшен, то работает таймер от предделителя тактовой частоты на 12. Если бит C/$\overline{T}$ установлен, то работает счетчик от внешних сигналов на входе Tx (x = 0, 1)
M0	TMOD.5 для T/C1 и TMOD.1 для T/C0	Бит номера режима работы (см. **Табл. 6.2**)
M1	TMOD.4 для T/C1, и TMOD.0 для T/C0	Бит номера режима работы (см. **Табл. 6.2**)

Таблица 6.2. **Выбор режима работы таймера-счетчика битами M**

M1	M0	Режим работы
0	0	Режим, совместимый с таймером семейства x48
0	1	16-битный таймер-счетчик. THx и TLx (x = 0, 1) включены последовательно
1	0	8-битный таймер-счетчик с автоперезагрузкой. THx хранит значение, которое автоматически перезагружается в TLx всякий раз, когда последний переходит из состояния 0FFh в состояние 000h
1	1	Таймер-счетчик 1 в этом режиме останавливается и не работает. Таймер-счетчик 0 работает так: TL0 работает как 8-битный таймер-счетчик, и его режим определяется управляющими битами таймера-счетчика $\overline{T}$/C0. «TH0» работает только как 8-битный таймер, и его режим определяется управляющими битами таймера-счетчика $\overline{T}$/C1

Таблица 6.3. **Структура регистра TCON (старшие 4 бита)**

Символ	Позиция	Имя и назначение
TF1	TCON.7	Флаг переполнения таймера-счетчика 1. Устанавливается аппаратурно при переполнении таймера-счетчика 1. Сбрасывается аппаратурно при обслуживании прерывания
TR1	TCON.6	Бит управления таймера-счетчика 1. Устанавливается/сбрасывается программой для пуска/останова таймера-счетчика 1
TF0	TCON.5	Флаг переполнения таймера-счетчика 0. Устанавливается аппаратурно при переполнении таймера-счетчика 0. Сбрасывается аппаратурно при обслуживании прерывания
TR0	TCON.4	Бит управления таймера-счетчика 0. Устанавливается/сбрасывается программой для пуска/останова таймера-счетчика 0

Как видите, управление запуском/остановкой таймеров-счетчиков определяется битами GATE, TRx и уровнем сигнала на входе $\overline{INT}$x (P3.2 — это $\overline{INT0}$, P3.3 — это $\overline{INT1}$). При GATE = 0 пуск/останов осуществляется только установкой/сбросом бита TRx, независимо от состояния на входе $\overline{INT}$x. Такой режим обычно используется, когда нам нужно формировать задержки или временные интервалы заданной длительности.

При GATE = 1 пуск/останов также осуществляется установкой/сбросом бита TRx. Однако при этом таймер-счетчик будет считать входные сигналы только в том случае, когда на его входе $\overline{INT}$x будет присутствовать сигнал единичного уровня. Чаще всего такой режим используют в том случае, когда хотят измерить длительность какого-либо импульса. С этой целью таймер-счетчик устанавливают битом C/$\overline{T}$x в режим таймера (т. е. на вход его поступают импульсы с предделителя на 12), сбрасывают в 0 регистры TLx и THx, устанавливают GATE и TRx в 1, а измеряемый импульс положительной полярности подают на вход $\overline{INT}$x. Если тактовая частота

микроконтроллера равна 12 МГц, т. е. на вход таймера поступают импульсы с частотой 1 МГц, то длительность измеряемого импульса в микросекундах равна числу, которое будет в TLx и THx после окончания импульса.

Следующий важный момент — режимы работы таймеров-счетчиков. Как видите, их 4. Однако для нас наибольшую практическую ценность представляют всего два из них — второй и третий. Первый режим — это анахронизм, оставленный для совместимости со всеми уже забытыми микроконтроллерами семейства x48, а четвертый — экзотика: 16-битный таймер-счетчик $C/\overline{T}0$ превращается в один 8-битный счетчик и в один 8-битный таймер. Кстати, именно работой в четвертом режиме таймеры-счетчики 0 и 1 отличаются друг от друга.

Во втором режиме (M1 = 0, M0 = 1) как $C/\overline{T}0$, так и $C/\overline{T}1$ работают следующим образом. Каждый из импульсов, поступающих на вход $C/\overline{T}x$, увеличивает на 1 содержимое регистра TLx. Когда происходит переполнение последнего (т. е. содержимое TLx меняется с 0FFh на 0), происходит увеличение на 1 регистра THx. Естественно, увеличение TLx и THx возможно только тогда, когда TRx установлен в 1 и $\overline{INT}x$ = 1 при соответствующем GATE = 0.

Когда происходит переполнение THx (т. е. содержимое THx меняется с 0FFh на 0), то устанавливается в 1 соответствующий флаг переполнения таймера TFx. Здесь я чуть забегу вперед и скажу, что если написанная нами программа к этому моменту разрешила так называемые прерывания, то установка в 1 упомянутого флага автоматически вызовет соответствующую подпрограмму, именуемую подпрограммой обработки данного прерывания. Именно этот процесс я и имел ввиду, когда говорил, что таймер-счетчик может сообщить нашему микроконтроллеру о том, что он досчитал до 0FFFFh — микроконтроллеру не нужно постоянно контролировать состояние таймера-счетчика, переполнение последнего вызывает соответствующую подпрограмму автоматически, вне зависимости от того, чем в этот момент занят микроконтроллер.

В третьем режиме (M1 = 1, M0 = 0) $C/\overline{T}0$ и $C/\overline{T}1$ также работают одинаково. Каждый из импульсов, поступающих на вход $C/\overline{T}x$, увеличивает на 1 содержимое регистра TLx. Когда происходит переполнение последнего (т. е. содержимое TLx меняется с 0FFh на 0), то устанавливается в 1 соответствующий флаг переполнения таймера TFx. Одновременно с этим происходит занесение в TLx числа, хранящегося в регистре THx, при этом содержимое THx остается неизменным (**Рис. 6.2**). И в этом случае установка в 1 соответствующего флага переполнения таймера TFx вызывает переход на выполнение подпрограммы обработки прерывания (если, естественно, прерывания были разрешены; о том, как это делается, будет сказано чуть ниже).

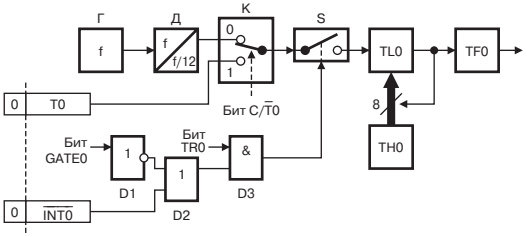


Рис. 6.2. Таймер-счетчик C/$\overline{T}$0 в режиме с автоперезагрузкой

Вот, собственно, и все, что я хотел сказать о таймерах-счетчиках. Осталось, разве что, добавить, что запись информации в упомянутые регистры должна осуществляться при помощи команд `MOV TL0,#data8`, `MOV TH0,#data8` (и им аналогичными для регистров таймера-счетчика 1), `MOV TMOD,#data8` и `MOV TCON,#data8`. Адреса упомянутых регистров приведены выше. Если ваш ассемблер, подобно моему TASM'у, не знает об их существовании, эти адреса нужно объявить в разделе адресов и символических имен в начале программы.

Кроме того, биты регистра TCON входят в битовое адресное пространство микроконтроллеров семейства x51, следовательно, их можно устанавливать и сбрасывать командами `SETB` и `CLR`.

Практический пример использования таймера-счетчика мы рассмотрим ниже, после знакомства с системой прерывания.

6.2. Система прерываний микроконтроллеров семейства x51

В предыдущем разделе мы уже упомянули о том, что переполнение таймеров-счетчиков может (в определенных условиях) вызвать соответствующую подпрограмму, называемую подпрограммой обработки прерывания от таймера-счетчика. Настала пора подробнее познакомиться с системой прерываний, узнать, какие еще подпрограммы обработки прерываний могут входить в нее, как разрешить их вызов и т. д.

Однако прежде, чем это сделать, еще раз давайте обсудим саму идеологию прерываний. Как было уже сказано, во многих задачах мы не можем себе позволить, чтобы микроконтроллер отвлекался от выполнения основного алгоритма, проверяя, произошло то или иное событие (пришел ожидаемый импульс, таймер досчитал до 0FFFFh и т. д.). И в то же время нам необходимо, чтобы контроллер как-то реагировал на упомянутые события. Типичный пример — быстродействующий измерительный прибор, снаб-

женный клавиатурой. С одной стороны он должен не отвлекаясь производить запланированные измерения. С другой же стороны, надо реагировать на нажатие клавиш — переключать режимы, коэффициенты и т. д. А как узнать о том, что та или иная клавиша нажата, если нет возможности отвлекаться от основной программы через каждые 15...20 мс и проверять, есть ли нажатие?

Выход из создавшейся ситуации нашли разработчики самых первых микропроцессоров. Они снабдили свои изделия входами, появление на которых оговоренного в спецификации сигнала (ну, например, импульса отрицательной полярности) приводило к автоматическому вызову расположенной строго в определенном месте подпрограммы. Таким образом, процессор выполнял свою программу до тех пор, пока на его вход прерывания не приходил упомянутый отрицательный импульс. С его появлением выполнение основной программы прерывалось (отсюда и термин — прерывание), адрес следующей команды из прерванной программы запоминался в стеке, и вызывалась вышеупомянутая подпрограмма. Применительно к рассмотренному случаю (прибор с клавиатурой) процессор должен был бы в этой подпрограмме определить, какая клавиша нажата, выполнить соответствующие ей действия и вернуться к выполнению прерванной программы. Таким образом, конфликт разрешался — микроконтроллер, не отвлекаясь на анализ состояния клавиатуры, выполнял поставленную задачу, но при этом, как только нажималась та или иная клавиша, он мгновенно «узнавал» об этом и реагировал, прервав выполнение основной программы и выполнив соответствующую подпрограмму. После ее завершения он возвращался к основной программе и продолжал ее выполнять вплоть до следующего прерывания.

Описанная идея оказалась весьма плодотворной, и в современных микроконтроллерах прерывания вызываются уже не только при появлении на том или ином входе прерывающего импульса, но и при достижении встроенным счетчиком своего максимального значения, при приеме или передаче байта встроенным приемопередатчиком, при снижении напряжения питания ниже заданного уровня... Каждому из этих прерывающих факторов обычно соответствует своя подпрограмма. Чаще всего эти подпрограммы привязаны к конкретным адресам памяти программ: например, в семействе x51 подпрограмма обработки прерывания от таймера 0 обязательно должна начинаться с адреса 000Bh, от таймера 1 — с 001Bh, от отрицательного импульса на входе $\overline{\text{INT0}}$— с адреса 0003h, от импульса на входе $\overline{\text{INT1}}$— с адреса 13h и т. д. Чуть ниже я покажу, как это сделать. Кстати, описанный способ связи между причиной прерывания и адресом подпрограммы его обработки получил название векторной системы прерывания, а начальные адреса подпрограмм обработки при этом именуют векторами прерывания. На мой взгляд, названия эти — неудачные, ибо вектор,

как нас когда-то учили в школе на уроках математики — это объект, характеризуемый двумя параметрами, величиной и направлением. И где разработчики x51 нашли в адресах подпрограмм величину и направление? Наверное, они не очень-то дружили со школьной математикой...

Но что сделано, то сделано, и с их легкой руки весь мир, говоря об адресах подпрограмм обработки прерываний, называет их векторами. Так что и нам придется если не придерживаться этой терминологии, то по крайней мере понимать, что под этими словами подразумевается.

Следующий момент, который нужно понять, прежде чем вникать в особенности системы прерываний — это как и зачем осуществляется запрещение/разрешение прерываний (вместо «запрещение» еще иногда говорят «маскирование» прерываний). Бывают ситуации, когда нам нежелательно, чтобы происходили прерывания. Рассмотрим такой пример. Подпрограмма обработки прерывания измеряет какую-то величину и заносит двухбайтный результат измерения в две расположенные рядом ячейки внутреннего ОЗУ. Основная же программа преобразовывает число из этих двух ячеек в двоично-десятичный формат, а затем отображает его на дисплее. При этом она вначале переносит число в регистры R2 и R3, а затем осуществляет соответствующее преобразование. Так вот, если прерывание наступит после того, как мы перенесли байт из первой ячейки в регистр R2, но до того, как мы перенесем второй байт в R3, и прерывающая программа изменит результат в упомянутых двух ячейках ОЗУ, то перенеся после этого второй байт в R3 и осуществив преобразование, мы получим ошибочный результат, не имеющий ничего общего ни с предшествующим измерением, ни с последним.

Ошибки не будет, если перед переносом числа из ячеек ОЗУ в регистры R2 и R3 мы запретим прерывания, а после переноса — разрешим их. Как это сделать? Для этого предусмотрен соответствующий механизм. Установка в 0 бита EA в регистре IE (подробнее о них я скажу чуть ниже) запрещает все прерывания.

Но не все так просто. В ряде случаев бывает, что в системе могут быть задействованы несколько прерываний, например, и от первого таймера-счетчика, и от второго. Положим, в какой-то момент нам желательно запретить прерывание от одного из них, но оставить прерывание от второго. Установив в 0, как сказано выше, бит EA, мы запрещаем оба. А нам хотелось, чтобы одно из них функционировало. Как быть?

Каждому прерыванию поставлен в соответствие бит (все они находятся в упомянутом регистре IE), установка которого в 0 запрещает соответствующее прерывание, не влияя на остальные. При этом запрещение происходит независимо от состояния бита EA. Так что если нам вдруг вздумается запретить лишь одно из прерываний, оставив разрешенными все остальные, то нужно сбросить в 0 бит, соответствующий запрещаемому прерыванию,

оставив в 1 биты, соответствующие разрешенным. Такую операцию и называют маскированием прерывания. Установив биты запрещаемых прерываний в нули, а разрешаемых — в единицы, мы как бы «надели маску» на систему прерываний, после чего она на некоторые из них перестала реагировать.

И последнее, о чем я хочу сказать перед тем, как мы перейдем к более подробному изучению системы прерываний — при старте микроконтроллеров семейства x51 все прерывания в них изначально запрещены. Следовательно, если нам понадобится их использовать, то мы должны в своей программе предусмотреть установку в 1 битов, соответствующих разрешаемым прерываниям, и установить в 1 уже упоминавшийся бит EA, действие которого (разрешение или запрещение) распространяется на все прерывания одновременно.

Итак, мы уже знаем, что прерывания — это такой внутренний механизм, который позволяет микроконтроллеру реагировать на те или иные внешние в отношении к выполняемой программе воздействия — приход импульса, переполнение таймера-счетчика, завершение приема или передачи байта встроенным приемопередатчиком. Реакция заключается в вызове соответствующей подпрограммы (при условии, что как все вместе, так и конкретное вызываемое прерывание в отдельности, разрешены). Вызываемые подпрограммы должны начинаться со строго определенных адресов. Разрешение или запрещение прерываний осуществляется путем записи нулей или единиц в биты регистра IE.

Теперь обо всем этом более подробно. Большинство микроконтроллеров семейства x51 имеет 5 прерываний: от таймера 0, от таймера 1, от сигнала нулевого логического уровня (или от перепада из 1 в 0) на входе $\overline{\text{INT0}}$, от сигнала нулевого логического уровня (или от перепада из 1 в 0) на входе $\overline{\text{INT1}}$ и от встроенного последовательного приемопередатчика (в последнем случае — при завершении операции приема или передачи байта). Соответственно, подпрограммы обработки этих прерываний должны начинаться с адресов 000Bh, 001Bh, 0003h, 0013h и 0023h. Как и всякие подпрограммы, они должны заканчиваться командой возврата из подпрограммы. Но в подпрограммах обработки прерываний с этой целью нужно использовать не до боли знакомую всем программистам команду `RET`, а ее модификацию `RETI`. Отличие последней от первой состоит в том, что она, помимо возврата, осуществляет автоматическое разрешение прерываний, которые также автоматически были запрещены после перехода микроконтроллера на выполнение подпрограммы обработки прерывания.

Источник прерывания (переполнившийся таймер-счетчик, нулевой сигнал на входе $\overline{\text{INT0}}$ или $\overline{\text{INT1}}$, принявший байт приемопередатчик) вызывает свою подпрограмму обработки путем установки в1 соответствующего ему бита (флага). Для таймера-счетчика 0 этот флаг называется

TF0 (бит TCON.7 в регистре TCON, см. **Табл. 6.3**). Для таймера-счетчика 1 это флаг TF1 (бит TCON.5 в регистре TCON). Для сигналов на входах $\overline{\text{INT0}}$ и $\overline{\text{INT1}}$ эти флаги называются IE0, IE1 и также находятся в регистре TCON (биты TCON.1 и TCON.3 соответственно, см. **Табл. 6.4**).

Таблица 6.4. **Структура регистра TCON (младшие 4 бита)**

Символ	Позиция	Имя и назначение
IE1	TCON.3	Флаг фронта прерывания 1. Устанавливается аппаратно, когда детектируется спад внешнего сигнала $\overline{\text{INT1}}$. Сбрасывается при обслуживании прерывания
IT1	TCON.2	Бит управления типом прерывания 1. Устанавливается/сбрасывается программно для выбора спада/низкого уровня как источника прерывания $\overline{\text{INT1}}$
IE0	TCON.1	Флаг фронта прерывания 0. Устанавливается аппаратно, когда детектируется спад внешнего сигнала $\overline{\text{INT0}}$. Сбрасывается при обслуживании прерывания
IT0	TCON.0	Бит управления типом прерывания 0. Устанавливается/сбрасывается программно для выбора спада/низкого уровня как источника прерывания $\overline{\text{INT0}}$

Прием байта приемопередатчиком или выдача байта устанавливает флаги RI и TI в регистре SCON (о нем — позже).

Имейте еще в виду следующее. Внешние прерывания (от входов $\overline{\text{INT0}}$ и $\overline{\text{INT1}}$) могут быть вызваны либо нулевым уровнем на этих входах, либо переходом сигнала на них из 1 в 0. Это определяется состоянием битов IT0 и IT1, также находящихся в регистре TCON (биты TCON.0 и TCON.2 соответственно, **Табл. 6.4**). Как уже было сказано, при этом автоматически (еще говорят — аппаратно, т. е. без вмешательства каких-либо команд) устанавливаются флаги IE0, IE1. Сброс же этих флагов выполняется при переходе к подпрограмме обработки прерывания аппаратно (автоматически) только в том случае, когда прерывания были вызваны переходом сигнала из 1 в 0. Если же такое прерывание было вызвано нулевым уровнем сигнала на входе, то сброс флага IEx (x = 0, 1) осуществится лишь тогда, когда подпрограмма обработки прерывания воздействует на источник, сформировавший этот нулевой сигнал, и заставит его установить на входе $\overline{\text{INT}}$x (x = 0, 1) вместо вызвавшего прерывание нуля единицу.

Флаги запросов прерываний от таймеров TF0 и TF1 автоматически устанавливаются при их переполнении и также автоматически сбрасываются при переходе микроконтроллера на выполнение подпрограммы обслуживания этих прерываний. Также автоматически устанавливаются флаги запросов прерываний RI и TI в регистре SCON, но сбрасывать их должны соответствующие команды, специально включенные для этого в подпрограмму обслуживания прерывания.

Кстати, а зачем эти флаги нужно сбрасывать (т. е. устанавливать в 0 упомянутые биты)? Да затем, что пока соответствующий флаг не сброшен, микроконтроллер не увидит новый запрос от этого источника и может пропустить ожидаемое событие.

Итак, с флагами запросов прерываний мы разобрались. Теперь самое время познакомиться с уже неоднократно упоминавшимся регистром IE. Его структура приведена в **Табл. 6.5**.

Таблица 6.5. **Структура регистра IE (адрес — 0A8h)**

Символ	Позиция	Имя и назначение
EA	IE.7	Глобальная блокировка прерываний. Установка/сброс осуществляются программным путем. Сброс флага запрещает все прерывания, вне зависимости от IE0... IE4
–	IE.6	Не используется
–	IE.5	Не используется
ES	IE.4	Бит разрешения/запрета прерываний от приемопередатчика
ET1	IE.3	Бит разрешения/запрета прерываний от таймера-счетчика 1
EX1	IE.2	Бит разрешения/запрета прерываний от входа $\overline{INT1}$
ET0	IE.1	Бит разрешения/запрета прерываний от таймера-счетчика 0
EX0	IE.0	Бит разрешения/запрета прерываний от входа $\overline{INT0}$

Как видите, все просто. Установка EA (IE.7) в 1 разрешает все прерывания (при наличии единиц в IE0...IE4). Так что если вы хотите использовать прерывания в своих программах, не забывайте в начале программы устанавливать все эти биты в единицы. Сброс EA в 0 запрещает все прерывания, независимо от того, что было их причиной. Если же нам нужно запрещать прерывания выборочно, то EA нужно оставлять в единичном состоянии, и устанавливать в нули те из IE0...IE4, которые соответствуют маскируемым прерываниям.

Итак, надеюсь, что вам ясно, что такое прерывания, зачем они нужны, сколько их, как они вызываются, как разрешаются и запрещаются, что такое флаги прерывания, и как они сбрасываются. Осталось обсудить лишь одно понятие — приоритеты прерываний, т. е. их степень важности и, как следствие этого, последовательность реакции на них со стороны нашего микроконтроллера.

Давайте на секунду предположим, что одновременно, в один и тот же момент возникли запросы на два различных прерывания — например, переполнился таймер-счетчик 1, и на входе $\overline{INT}$ появился сигнал нулевого уровня. Как быть нашему бедному микроконтроллеру, какой из запросов обслужить раньше, а какой — позже?

Или еще ситуация. Микроконтроллер обрабатывает запрос на прерывание от внешнего входа $\overline{INT0}$, а тут приходит запрос на прерывание от таймера-счетчика. Должен ли микроконтроллер прерывать обработку прерывания от $\overline{INT0}$ и заниматься таймером-счетчиком?

Для того, чтобы ответить на эти вопросы, нужно знать, что каждому из пяти упомянутых прерываний мы можем присвоить как высокий, так и низкий приоритет. Ясно, что прерывание с высоким приоритетом должно быть обработано в первую очередь, а с низким — во вторую. Далее, при старте микроконтроллера все прерывания имеют низкий (равный друг другу) приоритет. При этом, если два прерывания с одинаковым приоритетом возникают одновременно, микроконтроллер x51 обслужит сначала то, которое раньше упомянуто в следующем списке: от сигнала нулевого логического уровня (или от перепада из 1 в 0) на входе $\overline{INT0}$, от таймера 0, от сигнала нулевого логического уровня (или от перепада из 1 в 0) на входе $\overline{INT1}$, от таймера 1, от встроенного последовательного приемопередатчика. Естественно, если переполнился таймер-счетчик 1 и на входе $\overline{INT0}$ появился сигнал нулевого уровня, а приоритеты у этих прерываний одинаковые, то вначале обслуживается прерывание от $\overline{INT0}$. Для того, чтобы заставить микроконтроллер вначале обрабатывать прерывание от таймера-счетчика 1, а только затем все остальные, прерыванию от таймера нужно присвоить высокий приоритет — установить в 1 соответствующий бит в регистре приоритетов IP (см. **Табл. 6.6**). Ясно, что приоритеты остальных прерываний нужно оставить низкими.

Таблица 6.6. **Структура регистра IP (адрес — 0B8h)**

Символ	Позиция	Имя и назначение
–	IP.7	Не используется
–	IP.6	Не используется
–	IP.5	Не используется
PS	IP.4	Бит приоритета приемопередатчика (0 — низкий, 1 — высокий)
PT1	IP.3	Бит приоритета таймера-счетчика 1 (0 — низкий, 1 — высокий)
PX1	IP.2	Бит приоритета внешнего прерывания 1 (0 — низкий, 1 — высокий)
PT0	IP.1	Бит приоритета таймера-счетчика 0 (0 — низкий, 1 — высокий)
PX0	IP.0	Бит приоритета внешнего прерывания 0 (0 — низкий, 1 — высокий)

Прерывание подпрограммы обработки прерываний (простите за каламбур) возможно лишь тогда, когда требует внимания прерывание с приоритетом более высоким, чем то, которое обрабатывается. Если же возникает прерывание с приоритетом, равным или более низким, чем то, которое в настоящий момент обрабатывается, то пришедшее позже

запрещается до момента завершения обработки первого и разрешается лишь тогда, когда микроконтроллер выполнит команду `RETI`.

Вот и все, что нужно знать о прерываниях на первом этапе знакомства с ними. Попробуем теперь применить полученные знания на практике — проанализировать программку, которая использует таймер-счетчик и систему прерываний.

6.3. Программа, использующая таймер-счетчик и прерывание

Предлагаемая программа носит скорее демонстрационный, чем прикладной характер. Она показывает, как с использованием таймера-счетчика 0 и связанного с ним прерывания можно, практически не используя при этом вычислительных ресурсов микроконтроллера, на одном из его выводов сформировать меандр, частота которого, в зависимости от числа, заносимого в TH0, TL0, может варьироваться от 20 Гц до нескольких килогерц. Это может быть полезно при работе с АЦП, подобными DDC112 от Burr-brown.

Программа формирования меандра при помощи таймера-счетчика и прерывания

```
;                          АДРЕСА И КОНСТАНТЫ
;                          *****
;
;
  #INCLUDE "DEFS.INC"
;
CHISLO1 .EQU   100
CHISLO  .EQU   65536-CHISLO1
;
;
;                          НАЧАЛО ПРОГРАММЫ:
;                          *****
;
        .ORG   0
;
        LJMP   WORK
;
;
        .ORG   0BH         ;СЛЕДУЮЩАЯ КОМАНДА ПО АДРЕСУ 0BH
;
        LJMP   TIMER0      ;ПЕРЕХОД К ПОДПРОГРАММЕ ОБРАБОТКИ
;                          ;ПРЕРЫВАНИЯ ОТ ТАЙМЕРА-СЧЕТЧИКА 0
;
        .ORG   100H        ;СЛЕДУЮЩАЯ КОМАНДА ПО АДРЕСУ 100H
;
WORK:                      ;НАСТРОЙКА ТАЙМЕРА-СЧЕТЧИКА И
```

```
                                ;СИСТЕМЫ ПРЕРЫВАНИЙ
        CLR    TR0
        CLR    TR1              ;ТАЙМЕРЫ ОСТАНОВЛЕНЫ
        MOV    IP,#0            ;ВСЕ ПРИОРИТЕТЫ НИЗКИЕ
        MOV    IE,#0            ;ВСЕ ПРЕРЫВАНИЯ ЗАПРЕЩЕНЫ
        MOV    TMOD,#00000001B  ;0-GATE1, 0-C/T1#, 00-РЕЖИМ ТАЙМЕРА 1
                                ;0-GATE0, 0-C/T0#, 01-РЕЖИМ ТАЙМЕРА 0
        MOV    DPTR,#CHISLO
        MOV    TL0,DPL
        MOV    TH0,DPH          ;ТАЙМЕР 0 ЗАРЯЖАЕМ
                                ;ЧИСЛОМ 65535-CHISLO1
        SETB   ET0              ;РАЗРЕШИЛИ ПРЕРЫВАНИЕ ОТ T0
        SETB   EA               ;ГЛОБАЛЬНО РАЗРЕШИЛИ ПРЕРЫВАНИЯ
        SETB   TR0              ;СТАРТ ТАЙМЕРА 0
;
;ДАЛЕЕ ОСНОВНАЯ ПРОГРАММА, ВЫПОЛНЯЮЩАЯ ТЕ ИЛИ ИНЫЕ ДЕЙСТВИЯ, И
;ПРЕРЫВАЕМАЯ ПРИ ПЕРЕХОДЕ ЗНАЧЕНИЯ В TL0,TH0 ЧЕРЕЗ 0FFFFH
;
        SJMP   $
;
;
TIMER0:
        CLR    TR0              ;ТАЙМЕР ОСТАНОВЛЕН
        CLR    EA               ;ЗАПРЕЩЕНЫ ПРЕРЫВАНИЯ
        PUSH   PSW
        PUSH   ACC              ;СОХРАНИМ РЕГИСТРЫ В СТЕКЕ
        PUSH   DPL
        PUSH   DPH
;
        MOV    C,P1.0
        MOV    P1.1,C           ;В P1.1 ВЫВЕЛИ ТО, ЧТО БЫЛО В P1.0
        CPL    C                ;ИНВЕРТИРОВАЛИ ПРЕДЫДУЩЕЕ P1.0
        MOV    P1.0,C           ;ВЫВЕЛИ В P1.0 ИНВЕРТИРОВАННОЕ P1.0
;
        MOV    DPTR,#CHISLO
        MOV    TL0,DPL
        MOV    TH0,DPH          ;ВНОВЬ ЗАРЯЖАЕМ ТАЙМЕР 0
;
        POP    DPH
        POP    DPL
        POP    ACC
        POP    PSW              ;ВОССТАНОВИЛИ РЕГИСТРЫ ИЗ СТЕКА
        SETB   EA               ;РАЗРЕШИЛИ ПРЕРЫВАНИЯ
        SETB   TR0              ;СТАРТ ТАЙМЕРА 0
        RETI                    ;ВОЗВРАТ ИЗ ПРЕРЫВАНИЯ
                                ;С РАЗРЕШЕНИЕМ СЛЕДУЮЩЕГО
;
        .END
```

Обращаю ваше внимание на следующее. Во-первых, вместо длинного перечисления имен регистров и их битов, которые незнакомы моему ас-

семблеру TASM, я поставил директиву #INCLUDE «DEFS.INC». Действие ее состоит в том, что при трансляции она вместо строки с указанной директивой вставляет содержимое файла DEFS.INC, в который я вынес объявление адресов этих регистров и битов.

Далее приведено содержимое файла DEFS.INC, в который, дабы не перегружать тексты рассматриваемых далее программ, я вынес эти адреса.

Содержимое файла DEFS.INC

```
;
R15     .EQU  15
R14     .EQU  14
R13     .EQU  13
R12     .EQU  12
R11     .EQU  11
R10     .EQU  10
R9      .EQU  9
R8      .EQU  8
R7      .EQU  7
R6      .EQU  6
R5      .EQU  5
R4      .EQU  4
R3      .EQU  3
R2      .EQU  2
R1      .EQU  1
R0      .EQU  0
ACC     .EQU  0E0H
B       .EQU  0F0H
PSW     .EQU  0D0H
SP      .EQU  81H
DPL     .EQU  82H
DPH     .EQU  83H
P0      .EQU  80H
P1      .EQU  90H
P2      .EQU  0A0H
P3      .EQU  0B0H
B.0     .EQU  0F0H
B.1     .EQU  0F1H
B.2     .EQU  0F2H
B.3     .EQU  0F3H
B.4     .EQU  0F4H
B.5     .EQU  0F5H
B.6     .EQU  0F6H
B.7     .EQU  0F7H
ACC.0   .EQU  0E0H
ACC.1   .EQU  0E1H
ACC.2   .EQU  0E2H
ACC.3   .EQU  0E3H
ACC.4   .EQU  0E4H
ACC.5   .EQU  0E5H
```

```
ACC.6   .EQU   0E6H
ACC.7   .EQU   0E7H
PSW.0   .EQU   0D0H
PSW.1   .EQU   0D1H
PSW.2   .EQU   0D2H
PSW.3   .EQU   0D3H
PSW.4   .EQU   0D4H
PSW.5   .EQU   0D5H
PSW.6   .EQU   0D6H
PSW.7   .EQU   0D7H
P0.0    .EQU   080H
P0.1    .EQU   081H
P0.2    .EQU   082H
P0.3    .EQU   083H
P0.4    .EQU   084H
P0.5    .EQU   085H
P0.6    .EQU   086H
P0.7    .EQU   087H
P1.0    .EQU   090H
P1.1    .EQU   091H
P1.2    .EQU   092H
P1.3    .EQU   093H
P1.4    .EQU   094H
P1.5    .EQU   095H
P1.6    .EQU   096H
P1.7    .EQU   097H
P2.0    .EQU   0A0H
P2.1    .EQU   0A1H
P2.2    .EQU   0A2H
P2.3    .EQU   0A3H;
P2.4    .EQU   0A4H;
P2.5    .EQU   0A5H;
P2.6    .EQU   0A6H;
P2.7    .EQU   0A7H;
P3.0    .EQU   0B0H;
P3.1    .EQU   0B1H;
P3.2    .EQU   0B2H;
P3.3    .EQU   0B3H;
P3.4    .EQU   0B4H;
P3.5    .EQU   0B5H;
P3.6    .EQU   0B6H;
P3.7    .EQU   0B7H;
;
TMOD    .EQU   089H
TCON    .EQU   088H
TL0     .EQU   08AH
TH0     .EQU   08CH
IE      .EQU   0A8H
IP      .EQU   0B8H
TR0     .EQU   08CH
TR1     .EQU   08EH
```

```
ET0     .EQU   0A9H
EA      .EQU   0AFH
;
;
```

Далее обратите внимание на три идущих последовательно директивы `.OGR` . Первая присваивает идущей после нее команде безусловного перехода на метку `WORK (LJMP WORK)` адрес 0. То есть, эта команда расположена первой в транслируемой программе, и именно с нее микроконтроллер при старте начнет свои действия — перейдет на выполнение инициализации ресурсов микроконтроллера, идущих с упомянутой метки.

Вторая директива `.OGR` устанавливает счетчик адресов транслируемой программы в 0Bh. Следовательно, первый байт идущей после нее команды `LJMP TIMER0` окажется расположенным именно по этому адресу. Как мы помним, по этому адресу должна располагаться первая команда подпрограммы обработки прерывания от таймера-счетчика 0. В нашем же случае здесь расположена команда безусловного перехода к этой подпрограмме. Как вы понимаете, это то же самое — перейдя после переполнения таймера на расположенную по этому адресу команду, микроконтроллер тут же сделает переход на требуемую подпрограмму. Но зачем я по адресу 0Bh разместил не всю подпрограмму, а лишь команду перехода на нее?

Ответ прост. Да, при переполнении таймера-счетчика 0 микроконтроллер прервет основную программу, спрячет в стек адрес следующей команды основной программы, которую он должен будет выполнять по завершении прерывания, и перейдет на выполнение команды с адреса 0Bh. Но не будем забывать, что с адреса 13h (т. е. через 8 байт от адреса 0Bh) должна начинаться подпрограмма обработки внешнего прерывания (от входа $\overline{\text{INT1}}$). Поскольку подпрограмма обработки прерывания от таймера-счетчика 0 гораздо длиннее 8 байт, она «ляжет» на то место, где должны находиться и подпрограмма обработки внешнего прерывания (от входа $\overline{\text{INT1}}$), и подпрограмма обработки прерывания от таймера-счетчика 1, и подпрограмма обработки прерывания от приемопередатчика. Поэтому, чтобы такого не происходило, по адресам 03h, 0Bh, 13h, 1Bh, 23h обычно располагают не сами подпрограммы обработки соответствующих прерываний, а команды переходов к этим подпрограммам. Последние же в таком случае могут располагаться в любом месте программы — все равно в ходе обработки прерывания контроллер до них доберется.

Конечно, в рассматриваемой подпрограмме мы не используем ни внешние прерывания от входов $\overline{\text{INT0}}$, $\overline{\text{INT1}}$, ни прерывания от таймера-счетчика 1, ни прерывания от приемопередатчика. Так что если бы с адреса 0Bh мы расположили не команду перехода на подпрограмму обработки прерывания от таймера-счетчика 0, а саму эту подпрограмму, ничего плохого ровным счетом не произошло бы. Но я специально поставил команду

перехода, чтобы продемонстрировать вам, как нужно располагать подпрограммы обработки прерываний в том случае, если используется больше одного прерывания — в адресах 03h, 0Bh, 13h, 1Bh, 23h нужно располагать не сами подпрограммы обработки соответствующих прерываний, а команды переходов к этим подпрограммам.

Третья директива `.OGR` устанавливает счетчик адресов транслируемой программы в 100h. Следовательно, идущая после нее метка `WORK`: окажется расположенной по этому адресу, и именно на команду, расположенную по этому адресу, микроконтроллер перейдет после выполнения команды `LJMP WORK`.

Перейдя туда, микроконтроллер выполнит следующие действия. Вначале командами `CLR TR0` и `CLR TR1` он остановит таймеры-счетчики. Правда, этого можно, конечно, и не делать — при старте они обычно все равно стоят. Но не стоит полагаться на значения по умолчанию — завтра вы перенесете эту программу на какой-нибудь новый микроконтроллер, а на нем по умолчанию они не будут гарантированно выключены, и вы убьете немало времени на поиск возникшей проблемы. Так что не бойтесь иногда перестраховываться — это не повредит.

Также страховочными являются следующие две команды, заносящие нули в регистры IE и IP. Вначале мы полностью запрещаем все прерывания и присваиваем им всем одинаковый низкий приоритет. Затем в регистр TMOD мы заносим число, зануляющее бит GATE0, устанавливающее таймер-счетчик 0 в режим таймера и задающее второй режим работы. Далее при помощи регистра DPTR мы предустанавливаем таймер, занося в TH0, TL0 число, равное 65535 – CHISLO1. Его значение и определяет частоту меандра. Если CHISLO1 = 100, то мы предустанавливаем таймер значением 64535, и как только он перейдет в режим счета, то спустя всего 100 мкс (на 12 МГц) он переполнится и вызовет прерывание. Еще почти 50 мкс будет выполняться сама подпрограмма обработки прерывания, следовательно, полупериод меандра составит 150 мкс, что соответствует частоте, примерно равной 3.3 кГц.

Аналогично рассуждая, можно получить, что если CHISLO1 = 2500, то полупериод меандра составит 2550 мкс, что соответствует частоте, примерно равной 190 Гц и т. д. Другими словами, изменяя значение константы CHISLO1, можно в довольно широких пределах варьировать частотой меандра.

Следующие три команды разрешают прерывания и запускают на счет таймер 0. После этого идет основная программа (в данном случае вместо нее мы поставили бесконечный цикл — команду `SJMP $`, которая предписывает контроллеру, если кто подзабыл, перейти на выполнение команды, расположенной по тому же адресу, что и текущая, т. е. на саму себя).

Итак, мы рассмотрели фрагмент основной программы, который инициализирует (настраивает) таймер-счетчик с системой прерываний и запускает таймер на счет. Теперь посмотрим, что же делает подпрограмма обработки прерывания от таймера 0 `TIMER0`.

Первыми двумя командами мы останавливаем таймер-счетчик и на всякий случай запрещаем прерывания. Далее мы сохраняем в стеке регистры, содержание которых изменяется в рассматриваемой подпрограмме. Затем читаем в бит переноса сигнал (нолик или единичку) из регистра-защелки линии P1.0. Прочитанное выводим в регистр-защелку линии P1.1, затем инвертируем и возвращаем проинвертированный сигнал в P1.0. Таким образом, каждое прерывание меняет уровень сигнала как на P1.0, так и на P1.1 на противоположный (с 0 на 1 и обратно). Обратите также внимание, что меандр на P1.1 инвертированный в сравнении с таковым на P1.0 — когда на первом 0, на втором 1, и наоборот.

Далее все просто — таймер-счетчик вновь предустанавливается, подобно тому, как это было сделано при его инициализации, восстанавливаются из стека регистры (в последовательности, строго обратной той, которая была при записи их в стек), разрешаются прерывания, и командой `SETB TR0` таймер-счетчик 0 вновь запускается на счет. Последней идет команда возврата из обработчика прерываний `RETI`.

6.4. Краткие выводы

Итак, подведем итоги. Помимо ресурсов, рассмотренных в предыдущих главах, в состав микроконтроллера входит обычно еще ряд дополнительных устройств, которые при правильном их использовании не только облегчают решение поставленных задач, но иногда даже помогают справиться с задачами, которые без их применения попросту неразрешимы. К ним (т. е. к этим устройствам) относятся таймеры-счетчики и последовательный приемопередатчик (подробно он будет рассмотрен в одной из последующих глав). Высокая эффективность их использования обеспечивается механизмом прерываний.

Прерывания позволяют микроконтроллеру выполнять основную задачу, не отвлекаясь на анализ состояния упомянутых устройств и сигналов на входах $\overline{\text{INT0}}$ и $\overline{\text{INT1}}$. Если прерывания разрешены, то микроконтроллер тут же «узнает» о том, что произошло переполнение того или иного таймера-счетчика, передан или принят байт приемопередатчиком или что на одном из упомянутых входов появился сигнал нулевого логического уровня (либо произошел перепад из 1 в 0, в зависимости от настроек). Сразу после этого микроконтроллер прерывает выполнение основной программы, сохранив в стеке адрес возврата в нее, и переходит на выполнение подпрограммы, которая должна быть расположена с адресов 03h и 13h (для сигна-

лов на входах $\overline{INT0}$ и $\overline{INT1}$), 0Bh и 1Bh (для переполнившихся таймеров-счетчиков 0 и 1) и 23h (для приемопередатчика). Эти подпрограммы носят названия обработчиков или подпрограмм обработки соответствующих прерываний. Позаботиться о том, чтобы они оказались в упомянутых адресах памяти программ — наша с вами задача.

При старте микроконтроллера все прерывания запрещены, и если мы собираемся их использовать, мы не только должны написать обработчики соответствующих прерываний, но и инициализировать систему прерываний, установив в единицы соответствующие биты в регистре IE, и присвоив всем используемым прерываниям высокий или низкий приоритет. При необходимости мы можем запретить как все прерывания одновременно, так и любые из них по выбору и в любом количестве.

В случае, если в системе разрешено более чем одно прерывание, из пришедших одновременно микроконтроллер начинает обрабатывать то, у которого более высокий приоритет. Если же приоритеты равны, то вначале обрабатывается прерывание от входа $\overline{INT0}$, затем от таймера-счетчика 0, затем от входа $\overline{INT1}$, затем от таймера-счетчика 1 и в последнюю очередь от приемопередатчика. Прервать выполняемую подпрограмму обработки прерывания может лишь прерывание с более высоким приоритетом, чем обрабатываемое. Остальные же ждут, когда подпрограмма обработки текущего прерывания будет завершена, и стоящая в ее конце команда `RETI` разрешит вызов обработчика следующего прерывания из очереди на обслуживание. Если вы забудетесь и вместо `RETI` поставите в конце подпрограммы обработки прерывания `RET`, то ни одно из пришедших после ее выполнения прерываний не сможет быть обслужено. Так что не забывайте в конце обработчиков прерываний ставить `RETI`.

Таймеры-счетчики являются самыми распространенными из периферийных устройств микроконтроллеров. В стандартных микроконтроллерах семейства x51 их два. Каждый из счетчиков может работать в 4-х различных режимах, а также считать как мегагерцовые импульсы (с выхода делителя тактовой частоты на 12), так и импульсы, поступающие на соответствующий вывод микроконтроллера. Кстати, отсюда и его название — таймер-счетчик: когда на его вход поступают сигналы извне, то это — счетчик внешних импульсов, а когда с тактового генератора — то таймер, формирующий, как было описано, те или иные задержки. В любой из таймеров-счетчиков можно перед началом работы записать число от 0 до 0FFFFh, а также любой из них можно программным путем остановить или запустить. Управление таймерами-счетчиками осуществляется при помощи битов, входящих в состав регистров TMOD (его адрес — 89h) и TCON (88h).

Таймеры-счетчики микроконтроллеров x51 имеют 4 режима работы. Однако для нас наибольшую практическую ценность представляют всего два из них — второй и третий. Во втором режиме (M1 = 0, M0 = 1) как $/C/\overline{T}0$,

так и C/$\overline{T}$1 работают следующим образом. Каждый из импульсов, поступающих на вход C/$\overline{T}$x, увеличивает на 1 содержимое регистра TLx. Когда происходит переполнение последнего (т. е. содержимое TLx меняется с 0FFh на 0), происходит увеличение на 1 регистра THx. Естественно, увеличение TLx и THx возможно только тогда, когда TRx в 1, и $\overline{INT}$x = 1 при соответствующем GATE = 1.

Когда происходит переполнение THx (т. е. содержимое THx меняется с 0FFh на 0), то устанавливается в 1 соответствующий флаг переполнения таймера TFx. Если написанная нами программа к этому моменту разрешила прерывания, то установка в 1 упомянутого флага автоматически вызовет соответствующую подпрограмму обработки данного прерывания.

В третьем режиме (M1 = 1, M0 = 0) C/$\overline{T}$0 и C/$\overline{T}$1 также работают одинаково. Каждый из импульсов, поступающих на вход C/$\overline{T}$x, увеличивает на 1 содержимое регистра TLx. Когда происходит переполнение последнего (т. е. содержимое TLx меняется с 0FFh на 0), то устанавливается в 1 соответствующий флаг переполнения таймера TFx. Одновременно с этим происходит занесение в TLx числа, хранящегося в регистре THx, при этом содержимое THx остается неизменным. И в этом случае установка в 1 соответствующего флага переполнения таймера TFx вызывает переход на выполнение подпрограммы обработки прерывания (если, естественно, прерывания были разрешены).

На всякий случай напомню, что занесение информации в регистры таймеров-счетчиков и системы прерываний должно осуществляться при помощи команд `MOV reg,#data8`. Адреса упомянутых регистров приведены выше. Если ваш ассемблер, подобно моему TASM'у, не знает об их существовании, эти адреса нужно объявить в разделе адресов и символических имен в начале программы.

Кроме того, биты регистра TCON входят в битовое адресное пространство микроконтроллеров семейства x51, следовательно, их можно устанавливать и сбрасывать командами `SETB` и `CLR`.

Глава 7. ПРАКТИЧЕСКИЕ ПРИМЕРЫ РАЗРАБОТКИ УСТРОЙСТВ НА МК X51

Наше знакомство с микроконтроллерами было бы неполным, если бы мы не ознакомились с несколькими примерами полностью законченных разработок. На этих примерах мы должны научиться проводить весь цикл разработки, с прорисовкой блок-схемы как всей разрабатываемой программы, так и отдельных подпрограмм, с написанием их текстов и т. д. Я предлагаю вашему вниманию несколько примеров разной степени сложности, как говорится, на разный вкус. Их назначение — не только показать вам, как это делается, но и создать основу для ваших самостоятельных разработок. Думаю, что приведенных примеров будет вполне достаточно, чтобы вы смогли выбрать из них тот, который максимально близок к стоящей перед вами задаче.

7.1. Противоугонное устройство на микроконтроллере семейства x51

Устройство разработано на основе конструкции, описание которой было опубликовано в журнале «Схемотехника», №1, 2000 г.

Несмотря на обилие появившихся в последние годы противоугонных систем (Alligator, Mongoose и т. д.), интерес к простым противоугонным системам не пропал. Причина проста — потенциальные угонщики хорошо знают промышленные системы, их уязвимые места, типичные ошибки при их установке и в состоянии угнать машину даже несмотря на функционирующую противоугонку. Конечно, не бывает систем защиты, с которыми не смог бы справиться подготовленный специалист-угонщик. Но если противостоящая система ему незнакома, вероятность ее взлома резко падает. С этой точки зрения самодельные системы, выполненные в единичных экземплярах и известные, чаще всего, только изготовившим их авторам, весьма эффективны.

Одна из идей состоит в том, чтобы установленная в режим охраны противоугонка никак себя внешне не проявляла, ее включение/отключение осуществлялось бы штатными средствами (т. е. теми, которые обычно ус-

тановлены на автомобиле в момент его продажи), а ее действие не вписывалось бы в типичные алгоритмы действия подобных систем. Этим требованиям во многом удовлетворяет устройство, схема которого приведена на **Рис. 7.1**. Его включение/отключение осуществляется штатным выключателем из тех, которыми включается вентилятор обогрева, ближний свет и т. д. При этом, однако, спустя несколько секунд после запуска автомобиля, когда «противоугонка» отключится, этот выключатель можно вернуть в исходное положение — не будешь же ездить с постоянно включенными фарами или отопителем.

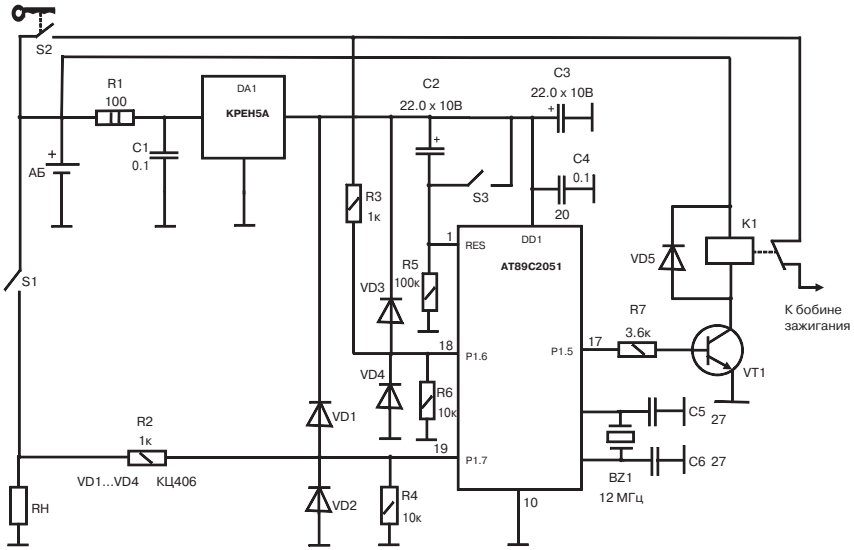


Рис. 7.1. Схема «противоугонки»

Основой устройства является микроконтроллер DD1 (AT89C2051 фирмы Atmel). Требуемые для DD1 5 В формирует стабилизатор DA1 (КРЕН5А), резистор R1 ограничивает на безопасном уровне протекающий через него ток. Один из выводов контроллера (в данном случае P1.5) сконфигурирован как выход, он управляет транзистором VT1 (КГ815А), в коллекторной цепи которого находится обмотка реле K1, разрывающего цепь зажигания. Выводы P1.6 и P1.7 являются входами, на них поступают ограниченные диодными ограничителями напряжения, возбуждающие бобину зажигания и (в нашем случае) включающие вентилятор отопителя Rн. К аккумуляторной батарее АБ устройство подключено постоянно. Остальные его элементы очевидны и не требуют пояснения.

Работает устройство следующим образом. Как только оно оказывается подключенным к питанию (или нажата кнопка сброса S3), микроконтроллер конфигурирует нужным образом свои выводы P1.5...P1.7 и устанавливает на выходе P1.5 логический 0. Транзистор VT1 закрывается, K1 обесточивается, и через его нормально замкнутые контакты бобина зажигания оказывается подключенной к выводу замка зажигания S2. После этого микроконтроллер начинает постоянно опрашивать состояние входа P1.6 на предмет определения момента включения зажигания. Как только он определил, что ключ зажигания повернут, и напряжение подано на бобину, он анализирует уровень сигнала на Rн (точнее, на P1.7). Если на P1.7 в этот момент оказывается 1, микроконтроллер трактует это как подтверждение того, что «противоугонный режим» включать не надо, сохраняет P1.5 в 0 и начинает опрашивать P1.6 с целью определения момента выключения зажигания (т. е. когда на P1.6 устанавливается 0). Вентилятор Rн после этого в любой момент можно выключить — его включение/выключение уже не влияет на работу устройства. Как только DD1 определит, что S2 разомкнулся, цикл повторится.

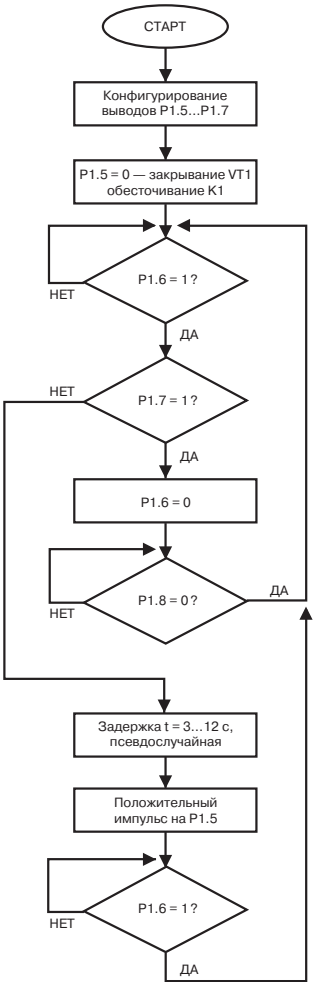


Рис. 7.2. Блок-схема программы противоугонной системы

Если в момент включения S2 на P1.7 окажется 0, DD1 трактует это как несанкционированный запуск двигателя, и спустя некоторое время (от 4 до 8 с, значение выбирается псевдослучайным образом) формирует на P1.5 секундный положительный импульс, открывающий реле и глушащий двигатель. После этого DD1 начинает опрашивать P1.6 с целью определения момента выключения зажигания, и как только определит, что S2 разомкнулся, цикл повторится. Блок-схема программы приведена на **Рис. 7.2**.

Как видите, она весьма проста. Собственно, в этом нет ничего удивительного — для ее описания понадобилось всего два абзаца. Но ее простота

— это не повод отказываться от привычки рисовать в начале разработки блок-схему разрабатываемой программы. Постарайтесь сформировать у себя эту привычку — она избавит вас от ненужных потерь времени.

Обратите внимание — приведенная блок-схема никак не привязана к особенностям микроконтроллеров семейства x51, т. е. в некотором смысле она универсальна. Иными словами, эта блок-схема подходит для любого контроллера, у которого есть как минимум один выход и два входа.

Пошли дальше. Как определить, включено или выключено зажигание, включен или выключен вентилятор? На первый взгляд довольно просто, надо опросить состояние входов P1.6 и P1.7 соответственно — если там нули, то выключено, если единицы, то включено. Но есть одна проблема, именуемая дребезгом контактов. Так, при переводе любого переключателя из состояния «выключено» в состояние «включено» и наоборот, всегда есть моменты, когда напряжение несколько раз скачкообразно меняется между своими крайними значениями (**Рис. 7.3**). В зависимости от конструкции тумблера момент дребезга может составлять от сотен микросекунд до десятков миллисекунд. Для надежной фиксации состояния выключателя мы должны уметь подавлять этот дребезг контактов. В допроцессорную эпоху с этой целью использовались специально разработанные схемы на простых логических микросхемах. С появлением микроконтроллеров выполнение этой функции было возложено на них, и подавление дребезга контактов в большинстве случаев осуществляется программным путем.

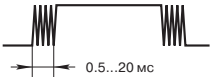


Рис. 7.3. К вопросу о подавлении дребезга контактов

А как реализовать это программное подавление дребезга? Да очень просто. Опросите состояние тумблера, запомните его, выждите от 10...20 до 50...100 мс и повторите опрос. Если при обоих опросах состояние тумблера одинаковое, то можно считать, что дребезг отсутствует или подавлен, и это состояние и есть результат опроса. Если состояние тумблера при первом и втором опросах оказалось различным, то повторите процедуру, пока состояние при обоих опросах не станет одинаковым.

В большинстве случаев оказывается достаточно двух опросов состояния тумблера, но в ответственных моментах количество опросов увеличивают до трех, а иногда и до четырех раз. Именно так я сделал в рассматриваемой в настоящем разделе программе (противоугонка — вещь ответственная, поэтому увеличение количества опросов при анализе состояния тумблеров в данном случае вполне оправдано). В случае анализа состояния кнопок на обычном изделии, например, на измерительном приборе, достаточно и двух опросов.

С учетом сказанного, блок-схема рассматриваемой программы выглядит следующим образом (**Рис. 7.4**). По-прежнему она никак не привязана к особенностям нашего микроконтроллера.

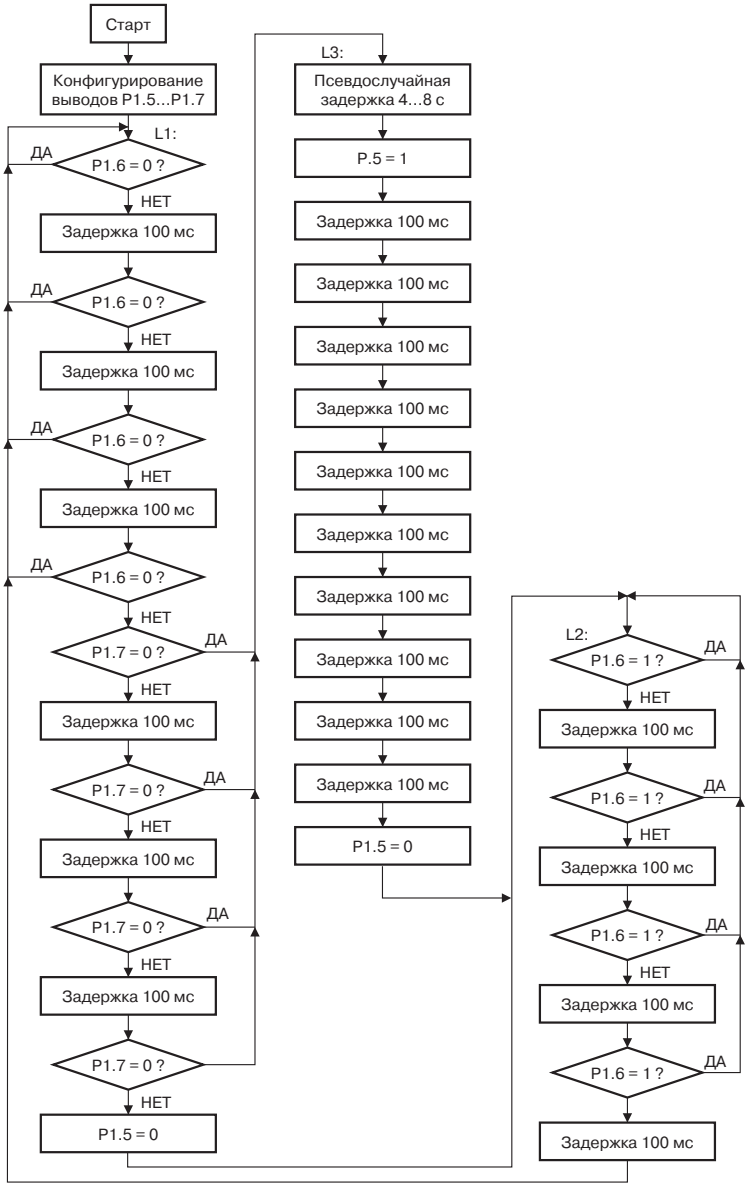


Рис. 7.4. Расширенная блок-схема программы противоугонной системы

Собственно, о блок-схеме программы говорить больше нечего. Дальше осталось самое простое — написать программу в соответствии с этой блок-схемой. Сейчас вам, возможно, покажется удивительным, но когда вы наберетесь некоторого опыта, вы убедитесь, что правильно составить блок-схему гораздо сложнее, чем написать программу, ей соответствующую. Первый процесс — творческий, второй же — скорее рутинный, почти механический.

Я постарался снабдить программу достаточным количеством комментариев, так что большая ее часть для вас должна быть совершенно очевидной. Как и в примере, приведенном в предыдущей главе, директивой `#INCLUDE «DEFS.INC»` я сообщил используемому мной ассемблеру TASM адреса уже знакомых нам, но незнакомых ему регистров и их отдельных битов.

Текст программы противоугонной системы

```
;
;       &&&&
```

```
        ANL    A,#01000000B
        JZ     L1                    ;ЕСЛИ P1.6=0, ЭТО ДРЕБЕЗГ
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#01000000B
        JZ     L1                    ;ЕСЛИ P1.6=0, ЭТО ДРЕБЕЗГ
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#01000000B
        JZ     L1                    ;ЕСЛИ P1.6=0, ЭТО ДРЕБЕЗГ
;
;ПРОВЕРКА НА ВКЛЮЧЕНИЕ ПЕЧКИ, Т.Е. НА P1.7=1
;
        MOV    A,P1
        ANL    A,#10000000B
        JZ     L3                    ;ЕСЛИ P1.7=0 (ВЫКЛ), ПЕРЕХОД
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#10000000B
        JZ     L3                    ;ЕСЛИ P1.7=0 (ВЫКЛ), ПЕРЕХОД
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#10000000B
        JZ     L3                    ;ЕСЛИ P1.7=0 (ВЫКЛ), ПЕРЕХОД
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#10000000B
        JZ     L3                    ;ЕСЛИ P1.7=0 (ВЫКЛ), ПЕРЕХОД
;
;ПЕЧКА ВКЛЮЧЕНА. ЕЩЕ РАЗ НА ВСЯКИЙ СЛУЧАЙ УСТАНОВИМ P1.5=0,
;ЗАТЕМ ИЩЕМ МОМЕНТ ВЫКЛЮЧЕНИЯ ЗАЖИГАНИЯ (P1.6=0)
;
        CLR    P1.5
;
;ПРОВЕРКА НА ВЫКЛЮЧЕНИЕ ЗАЖИГАНИЯ, Т.Е. НА P1.6=0
;
L2:
        MOV    A,P1
        ANL    A,#01000000B
        JNZ    L2                    ;ЕСЛИ P1.6=1, СНОВА ПРОВЕРЯТЬ
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#01000000B
        JNZ    L2                    ;ЕСЛИ P1.6=1, СНОВА ПРОВЕРЯТЬ
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#01000000B
        JNZ    L2                    ;ЕСЛИ P1.6=1, СНОВА ПРОВЕРЯТЬ
        LCALL  DEL100
        MOV    A,P1
        ANL    A,#01000000B
```

```
        JNZ    L2                    ;ЕСЛИ P1.6=1, СНОВА ПРОВЕРЯТЬ
        LCALL  DEL100
;
;НАЙДЕНО СОСТОЯНИЕ P1.6=0, Т.Е. ЗАЖИГАНИЕ ВЫКЛЮЧЕНО. ТЕПЕРЬ
;ВСЕ ПО НОВОЙ, Т.Е. НА СТАРТ
;
        LJMP   L1
;
;А ЭТО ПРИ УГОНЕ, Т.Е. ПРИ ВКЛЮЧЕНИИ ЗАЖИГАНИЯ ПРИ НЕВКЛЮЧЕННОЙ
;ПЕЧКЕ, Т.Е. P1.6=1, P1.7=0
;
L3:
        LCALL  PSEUDO
        SETB   P1.5                  ;ОТКРЫВАЕМ VT1, ВКЛЮЧАЕМ K1
        LCALL  DEL100                ;СНИМАЯ, ТАКИМ ОБРАЗОМ, ВЫСОКОЕ
        LCALL  DEL100                ;С КАТУШКИ ЗАЖИГАНИЯ
        LCALL  DEL100
        LCALL  DEL100
        LCALL  DEL100
        LCALL  DEL100
        LCALL  DEL100
        LCALL  DEL100
        LCALL  DEL100
        LCALL  DEL100                ;СЕКУНДНАЯ ЗАДЕРЖКА ПРИ СНЯТИИ
        LCALL  DEL100                ;ВЫСОКОГО
        CLR    P1.5                  ;ЗАКРЫВАЕМ VT1, ОТКЛЮЧАЕМ K1
        SJMP   L2                    ;НА ПОИСК МОМЕНТА ВЫКЛ. ЗАЖИГАНИЯ
;
;ПСЕВДОСЛУЧАЙНАЯ ЗАДЕРЖКА
;
PSEUDO:
        INC    R5
        MOV    A,R5
        MOV    DPTR,#BAZE
        MOVC   A,@A+DPTR             ;ПСЕВДОСЛУЧАЙНОЕ ЧИСЛО В АККУМУЛЯТОРЕ
        MOV    R4,A                  ;ПСЕВДОСЛУЧАЙНОЕ ЧИСЛО В R4
L100:   LCALL  DEL100                ;ЗАДЕРЖКА 100 МС
        DJNZ   R4,L100               ;ВЫПОЛНЯТЬ ПОКА R4 НЕ СТАНЕТ = 0
        RET
;
;ЗАДЕРЖКА 100 МС
;
DEL100:
        MOV    R7,#200
DLY1:   MOV    R6,#250
        DJNZ   R6,$
        DJNZ   R7,DLY1
        RET
;
;НАБОР ПСЕВДОСЛУЧАЙНЫХ ЧИСЕЛ
;
```

```
BAZE:
;
;256 random numbers. minimum value: 40, maximum value: 80
;
.DB     79,74,54,78,56,46,45,53,77,45,48,79,64,54,77,76
.DB     41,75,74,47,76,73,43,76,44,54,76,61,48,49,78,75
.DB     72,40,61,75,41,77,79,42,78,57,49,61,60,70,52,61
.DB     45,76,53,54,73,70,69,78,49,47,79,40,58,67,47,58
.DB     41,69,67,67,68,45,77,46,77,78,62,74,80,72,42,59
.DB     63,45,78,70,63,44,40,49,78,53,68,71,80,51,41,72
.DB     76,75,40,62,70,62,75,66,51,76,70,74,44,66,54,48
.DB     80,58,61,54,72,76,47,42,74,71,43,54,75,50,72,52
.DB     74,73,59,76,63,76,80,60,54,55,64,47,57,42,46,65
.DB     49,44,61,59,44,47,79,63,53,47,46,79,66,53,67,74
.DB     68,48,64,52,63,41,78,66,42,51,80,48,40,52,55,42
.DB     58,64,44,62,46,42,55,79,61,47,46,62,59,75,69,72
.DB     56,79,63,64,45,76,69,79,49,54,79,79,59,51,73,53
.DB     40,74,79,49,65,72,75,53,53,44,57,72,77,72,79,76
.DB     50,56,48,72,68,59,67,51,55,46,67,46,79,58,44,68
.DB     64,41,62,55,47,66,51,68,56,50,58,40,44,65,49,55
;
;
        .ORG   00400H
        .END
```

Пожалуй, единственное, что стоит прокомментировать в приведенной программе — процесс формирования псевдослучайной задержки. Начиная с метки `BAZE:`, в тексте программы идет массив из 256 псевдослучайных чисел из диапазона от 40 до 80, сформированный при помощи паскалевской программы, исходный текст которой приведен ниже.

Программа для формирования файла с псевдослучайными числами

```
var n, min, max, i : integer;
  name : string;
  t : text;

begin
  write («how many random numbers: «); readln (n);
  write («enter minimum value    : «); readln (min);
  write («enter maximum value    : «); readln (max);
  write («enter output file name : «); readln (name);
  assign (t, name);
  rewrite (t);
  randomize;
  writeln (t, «; «,n,» random numbers. minimum value: «,min,», maximum
value: «,max);
  for i := 1 to n do
    writeln (t, «.DB»#9, min+random(max-min+1));
  close (t);
end.
```

Подпрограмма формирования псевдослучайной задержки `PSEUDO` осуществляет следующие действия. При каждом ее вызове она увеличивает на 1 содержимое регистра R5, после чего переносит его в аккумулятор, и командой `MOVC A,@A+DPTR` считывает в последний число, порядковый номер которого в массиве `BAZE`: соответствует содержимому R5. Далее это число переносится в регистр R4, где оно играет роль счетчика — команда `DJNZ R4,L100` осуществит вызов идущей с метки `L100`: подпрограммы 100-миллисекундной задержки ровно то число раз, которое записано в R4. Поскольку оно лежит в пределах между 40 и 80, то время выполнения этого цикла займет от 4 до 8 секунд. Собственно, что и требовалось...

В заключение — небольшое замечание для тех, кто решит использовать описанную конструкцию по ее прямому назначению. У нее есть один недостаток — если ваша машина заводится после поворота ключа зажигания дольше, чем 4 секунды, описанная система ее не защитит. В этом случае вам необходимо снабдить ее еще каким-то дополнительным датчиком, индицирующим, что двигатель завелся, и отсчет псевдослучайной задержки осуществлять после того, как датчик просигналит, что двигатель работает. Соответственно, вам потребуется доработать и принципиальную схему устройства, и программу. Но вам это уже вполне по силам. Так что дерзайте.

7.2. Простой термостабилизатор на микроконтроллере AT89C2051

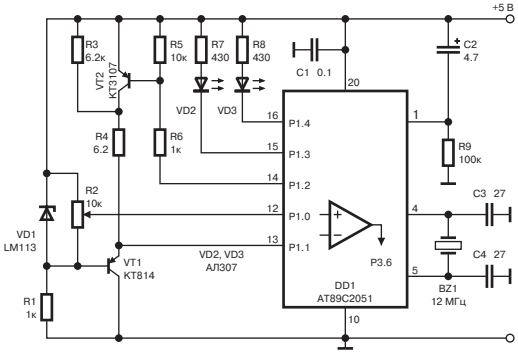


Рис. 7.5. Схема термостабилизатра

Второе устройство, которое мы с вами рассмотрим — простой маломощный термостабилизатор. Схема его приведена на **Рис. 7.5.**

В ряде случаев возникает необходимость поддерживать постоянной температуру объекта небольшого размера. Типичный пример — источник опорного напряжения (ИОН) для того или иного измерительного прибора. Очень многие ИОНы выпускаются в 4...8-выводных корпусах типа SOIC или микроSOIC, площадь их корпуса составляет десятые доли квадратного

сантиметра. В рассматриваемом случае мала не только площадь термостабилизируемой поверхности, но и мощность, требуемая для поддержания температуры на заданном уровне. В свете этого возникает желание сделать устройство как можно более простым, а также не содержащим громоздких терморезисторов. Приведенная на **Рис. 7.5** схема удовлетворяет этим требованиям. Она содержит небольшое количество элементов, а кроме того, функции как нагревателя, так и термодатчика в нем выполняет один и тот же элемент — транзистор VT1.

Идея заключается в следующем. В зависимости от состояния транзистора VT2, через VT1 может протекать ток 100 мкА (когда VT2 закрыт) или 100 мА (когда VT2 открыт). Пропуская малый ток, мы переводим VT1 в режим измерения температуры *p-n*-перехода — напомню, увеличение температуры эмиттерного перехода транзистора на 1°С приводит к снижению падения напряжения на нем (при неизменном токе через него) примерно на 2.2...2.4 мВ. Когда через VT1 мы пропускаем большой ток, на нем рассеивается мощность 3.8 В × 0.1 А = 0.38 Вт, что увеличивает температуру коллекторного перехода рассматриваемого транзистора.

Дальше нам нужен только компаратор, на один из входов которого будет подан сигнал с эмиттера VT1, а на второй — опорное напряжение, соответствующее заданной температуре. Пусть для определенности сигналы будут сфазированы таким образом, чтобы при температуре перехода ниже заданной на выходе компаратора устанавливался бы нулевой уровень, а если выше — единичный. Давайте также сделаем так, чтобы микроконтроллер опрашивал состояние выхода этого компаратора с периодичностью, например, 100 мс, и при обнаружении на выходе нуля (т. е. состояния, когда VT1 недогрет) до конца рассматриваемого 100-миллисекундного интервала он установил бы 100-миллиамперный ток через VT1, при котором температура коллекторного перехода повысится. Если же в начале этого интервала на выходе компаратора будет обнаружена единица, то пусть микроконтроллер до конца интервала задает через VT1 малый 100-микроамперный ток, при котором транзистор будет остывать. Чередуя в зависимости от температуры транзистора интервалы его нагрева и охлаждения, мы смогли бы поддерживать температуру *p-n*-перехода на заданном уровне.

Так-то оно так, скажете вы, но для реализации описанного алгоритма нам потребуются не только транзисторы и микроконтроллер, но и компаратор. И полученная при этом схема, содержащая уже две микросхемы, вряд ли может считаться такой уж простой. Согласны?

Да, согласен. Но не все так плохо, как кажется на первый взгляд. Дело в том, что помимо совсем уж стандартных микроконтроллеров семейства x51, которые мы рассматривали до сих пор, фирма Atmel выпускает их разновидность, упакованную в 20-выводный корпус, у которой отсутствуют

выводы линий портов P0 и P2, но зато есть встроенный компаратор. Входами этого компаратора «по совместительству» являются две линии порта P1 (P1.0 и P1.1), а выход соединен со входной линией порта P3.6. Для использования P1.0 и P1.1 в качестве входов компаратора нужно предварительно в их выходные буферы занести единички. Если же вы хотите эти линии использовать как входные или выходные линии порта, не забудьте на плате предусмотреть соединение их с шиной питания через резисторы номиналом 10...20 кОм. Однажды я забыл это сделать, и полдня пытался понять, почему одна и та же программа в плате с 40-выводным контроллером нормально работала, а в плате с 20-выводным — никак. А все оказалось до обидного просто — я не установил эти резисторы, и выходы P1.0, P1.1, оканчивающиеся транзисторами с открытым стоком, естественно не могли переключать в единицы соединенные с ними входы КМОП-мультиплексора.

Итак, вы узнали, что помимо как бы совсем стандартных микроконтроллеров x51 существуют их модификации, включающие, в частности, встроенный компаратор. Скажу по секрету, именно для этого я и разместил в этой главе рассматриваемое устройство — по части простоты оно здорово уступает термометру/термостату DS1821. Но на этом примере вы увидите, как можно использовать упомянутый встроенный компаратор — есть задачи, где он оказывается весьма кстати. Так что давайте запомним, что он присутствует в микросхемах AT89C1051, AT89C2051 и AT89C4051 и пойдем дальше. Точнее, вернемся к принципиальной схеме.

Движок переменного резистора R2, включенного параллельно прецизионному источнику опорного напряжения 1.22 В, соединим с неинвертирующим входом компаратора (P1.0), а эмиттер транзистора VT1 — с инвертирующим входом. Выход компаратора соединен с P3.6 внутри микроконтроллера, так что нам об этом соединении беспокоиться не надо, оно уже есть. Базу транзистора VT2, управляющего током через VT1, соединим через резистор R6 с выводом P1.2 микроконтроллера. Цепи сброса и тактового генератора DD1 аналогичны тем, которые имеются в рассмотренных ранее 40-выводных контроллерах семейства x51. Отмечу также, что поскольку у рассматриваемых в настоящем подразделе 20-выводных микроконтроллеров отсутствуют выводы портов P2 и P0, они не могут работать с внешней памятью программ и данных. В связи с этим у AT89C1051, AT89C2051 и AT89C4051 нет выводов ALE, $\overline{PSEN}$, $\overline{EA}$, последний из которых у 40-выводных микроконтроллеров мы соединяли через 20-килоомный резистор с шиной питания.

Светодиоды VD2 и VD3 нужны лишь для демонстрации того, что устройство работает — один из них загорается тогда, когда транзистор VT1 нагревается 100-миллиамперным током, а второй — когда через транзистор пропускается малый 100-микроамперный ток. При включении вначале

некоторое время горит лишь первый из упомянутых светодиодов (пока VT1 еще недогрет). Далее наступает момент, когда сперва редко, но затем все чаще и чаще загорается второй светодиод (светодиоды работают в противофазе, т. е. когда один горит, другой погашен, и наоборот). Наконец, когда устанавливается режим стабилизации, светодиоды включаются поочередно, и промежуток свечения каждого из них с течением времени уже не меняется.

Ну вот и все о принципиальной схеме устройства. Теперь надо бы нарисовать блок-схему требуемой программы, а затем написать последнюю. Давайте этим и займемся.

Блок-схема программы приведена на **Рис. 7.6.** Она настолько проста, что комментировать ее не имеет смысла, все и так очевидно. Я привел ее лишь для того, чтобы еще раз напомнить, что рисовать блок-схему программы перед написанием ее текста всегда полезно, даже если она столь примитивна. Пока вы учитесь, вы будете делать ошибки даже в самых простых программах, так что не пренебрегайте тем, что поможет вам минимизировать их число. А правильно разработанная блок-схема программы — самое эффективное средство против серьезных ошибок.

Далее приведен текст самой программы. Она снабжена достаточным количеством комментариев, так что описывать ее нет необходимости.

Рис. 7.6. Блок-схема программы термостабилизатора

Текст программы термостабилизатора

```
;
;       &&&&
```

```
;               *****
;
NACHPZU .EQU   0000H
NACHOZU .EQU   0000H
AD00    .EQU   NACHOZU+26H
;
;       НАЧАЛО ПРОГРАММЫ:
;               *****
;
        .ORG   NACHPZU
;
;
START:
        MOV    SP,#07H
        MOV    P1,#0FFH
        MOV    P3,#0FFH
LSTART:
        SETB   P1.2        ;ЗАКРЫЛИ VT2, ЧЕРЕЗ VT1 100 МКА
        LCALL  DEL10MS
        MOV    C,P3.6      ;В CY - СОСТОЯНИЕ ВЫХОДА КОМПАРАТОРА
        JC     PEREGREV    ;ЕСЛИ CY=1, VT1 ПЕРЕГРЕТ
NEDOGREV:                  ;НИЖЕСЛЕДУЮЩЕЕ ДЕЛАЕМ ПРИ НЕДОГРЕВЕ
        CLR    P1.2        ;ОТКРЫЛИ VT2, ПРОПУСКАЕМ ЧЕРЕЗ VT1 100 МА
        CLR    P1.3        ;ЗАЖГЛИ СВЕТОДИОД НА P1.3
        SETB   P1.4        ;ПОГАСИЛИ СВЕТОДИОД НА P1.4
        SJMP   D90         ;НА 90-МИЛЛИСЕКУНДНУЮ ЗАДЕРЖКУ
PEREGREV:                  ;НИЖЕСЛЕДУЮЩЕЕ ДЕЛАЕМ ПРИ ПЕРЕГРЕВЕ
        SETB   P1.2        ;ЗАКРЫЛИ VT2,ПРОПУСКАЕМ ЧЕРЕЗ VT1 100 МКА
        CLR    P1.4        ;ЗАЖГЛИ СВЕТОДИОД НА P1.4
        SETB   P1.3        ;ПОГАСИЛИ СВЕТОДИОД НА P1.3
D90:
        LCALL  DEL90MS     ;90-МИЛЛИСЕКУНДНАЯ ЗАДЕРЖКА
        SJMP   LSTART      ;ЗАЦИКЛИВАНИЕ ПРОГРАММЫ
;

;               ПОДПРОГРАММЫ ЗАДЕРЖЕК
;                          *****
;
;ЗАДЕРЖКА 1 МС
;
DEL1MS:
        MOV    R1,#25H
LPEX:   MOV    R2,#18H
LPIN:   DJNZ   R2,LPIN
        DJNZ   R1,LPEX
        RET
;
;ЗАДЕРЖКА 10 МС
;
DEL10MS:
        ACALL  DEL1MS
```

```
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        ACALL  DEL1MS
        RET
;
;ЗАДЕРЖКА 90 МС
;
DEL90MS:
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        ACALL  DEL10MS
        RET
;
        .ORG   0800H
        .END
```

И последнее — как настроить описанное устройство на заданную температуру стабилизации. Как я уже говорил в одной из первых глав, чаще всего микроконтроллеры на платах устанавливают в панельках, дабы их можно было легко извлекать из них для программирования, а затем вставлять обратно. При наладке описанного устройства микроконтроллер нужно удалить из панельки. Затем подайте на схему питание, и измерьте напряжение между выводами резистора R2, оно должно быть примерно равно 1.22 В. Далее измерьте температуру окружающей среды и падение напряжения между базой и эмиттером VT1. Предположим, температура равна 22°С, а падение напряжения на эмиттерном переходе VT1 составляет 590 мВ. Далее найдем разность между требуемой температурой стабилизации (пусть она равна 35°С) и текущей температурой: 35 – 22 = 13°С. Затем умножим полученные 13°С на 2.3 мВ/°С: 13°С × 2.3 мВ/°С = 29.9 мВ ≈ 30 мВ. (Напомню, что 2.3 мВ/°С — это среднее значение крутизны зависимости падения напряжения на эмиттерном переходе от температуры перехода.) С учетом того, что при нагревании падение напряжения на *p-n*-переходе уменьшается, в нашем случае при температуре 35°С оно должно быть равно 590 – 30 = 560 мВ. Вращением движка резистора R2 установим на выводе P1.0 отсутствующего по-

ка в панельке микроконтроллера напряжение 560 мВ относительно базы транзистора VT1. После этого отключим питание, вернем микроконтроллер с занесенной в него программой в панельку, и снова включим питание — устройство начнет работать, о чем, как говорилось, можно судить по состоянию светодиодов VD2 и VD3.

7.3. Подпрограммы целочисленного многобайтного умножения и деления

Прежде чем мы рассмотрим следующее устройство, нам нужно познакомиться с тем, как микроконтроллеры реализуют такие математические операции, как умножение и деление многобайтных чисел. В самом деле, со сложением или вычитанием все просто — для их реализации есть команды `ADD`, `ADDC` и `SUBB`. Если нам, к примеру, нужно сложить два трехбайтных числа (напомню, что диапазон изменений трехбайтных чисел — от 0 до 0FFFFFFh = 16777215d), одно из которых расположено в регистрах R2, R1 и R0 (младший байт в R0), а другое — в R5, R4 и R3, и результат нужно сохранить в R6, R5, R4 и R3, это делается следующим образом:

Пример подпрограммы сложения двух трехбайтных чисел

```
SUMMA3&3:
  MOV   A,R3
  ADD   A,R0
  MOV   R3,A
  MOV   A,R4
  ADDC  A,R1
  MOV   R4,A
  MOV   A,R5
  ADDC  A,R2
  MOV   R5,A
  MOV   A,R6
  ADDC  A,#0
  MOV   R6,A
  RET
```

Отмечу, что последнее сложение (регистра R6 с нулем) нужно для того, чтобы учесть перенос, который возникнет, если сумма слагаемых окажется больше 16777215 (т. е. сумма выйдет за рамки трехбайтного числа и будет четырехбайтной). Если кому смысл последнего предложения оказался не совсем понятным, сложите при помощи Windows-калькулятора два довольно больших трехбайтных числа, например 0FF2594h = 16721300d и 0FE7FF4h = 16678900d. В результате вы должны получить 1FDA588h = 33400200d — число явно четырехбайтное (младший байт — 88h, второй — 0A5h, третий — 0FDh и четвертый — 1h).

По образу и подобию подпрограммы сложения двух трехбайтных чисел, строятся подпрограммы сложения чисел любой длины — от двухбайтных до, к примеру, 256-байтных (где могут быть использованы последние — ума не приложу). Аналогично строятся и подпрограммы многобайтного вычитания:

Пример подпрограммы вычитания двух трехбайтных чисел

```
RAZN3&3:
  CLR   C
  MOV   A,R3
  SUBB  A,R0
  MOV   R3,A
  MOV   A,R4
  SUBB  A,R1
  MOV   R4,A
  MOV   A,R5
  SUBB  A,R2
  MOV   R5,A
  RET
```

Здесь надо помнить, что поскольку у микроконтроллеров x51 отсутствует команда вычитания без учета переноса (заема), то перед тем, как осуществить первое вычитание, вы должны принудительно обнулить бит переноса командой `CLR C`.

Ну а как умножать и делить? Да, у нашего микроконтроллера есть команда умножения `MUL AB`, но она распространяется только на два однобайтных числа. А если хотя бы один из сомножителей двухбайтный? О команде деления я вообще помолчу — использовать ее для многобайтного деления весьма проблематично, во всяком случае я плохо себе представляю, где она там может пригодиться.

Итак, вопрос формулируется следующим образом: как при помощи системы команд, не содержащей специальные команды для умножения и деления, реализовать умножение и деление многобайтных чисел? Да очень просто! Нужно только вспомнить, как нас в начальной школе учили умножать и делить «в столбик», да соединить полученные тогда знания с теми, которые у вас появились в процессе знакомства с микроконтроллерной техникой. Не верите? Тогда смотрите и старайтесь не терять нить рассуждений.

Ниже записан пример умножения «в столбик» двух десятичных чисел — 11011 и 10101. Почему я для примера выбрал именно такие числа, вы поймете чуть позже. Пока же убедитесь, что этот пример выполнен в пол-

ном соответствии с тем, как всех вас, и меня вместе с вами, учили в начальной школе.

Пример умножения двух пятизначных десятичных чисел

```
     11101          (множимое)
   x 10111          (множитель)
     11101          (первое частичное произведение)
    11101           (второе частичное произведение)
   11101            (третье частичное произведение)
  00000             (четвертое частичное произведение)
 11101              (пятое частичное произведение)
 111332221          (произведение)
```

Обратите внимание на то, как выполнялось умножение. Вначале мы умножили множимое на младший, самый правый разряд множителя, и записали результат этого умножения в виде первого частичного произведения прямо под множителем. Кстати, поскольку в этом правом разряде множителя стояла единица, то первое частичное произведение оказалось в точности равно множимому (думаю, нет смысла долго задерживаться на том, что умножение любого числа на 1 оставляет его неизменным).

После этого мы взяли второй справа разряд множителя (в нашем примере там тоже оказалась единица) и умножили множимое на эту единицу. Однако получившееся при этом второе частичное произведение мы записали не прямо под первым, а со сдвигом влево на 1 десятичный разряд. Далее мы повторили операцию для третьего справа разряда множителя, и третье частичное произведение записали под вторым также со сдвигом влево (уже на два разряда относительно первого). То же мы проделали и для четвертого, и для пятого разрядов множителя, здесь все должно быть очевидно. Обратите внимание только на то, что поскольку четвертый разряд множителя — нуль, то четвертое частичное произведение состоит из нулей (что на 0 не умножай...). Ну и после всего этого нам лишь осталось сложить между собой все пять полученных частичных произведений (с учетом сдвигов) — ответ готов!

Ну и что нового, спросите вы? Еще ничего. До тех пор, пока я не сказал, что принципы умножения многоразрядных чисел одинаковы и в десятичной, и в шестнадцатеричной, и в двоичной системе. В любой из этих систем мы вначале умножаем множимое на самый младший (правый) разряд множителя и получаем первое частичное произведение; дальше умножаем множимое на второй разряд, сдвигаем полученное второе частичное произведение на одну позицию влево и складываем с первым частичным произведением. К полученной сумме прибавляем сдвинутое

на два разряда влево третье частичное произведение, затем — сдвинутое на три разряда влево четвертое частичное произведение, сдвинутое на четыре разряда влево пятое частичное произведение и т.д. Словом, ровно то же самое, что показано в примере умножения двух пятизначных чисел, разве что там мы вначале записали все частичные произведения, а затем их сложили, а в только что сказанном я каждое полученное частичное произведение предложил суммировать с первым сразу после сдвига на нужное число разрядов. Но такое изменение последовательности действий, как вы понимаете, никоим образом не может повлиять на окончательный результат.

Да, это так, скажете вы, возможно для того, чтобы умножить два многоразрядных числа друг на друга, действительно нужно всего лишь получить упомянутые частные произведения, сдвинуть их правильным образом да сложить друг с другом. Но ведь эти частные произведения получаются путем *умножения* множимого на тот или иной разряд множителя. А с умножением-то у микроконтроллера, как упоминалось выше, туговато! Так как быть?

Никак. Вспомните для начала, что микроконтроллеры оперируют не с десятичными, не с шестнадцатеричными, а с двоичными числами, составленными *только* из ноликов и единичек. А дальше... догадались? Если нет, то обратитесь еще раз к примеру умножения на стр. 242. Вот он, наш спасительный канатик: частичное произведение множимого на разряд множителя равно нулю, если этот разряд равен нулю, и равно самому (ударение на последнем слоге) множителю, если разряд равен единице. Других вариантов частичных произведений в двоичной системе быть не может! Или нуль, или само множимое, третьего, к счастью, не дано.

Итак, теперь у нас есть все, чтобы реализовать умножение друг на друга двоичных чисел любой длины (или, что то же самое, любой разрядности). Вначале анализируем младший разряд (бит) множителя — если он равен 0, то первое частичное произведение (назовем его накапливаемой суммой) равно 0, если он равен 1, то первое частичное произведение (накапливаемая сумма) равно множимому. Далее сдвигаем множимое на 1 бит *влево* и смотрим второй *справа* бит множителя. Если он равен 0, ничего не делаем, если равен 1— прибавляем это сдвинутое множимое к накапливаемой сумме, определенной на предыдущем шаге. Далее снова сдвигаем множимое на 1 бит *влево* и смотрим третий справа бит множителя. Если он равен 0, ничего не делаем, если равен 1 — прибавляем это дважды сдвинутое множимое к сумме, полученной на предыдущем шаге. И так до тех пор, пока не переберем все биты множителя. Сказанное иллюстрируется **Рис. 7.7**.

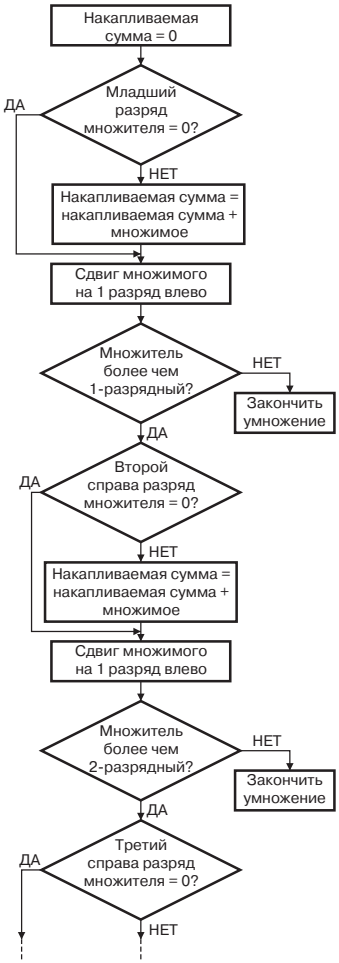


Рис. 7.7. Алгоритм умножения двоичных чисел

Чтобы окончательно убедить вас в том, что вы постигли эту премудрость, я предлагаю вам перемножить в соответствии с приведенным на **Рис. 7.7** алгоритмом два 4-разрядных двоичных числа — 1101b (13d) и 1001b (9d).

Пример умножения двух четырехзначных двоичных чисел

```
   1101   (множимое)
 × 1001   (множитель)
   1101   (первое част.произведение или накапл.сумма)
 [1101]   (сдвинули множитель, но не прибавили)
   1101   (накапл. сумма осталась без изменений)
[1101]    (сдвинули множитель, но не прибавили)
   1101   (накапл. сумма осталась без изменений)
1101      (сдвинули множитель, и прибавили)
1110101   (произведение)
```

Итак, поскольку крайний справа бит множителя равен 1, в качестве накапливаемой суммы берем значение множимого. Далее сдвигаем множимое на 1 позицию влево и анализируем второй справа бит множителя. Этот бит равен 0, следовательно, суммировать сдвинутое множимое с накапливаемой суммой не надо (чтобы это подчеркнуть, я заключил это сдвинутое множимое в квадратные скобки; накапливаемая сумма, как видите, осталась без изменений). Далее сдвигаем множимое еще на 1 позицию влево и анализируем третий справа бит множителя. Этот бит опять-таки равен 0, следовательно, суммировать сдвинутое множимое с накапливаемой суммой также не надо (снова я заключил это сдвинутое множимое в квадратные скобки; накапливаемая сумма, как видите, опять осталась без изменений). Еще раз сдвигаем множимое влево и смотрим, чему равен четвертый справа бит множителя. Он равен 1, следовательно, это уже трижды сдвинутое множимое нужно прибавить к накапливаемой сумме. Не забудьте только, что мы

работаем с двоичными числами, у которых 1+1 не 2, а 0 с единицей в бите переноса.

Итак, что же мы получили в результате? Произведение 13 и 9 оказалось равным 1110101b или (проверьте по Windows-калькулятору) 117d. Похоже на правду?

Вот здесь я закончу рассказ о том, как в микроконтроллерах осуществляется многобайтное умножение, и в качестве примера приведу две подпрограммы, одна из которых осуществляет перемножение двух трехбайтных чисел с получением шестибайтного результата, а другая — двух двухбайтных чисел с получением четырехбайтного результата. Вы можете, конечно, не разбираться с ними, а просто использовать их в своих программах — работать они будут, никуда не денутся. Но тем, кто собирается серьезно работать с микроконтроллерами, я бы посоветовал постараться понять их, это поможет вам в дальнейшем при написании других программ. И перед тем, как оставить вас один на один с этими подпрограммами, скажу об одной особенности их реализации. В них вместо сдвига множимого влево после каждого его суммирования с накапливаемой суммой (или пропуска суммирования, если соответствующий бит равен 0) осуществляется сдвиг вправо накапливаемой суммы. При перемножении сдвигающаяся вправо накапливаемая сумма еще к тому же «вытесняет» множитель. Подобные программистские «выверты» позволили осуществить такие алгоритмы умножения, при которых задействованы лишь регистры микроконтроллера, и нет обращений к внутренней или внешней памяти данных. Благодаря этому скорость выполнения умножения достаточно высока, т. к. операции с регистрами гораздо быстрее аналогичных операций с ячейками ОЗУ.

Подпрограмма перемножения двух трехбайтных чисел с получением шестибайтного результата

```
;
;ПОДПРОГРАММА УМНОЖЕНИЯ ДВУХ ТРЕХБАЙТНЫХ ЧИСЕЛ С
;ПОЛУЧЕНИЕМ НА ВЫХОДЕ 6-БАЙТНОГО РЕЗУЛЬТАТА.
;СОМНОЖИТЕЛИ В B(СТ.БАЙТ)R1R0 И В R4(СТ.БАЙТ)R3R2,
;РЕЗУЛЬТАТ В R7R6R5R4R3R2.
;
MUL63:
  MOV   A,#24              ;МНОЖИТЕЛЬ ТРЕХБАЙТНЫЙ
  MOV   R7,#0              ;ПОСЛЕ 24 СДВИГОВ ЭТИ
  MOV   R6,#0              ;3 БАЙТА БУДУТ ТРЕМЯ
  MOV   R5,#0              ;СТАРШИМИ БАЙТАМИ ПРОИЗВЕДЕНИЯ
MUL63_1:
  PUSH  ACC
  MOV   A,R2               ;В A - МЛ.БАЙТ МНОЖИТЕЛЯ
```

```
  RRC   A                   ;АНАЛИЗ ЕГО МЛ. БИТА
  JNC   MUL63_2             ;ЕСЛИ ОН НЕ 0, НЕ СУММИРОВАТЬ
  MOV   A,R5                ;ПЛЮСУЕМ МНОЖИМОЕ К НАКАПЛИВАЕМОЙ
  ADD   A,R0                ;СУММЕ, КОТОРАЯ ПОСЛЕ ЗАВЕРШЕНИЯ
  MOV   R5,A                ;П/П И БУДЕТ ПРОИЗВЕДЕНИЕМ
  MOV   A,R6
  ADDC  A,R1
  MOV   R6,A
  MOV   A,R7
  ADDC  A,B
  MOV   R7,A
MUL63_2:
  MOV   A,R7
  RRC   A
  MOV   R7,A
  MOV   A,R6
  RRC   A
  MOV   R6,A
  MOV   A,R5
  RRC   A
  MOV   R5,A
  MOV   A,R4
  RRC   A
  MOV   R4,A
  MOV   A,R3
  RRC   A
  MOV   R3,A
  MOV   A,R2                ;СДВИНУЛИ МНОЖИТЕЛЬ И
  RRC   A                   ;НАКАПЛИВАЕМУЮ СУММУ
  MOV   R2,A                ;НА 1 БИТ ВПРАВО
  POP   ACC
  DJNZ  ACC,MUL63_1
  RET
;
```

Подпрограмма перемножения двух двухбайтных чисел с получением четырехбайтного результата

```
;
;ПОДПРОГРАММА УМНОЖЕНИЯ ДВУХ ДВУХБАЙТНЫХ ЧИСЕЛ С
;ПОЛУЧЕНИЕМ НА ВЫХОДЕ 4-БАЙТНОГО РЕЗУЛЬТАТА.
;СОМНОЖИТЕЛИ В R1(СТ.БАЙТ)R0 И В R3(СТ.БАЙТ)R2,
;РЕЗУЛЬТАТ В R7R6R5R4.
;
MUL42:
  MOV   A,#16               ;МНОЖИТЕЛЬ ДВУХБАЙТНЫЙ
  MOV   R7,#0               ;ПОСЛЕ 24 СДВИГОВ ЭТИ
  MOV   R6,#0               ;4 БАЙТА БУДУТ
  MOV   R5,#0               ;БАЙТАМИ ПРОИЗВЕДЕНИЯ
  MOV   R4,#0
MUL42_1:
```

```
  PUSH  ACC
  MOV   A,R2              ;В А - МЛ.БАЙТ МНОЖИТЕЛЯ
  RRC   A                 ;АНАЛИЗ ЕГО МЛ. БИТА
  JNC   MUL42_2           ;ЕСЛИ ОН НЕ 0, НЕ СУММИРОВАТЬ
  MOV   A,R6              ;ПЛЮСУЕМ МНОЖИМОЕ К НАКАПЛИВАЕМОЙ
  ADD   A,R0              ;СУММЕ, КОТОРАЯ ПОСЛЕ ЗАВЕРШЕНИЯ
  MOV   R6,A              ;П/П И БУДЕТ ПРОИЗВЕДЕНИЕМ
  MOV   A,R7
  ADDC  A,R1
  MOV   R7,A
MUL42_2:
  MOV   A,R7
  RRC   A
  MOV   R7,A
  MOV   A,R6
  RRC   A
  MOV   R6,A
  MOV   A,R5
  RRC   A
  MOV   R5,A
  MOV   A,R4
  RRC   A
  MOV   R4,A
  MOV   A,R3
  RRC   A
  MOV   R3,A
  MOV   A,R2              ;СДВИНУЛИ МНОЖИТЕЛЬ И
  RRC   A                 ;НАКАПЛИВАЕМУЮ СУММУ
  MOV   R2,A              ;НА 1 БИТ ВПРАВО
  POP   ACC
  DJNZ  ACC,MUL42_1
  RET
;
```

Как видите, подпрограммы похожи друг на друга и довольно просты, во всяком случае занимают менее 40 строк ассемблерного текста каждая. Написав со временем небольшую библиотечку подобных подпрограмм, в которые войдут многобайтные умножение, деление, преобразование из двоичного представления в двоично-десятичное и наоборот, табличное преобразование и т. д., вы будете перетаскивать ее из программы в программу, не тратя в дальнейшем время на создание подобных «мелочей», после чего не задумываясь будете использовать эти подпрограммы по мере необходимости.

Теперь перейдем к делению многобайтных чисел. Думаю, некоторые из вас уже догадались, что оно осуществляется на основе вычитания делителя из делимого со сдвигом делителя вправо после каждого вычитания. Поясню это поподробнее.

Пусть нам нужно разделить 782 на 23. Нетрудно посчитать, что частное от деления 782 на 23 будет равно 34. Но это в хорошо знакомых нам десятичных числах, с использованием калькулятора или бумаги с ручкой. А как в двоичном представлении?

Ниже приведен пример интересующего нас процесса деления. Он начинается с того, что делитель записывается под делимым, причем так, чтобы старший бит делителя был под старшим битом делимого. Разрядность делителя больше разрядности делимого, поэтому при вычитании второго из первого будем принимать во внимание только те биты делимого, под которыми есть биты делителя, остальные биты делимого мы будем как бы «не замечать».

Пример деления двух двоичных чисел

```
  1100001110b  (делимое)
- 10111   b    (делитель)
  ----------
  0000101110   (перв.разность;CY = 0,ст.бит частного = 1)
  [10111]      (сдвинули делитель, но не вычли, ибо при
               вычитании его из первой разности
               CY будет = 1)
  0000101110   (разн.осталась без изменений, и
               следующий бит частного = 0)
   [10111]     (сдвинули делитель, но не вычли,
               ибо при вычитании CY будет = 1)
  0000101110   (разн.осталась без изменений, и
               следующий бит частного = 0)
     [10111]   (сдвинули делитель, но не вычли,
               ибо при вычитании CY будет = 1)
  0000101110   (разн.осталась без изменений, и
               следующий бит частного = 0)
      10111    (сдвинули делитель; вычитать можно)
  0000000000   (пятая разность;при вычитании CY
              будет = 0, поэтому следующий
               бит частного = 1)
     [10111]   сдвинули делитель, но не вычли,
               т.к.пятая разность = 0,из нее ничего не
              вычтешь,последний бит частного = 0)
  ----------
  0000000000   (остаток, в нашем случае он = 0)
    100010b    (частное)
```

Осуществим вычитание делителя из делимого. Если в результате вычитания бит заема CY окажется равен 0, т. е. заем не потребуется, установим первый полученный таким образом бит частного в 1. Если бит заема в результате такого вычитания оказался бы равным 1, то вычитание на этом этапе делать нельзя, и полученный бит частного нужно взять равным 0.

Обращаю ваше внимание на то, что микроконтроллеру для того, чтобы проверить, будет ли бит заема в результате вычитания равным 1, обязательно нужно это самое вычитание совершить. Следовательно, о том, что вычитать не нужно, он узнает лишь после того, как завершит эту операцию. Поэтому, если CY оказался равным 1, нужно к полученной в результате вычита-

ния делителя из делимого разности прибавить делитель, дабы восстановить делимое. Поэтому описываемый алгоритм деления носит название деления с восстановлением остатка. Существуют и иные алгоритмы реализации деления, но их рассмотрение выходит за рамки настоящей главы.

Итак, мы произвели (или не произвели) первое вычитание, и получили старший бит частного, равный соответственно 1 или 0. Далее сдвинем делитель на один бит вправо. Снова осуществим вычитание делителя из делимого. По-прежнему, если в результате вычитания бит заема CY окажется равен 0, т. е. заем не потребуется, установим следующий по счету бит частного в 1. Если бит заема в результате такого вычитания оказался бы равным 1, то вычитание на этом этапе делать нельзя, нужно восстановить делимое и полученный бит частного взять равным 0. В рассматриваемом примере деления (стр. 248) вычитание на этом этапе даст CY = 1, поэтому его делать не нужно, и второй (слева) бит частного оказывается равным 0. Чтобы подчеркнуть, что на этом этапе вычитание не выполняется, я заключил делитель в квадратные скобки.

Снова сдвинем делитель и проверим, можно ли осуществить вычитание. В рассматриваем нами случае опять нельзя, в связи с чем третий (слева) полученный бит частного оказался равным 0. Еще раз сдвинем делитель и повторим описанные операции — вычитать опять нельзя, четвертый бит частного, как и два предыдущих, оказался нулем. После пятого сдвига в результате вычитания бит заема не установится в 1, поэтому вычитать можно, и мы осуществим его, установив в 1 пятый (слева) бит частного.

До каких пор мы должны сдвигать делитель вправо? До тех пор, пока его младший бит не установится под младшим битом делимого. Именно это и произойдет, когда мы в очередной раз сдвинем делитель вправо (напомню, что все сказанное относится к примеру на стр. 248). Разность, полученная в результате осуществленного на предыдущем этапе вычитания, равна 0, следовательно, вычитать из нее делитель нельзя, и младший бит частного оказался равным 0. Что мы в итоге получили? Десятичный эквивалент двоичного 100010b — это 34d, то самое число, которое мы и должны были получить! Вот и все! Как видите, вполне доступно для понимания, не сложнее рассмотренного выше умножения.

Да, а что будет, если делимое не делится нацело? Например, на 23 нам нужно было бы разделить не 782, а 788? Отвечаю — после завершения вычитаний последняя разность будет содержать остаток, в нашем случае он равен 6. Если хотите — можете использовать его для округления результата — когда он больше половины делителя, увеличьте частное на 1, в противном случае попросту отбросьте его.

Далее приведены подпрограммы деления шестибайтного числа на трехбайтное и четырехбайтного на двухбайтное. Они построены на основе описанного выше алгоритма с восстановлением остатка. Отмечу, что для пер-

вой из них обязательно, чтобы трехбайтный делитель был бы больше числа из старших трех байт шестибайтного делимого, а для второй — чтобы двухбайтный делитель был бы больше числа их старших двух байт четырехбайтного делимого. Если это будет не так, то произойдет переполнение частного. Это значит, что в первом случае частное будет больше самого большого из трехбайтных чисел (0FFFFFFh), а во втором — больше самого большого из двухбайтных (0FFFFh). Проверьте это самостоятельно на Windows-калькуляторе. Отмечу также, что приведенные ниже подпрограммы не проверяют, удовлетворяют ли делимое и делитель приведенным ограничениям, поэтому при использовании их в своих программах проанализируйте, возможна ли ситуация, которая приведет к переполнению частного, и если да, то примите те или иные меры. Например, перед выполнением деления проверьте соотношение между делителем и старшей половиной делимого, и если делитель больше, продумайте, каким образом микроконтроллер сообщит вам о возникновении подобной нештатной ситуации.

Подпрограмма деления шестибайтного числа на трехбайтное с получением трехбайтного частного

```
;ПОДПРОГРАММА ДЕЛЕНИЯ ШЕСТИБАЙТНОГО ЧИСЛА ИЗ R7R6R5R4R3R2
;НА ТРЕХБАЙТНОЕ В BR1R0 С ПОЛУЧЕНИЕМ ТРЕХБАЙТНОГО ЧАСТНОГО
;В R4R3R2
;
DIV63:
        MOV    A,#24
DCLK_63:
        PUSH   ACC
        CLR    C
        MOV    A,R2
        RLC    A
        MOV    R2,A
        MOV    A,R3
        RLC    A
        MOV    R3,A
        MOV    A,R4
        RLC    A
        MOV    R4,A
        MOV    A,R5
        RLC    A
        MOV    R5,A
        MOV    A,R6
        RLC    A
        MOV    R6,A
        MOV    A,R7
        RLC    A
        MOV    R7,A
        PUSH   PSW
PER1_63:
```

```
        CLR    C
        MOV    A,R5
        SUBB   A,R0
        MOV    R5,A
        MOV    A,R6
        SUBB   A,R1
        MOV    R6,A
        MOV    A,R7
        SUBB   A,B
        MOV    R7,A
        JC     PER2_63
        POP    PSW
PER3_63:
        INC    R2
        SJMP   PER4_63
PER2_63:
        POP    PSW
        JC     PER3_63
        MOV    A,R5
        ADD    A,R0
        MOV    R5,A
        MOV    A,R6
        ADDC   A,R1
        MOV    R6,A
        MOV    A,R7
        ADDC   A,B
        MOV    R7,A
PER4_63:
        POP    ACC
        DJNZ   ACC,DCLK_63
        RET
;
```

Подпрограмма деления четырехбайтного числа на двухбайтное с получением двухбайтного частного

```
;ПОДПРОГРАММА ДЕЛЕНИЯ ЧЕТЫРЕХБАЙТНОГО ЧИСЛА ИЗ R7R6R5R4
;НА ДВУХБАЙТНОЕ В R1R0 С ПОЛУЧЕНИЕМ ДВУХБАЙТНОГО ЧАСТНОГО
;В R5R4
;
DIV42:
        MOV    A,#16
DCLK_42:
        PUSH   ACC
        CLR    C
        MOV    A,R4
        RLC    A
        MOV    R4,A
        MOV    A,R5
        RLC    A
        MOV    R5,A
```

```
        MOV    A,R6
        MOV    R6,A
        MOV    A,R7
        RLC    A
        MOV    R7,A
        PUSH   PSW
PER1_42:
        CLR    C
        MOV    A,R6
        SUBB   A,R0
        MOV    R6,A
        MOV    A,R7
        SUBB   A,R1
        MOV    R7,A
        JC     PER2_42
        POP    PSW
PER3_42:
        INC    R4
        SJMP   PER4_42
PER2_42:
        POP    PSW
        JC     PER3_42
        MOV    A,R6
        ADD    A,R0
        MOV    R6,A
        MOV    A,R7
        ADDC   A,R1
        MOV    R7,A
PER4_42:
        POP    ACC
        DJNZ   ACC,DCLK_42
        RET
;
```

И перед тем, как мы обратимся к следующему примеру, я познакомлю вас еще с одной «программистской хитростью», часто используемой теми, кто пишет программы для микроконтроллеров. Все рассмотренные нами выше арифметические операции с многобайтными числами — сложение, вычитание, умножение и деление — работают с целыми числами. А если нам нужно оперировать с дробными? Пусть нам необходимо умножить полученное в результате той или иной операции целое двухбайтное число на дробное, например, на 1.41421 (квадратный корень из 2). Как это сделать?

На самом деле осуществить это довольно просто. Нужно только подобрать два двухбайтных числа, отношение которых равно нашему дробному числу.

Разделите 56570 на 40001. Что получится? Правильно, 1.4142146 (восьмой и последующие знаки после точки отбрасываем). А корень из двух с точностью до того же седьмого знака равен 1.4142135. Погрешность при

замене одного числа на другое равна 1.08226E-6, или около 0.0001%, чего вполне достаточно для подавляющего большинства решаемых задач. Поэтому, если вам нужно умножить имеющийся результат на корень из двух, умножьте его вначале на 56570, а затем разделите на 40001. Перед этой операцией не забудьте удостовериться в том, что исходный результат не превышает 46340 — большее число после умножения на 1.4142135 превысит максимальное двухбайтное число (0FFFFh = 65535d), поэтому после деления произведения на 40001 произойдет переполнение частного.

Обратите внимание, я вначале умножил число на 56570, а только затем разделил его на 40001, а не наоборот. Почему? Да потому, что если вначале осуществить деление, вы изрядно потеряете в точности. Попробуйте убедиться в этом сами, используя все тот же Windows-калькулятор. Вычисление проводите в HEX-счислении: 56570d = 0DCFAh, 40001d = 9C41h. Убедились? Так что запомните — чтобы не потерять в точности, сначала умножайте, а затем делите, но не наоборот.

Вот и все. Как видите, ничего сложного, разве что трудно без навыка подобрать подходящую пару чисел, отношение которых равно с хорошей степенью точности нашей дроби. Но дорогу осилит идущий — десять минут сидения за Windows-калькулятором, и вы найдете требуемую пару. А после пятого такого подбора эта операция и вовсе перестанет создавать для вас какие-либо трудности — на подбор этих чисел вы будете затрачивать от силы две минуты.

7.4. Милливольтметр постоянного тока на АЦП AD7894 и МК семейства x51

Умение умножать и делить придется очень кстати в рассматриваемом примере — милливольтметре постоянного тока на основе 14-битного АЦП AD7894.

Вообще рассматриваемый АЦП довольно быстродействующий — цикл измерения и считывания из него информации составляет порядка 10 мкс. Однако младшие 3...4 бита нестабильны от измерения к измерению, и если вам надо получать при измерении стабильные 13...14 битов, вы должны осуществлять операцию накопления с последующим усреднением — например, осуществить сотню измерений, все результаты сложить и затем разделить на 100. Правда, на практике обычно накапливают не 10 или 100 измерений, а 16, 32, 64, 128 или 256. Как вы думаете, почему? Правильно, разделить накопленный результат на число, равное той или иной степени двойки, гораздо быстрее, чем на любое иное. Деление на 16 сводится к четырем последовательным сдвигам результата на один бит вправо, на 32 — к пяти сдвигам, на 64 — к шести сдвигам, на 128 — к семи сдвигам. Ну а раз-

делить результат на 256 и вовсе проще пареной репы — нужно всего-навсего отбросить его младший байт.

Опыт показывает, что если не прибегать к особо тщательным мерам по фильтрации результата, усреднения по 16...32 результатам оказывается достаточно, чтобы в 12-битных АЦП подобного типа получать стабильные 10 битов, а в 14-битных — стабильные 12. Для получения 14 стабильных битов в 14-битных АЦП (или 12 в 12-битных) приходится усреднять по 128...256 результатам. При этом, конечно, от 10-микросекундного быстродействия мало что остается, но все равно скорость измерения оказывается повыше, чем в большинстве сигма-дельта АЦП.

Схема устройства приведена на **Рис. 7.8.**

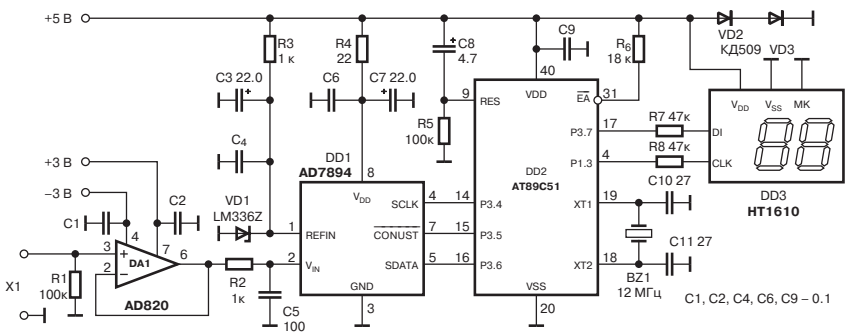


Рис. 7.8. Схема милливольтметра на АЦП AD7894

На ней вы увидите все те же микроконтроллер и HT1610. Однако есть и некоторое «ноу-хау», заключающееся в каскаде, стоящем перед АЦП. История его появления следующая.

Когда я впервые попытался использовать рассматриваемый АЦП, я с удивлением обнаружил, что он нелинеен, и при измерении занижает результаты в средней части динамического диапазона на 100...150 единиц, что во много раз больше допустимых по паспорту погрешностей. Мои обращения к службе технической поддержки у фирмы-дистрибьютора ни к чему не привели — там подтвердили, что каких-либо явных ошибок мной не сделано, и предложили обменять микросхему как бракованную. Однако спустя пару дней после этого предложения из беседы с одним из опытных разработчиков я узнал, что проблема состоит в устройстве выборки/хранения на входе рассматриваемого АЦП. В момент кратковременного подключения конденсатора к выходу каскада, на котором мы производим измерения, ток заряда конденсатора «просаживает» выход, даже если каскад этот состоит из повторителя напряжения на стандартном ОУ (операционный усилитель). Увы, о подобном источнике ошибок разработчики АЦП

нигде в своей документации даже словом не обмолвились. Так же как и о том, как с ним бороться.

Выход оказался довольно прост — между ОУ и входом АЦП пришлось поставить интегрирующую цепочку из килоомного резистора и 100-пикофарадной емкости. После этого характеристика АЦП (зависимость считанного кода от измеряемого напряжения) стала линейной. Правда, у конкретного образца микросхемы она оказалась еще смещенной на 28 единиц (т. е. описывающая эту зависимость прямая пересекала ось ординат не в начале координат, как это должно было быть, а 28-ю единицами ниже). Поэтому к результату измерения мне пришлось еще добавлять 28 единиц. Вот так мне удалось почти полностью «вылечить» AD7894.

Кстати, подобными болезнями в той или иной степени страдают не только AD7894 и схожие с ней 12-битные АЦП от Analog Devices, но и аналогичные АЦП других производителей — MAX187, MAX1241 от Maxim, LTC1286, LTC1292 от Linear Technology, SP8528, SP8531 от Sipex и ряд других. Поэтому я и включил в рассмотрение этот пример — здесь мы не только производим измерение и отображаем измеренное, но и исправляем некоторые присущие используемым АЦП «глюки», да простят нас создатели этих микросхем за такие неуважительные слова в адрес их изделий.

Ну вот и все об аппаратной части рассматриваемого устройства. Перейдем к программе, «оживляющей» это «железо».

Программа для вольтметра на AD7894 и МК семейства x51

```
;
;       &&&&
```

```
DATIND1 .EQU   P3.7
;
NACHPZU .EQU   0000H
NACHOZU .EQU   0000H
AD00    .EQU   NACHOZU+26H
;
;       НАЧАЛО ПРОГРАММЫ:
;              *****
;
        .ORG   NACHPZU
;
;
START:
        LCALL  IZMN2563    ;14-БИТНЫЙ РЕЗУЛЬТАТ В R5R4
        MOV    A,R4
        ADD    A,#28
        MOV    R4,A
        MOV    A,R5
        ADDC   A,#0
        MOV    R5,A        ;РЕЗУЛЬТАТ = РЕЗУЛЬТАТ + 28 ДЕС.
        MOV    DPTR,#24814
        MOV    R3,DPH
        MOV    R2,DPL      ;DPTR=R3R2=24814
        LCALL  MUL42       ;14-БИТ.РЕЗ.*24814 В R7R6R5R4
;
;ДАЛЕЕ РЕЗУЛЬТАТ НАДО РАЗДЕЛИТЬ НА 16383
;
        MOV    DPTR,#16383
        MOV    R3,DPH
        MOV    R2,DPL      ;DPTR=R3R2=16383
        LCALL  DIV42       ;РЕЗУЛЬТАТ В 10*МВ (от 0 до 24814) В R5R4
        MOV    R3,R5
        MOV    R2,R4
        LCALL  BN2BCD      ;R3R2 - РАСПАКОВАЛИ В R6R5R4
;
;РАСПАКОВКА НА ЭКРАН В HT1610
;
        MOV    A,#0FH
        LCALL  SIMBOL1     ;10-Й
        MOV    A,#0FH
        LCALL  SIMBOL1     ;9-Й
        MOV    A,#0FH
        LCALL  SIMBOL1     ;8-Й
        MOV    A,#0FH
        LCALL  SIMBOL1     ;7-Й
        MOV    A,#0FH
        LCALL  SIMBOL1     ;6-Й
;
        MOV    A,R6
        ANL    A,#00001111B
        LCALL  SIMBOL1     ;5-Й
```

```
        MOV   A,R5
        SWAP  A
        ANL   A,#00001111B
        LCALL SIMBOL1        ;4-Й
        MOV   A,R5
        ANL   A,#00001111B
        LCALL SIMBOL1        ;3-Й
        MOV   A,R4
        SWAP  A
        ANL   A,#00001111B
        LCALL SIMBOL1        ;2-Й
        MOV   A,R4
        ANL   A,#00001111B
        LCALL SIMBOL1        ;1-Й
;
        SJMP  START
;
SIMBOL1:
        ANL   A,#00001111B
        CJNE  A,#0,SIMB11
        MOV   A,#10
        SJMP  SIMB12
SIMB11: CJNE  A,#0FH,SIMB12
        MOV   A,#0
SIMB12: CLR   CLKIND1
        SWAP  A
        ACALL BIT1
        ACALL BIT1
        ACALL BIT1
        ACALL BIT1
        RET
;
BIT1:   RLC   A
        MOV   DATIND1,C      ;ВЫВ. ДАННЫХ В ЖК-ДИСПЛЕЙ
        SETB  CLKIND1        ;ИМПУЛЬС ЗАЩЕЛКИВАНИЯ
        CLR   CLKIND1
        RET
;
;
;ПОДПРОГРАММА ИЗМЕРЕНИЯ СИГНАЛА С АЦП С УСРЕДНЕНИЕМ ПО 256
;ИЗМЕРЕНИЯМ.
;
IZMN2563:
        MOV   R7,#0
        MOV   R6,#0
        MOV   R3,#0
        MOV   R2,#0
IZMN256C:
        LCALL ADC7894
        MOV   A,R6
        MOV   R6,A
```

```
        MOV    A,R7
        ADDC   A,R5
        MOV    R7,A
        MOV    A,R3
        ADDC   A,#0
        MOV    R3,A
        DJNZ   R2,IZMN256C
        MOV    R5,R3
        MOV    R4,R7
        RET
;
;               ПОДПРОГРАММЫ УМНОЖЕНИЯ
;
;П/П MUL21M ОСУЩЕСТВЛЯЕТ УМНОЖЕНИЕ ЦЕЛЫХ ДВОИЧНЫХ ЧИСЕЛ
;БЕЗ ЗНАКА ФОРМАТА 16*8=24 (БЕЗ ПРОВЕРКИ СОМНОЖИТЕЛЕЙ НА
;НОЛЬ). МНОЖИТЕЛЬ-В R1, МНОЖИМОЕ-В R3R2, ПРОИЗВЕДЕНИЕ-В (R1R5R4)
;ИСПОЛЬЗУЮТСЯ ВСЕ РЕГИСТРЫ,КРОМЕ R0,R6,R7, СОХРАНЯЮТСЯ R2,R3.
;
MUL21M:
        MOV    A,R1
        MOV    B,R3
        MUL    AB
        MOV    R5,A        ;МЛ. БАЙТ ПРОИЗВ. СОХР. В R5
        MOV    A,R1        ;МНОЖИТЕЛЬ В ОСВОБ. АККУМУЛЯТОР
        MOV    R1,B        ;СТ. БАЙТ ПРОИЗВ. СОХР. В R1
        MOV    B,R2        ;МЛ. БАЙТ МНОЖИМОГО В B
        MUL    AB
        MOV    R4,A        ;МЛ. БАЙТ ОКОНЧ. РЕЗ-ТАТА В R4
        MOV    A,R5
        ADD    A,B
        MOV    R5,A        ;СР. БАЙТ - СУММА МЛ. БАЙТА ВТОРОГО И СТ.
;БАЙТА ПЕРВОГО ПРОИЗВ.
        MOV    A,R1
        ADDC   A,#0
        MOV    R1,A        ;СТ. БАЙТ - СТ. БАЙТ ВТОРОГО И ПЕРЕНОС
        RET
;
;ПОДПРОГРАММА УМНОЖЕНИЯ ДВУХ ДВУХБАЙТНЫХ ЧИСЕЛ
;ИЗ (R5R4) И (R3R2), РЕЗУЛЬТАТ В R7R6R5R4
;
MUL42:
        MOV    R1,R5
        MOV    R0,R4
;
;ПОДПРОГРАММА УМНОЖЕНИЯ ДВУХ ДВУХБАЙТНЫХ ЧИСЕЛ
;ИЗ (R1R0) И (R3R2), РЕЗУЛЬТАТ В R7R6R5R4
;
MUL42_0:
        LCALL  MUL21M      ;РЕЗ. В R1R5R4
        MOV    A,R1
        MOV    R1,R0       ;R0 <- R1
```

```
        MOV    R0,A        ;В R0 -СТ.БАЙТ ПЕРВОГО ПРОИЗВЕДЕНИЯ
        MOV    R6,R4
        MOV    R7,R5       ;ПЕРВ. ПРОИЗВ. СОХР. В R0R7R6
        LCALL  MUL21M
        MOV    A,R5
        ADD    A,R6
        MOV    R5,A
        MOV    A,R1
        ADDC   A,R7
        MOV    R6,A
        MOV    A,#0
        ADDC   A,R0
        MOV    R7,A
        RET
;
;ПРЕОБРАЗОВАНИЕ ИЗ ДВОИЧНОГО В ДВОИЧНО-ДЕСЯТИЧНЫЙ ФОРМАТ
;ВХОДНЫЕ ДАННЫЕ В R3R2, ВЫХОДНЫЕ В R6R5R4, R0 И R1 СОХРАНЯЮТСЯ
;
BN2BCD:                    ;R3R2 -> R6R5R4, R0 И R1 СОХР.
        MOV    R7,#10H
        MOV    A,#0
        MOV    R4,A
        MOV    R5,A
        MOV    R6,A
BN2CKL: MOV    A,R2
        ADD    A,R2
        MOV    R2,A
        MOV    A,R3
        ADDC   A,R3
        MOV    R3,A
;ДВОИЧНО-ДЕСЯТИЧНОЕ УДВОЕНИЕ СУММЫ С УЧЕТОМ ПЕРЕНОСА:
        MOV    A,R4
        ADDC   A,R4
        DA     A
        MOV    R4,A
        MOV    A,R5
        ADDC   A,R5
        DA     A
        MOV    R5,A
        MOV    A,R6
        ADDC   A,R6
        DA     A
        MOV    R6,A
;ПРОВЕРКА КОНЦА ЦИКЛА:
        DJNZ   R7,BN2CKL
        RET
;
;              ПОДПРОГРАММА ДЕЛЕНИЯ ЦЕЛЫХ ЧИСЕЛ
;                          ******
;
;П/П DIV42 ОСУЩЕСТВЛЯЕТ ДЕЛЕНИЕ ЦЕЛЫХ ДВОИЧНЫХ ЧИСЕЛ БЕЗ
```

```
;ЗНАКА ФОРМАТА 32:16=16.МЕТОД ДЕЛЕНИЯ-С ВОССТАНОВЛЕНИЕМ
;ОСТАТКА.ВХОДНЫЕ ПАРАМЕТРЫ- ДЕЛИМОЕ В R7R6R5R4,ДЕЛИТЕЛЬ
;- В R3R2.ВЫХОДНЫЕ ПАРАМЕТРЫ- ЧАСТНОЕ В R5R4,ОСТАТОК - В
;R7R6,CY=0 - ПРИЗНАК ПЕРЕПОЛНЕНИЯ ЧАСТНОГО.ИСПОЛЬЗУЮТСЯ
;ВСЕ РЕГИСТРЫ И СТЕК,R3R2 - СОХРАНЯЕТСЯ.
;
DIV42:  MOV    R0,#10H
DCLK:   LCALL  HL2
        MOV    R1,0D0H     ;СОХРАНИМ PSW В R1
        LCALL  DE2
        JNC    PER1
        LCALL  INXH
;ВЫЧИТАНИЕ ВЫЧИТАЕМОГО ИЗ УМЕНЬШАЕМОГО
PER1:   LCALL  HLMNBC
        JC     PER2
        MOV    0D0H,R1     ;ВОССТАНОВИМ PSW ИЗ R1
PER3:   LCALL  INXD
        SJMP   PER4
;ПРОВЕРКА БИТА CY
PER2:   MOV    0D0H,R1
        JC     PER3
;ВОССТАНОВЛЕНИЕ УМЕНЬШАЕМОГО
        LCALL  DADB
;ПРОВЕРКА КОНЦА ЦИКЛА
PER4:   DJNZ   R0,DCLK
        RET
;

;              ПОДПРОГРАММЫ ЗАДЕРЖЕК
;                          *****
;
;ЗАДЕРЖКА 20 МС
;
DEL20:
        MOV    R1,#0FDH
LPEX:   MOV    R2,#026H
LPIN:   DJNZ   R2,LPIN
        DJNZ   R1,LPEX
        RET
;
;ЗАДЕРЖКА 600 МС
;
DEL05S:
        ACALL  DEL02S
        ACALL  DEL02S
        ACALL  DEL02S
        RET
;
;ЗАДЕРЖКА 200 МС
;
DEL02S:
```

```
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        ACALL  DEL20
        RET
;
;ПОДПРОГРАММКИ
;
HLMNBC: CLR    C
        MOV    A,R6
        SUBB   A,R2
        MOV    R6,A
        MOV    A,R7
        SUBB   A,R3
        MOV    R7,A
        RET
;
DADB:   MOV    A,R6
        ADD    A,R2
        MOV    R6,A
        MOV    A,R7
        ADDC   A,R3
        MOV    R7,A
        RET
;
DE2:    CLR    C
        MOV    A,R4
        RLC    A
        MOV    R4,A
        MOV    A,R5
        RLC    A
        MOV    R5,A
        RET
HL2:    CLR    C
        MOV    A,R6
        RLC    A
        MOV    R6,A
        MOV    A,R7
        RLC    A
        MOV    R7,A
        RET
;
INXD:   MOV    A,R4
        ADD    A,#1
        MOV    R4,A
```

```
        MOV    A,R5
        ADDC   A,#0
        MOV    R5,A
INXDEN: RET
;
INXH:   MOV    A,R6
        ADD    A,#1
        MOV    R6,A
        MOV    A,R7
        ADDC   A,#0
        MOV    R7,A
INXHEN:RET
;
;ОПРОС АЦП ADC7894. РЕЗУЛЬТАТ В РЕГИСТРАХ R5,R4
;
ADC7894:
        SETB   DATADC      ;P3.6
        SETB   CSADC       ;P3.5
        CLR    CLKADC      ;P3.4
ADC_12:
        CLR    CSADC       ;CS=0 - START
        MUL    AB          ;4 МКС НА 12 МГЦ
        MUL    AB          ;4 МКС
        SETB   CSADC       ;CS=1 - КОНЕЦ СТАРТОВОГО ИМПУЛЬСА
                           ;ДЛЯ AD7894
        MUL    AB          ;4 МКС
        MUL    AB          ;4 МКС
        MOV    A,#0
        MOV    B,A
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ НУЛЕВОГО (16-ГО) БИТА
        MOV    B.7,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ НУЛЕВОГО (15-ГО) БИТА
        MOV    B.6,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ СТАРШЕГО (14-ГО) БИТА
        MOV    B.5,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 13-ГО БИТА
        MOV    B.4,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 12-ГО БИТА
        MOV    B.3,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 11-ГО БИТА
        MOV    B.2,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 10-ГО БИТА
        MOV    B.1,C
;
```

```
        ACALL  CLK_ADC     ;ЧТЕНИЕ 9-ГО БИТА
        MOV    B.0,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 8-ГО БИТА
        MOV    ACC.7,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 7-ГО БИТА
        MOV    ACC.6,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 6-ГО БИТА
        MOV    ACC.5,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 5-ГО БИТА
        MOV    ACC.4,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 4-ГО БИТА
        MOV    ACC.3,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 3-ГО БИТА
        MOV    ACC.2,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ 2-ГО БИТА
        MOV    ACC.1,C
;
        ACALL  CLK_ADC     ;ЧТЕНИЕ МЛАДШЕГО(1-ГО) БИТА
        MOV    ACC.0,C
;
        MOV    R4,A
        MOV    A,B
        ANL    A,#00111111B;ЗАНУЛЕНИЕ 16 И 15 БИТОВ
        MOV    R5,A
;
        MUL    AB          ;4 МКС НА 12 МГЦ
        MUL    AB          ;4 МКС
        MUL    AB          ;4 МКС
        MUL    AB          ;4 МКС
        RET
;
CLK_ADC:
        SETB   CLKADC      ;
        MOV    C,DATADC    ;ЧТЕНИЕ БИТА В CY
        CLR    CLKADC
        RET
;
        .ORG   01000H
        .END
```

На самом деле она довольно проста. Вначале мы вызываем подпрограмму `IZMN2563`. Она делает следующее — 256 раз вызывает подпрограмму запуска АЦП и считывания из него результата измерения, накапливая их в регистрах R3 (старшие биты), R7 и R6. Счетчик до 256 организован в

регистре R2. Затем старшие два байта накопленной суммы переносятся в регистры R5 и R4, а младший отбрасывается — напомню, что так мы осуществляем деление накопленной суммы на 256.

После этого к хранимому в R5 и R4 результату мы прибавляем 28, о необходимости чего говорилось несколькими абзацами выше. Затем нам необходимо произвести масштабирование результата. Что это такое?

Как вы помните, используемый АЦП — 14-битный, т. е. считываемый результат выдается им в виде числа от 0 до 16383 (от 0 до 3FFFh). Следовательно, напряжению, равному опорному (в нашем случае оно, измеренное при помощи хорошего поверенного вольтметра постоянного тока, оказалось равным 2481.4 мВ), соответствует результат 16383 единиц. Но мы ведь хотим увидеть на дисплее при этом не 16383, а 2481 (или 2481.4), коль скоро мы договорились, что рассматриваемое устройство — милливольтметр постоянного тока, а не просто абстрактный измеритель входного сигнала. Поэтому перед отображением нам необходимо умножить полученный результат на 2481/16383 (или на 24814/16383, отделив затем при отображении крайний справа знак точкой). Эти действия осуществляются при помощи подпрограмм `MUL42` и `DIV42`. Они, особенно первая, несколько отличаются от тех, которые мы рассмотрели в предыдущем подразделе. Для тех, кто действительно хочет научиться писать программы на ассемблере, рекомендую разобраться с тем, как работают подпрограммы — это будет неплохой практикой.

Кстати, еще раз обращаю ваше внимание — вначале я умножил результат на 24814, после чего разделил его на 16383, а не наоборот. Причина такой последовательности действий — стремление не потерять в точности вычислений.

Дальше все совсем просто — мы переносим масштабированный результат в регистры R3 и R2, после чего подпрограммой `BN2BCD` преобразуем его в двоично-десятичную форму и отображаем на HT1610. Далее командой `SJMP START` зацикливаем программу — вольтметр будет непрерывно измерять и отображать измеренное, пока на него будет подаваться питающее напряжение.

7.5. Милливольтметр постоянного тока на АЦП AD7714 и МК семейства x51

Милливольтметр разработан на базе устройства, описание которого было опубликовано в журнале «Схемотехника», №1, 2002 г.

В течение нескольких лет фирма Analog Devices выпускает семейство 24-битных сигма-дельта АЦП (AD7710...AD7714, AD7730, AD7731). Применение этих микросхем резко упростило схемотехнику построения пре-

цизионных измерительных систем в областях, не требующих высокого быстродействия, таких, например, как термометрия, тензометрия и т. д. Однако программирование этих микросхем — непростая задача, особенно для тех, у кого опыт подобной работы невелик. В связи с этим в настоящем подразделе приведен пример конкретной разработки на одной из таких микросхем — AD7714, который может быть взят за основу при создании системы на одной из перечисленных микросхем.

Отмечу также, что выпускаются и 16-битные сигма-дельта АЦП с подобной структурой. Рассматриваемый пример может также быть использован и для работы с этими микросхемами.

В предложенном примере на AD7714 выполнена основная часть прецизионного низкочастотного вольтметра на 6...8 десятичных разрядов. Для полного вольтметра описываемое устройство необходимо снабдить входным каскадом с делителем напряжения и преобразованием сигнала отрицательной полярности. Но поскольку в очень многих вариантах применения рассматриваемой микросхемы входной каскад с делителем не нужен, а измеряемое напряжение никогда не меняет своей полярности, я решил ограничиться описанием системы без подобного входного каскада — сделать его по силам любому подготовленному разработчику.

Сама AD7714 довольно подробно описана в приложении 8 (см. том 2). В связи с этим здесь я не стану рассказывать о структуре ее регистров, всех возможных вариантах использования ее входов, всех формах калибровок и т. д., предполагая, что читатели уже знакомы с этой микросхемой или самостоятельно познакомятся с ней по ее фирменному техническому описанию и материалам, приведенным в приложении.

Принципиальная схема устройства приведена на **Рис. 7.9.** Оно, помимо AD7714, содержит прецизионный источник опорного напряжения DA1(REF192), согласующий каскад на ОУ DA2(AD8551), микроконтроллер DD1(AT89C51) и индикатор DD2(HT1610).

Источник опорного напряжения вырабатывает напряжение, примерно равное 2.48 В. Температурная стабильность источника — 8 ppm/°C. Столь низкое значение собственного теплового дрейфа позволяет во многих случаях обойтись без его термостабилизации, однако если вы хотите надежно измерять сигналы величиной менее 100 мкВ, источник необходимо термостабилизировать.

Поскольку напряжение стабилизации DA1 имеет двухпроцентный разброс, его необходимо как-то компенсировать. Это можно делать как программным путем, так и аппаратно. Второй путь более оперативный, т. к. подкрутить движок переменного резистора гораздо быстрее, чем провести необходимые измерения и перепрограммировать микроконтроллер.

Для возможности этой компенсации, т. е. фактически для калибровки вольтметра, служит согласующий каскад на ОУ DA2. Поскольку именно с

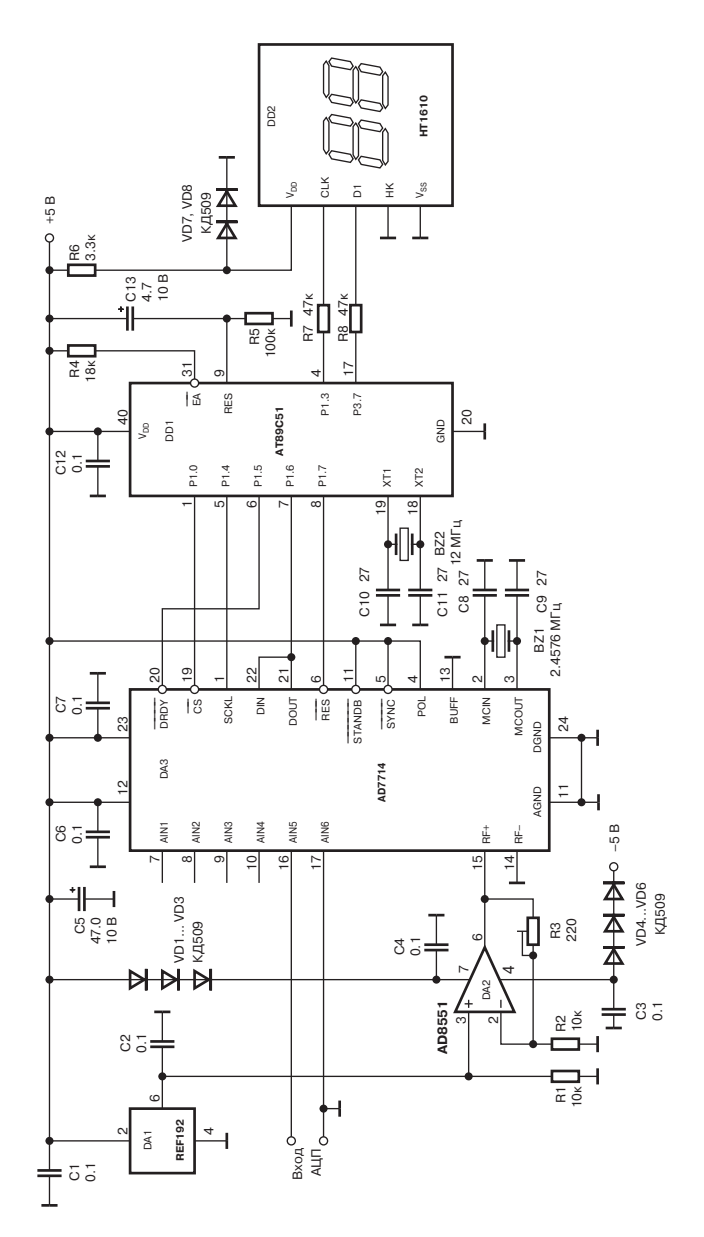


Рис. 7.9. Принципиальная схема милливольтметра на AD7714

его выхода сигнал поступает на опорный вход АЦП, дрейфовые характеристики этого ОУ должны быть по крайней мере не хуже таковых у DA1. Такому условию удовлетворяют ОУ AD8551, максимальный паспортный тепловой дрейф которых составляет 40 нВ/°С (а реальный, полученный мной при измерении имевшихся в наличии двух экземпляров и того ниже — 25 нВ/°С). Подстроечный резистор R3 (СП5-2) позволяет регулировать напряжение на выходе ОУ AD8551 в пределах ±1%.

Работа системы организуется микроконтроллером DD1, который вначале настраивает, а затем опрашивает АЦП DA3, обрабатывает полученные результаты и отображает их на индикаторе DD2. Схема сопряжения микроконтроллера с индикатором — стандартная и каких-либо особенностей не имеет.

Управление АЦП осуществляется микроконтроллером по входам $\overline{\text{DRDY}}$, $\overline{\text{CS}}$, SCLK, DIN и $\overline{\text{RESET}}$, данные считываются с выхода DOUT. Эти выводы соединены с линиями P1.5, P1.0, P1.4, P1.6 и P1.7 микроконтроллера соответственно. Сигнал для оцифровки подается на входы AIN5 и AIN6 АЦП. Входной буфер АЦП отключен за счет подачи нулевого потенциала на вход BUFFER.

Микросхема AD7714 характеризуется большим количеством разнообразных режимов настройки и функционирования, поэтому процесс ее программирования довольно сложен. Одна из причин, почему я включил в рассмотрение пример с ее использованием — желание подробно описать этот процесс, чтобы те из читателей, кому придется работать с этой микросхемой, могли легко разобраться в нем и при необходимости модифицировать программу под свои задачи.

Алгоритм настройки AD7714 и считывания с нее информации изображен на **Рис. 7.10.** Как следует из рисунка, вначале микросхему нужно настроить соответствующим образом, задав аналоговые входы, сигнал с которых будет оцифровываться, режим калибровки, коэффициент усиления внутреннего усилителя, частоту среза цифрового фильтра, входной диапазон (биполярный или униполярный), разрядность считываемого результата (16 или 24 бита), режим потребления, включения/выключения цифрового фильтра и проверки работоспособности преобразователя. Это осуществляется тремя последовательными циклами записи соответствующей информации в верхний регистр фильтра, нижний регистр фильтра и регистр режима. Напомню, что каждый из упомянутых циклов записи предваряется записью в регистр обмена, так что настройка AD7714 представляет собой шесть последовательных циклов записи. После этого возможно считывание информации.

Обмен информацией между микроконтроллером и АЦП возможен двумя способами. Первый — с использованием последовательного порта микроконтроллера. Пример реализации такого способа приведен в фир-

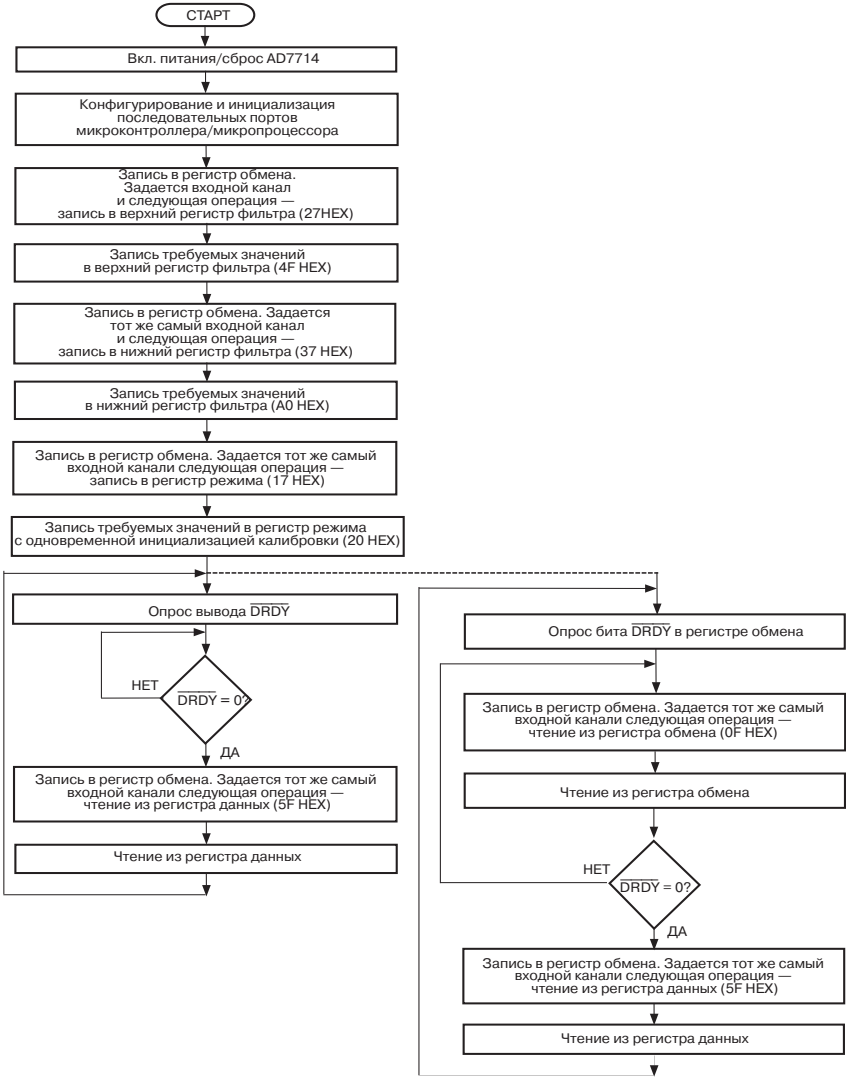


Рис. 7.10. Алгоритм настройки и считывания из AD7714

менном описании на схожую с AD7714 микросхему AD7711. Однако этот способ довольно сложен в настройке и требует перекодировки данных, передаваемых или принимаемых микроконтроллером. Поэтому в рассматриваемом примере выбран второй способ, без использования последовательного порта, при котором все требуемые сигналы формируются на линиях порта программным способом. Отмечу, что AD7714 никогда не работает в полностью дуплексном режиме и может либо только передавать данные, либо только принимать. Благодаря этому оказалось возможным объединить вход DIN и выход DOUT АЦП. Также отмечу, что в выбранном примере проверка готовности АЦП осуществляется аппаратно, по уровню сигнала на выходе $\overline{\text{DRDY}}$.

Особо нужно отметить то, что микроконтроллер при включении формирует сигнал сброса АЦП по входу $\overline{\text{RESET}}$. Хотя в фирменном описании на микросхему ничего не сказано о необходимости реализации такого сброса, практика показала, что обычная цепь сброса, содержащая резистор и конденсатор соответствующих номиналов, не в состоянии надежно сбросить АЦП, и он может не заработать при подаче питания. Описанный способ формирования сигнала сброса позволяет добиться надежного старта AD7714.

Как я уже упоминал, AD7714 настраивается на работу со входами AIN5 и AIN6. Тактовая частота работы микросхемы — 2.4576 МГц. Первую частоту режекции выбираем равной 50 Гц. С этой целью в регистры фильтра (при выбранной тактовой частоте) нужно занести число 384d = 001100000000b (0011b — в старший регистр фильтра, 00000000b — в младший). Работаем в униполярном входном режиме, считываем 24-битный результат. Коэффициент усиления внутреннего усилителя выберем равным 1, перед измерением будем запускать режим автокалибровки, подстраивающий и ноль, и коэффициент усиления полной шкалы.

Анализ таблицы из фирменного технического описания, представляющей зависимость выходного шума/разрешения AD7714 от первой частоты режекции, коэффициента усиления внутреннего усилителя и тактовой частоты показывает, что при выбранных параметрах типовое значение выходного шума должно составлять 4.3 мкВ, а эффективное разрешение — 20 битов. Другими словами, 4 младших бита имеют право «гулять» от измерения к измерению. Если мы хотим добиться более высокого разрешения, необходимо проводить серию из как минимум нескольких десятков измерений с последующим усреднением. Так же, как и в рассмотренном выше случае, удовлетворительные результаты получаются при усреднении 64...128 измерений (эффективное разрешение — 22...23 бита). В настоящем примере я усредняю по 256 измерениям, при этом результат измерения постоянного сигнала от измерения к измерению либо не меняется, либо меняется на 1 единицу младшего значащего разряда. Увеличение числа измерений для усреднения сверх

256 уже практически ничего не дает. Отмечу, что быстродействие вольтметра, усредняющего по 256 измерениям, составляет примерно 0.3 Гц (1 измерение в 3 секунды). Естественно, измеряемый сигнал за время измерения должен оставаться неизменным. Если это по тем или иным причинам не так, и скорость его изменения более высока, нужно соответствующим образом уменьшить количество усредняемых измерений.

При измерении напряжения, равного опорному (2.5 В), и последующем считывании с усреднением контроллер получит, как нетрудно догадаться, результат, равный FFFFFFh. После преобразования его в двоично-десятичную форму получается (проверьте по Windows-калькулятору) 16777215. Ясно, что перед отображением результата на индикаторе его надо масштабировать — либо умножить на 1.4901162, либо разделить на 6.710886. В первом случае, мы получим число 24999999, которое затем можно отобразить в формате 2.4999999 В, что соответствует 8-декадному цифровому вольтметру, младшая значащая цифра которого является разрядом сотен нВ. Во втором случае, мы получим 2500000, которое затем можно отобразить в формате 2.500000 В. В этом случае, разрешение получается на порядок хуже, но все равно очень большим — 7 десятичных разрядов. В настоящем примере реализован первый случай — 8-декадный вольтметр. При этом, как и ранее, дабы обойтись без использования программ с плавающей или фиксированной десятичной точкой, умножение на 1.4901162 осуществляется за счет умножения результата измерения на 94302 с последующим делением на 63285 (94302/63285 ≈ 1.49011614). Далее полученный результат преобразовывается в двоично-десятичный формат и отображается на экране индикатора.

Еще один момент. Если при оцифровке получится результат ABCC76h или больший, то после умножения на 94302 и деления на 63285 он превысит 16777215d = FFFFFFh, т. е. будет уже не трехбайтный, а четырехбайтный. А в нашем распоряжении есть лишь подпрограмма деления шестибайтного числа на трехбайтное с получением *трехбайтного* частного. Следовательно, если не принять мер, то масштабирование результата, большего чем ABCC76h, приведет к ошибке из-за переполнения частного, которое не впишется в отведенные ему три байта. Как можно выйти из этого положения?

Я поступил следующим образом. Результат оцифровки перед делением на 63285 я умножил не на 94302, а на 94302/3 = 31434. Таким образом, при масштабировании я получил результат, ровно втрое меньший, чем требуется. Зато он вписывается в трехбайтные рамки, и после упомянутых умножения с последующим делением мы ни при каком результате оцифровки не столкнемся с переполнением частного. А после такого масштабирования мне останется либо умножить частное на 3, либо (что гораздо быстрее)

дважды сложить его с самим собой. В настоящей программе я обошелся суммированием.

Этот пример демонстрирует вам, что означают вышесказанные мною слова о том, что рассмотренные в настоящей главе подпрограммы многобайтного деления не отслеживают переполнение частного, и недопущение переполнения входит в обязанности программиста. Еще раз подчеркну — если вы собираетесь воспользоваться подпрограммами деления, обязательно проверьте, какой результат получится при масштабировании максимального и минимального из исходных результатов, и если обнаружите, что хотя бы в одном из случаев частное больше максимально допустимого, предпринимайте меры, аналогичные описанным в предыдущем абзаце.

Следует отметить, что используемый индикатор (HT1610) не отображает точку. В связи с этим в качестве разделителя целой и дробной части числа используется пробел, т. е. упомянутое 2.4999999 отображается в виде 2_4999999, где символ нижнего подчеркивания означает пробел, т. е. пустое знакоместо на индикаторе. При необходимости отображать результат не с пробелом, а с десятичной точкой нужно применить иной индикатор, например, MT10T7 (см. «Схемотехника», №2/2000 г., с. 16—17), переписав соответствующую подпрограмму вывода информации на индикатор.

Программа для вольтметра на AD7714 и МК семейства x51

```
;
;       &&&&
```

```
NACHPZU .EQU   0000H
BUFFER  .EQU   NACHPZU+20H
COUNT   .EQU   NACHPZU+23H
SWAPREG .EQU   NACHPZU+24H
COMDREG .EQU   NACHPZU+25H
ASTK    .EQU   NACHPZU+15
BUFFER2 .EQU   NACHPZU+30H
DRDY    .EQU   P1.5
CS      .EQU   P1.0
SCLK    .EQU   P1.4
DIN     .EQU   P1.6
DOUT    .EQU   P1.6
RESADC  .EQU   P1.7
SDATA   .EQU   P1.6
;
;
;
;
;               НАЧАЛО ПРОГРАММЫ:
;                   *****
;
        .ORG   NACHPZU
;
RESET:
        MOV    SP,#10H
        MOV    P1,#11111110B          ;#01111111B
        MOV    P3,#11111111B          ;#10010011B
;
        CLR    RESADC
        NOP
        SETB   RESADC
        JB     DRDY,$
;
START_24:
        LCALL  ADSETP56
S1:
        MOV    BUFFER2,#0
        MOV    BUFFER2+1,#0
        MOV    BUFFER2+2,#0
        MOV    BUFFER2+3,#0
        MOV    BUFFER2+4,#0
        MOV    BUFFER2+5,#0
;
        MOV    R2,#0                  ;R2=256
;
S2:
        LCALL  ADC_RD56
        LCALL  SUMMIP                 ;
        DJNZ   R2,S2
;
;B BUFFER2-BUFFER2+3 СУММА 256-и 24-БИТНЫХ ОТСЧЕТОВ
```

```
;
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 2 РАЗА
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 4 РАЗА
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 8 РАЗ
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 16 РАЗ
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 32 РАЗА
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 64 РАЗА
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 128 РАЗ
        LCALL  RRC_BUF2              ;УМЕНЬШИЛ ЕЕ В 256 РАЗ
;
;МАСШТАБИРУЕМ В 1,5 РАЗА, ПЕРЕНЕСЕМ В BR7R6R5 И ОТОБРАЗИМ
;
        MOV    B,#0
        MOV    R1,#07AH
        MOV    R0,#0CAH              ;BR1R0=007ACA=31434D
        MOV    R4,BUFFER2+2
        MOV    R3,BUFFER2+1
        MOV    R2,BUFFER2+0
        LCALL  MUL63                 ;R7R6R5R4R3R2=(BUFFER2+2...+0)*(31434D)
        MOV    B,#0
        MOV    R1,#0F7H
        MOV    R0,#035H              ;BR1R0=00F735H=63285D
        LCALL  DIV63                 ;R4R3R2=(BUFFER2+2...+0)*(31434D)/(63285D)
;
        MOV    BUFFER2+2,R4
        MOV    BUFFER2+1,R3
        MOV    BUFFER2+0,R2          ;ВОЗВРАТИЛИ СКОРРЕКТИРОВАННОЕ
;
        MOV    R7,BUFFER2+2
        MOV    R6,BUFFER2+1
        MOV    R5,BUFFER2+0
;
        MOV    B,#0
;
        MOV    A,R5
        ADD    A,BUFFER2+0
        MOV    R5,A
        MOV    A,R6
        ADDC   A,BUFFER2+1
        MOV    R6,A
        MOV    A,R7
        ADDC   A,BUFFER2+2
        MOV    R7,A
        MOV    A,B
        ADDC   A,#0
        MOV    B,A
;
        MOV    A,R5
        ADD    A,BUFFER2+0
        MOV    R5,A
        MOV    A,R6
```

```
        ADDC   A,BUFFER2+1
        MOV    R6,A
        MOV    A,R7
        ADDC   A,BUFFER2+2
        MOV    R7,A
        MOV    A,B
        ADDC   A,#0
        MOV    B,A
;
        LCALL  BN25BCD
;
        LCALL  R7654IN2
;
        LJMP   S1
;
RRC_BUF2:
        CLR    C
        MOV    A,BUFFER2+3
        RRC    A
        MOV    BUFFER2+3,A
        MOV    A,BUFFER2+2
        RRC    A
        MOV    BUFFER2+2,A
        MOV    A,BUFFER2+1
        RRC    A
        MOV    BUFFER2+1,A
        MOV    A,BUFFER2+0
        RRC    A
        MOV    BUFFER2+0,A
        RET
;
SUMMIP:
        MOV    A,BUFFER2+0          ;МЛАДШИЙ
        ADD    A,BUFFER+2           ;МЛАДШИЙ
        MOV    BUFFER2+0,A
;
        MOV    A,BUFFER2+1          ;СРЕДНИЙ
        ADDC   A,BUFFER+1           ;СРЕДНИЙ
        MOV    BUFFER2+1,A
;
        MOV    A,BUFFER2+2          ;СТАРШИЙ
        ADDC   A,BUFFER             ;СТАРШИЙ
        MOV    BUFFER2+2,A
;
        MOV    A,BUFFER2+3          ;СОВСЕМ СТАРШИЙ
        ADDC   A,#0
        MOV    BUFFER2+3,A
        RET
;
;
R7654IN2:                           ;ФОРМАТ - _2_5102162
```

```
        MOV    A,#0FH
        LCALL  SIMBOL1              ;10-Й
;
        MOV    A,R7
        SWAP   A
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;9-Й
;
        MOV    A,#0FH
        LCALL  SIMBOL1              ;8-Й
;
        MOV    A,R7
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;7-Й
;
        MOV    A,R6
        SWAP   A
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;6-Й
        MOV    A,R6
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;5-Й
;
        MOV    A,R5
        SWAP   A
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;4-Й
        MOV    A,R5
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;3-Й
;
        MOV    A,R4
        SWAP   A
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;2-Й
        MOV    A,R4
        ANL    A,#00001111B
        LCALL  SIMBOL1              ;1-Й
RETR:
        RET
;
;
ADSETP56:
        MOV    SWAPREG,#00100110B     ;0 - ЭТО 0
                                      ;010 - ВЕР.РЕГ.ФИЛЬТРА
                                      ;0 - ЭТО WR
                                      ;110 - AIN5,AIN6
        MOV    COMDREG,#11000001B     ;0 - ЭТО БИПОЛЯРНЫЙ,1-униполярный
                                      ;1 - ЭТО 24 БИТА
                                      ;0 - ЭТО СНИЖ.ПОТРЕБЛЕНИЯ
                                      ;0 - ЭТО ZERO
```

```
                                        ;0001 - СТ.4 БИТА F11-F0
        LCALL  ADC_WR
;
        MOV    SWAPREG,#00110110B       ;0 - ЭТО 0
                                        ;011 - НИЖ.РЕГ.ФИЛЬТРА
                                        ;0 - ЭТО WR
                                        ;110 - AIN5,AIN6
        MOV    COMDREG,#10000000B       ;ЭТО МЛ.8 БИТ F7-F0
                                        ;В ФИЛЬТРЕ 000110000000=384D
                                        ;ЧАСТОТА ОБНОВЛЕНИЯ =
                                        ;2457600/128/384=19200/384=50ГЦ
                                        ;0001 - СТ.4 БИТА F11-F0
        LCALL  ADC_WR
;
        MOV    SWAPREG,#00010110B       ;0 - ЭТО 0
                                        ;001 - РЕГ.РЕЖИМА
                                        ;0 - ЭТО WR
                                        ;110 - AIN5,AIN6
        MOV    COMDREG,#00100000B       ;001 - ЭТО АВТОКАЛИБРОВКА
                                        ;000 - ЭТО Kyc=1
                                        ;0 - ЭТО Iконтр ОТКЛЮЧЕН
                                        ;0 - ЭТО ФИЛЬТР ВКЛЮЧЕН
        LCALL  ADC_WR
;
        JB     DRDY,$
        RET
;
;
ADC_WR:
        MOV    A,SWAPREG
        LCALL  TX_ADC
        MOV    A,COMDREG
        LCALL  TX_ADC
        RET
;
ADC_RD56:
        MOV    SWAPREG,#01011110B       ;0 - ЭТО 0
                                        ;101 - РЕГ.РЕЗУЛЬТАТА
                                        ;1 - ЭТО RD
                                        ;110 - AIN5,AIN6
ADC_RD:
;       CLR    EA
        MOV    R0,#BUFFER
        JB     DRDY,$
        MOV    A,SWAPREG
        LCALL  TX_ADC
        LCALL  RX_ADC
        MOV    @R0,A
        INC    R0
        LCALL  RX_ADC
        MOV    @R0,A
```

```
        INC    R0
        LCALL  RX_ADC
        MOV    @R0,A
;       SETB   EA
        RET
;
TX_ADC:
        MOV    COUNT,#8
TX_ADC_N:
        CLR    SCLK
        RLC    A
        MOV    SDATA,C
        NOP
        NOP
        SETB   SCLK
        NOP
        NOP
        NOP
        DJNZ   COUNT,TX_ADC_N
        SETB   SDATA
        RET
;
RX_ADC:
        MOV    COUNT,#8
RX_ADC_N:
        CLR    SCLK
        NOP
        NOP
        MOV    C,SDATA
        RLC    A
        SETB   SCLK
        NOP
        NOP
        DJNZ   COUNT,RX_ADC_N
        RET
;
;
;ПРЕОБР. В ДВ.-ДЕС. ФОРМАТ ЧИСЛА ИЗ BR7R6R5 (0...17FFFFD)
;
BN25BCD:
        MOV    A,B
        JNZ    MO16MLN              ;ПЕР. ПРИ B>0
        LCALL  BN24BCD
        SJMP   RETBN25
MO16MLN:
        CLR    C
        MOV    A,R5
        SUBB   A,#80H               ;10 000 000D = 989680H
        MOV    R5,A
        MOV    A,R6
        SUBB   A,#96H
```

```
        MOV    R6,A
        MOV    A,R7
        SUBB   A,#98H
        MOV    R7,A
        MOV    A,B
        SUBB   A,#0
        MOV    B,A              ;РЕГ. В ДОЛЖЕН БЫТЬ РАВЕН 0!
        LCALL  BN24BCD
        MOV    A,R7
        ADD    A,#10H
        MOV    R7,A             ;ДЕСЯТКИ МИЛЛИОНОВ=ДЕСЯТКИ МИЛЛИОНОВ + 1
RETBN25:
        RET
;
;ТО ПРИ ВХОДЕ В ЭТУ П/П
;ТРЕХБАЙТНЫЙ РЕЗУЛЬТАТ В R7R6R5, А ДВОИЧНО-
;ДЕСЯТИЧНЫЙ ОТВЕТ - В R7R6R5R4
;
BN24BCD:
        MOV    R0,#0
        MOV    R1,#0            ;ВНАЧАЛЕ СЮДА ПОМЕСТИМ 4 СТАРШИЕ ЦИФРЫ
;
        MOV    R2,#80H
        MOV    R3,#69H
        MOV    R4,#67H          ;676980H = - 10 000 000
DEC_MIL:
        LCALL  ADD765_432       ;ЧИСЛО - 10 000 000
        JNC    ED_MIL0          ;ЕСЛИ CY=0 ТО РАЗНОСТЬ < 0
        MOV    A,R1             ;А ЭТО ЕСЛИ РАЗНОСТЬ > 0
        ADD    A,#00010000B
        MOV    R1,A             ;К-ВО ДЕС.МИЛЛИОНОВ УВЕЛИЧИЛИ НА 1
        SJMP   DEC_MIL
ED_MIL0:
        MOV    R2,#80H
        MOV    R3,#96H
        MOV    R4,#98H          ;989680H = 10 000 000
        LCALL  ADD765_432
;
        MOV    R2,#0C0H
        MOV    R3,#0BDH
        MOV    R4,#0F0H         ;F0BDC0H = - 1 000 000
ED_MIL:
        LCALL  ADD765_432       ;ЧИСЛО - 1 000 000
        JNC    HUN_TH0          ;ЕСЛИ CY=0 ТО РАЗНОСТЬ < 0
        MOV    A,R1             ;А ЭТО ЕСЛИ РАЗНОСТЬ > 0
        ADD    A,#00000001B
        MOV    R1,A             ;К-ВО ЕД.МИЛЛИОНОВ УВЕЛИЧИЛИ НА 1
        SJMP   ED_MIL
HUN_TH0:
        MOV    R2,#40H
        MOV    R3,#42H
```

```
        MOV    R4,#0FH          ;0F4240H = 1 000 000
        LCALL  ADD765_432
;
        MOV    R2,#60H
        MOV    R3,#79H
        MOV    R4,#0FEH         ;FE7960H = - 100 000
HUN_TH:
        LCALL  ADD765_432       ;ЧИСЛО - 100 000
        JNC    DEC_TH0          ;ЕСЛИ CY=0 ТО РАЗНОСТЬ < 0
        MOV    A,R0             ;А ЭТО ЕСЛИ РАЗНОСТЬ > 0
        ADD    A,#00010000B
        MOV    R0,A             ;К-ВО СОТЕН ТЫСЯЧ УВЕЛИЧИЛИ НА 1
        SJMP   HUN_TH
DEC_TH0:
        MOV    R2,#0A0H
        MOV    R3,#86H
        MOV    R4,#01H          ;0186A0H = 100 000
        LCALL  ADD765_432
;
        MOV    R2,#0F0H
        MOV    R3,#0D8H
        MOV    R4,#0FFH         ;FFD8F0H = - 10 000
DEC_TH:
        LCALL  ADD765_432       ;ЧИСЛО - 10 000
        JNC    ED_TH0           ;ЕСЛИ CY=0 ТО РАЗНОСТЬ < 0
        MOV    A,R0             ;А ЭТО ЕСЛИ РАЗНОСТЬ > 0
        ADD    A,#00000001B
        MOV    R0,A             ;К-ВО ДЕС. ТЫСЯЧ УВЕЛИЧИЛИ НА 1
        SJMP   DEC_TH
ED_TH0:
        MOV    R2,#10H
        MOV    R3,#27H
        MOV    R4,#00H          ;002710H = 10 000
        LCALL  ADD765_432
;
        MOV    R2,R5
        MOV    R3,R6
        LCALL  BN2BCD
        MOV    R7,R1
        MOV    R6,R0
        RET
;
ADD765_432:
        CLR    C
        MOV    A,R5
        ADD    A,R2
        MOV    R5,A
        MOV    A,R6
        ADDC   A,R3
        MOV    R6,A
        MOV    A,R7
```

```
        ADDC   A,R4
        MOV    R7,A
        RET
;
SIMBOL1:
        ANL    A,#00001111B
        CJNE   A,#0,SIMB11
        MOV    A,#10
        SJMP   SIMB12
SIMB11: CJNE   A,#0FH,SIMB12
        MOV    A,#0
SIMB12: CLR    CLKIND1
        SWAP   A
        ACALL  BIT1
        ACALL  BIT1
        ACALL  BIT1
        ACALL  BIT1
        RET
;
BIT1:   RLC    A
        MOV    DATIND1,C        ;ВЫВ. ДАННЫХ В ЖК-ДИСПЛЕЙ
        SETB   CLKIND1          ;ИМПУЛЬС ЗАЩЕЛКИВАНИЯ
        CLR    CLKIND1
        RET
;
;
;ПОДПРОГРАММА УМНОЖЕНИЯ ДВУХ ТРЕХБАЙТНЫХ ЧИСЕЛ С
;ПОЛУЧЕНИЕМ НА ВЫХОДЕ 6-БАЙТНОГО РЕЗУЛЬТАТА.
;СОМНОЖИТЕЛИ В B(СТ.БАЙТ)R1R0 И В R4(СТ.БАЙТ)R3R2,
;РЕЗУЛЬТАТ В R7R6R5R4R3R2.
;
MUL63:
        MOV    A,#24            ;МНОЖИТЕЛЬ ТРЕХБАЙТНЫЙ
        MOV    R7,#0            ;ПОСЛЕ 24 СДВИГОВ ЭТИ
        MOV    R6,#0            ;3 БАЙТА БУДУТ ТРЕМЯ
        MOV    R5,#0            ;МЛАДШИМИ БАЙТАМИ ПРОИЗВЕДЕНИЯ
MUL63_1:
        PUSH   ACC
        MOV    A,R2             ;В A - МЛ.БАЙТ МНОЖИТЕЛЯ
        RRC    A                ;АНАЛИЗ ЕГО МЛ. БИТА
        JNC    MUL63_2          ;ЕСЛИ ОН НЕ 0, НЕ СУММИРОВАТЬ
        MOV    A,R5             ;ПЛЮСУЕМ МНОЖИМОЕ К НАКАПЛИВАЕМОЙ
        ADD    A,R0             ;СУММЕ, КОТОРАЯ ПОСЛЕ ЗАВЕРШЕНИЯ
        MOV    R5,A             ;П/П И БУДЕТ ПРОИЗВЕДЕНИЕМ
        MOV    A,R6
        ADDC   A,R1
        MOV    R6,A
        MOV    A,R7
        ADDC   A,B
        MOV    R7,A
MUL63_2:
```

```
        MOV    A,R7
        RRC    A
        MOV    R7,A
        MOV    A,R6
        RRC    A
        MOV    R6,A
        MOV    A,R5
        RRC    A
        MOV    R5,A
        MOV    A,R4
        RRC    A
        MOV    R4,A
        MOV    A,R3
        RRC    A
        MOV    R3,A
        MOV    A,R2              ;СДВИНУЛИ МНОЖИТЕЛЬ И
        RRC    A                 ;НАКАПЛИВАЕМУЮ СУММУ
        MOV    R2,A              ;НА 1 БИТ ВПРАВО
        POP    ACC
        DJNZ   ACC,MUL63_1
        RET
;
;ПОДПРОГРАММА ДЕЛЕНИЯ ШЕСТИБАЙТНОГО ЧИСЛА ИЗ R7R6R5R4R3R2
;НА ТРЕХБАЙТНОЕ В BR1R0 С ПОЛУЧЕНИЕМ ТРЕХБАЙТНОГО ЧАСТНОГО
;В R4R3R2
;
DIV63:
        MOV    A,#24
DCLK_63:
        PUSH   ACC
        CLR    C
        MOV    A,R2
        RLC    A
        MOV    R2,A
        MOV    A,R3
        RLC    A
        MOV    R3,A
        MOV    A,R4
        RLC    A
        MOV    R4,A
        MOV    A,R5
        RLC    A
        MOV    R5,A
        MOV    A,R6
        RLC    A
        MOV    R6,A
        MOV    A,R7
        RLC    A
        MOV    R7,A
        PUSH   PSW
PER1_63:
```

```
        CLR    C
        MOV    A,R5
        SUBB   A,R0
        MOV    R5,A
        MOV    A,R6
        SUBB   A,R1
        MOV    R6,A
        MOV    A,R7
        SUBB   A,B
        MOV    R7,A
        JC     PER2_63
        POP    PSW
PER3_63:
        INC    R2
        SJMP   PER4_63
PER2_63:
        POP    PSW
        JC     PER3_63
        MOV    A,R5
        ADD    A,R0
        MOV    R5,A
        MOV    A,R6
        ADDC   A,R1
        MOV    R6,A
        MOV    A,R7
        ADDC   A,B
        MOV    R7,A
PER4_63:
        POP    ACC
        DJNZ   ACC,DCLK_63
        RET
;
BN2BCD:                         ;R3R2 -> R6R5R4, R0 И R1 СОХР.
        MOV    R7,#10H
        MOV    A,#0
        MOV    R4,A
        MOV    R5,A
        MOV    R6,A
BN2CKL: MOV    A,R2
        ADD    A,R2
        MOV    R2,A
        MOV    A,R3
        ADDC   A,R3
        MOV    R3,A
;ДВОИЧНО-ДЕСЯТИЧНОЕ УДВОЕНИЕ СУММЫ С УЧЕТОМ ПЕРЕНОСА:
        MOV    A,R4
        ADDC   A,R4
        DA     A
        MOV    R4,A
        MOV    A,R5
        ADDC   A,R5
```

```
        DA     A
        MOV    R5,A
        MOV    A,R6
        ADDC   A,R6
        DA     A
        MOV    R6,A
;ПРОВЕРКА КОНЦА ЦИКЛА:
        DJNZ   R7,BN2CKL
        RET
;
;
;               ЗАДЕРЖКА
;                ******
;
DEL20:
        MOV    R1,#0FFH
LPEX:   MOV    R2,#05H
LPIN:   DJNZ   R2,LPIN
        DJNZ   R1,LPEX
        RET
;
        .ORG   00800H
        .END
```

Исходный ассемблерный текст снабжен большим количеством комментариев, которые позволяют при необходимости легко менять те или иные параметры настройки AD7714 — аналоговые входы, сигнал с которых будет оцифровываться, режим калибровки, коэффициент усиления внутреннего усилителя, частоту среза цифрового фильтра, входной диапазон (биполярный или униполярный), разрядность считываемого результата (16 или 24 бита), режим потребления и т. д. Можно также, как было сказано выше, менять количество измерений при усреднении, а также уменьшить число отображаемых десятичных разрядов.

В завершение отмечу, что при использовании микросхемы AD7714 нужно учитывать следующие моменты. Во-первых, несмотря на постоянное использование режима автокалибровки, при непрерывной работе микросхемы в течение 7...8 часов результат оцифровки неизменного сигнала может несколько подплывать (в использованном мной образце этот уход составлял 4...5 мкВ). Далее, от включения к включению результат оцифровки этого неизменного сигнала также не остается постоянным, и здесь разброс еще больше (у меня получилось в пределах 25...30 мкВ). Сказанное, например, означает, что использование AD7714 для прямой оцифровки сигнала с термопары, без предварительного усиления (а такие рекомендации имеются в datasheet'е) может внести в измерения заметную погрешность. Во всяком случае, прежде, чем использовать AD7714 с платиновыми термопарами без дополнительного входного уси-

лителя, нужно после монтажа и отладки вольтметра провести небольшое исследование, подобное тому, о котором я упомянул — в противном случае вы можете получить неконтролируемую погрешнось на уровне 4...5 градусов.

7.6. Краткие выводы

Мы рассмотрели четыре примера завершенных разработок систем на основе микроконтроллеров семейства x51. Они достаточно просты, хотя программа последней содержит почти 1000 строк. Конечно, четыре примера — это немного, но вариантов того или иного использования микроконтроллеров — тысячи, десятки тысяч, и сколько их не приведи, что-то окажется неохваченным. Поэтому я и не ставил перед собой задачу привести десять, двадцать примеров, это мало что добавило бы читателям. С другой стороны, глава, содержащая эти четыре примера, оказалась одной из самых больших. А какой бы она была, замахнись я на описание десятка конструкций!

На что я хотел бы обратить ваше внимание? Разработка микроконтроллерных систем имеет одну особенность. Обычно всем, начиная с проектирования аппаратных средств и кончая их программированием, занимается один специалист. В этом состоит существенное отличие от практики компьютерного программирования, где программист, как правило, смутно представляет тонкости аппаратных особенностей системы, для которой пишется программа. И здесь у разработчика системы на микроконтроллерах есть определенное преимущество. Он может варьировать как аппаратными, так и программными средствами, добиваясь их оптимального баланса. Оптимизация обычно идет в направлении минимизации времени разработки для единичных систем и минимизации аппаратных средств, времени сборки и наладки для систем, выпускаемых массовым тиражом. Стратегия этой оптимизации определяется самим разработчиком, и именно он сам принимает решение, где для достижения цели добавить какие-либо узлы на тех или иных микросхемах, транзисторах и т. д., а где добиться цели путем использования разнообразной микроконтроллерной периферии и написания программы, заставляющей ее функционировать нужным образом. Например, если вашей системе для выполнения определенной функции требуется счетчик, а вы только делаете первые шаги в микроконтроллерах, вам может оказаться проще поставить рядом с микроконтроллером дополнительную микросхему, чем разобраться с тем, как использовать таймер-счетчик микроконтроллера и отладить его работу. По мере накопления опыта вы будете все эффективнее использовать имеющиеся у микроконтроллера внутренние ресурсы, выбирать контроллеры с более развитой периферией, дабы иметь этих ресурсов как можно больше.

Но это будет потом, когда вы наберетесь опыта. А поначалу не бойтесь использовать все доступные вам средства. Пусть ваши первые конструкции будут иметь излишние с точки зрения опытного программиста аппаратные средства, пусть программа будет неказистой, это неважно. Важно, чтобы система функционировала в соответствии с вашим замыслом. А все остальное придет со временем — и программы станут изящнее, и схемы проще, да и процесс разработки и реализации перестанет занимать столь много времени, сколько займет создание первой самостоятельной конструкции. Все через это прошли, и вы — не исключение. Так что дерзайте.

Глава 8. ИСПОЛЬЗОВАНИЕ ПРИЕМОПЕРЕДАТЧИКА ДЛЯ СВЯЗИ С ПК

И последнее, с чем мы обязательно должны познакомиться — последовательный канал микроконтроллера, позволяющий, в частности, легко связать его с персональным компьютером (или с другим контроллером) по протоколу RS-232. В микроконтроллерах семейства x51 для этого существуют специально разработанное аппаратное средство (приемопередатчик), благодаря которому создание интерфейса RS-232 превращается в относительно простую задачку для программиста. С легкой руки разработчиков микроконтроллеров семейства x51, предусмотревших действительно удачные решения, подобный последовательный приемопередатчик стал стандартом де-факто в микроконтроллерах, и его (с незначительными модификациями) можно найти практически в любом микроконтроллере. Поэтому мы обязательно должны ознакомиться с тем, как его использовать, например, для связи микроконтроллера с персональным компьютером.

Простое описание приемопередатчика, похожее на то, которое приведено почти в любой книжке по микроконтроллерам, вы найдете в приложении 2 (см. том 2). Но для тех, кто только знакомится с микроконтроллерной техникой, этого совершенно недостаточно — мало знать, что есть внутри микроконтроллера, нужно еще увидеть, как это все правильно оживить. С этой целью я предлагаю вам материал, специально подготовленный для начинающих моим сыном, Алексеем Фрунзе. Этот материал задумывался как пример реализации разработки микроконтроллерного устройства, управляемого персональным компьютером по последовательному каналу. Разобравшись с тем, как ПК управляет микроконтроллером, отображает, обрабатывает и сохраняет полученную от него информацию, вы сможете применить эти знания для собственных разработок. К тому же, описанное устройство имеет еще и самостоятельную ценность — это управляемый цифровой вольтметр, результаты измерения которого перед отображением могут быть обработаны компьютером по заранее заданному алгоритму, а также сохранены в файле на винчестере вашего ПК, просмотрены впоследствии и распечатаны. Все это делает описанное устройство

основой для простой системы сбора, обработки и документирования данных, полезной для электронщиков, имеющих недостаточный для самостоятельных разработок уровень знания микроконтроллерной техники.

8.1. Как связывать микроконтроллер и компьютер по каналу RS-232

Как уже отмечалось выше, мы рассматриваем простейшее измерительное устройство на базе микроконтроллера семейства x51, которое могло бы обмениваться информацией с персональным компьютером. При разработке в устройстве предполагалось реализовать измеритель напряжения, который в дальнейшем мог бы быть дополнен различными приставками, преобразующими другие непосредственно измеряемые физические величины в напряжение. Подобное устройство позволило бы легко проводить серии измерений, будучи управляемым компьютером, а также накапливать результаты и проводить их компьютерную обработку. Подвергнутое непринципиальным изменениям, оно смогло бы легко превратиться в систему дистанционного контроля и управления оборудованием или иными приборами и устройствами.

По сути, оно представляет собой цифровой вольтметр. На входе вольтметра стоит операционный усилитель, имеющий высокое входное сопротивление. За операционным усилителем следует АЦП, позволяющий оцифровать интересующее нас напряжение для последующей передачи в микроконтроллер. Микроконтроллер является главным управляющим звеном устройства, т. к. он считывает информацию из АЦП и общается с персональным компьютером по последовательному каналу. В устройство также входят преобразователи питания для выработки напряжений +5 В для цифровой части и ±10 В для операционного усилителя, а также микросхема преобразования уровней (логические 0 и 1 в +15 и в –15 В, и обратно) для обмена информацией по последовательному каналу типа RS-232. Схема устройства приведена на **Рис. 8.1.**

Характеристики устройства следующие:

- входное напряжение: 0...+5 В;
- входное сопротивление: 1 МОм;
- разрешение АЦП: 12 бит;
- скорость обмена не менее 9600 бод.

Примечания:

1. Число, посылаемое в компьютер, лежит в диапазоне 0...4095 (что соответствует разрядности АЦП), при этом 0 соответствует входному уровню 0 В, 4095 — уровню 5 В, зависимость линейная.

2. Скорость обмена информацией может быть выбрана как меньше 9600 бод, так и выше — до 115200 бод. На достаточно старых компьютерах (типа 386 и более ранних) верхний предел гораздо ниже — 19200 бод. Это связано с тем, что микросхемы последовательного порта, установленные в компьютерах, не были рассчитаны на более высокие скорости.

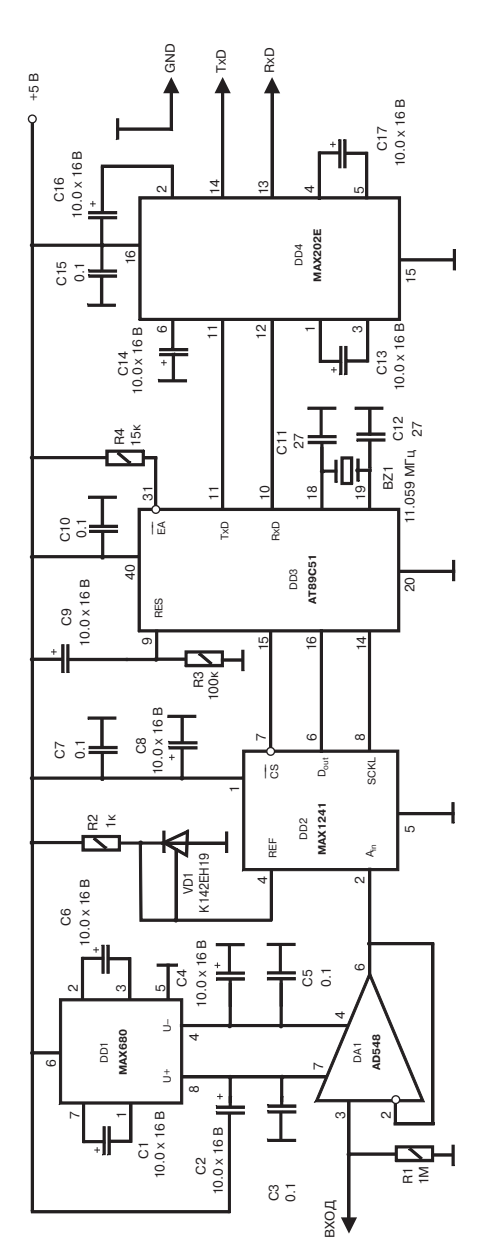


Рис. 8.1. Принципиальная схема устройства связи микроконтроллера с ПК

8.2. Описание микросхем

Перейдем к описанию незнакомых еще вам микросхем.

Операционные усилители обычно требуют подачи на них двухполярного питания (например, +10 и –10 В относительно общего провода). Радиолюбители, мало знакомые с современной элементной базой, используют обычно для получения такого напряжения трансформатор с двумя вторичными обмотками (или с одной, но с отводом от середины), два фильтрующих конденсатора, два стабилизатора и т. д. Однако, если у вас есть в распоряжении стабилизированное напряжение 5 В, а используемый операционник, требующий двухполярного питания, может обойтись всего ±7...10 В, потребляя при этом 1...2 мА, то упомянутые две обмотки и два стабилизатора не понадобятся. Достаточно использовать микросхему MAX680 фирмы Maxim. Отметим, что подобные микросхемы выпускает Linear Technology (LT1026) и ряд других известных фирм. На вход микросхемы подается напряжение $V_{вх}$ величиной от 3...5 до 6...10 В (в зависимости от типа), на выходах ее формируются напряжения, примерно равные $\pm 2V_{вх}$. Замечательно то, что во-первых, для формирования этих напряжений, помимо 8-выводных MAX680 или LT1026 нужно всего лишь 4 небольших электролитических конденсатора (**Рис. 8.1**), а во-вторых, при изменении входного напряжения удвоенные выходные напряжения изменяются синфазно, что практически не сказывается на выходном сигнале ОУ. Для более подробного ознакомления с подобными микросхемами рекомендуется обратиться к соответствующим фирменным описаниям.

Микросхема MAX1241 вам уже знакома по первым главам настоящей книги, поэтому здесь на ней мы останавливаться не будем, и перейдем к краткому описанию преобразователя уровней MAX202E.

Мало для кого является секретом, что в стандартной логике единица представляется уровнем напряжения от 2.4 до 5 В, а нуль — от 0 до 0.8 В. Однако начинающим может быть неизвестно, что при передаче по каналу RS-232 нуль и единица кодируются одинаковыми по величине (от 5 до 12 В), но разными по знаку сигналами. В рамках настоящей статьи не предполагается объяснять, почему принято делать так, а не иначе — мы ограничимся лишь константацией этого факта.

Коль скоро для передачи по RS-232 пятивольтовые логические сигналы должны быть преобразованы в сигналы другого уровня, мы должны предусмотреть в схеме соответствующие средства преобразования. Лет 10 назад для этой цели применялись специально разработанные каскады из трех-четырех транзисторов, пары диодов и почти десятка резисторов. Сейчас ситуация значительно изменилась — ведущие производители микросхем выпускают полностью законченные преобразователи, требующие минимального количества дополнительных элементов. К ним относятся

MAX202E от Maxim и полностью идентичная ей вплоть до цоколевки AD232 от Analog Devices. Обе микросхемы содержат преобразователь напряжения из +5 В в ±10 В, идентичный вышеописанному MAX680, и каскады, осуществляющие преобразование логических сигналов стандартного пятивольтового уровня в сигналы уровня по стандарту RS-232. Каждая из упомянутых микросхем содержит преобразователи логического уровня для двух приемников и двух передатчиков, из которых мы воспользуемся только одним приемопередающим каналом.

8.3. Режим работы микроконтроллера с последовательным каналом

Как известно (см., например, том 2, приложение 2), у микроконтроллеров семейства x51 существует четыре режима работы приемопередатчика. Нас будет интересовать режим 1, как наиболее простой и приемлемый.

Режим 1 характеризуется следующими параметрами:

- асинхронный обмен информацией;
- передаются 10 бит за один акт обмена (старт-бит (0), 8 бит данных и стоп-бит (1));
- скорость приема/передачи программируема и задается таймером.

Это удобный режим для программирования — требуется очень немного программного кода для настройки приемопередатчика и работы с ним. Хотя, если читателю захочется использовать и другие режимы работы, никто ему не помешает. Целью же настоящей главы является описание устройства, имеющего возможность общаться с персональным компьютером по одному из возможных протоколов.

Мы не будем приводить здесь описания того, как именно работает приемопередатчик. Эту информацию можно почерпнуть из упомянутого приложения или другой литературы.

8.4. Основные подпрограммы для микроконтроллера

Основными подпрограммами для микроконтроллера являются подпрограммы считывания данных из АЦП, инициализации приемопередатчика (в отечественной литературе его еще называют универсальным асинхронным приемопередатчиком или УАПП), подпрограммы приема байта и посылки байта.

Подпрограмма считывания данных из АЦП

```
GET_VOLT:
  SETB  DOUT              ;РАЗРЕШИЛИ ВВОД ДАННЫХ ИЗ ADC
  SETB  CS                ;УСТАНОВИЛИ НАЧАЛЬНОЕ СОСТОЯНИЕ ADC
```

```
   CLR   SCLK              ;УСТАНОВИЛИ НАЧАЛЬНОЕ СОСТОЯНИЕ ADC

   CLR   CS                ;СООБЩИЛИ О ЖЕЛАНИИ ПРОЧЕСТЬ ДАННЫЕ

   MUL   AB                ;4 МКС НА 12 МГЦ \
   MUL   AB                ;4 МКС            :
   MUL   AB                ;4 МКС            } ДОЖДАЛИСЬ КОНЦА
                           ;                   | ОЦИФРОВКИ
   MUL   AB                ;4 МКС             /
   MOV   R0, #12           ;СЧИТЫВАТЬ 12 БИТ
 GET_VC:
   SETB  SCLK              ;\
   NOP                     ; :
   NOP                     ; :
   CLR   SCLK              ; } СФОРМИРОВАЛИ ИМПУЛЬС ДЛЯ ЧТЕНИЯ БИТА
   NOP                     ; :
   NOP                     ;/
   MOV   C, DOUT           ;ПРОЧИТАЛИ БИТ
   MOV   A, R2             ;\
   RLC   A                 ; :
   MOV   R2, A             ; :
   MOV   A, R3             ; } ЗАДВИНУЛИ БИТ В СЛОВО
                           ; : РЕЗУЛЬТАТА - R3R2
   RLC   A                 ; :
   MOV   R3, A             ;/
   DJNZ  R0, GET_VC        ;ЗАЦИКЛИВАЕМСЯ
   ANL   A, #0Fh
   MOV   R3, A             ;ОЧИСТИЛИ СТАРШИЕ БИТЫ R3R2

   SETB  CS                ;БОЛЬШЕ НЕ ХОТИМ СЧИТЫВАТЬ
                            ;(ОСТАЛЬНЫЕ БИТЫ=0)
   MUL   AB                ;4 МКС НА 12 МГЦ \
   MUL   AB                ;4 МКС              :
   MUL   AB                ;4 МКС              :
   MUL   AB                ;4 МКС              } MIN ЗАДЕРЖКА
                           ;                   : ПЕРЕД СЛЕД.
   MUL   AB                ;4 МКС              :
   MUL   AB                ;4 МКС              :
   RET
```

Для того, чтобы настроить микроконтроллеры на обмен информацией по последовательному каналу, необходимо совершить следующие действия:

- запретить все прерывания, т. к. в их использовании необходимости нет;
- настроить таймер 1 на работу в режиме 2 с автоматической перезагрузкой содержимого — это нужно для задания скорости обмена информацией и поддержания ее постоянной;
- загрузить в счетчик таймера начальные значения;
- запустить таймер 1.

Ниже приведен пример программы, рассчитанной на скорость обмена 9600 бит/с для кварцевого резонатора с резонансной частотой равной 11.059 МГц.

Подпрограмма настройки микроконтроллера для обмена информацией по последовательному каналу

```
SERINIT:
  MOV   IE, #0            ;Запретить все прерывания
  MOV   TMOD, #20h        ;Установить режим 2 для таймера 1
  MOV   TH1, #REL96       ;Значение для автоперезагрузки счетчика
  MOV   TL1, #REL96       ;Начальное значение счетчика для 9600 бит/с
  ;при SMOD=0
  ANL   PCON, #7Fh        ;Очистили SMOD
  MOV   SCON, #50h        ;Режим для 8 бит данных и скорости передачи,
                          ;зависящей от таймера
  SETB  TR1               ;Старт таймера/счетчика 1
  RET
```

Примечание. `REL96` — константа, равная 0FDh.

Эта подпрограмма вызывается самой первой в основной программе микроконтроллера. В принципе, ее можно даже и не оформлять как подпрограмму, а просто поставить приведенный фрагмент сразу после команд настройки портов и стека.

Подпрограммы приема и посылки байта по последовательному каналу очень просты.

Считывать байт из порта ввода/вывода SBUF можно только при установленном бите RI регистра управления/статуса SCON, сигнализирующего о наличии байта в буфере приема. После считывания этого байта бит RI необходимо сбросить.

После записи байта в порт ввода/вывода нужно дождаться установления бита TI, который будет сигнализировать окончание посылки байта в линию. Затем бит TI так же будет нужно сбросить.

Подпрограмма приема байта

```
GETCH:
  JNB   RI, GETCH
  MOV   A, SBUF
  CLR   RI
  RET
```

Подпрограмма посылки байта

```
PUTCH:
  MOV   SBUF, A
SEND:
  JNB   TI, SEND
  CLR   TI
  RET
```

Должен также отметить, что никаких средств для обнаружения ошибок ввода/вывода микроконтроллер не имеет.

Для того, чтобы организовать проверку программно-аппаратным образом, можно расширить количество линий ввода/вывода, по которым будут передаваться дополнительные сигналы, и по ним можно будет определять состояния, в которых находятся участники диалога, а также выявлять ошибки.

Можно повысить надежность приема/передачи информации и другим путем. Например, передавать с восемью битами данных еще один бит — бит четности, вычисляющийся аналогично флагу паритета в слове состояния программы (бит 0 PSW), только вычисляться он должен для передаваемого или принятого байта. После принятия байта и бита четности необходимо сравнить их на соответствие друг другу. Если они не соответствуют, значит имела место ошибка ввода/вывода. Для передачи дополнительного 9-го информационного бита нужно использовать режимы 2 или 3 работы таймера/счетчика. В описываемой реализации мы не будем усложнять себе жизнь, и ограничимся передачей восьми информационных байт.

8.5. Общая программа для микроконтроллера. Диаграмма состояний устройства

Общая программа для микроконтроллеров базируется на нижеописанном алгоритме. Алгоритм довольно не простой, т. к. все же нужно каким-то образом выявлять ошибки ввода/вывода, хотя бы программным методом, и соответствующим образом реагировать на их появление.

Для большей наглядности к алгоритму, описанному обычными словами, прилагается картинка — так называемая диаграмма состояний устройства (**Рис. 8.2**).

На диаграмме приведены четыре основных состояния устройства с точки зрения обмена информацией с ПК.

Заранее оговорим тот факт, что наш микроконтроллер является ведомым, а персональный компьютер является ведущим при обмене данными. Иными словами, устройство само по себе, без приказа от ПК, ничего делать не должно, а должно всецело подчиняться управляющему компьютеру. Персональный компьютер выбран ведущим по той простой причине, что он обладает

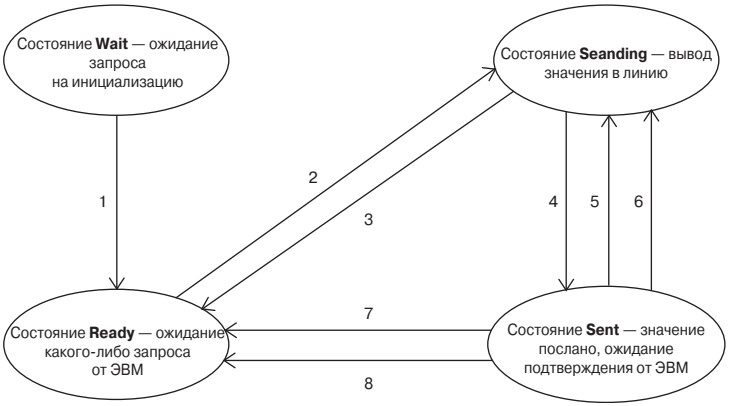


Рис. 8.2. Диаграмма состояний микроконтроллера

большей мощностью и способен без особенных проблем управлять устройством, плюс ко всему он может дать пользователю больше сервисных функций.

Состояние первое — **Wait**. В этом состоянии устройство оказывается сразу же после включения питающего напряжения. Здесь оно ожидает от компьютера запроса на инициализацию, который выражается в посылке компьютером символа NUL. Устройство же в свою очередь должно в ответ на полученный запрос включить и настроить, если требуется, дополнительные модули и ресурсы, а затем, если все прошло нормально, послать в ЭВМ символ ACK, в случае же ошибки нужно послать NAK. Таким образом, происходит первое «общение» двух «собеседников». Если хотите, они должны «обменяться приветствиями» или «пожать друг другу руки».

При удачной инициализации устройства с последующей посылкой символа ACK оно автоматически переходит в следующее состояние. Этот переход обозначен стрелкой 1 на диаграмме.

Состояние **Ready**. В этом состоянии наш микроконтроллер ожидает запроса ПК на посылку измеренного значения, считанного с АЦП. Запросом является символ XON. По принятии этого символа, устройство переходит в новое состояние — **Sending**. Переходу соответствует стрелка 2.

Состояние **Sending**. Попадая сюда, микроконтроллер считывает двоичное 12-битное число из АЦП ранее указанным методом и посылает частями в ПК. В данной реализации происходит преобразование двоичного числа в трехсимвольный шестнадцатеричный эквивалент, например, в 1FF для десятичного числа 511. Сначала посылается 1, затем F и еще F.

По окончании передачи значения в компьютер, микроконтроллер переходит в следующее состояние по стрелке 4.

Состояние **Sent**. Это состояние является последним и, как бы, замыкает круг единичного акта общения устройства с персоналкой. Здесь ожидается подтверждение от компьютера того, что он принял правильно значение, которое было ему адресовано.

Тут возможными являются несколько вариантов ответа ПК на посланное число: он может ответить об успешном приеме символом XOFF, который будет означать, что больше пока не требуется других значений, а может ответить символом XON, что будет означать, что нужно еще одно значение. Если принят XOFF, то устройство возвращается в состояние готовности **Ready** (переход 7 на диаграмме). Если же принят символ XON, то устройство опять оказывается в состоянии **Sending** (переход 5) и повторяет считывание из АЦП с последующей передачей числа в линию. Не рассмотренным оказался лишь тот случай, когда ПК не понравилось то, что он получил — например, вместо символов диапазона 0...9, A...F, он получил нечто иное, например, «G» или «,». В этой ситуации он посылает нашему устройству символ NAK, который должен трактоваться как запрос на повторную посылку последнего значения, что и делается — устройство опять переводится в состояние **Sending** (переход по стрелке 6).

Остались не описанными переходы, обозначенные стрелками 3 и 8 диаграммы. Что же это за переходы между состояниями? Какая тайна скрывается за этими простыми стрелками? А тайны на самом деле здесь никакой и нет. Если компьютер обнаружит серьезную ошибку ввода/вывода или ему понадобится прекратить обмен с устройством, то он просто пошлет инициализационный NUL, по котором произойдет инициализация устройства, и оно окажется в состоянии готовности **Ready**.

Таким образом, в каком бы состоянии не находилось наше устройство, оно обязано ответить на инициализационный запрос таким же образом, как и при первичной инициализации (см. пункт «Состояние **Wait**»). Если же микроконтроллер получил какой-то неожиданный или неверный символ или запрос, то он всегда должен ответить на него символом NAK.

Описанная стратегия является выигрышной, т. к. при подобной организации программы для устройства убивается сразу несколько зайцев — во-первых, микроконтроллер и ПК не будут играть в испорченный телефон, а во-вторых, они смогут просто и эффективно «общаться» друг с другом.

Ниже приведена полная программа для микроконтроллера устройства, изображенного на **Рис. 8.1**.

Программа для микроконтроллера цифрового вольтметра

```
IE      .EQU   0A8h
SCON    .EQU   098h
SBUF    .EQU   099h
TI      .EQU   099h
RI      .EQU   098h
```

```
TMOD     .EQU   089h
TL1      .EQU   08Bh
TH1      .EQU   08Dh
ET1      .EQU   0ABh
TR1      .EQU   08Eh
PCON     .EQU   087h
REL96    .EQU   0FDh

P3.0     .EQU   0B0h
P3.1     .EQU   0B1h
P3.2     .EQU   0B2h
P3.3     .EQU   0B3h
P3.4     .EQU   0B4h
P3.5     .EQU   0B5h
P3.6     .EQU   0B6h
P3.7     .EQU   0B7h

SCLK     .EQU   P3.4
CS       .EQU   P3.5
DOUT     .EQU   P3.6

NUL      .EQU   0
ACK      .EQU   6
XON      .EQU   17
XOFF     .EQU   19
NAK      .EQU   21

PHWAIT   .EQU   0
PHREADY  .EQU   1
PHSEND   .EQU   2
PHSENT   .EQU   3

.ORG     0
START:   MOV    IE, #0
         MOV    TMOD, #20h
         MOV    TH1, #REL96
         MOV    TL1, #REL96
         ANL    PCON, #7Fh
         CLR    ET1
         MOV    SCON, #50h
         SETB   TR1

         MOV    R7, #PHWAIT     ;PHASE
CYCLE:
         JB     RI, GOT_CHAR
         AJMP   NO_CHAR
GOT_CHAR:
         MOV    A, SBUF
         CLR    RI
         CJNE   A, #NUL, CASE
         MOV    A, #ACK
         ACALL  PUTCH
         MOV    R7, #PHREADY
         AJMP   CYCLE        ;INITIALIZED
CASE:
         CJNE   R7, #PHWAIT, CASE2
         MOV    A, #NAK
         ACALL  PUTCH
         AJMP   ENDCASE      ;ERROR
CASE2:
         CJNE   R7, #PHREADY, CASE3
```

```
        CJNE   A, #XON, CASE2E
LNEW:
        ACALL  GET_VOLT
;       MOV    R3, #0Fh
;       MOV    R2, #0FFh

        MOV    DPTR, #HEXTAB
        MOV    R1, #10
        MOV    A, R3
        ANL    A, #0Fh
        MOVC   A, @A+DPTR
        MOV    @R1, A
        INC    R1
        MOV    A, R2
        SWAP   A
        ANL    A, #0Fh
        MOVC   A, @A+DPTR
        MOV    @R1, A
        INC    R1
        MOV    A, R2
        ANL    A, #0Fh
        MOVC   A, @A+DPTR
        MOV    @R1, A
LOLD:
        MOV    R7, #PHSEND
        MOV    R1, #10      ;INDEX=0 (FOR THE HEX VALUE)
        AJMP   ENDCASE      ;GOT REQUEST TO SEND, SENDING
CASE2E:
        MOV    A, #NAK
        ACALL  PUTCH
        AJMP   ENDCASE      ;ERROR
CASE3:
        CJNE   R7, #PHSEND, CASE4
        MOV    A, #NAK
        ACALL  PUTCH
        AJMP   ENDCASE      ;ERROR
CASE4:
        CJNE   R7, #PHSENT, ENDCASE
        CJNE   A, #XON, CASE41
        AJMP   LNEW         ;»MORE VALUES NEEDED» REQUEST
CASE41:
        CJNE   A, #XOFF, CASE42
        MOV    R7, #PHREADY
        AJMP   ENDCASE      ;VALUE SENT SUCCESSFULY
CASE42:
        CJNE   A, #NAK, CASE4E
        AJMP   LOLD         ;»REPEAT» REQUEST
CASE4E:
        MOV    A, #NAK
        ACALL  PUTCH
        AJMP   ENDCASE      ;ERROR
ENDCASE:
        AJMP   ENDCYC
NO_CHAR:
        CJNE   R7, #PHSEND, ENDCYC
;       MOV    A, #'F'
        MOV    A, @R1
        ACALL  PUTCH        ;INDEXED CHARACTER SENT
        INC    R1           ;NEXT INDEX
        CJNE   R1, #13, ENDCYC
        MOV    R7, #PHSENT  ;VALUE SENT
```

```
ENDCYC:
        AJMP   CYCLE

;———————— ПОДПРОГРАММЫ ————————-

PUTCH:  MOV    SBUF, A
        JNB    TI, $
        CLR    TI
        RET

PUTSTR: CLR    A
        MOVC   A, @A+DPTR
        JZ     PUTSTR_E
        ACALL  PUTCH
        INC    DPTR
        SJMP   PUTSTR
PUTSTR_E:RET

GETCH:  JNB    RI, $
        MOV    A, SBUF
        CLR    RI
        RET

;ПОДПРОГРАММЫ ОБМЕНА С MAX 1241 (12-bit serial ADC) ---
GET_VOLT:
        SETB   DOUT         ;РАЗРЕШИЛИ ВВОД ДАННЫХ ИЗ ADC
        SETB   CS           ;УСТАНОВИЛИ НАЧАЛЬНОЕ СОСТОЯНИЕ ADC
        CLR    SCLK         ;УСТАНОВИЛИ НАЧАЛЬНОЕ СОСТОЯНИЕ ADC

        CLR    CS           ;СООБЩИЛИ О ЖЕЛАНИИ ПРОЧЕСТЬ ДАННЫЕ

        MUL    AB           ;4 МКС НА 12 МГЦ \
        MUL    AB           ;4 МКС            :
        MUL    AB           ;4 МКС            } ДОЖДАЛИСЬ КОНЦА ОЦИФРОВКИ
        MUL    AB           ;4 МКС           /
        MOV    R0, #12      ;СЧИТЫВАТЬ 12 БИТ
GET_VC:
        SETB   SCLK         ;\
        NOP                 ; :
        NOP                 ; :
        CLR    SCLK         ; } СФОРМИРОВАЛИ ИМПУЛЬС ДЛЯ ЧТЕНИЯ БИТА
        NOP                 ; :
        NOP                 ;/
        MOV    C,DOUT       ;ПРОЧИТАЛИ БИТ
        MOV    A, R2        ;\
        RLC    A            ; :
        MOV    R2, A        ; :
        MOV    A, R3        ; } ЗАДВИНУЛИ БИТ В СЛОВО РЕЗУЛЬТАТА — R3R2
        RLC    A            ; :
        MOV    R3, A        ;/
        DJNZ   R0, GET_VC   ;ЗАЦИКЛИВАЕМСЯ
        ANL    A, #0Fh
        MOV    R3, A        ;ОЧИСТИЛИ СТАРШИЕ БИТЫ R3R2

        SETB   CS           ;БОЛЬШЕ НЕ ХОТИМ СЧИТЫВАТЬ (ОСТАЛЬНЫЕ БИТЫ=0)
        MUL    AB           ;4 МКС НА 12 МГЦ \
        MUL    AB           ;4 МКС            :
        MUL    AB           ;4 МКС            :
        MUL    AB           ;4 МКС            } MIN ЗАДЕРЖКА ПЕРЕД СЛЕД.
        MUL    AB           ;4 МКС            :
        MUL    AB           ;4 МКС           /
```

```
        RET
;——--- КОНЕЦ ПОДПРОГРАММ ОБМЕНА С MAX 1241 (12-bit serial ADC) ——-

;—— ТЕКСТОВЫЕ СООБЩЕНИЯ ——-
HEXTAB  .TEXT   «0123456789ABCDEF»

        .ORG   4096

        .END
```

8.6. Общая программа для ПК. Диаграмма состояний ПК

Принципиально общая программа для компьютера не будет ни в чем отличаться от используемой в микроконтроллере. Алгоритм будет аналогичным, похожей будет и диаграмма состояний. Вот, собственно, и новая диаграмма, иллюстрирующая процесс получения одного значения из устройства (**Рис. 8.3**):

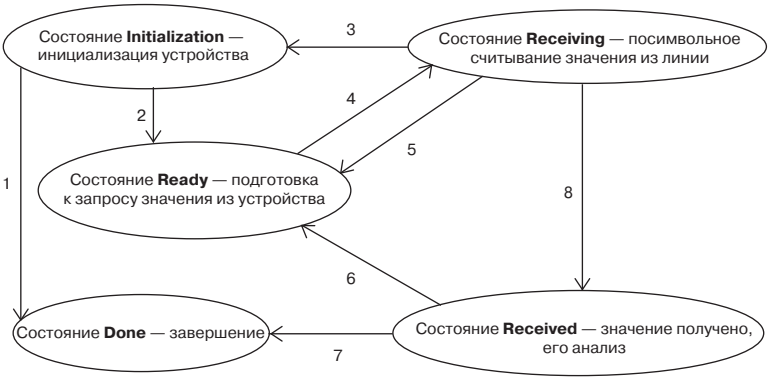


Рис. 8.3. Диаграмма состояний ПК

Состояние **Initialization**. Сюда компьютер попадает, когда пользователь нажимает на его клавиатуре клавишу, соответствующую принятию единственного значения. В этом состоянии компьютер посылает символ инициализации NUL в устройство и ожидает ответа не него символом ACK или NAK. Если был получен ACK в ответ, значит инициализация прошла нормально и можно продолжать работу — перейти в следующее состояние по стрелке 2 на диаграмме. В случае получения NAK работа должна прекратиться, и компьютеру следует перейти в заключительное состояние **Done** по стрелке 1.

Состояние **Ready**. В этом состоянии компьютер подготавливается к приему символов, из которых будет состоять запрошенное из микроконтроллера значение. Запросов на посылку значения существует два.

Первый — это обычный запрос значения, ему соответствует символ XON. Второй же запрос — это запрос на повторную посылку последнего значения. Это необходимо в том случае, если значение не было принято полностью за какое-то объективное время или были приняты неверные символы, не попадающие в диапазоны от 0 до 9 и от A до F. Для запроса на повторную посылку отводится символ NAK.

Далее после подготовки к приему символов значения происходит один из двух вышеуказанных запросов к нашему устройству, затем компьютер переходит по стрелке 4 в состояние приема значения.

Состояние **Receiving**. Здесь ПК просто считывает три символа значения, измеренного и преобразованного с помощью АЦП. Как было сказано ранее, существует некоторое объективное время ожидания символа компьютером. Если символ не был считан за это время, то такая ситуация интерпретируется как ошибочная, т. е. считается, что имела место ошибка ввода/вывода.

Кстати говоря, при довольно-таки высоких скоростях обмена информацией (больших 19200 бит/с) или при работе в операционной системе MS-Windows (любой версии) часто бывает, что компьютер из полагающихся ему трех символов принимает только два, а иногда и того меньше — один. Чтобы компьютер «не висел», ожидая бесконечно долго недостающего или пропущенного символа — и вводится некоторое время, ограничивающее его ожидание. К сожалению, эти пропуски аппаратным методом никак не выявляются.

В данной реализации определено два типа времени ожидания, которые могут быть заданы пользователем с клавиатуры. Первый тип — это время ожидания одного из трех символов, он позволяет устройству спокойно, никуда не торопясь, измерить, оцифровать необходимое нам число и преобразовать его в символьный эквивалент. Второй тип — это временной лимит на посылку второго и третьего символов.

Перейдем теперь к возможным переходам из состояния **Receiving** в другие состояния.

Если так и не были приняты все три символа значения за отведенное время, то компьютер должен попросить наше устройство послать ему значение повторно — этой ситуации соответствует переход по стрелке 5, т. е. компьютер делает запрос символом NAK и переходит обратно в состояние **Ready**.

Если в процессе приема компьютером была зафиксирована ошибка ввода/вывода (а у ПК есть такая возможность — достаточно проанализировать регистр состояния последовательного порта), то лучше привести и

компьютер, и микроконтроллер в исходное состояние, т. е. повторить инициализацию. Поэтому на диаграмме также присутствует и стрелка 3.

И, наконец, если компьютер получил от устройства все три символа, то он переходит в состояние анализа полученного значения — в состояние **Received** по стрелке 8.

Состояние **Received**. Тут компьютер проверяет полученные символы на принадлежность диапазону 0...9, A...F. Если хотя бы один из них не попадает в указанный диапазон, то ПК в праве потребовать от микроконтроллера повтора посылки, что он и делает, посылая NAK. Затем он возвращается в состояние **Ready** (стрелка 6).

Если же символы удовлетворяют наложенному ограничению на диапазон их значений, то компьютер подтверждает удачный прием значения, посылая в линию символ XOFF, и переходит на заключительную стадию приема значения. Это состояние **Done**, и переход в него обозначен стрелкой 7. В этом состоянии можно сохранить или отобразить полученное от устройства значение.

8.7. Основные подпрограммы для ПК

В этом подразделе настоящей главы вы не найдете ничего принципиально нового — здесь будут приведены и описаны подпрограммы для обмена информацией по последовательному каналу с точки зрения персонального компьютера.

Прежде чем настроить последовательный порт ПК на обмен с заданной скоростью и определить формат информационных пакетов, нужно выяснить, имеются ли на компьютере последовательные порты, и если имеются, то вычислить их адреса. Лучше всего сделать определение наличия COM-портов и их адресов универсальным, чтобы иметь возможность переносить программу с одного компьютера на другой, ничего в ней не меняя.

Информацию о последовательных портах, установленных на компьютере, можно почерпнуть из области данных BIOS'а. Напомню, что этой области отводится сегмент 40h памяти ПК.

И как раз то, что нас так интересует, расположено в первых четырех словах вышеуказанной области. В этих четырех словах памяти содержатся базовые адреса COM-портов для COM1, COM2, COM3 и COM4. Причем, если какой-то из этих портов отсутствует в компьютере, то в соответствующем слове содержится 0.

Приведу подпрограмму, демонстрирующую определение существующих коммуникационных портов и их базовых адресов:

Подпрограмма определения существующих коммуникационных портов и их базовых адресов

```
Procedure Show_COMs;
Var
  I : Integer;
  E : Boolean;
  Addr : Word;
Begin
  E := False;
  For I := 0 to 3 do Begin
    Addr := MemW[$40:I*2];
    If Addr <> 0 then Begin
      E := True;
      { Выводит порт и его базовый адрес }
      WriteLn ("COM", I+1, " - ", Addr);
    End;
  End;
  { Выводит сообщение, если не было обнаружено ни одного COM-порта }
  If not E then WriteLn ("No COM-ports available.");
End;
```

Примечание. Здесь был использован компилятор Borland Pascal 7.0.

Теперь несколько слов о портах последовательных каналов. Как уже отмечалось, каждый порт компьютера имеет свой базовый адрес. Это значит то, что для обмена информацией одного единственного однобайтного порта недостаточно, и вспомогательные порты располагаются сразу же за портом с базовым адресом. Т. е. если порт имеет базовый адрес 3F8h, то первый дополнительный будет иметь адрес 3F9h.

Процедура настройки порта

```
Procedure OpenCom (Base : Word; BaudRate : Word; Config : Byte);
Begin
{ Ожидание завершения обмена данными - на всякий случай. }
  While Port[Base+5] and $60 <> $60 do;
{ Настройка. }
  Port[Base+3] := $80;
  Port[Base+1] := BaudRate shr 8;
  Port[Base+0] := BaudRate and $FF;
  Port[Base+3] := Config;
  Port[Base+4] := 0;
  Port[Base+1] := 0;
End;
```

Здесь Base — базовый адрес COM-порта; BaudRate — константа, задающая скорость обмена, а в константе Config содержится информация о количестве бит, из которых состоят непосредственно данные, количестве стоповых бит и режиме проверки четности.

Константы, необходимые для функционирования порта

```
Const
{ Константы, определяющие скорость обмена. См константу BaudRate. }
  B_110     = 1040;
  B_150     =  768;
  B_300     =  384;
  B_600     =  192;
  B_1200    =   96;
  B_2400    =   48;
  B_4800    =   24;
  B_9600    =   12;
  B_19200   =    6;
  B_38400   =    3;
  B_57600   =    2;
  B_115200  =    1;

{ Константы, определяющие формат единичного информационного пакета }
{ и тип контроля четности. (константа Config.) }
  Bits_5      =    0;   { 5 бит данных }
  Bits_6      =    1;   { 6 бит данных }
  Bits_7      =    2;   { 7 бит данных }
  Bits_8      =    3;   { 8 бит данных - целый байт }
  Stops_1     =    0;   { 1 стоповый бит }
  Stops_2     =    4;   { 2 стоповых бита }
  Parity_No   =    0;   { нет контроля четности }
  Parity_Even =  $18;   { контроль на четность }
  Parity_Odd  =    8;   { контроль на нечетность }
```

В нашем случае используется скорость обмена 9600 бод, целый байт данных, 1 стоповый бит, и отсутствует контроль четности. Значит, нам необходимо вызывать процедуру следующим образом (для порта с базовым адресом 3F8h):

```
OpenCom ($3F8, BB_9600, Bits_8+Stops_1+Parity_No);
```

Подпрограммы приема и посылки байта выглядят следующим образом.

Процедура посылки байта

```
Procedure SendChar (Base : Word; Value : Char);
Begin
{ Ожидание момента конца посылки предыдущего символа. }
  While Port[Base+5] and $20 = 0 do;
{ Посылка символа. }
  Port[Base] := Byte(Value);
End;
```

Здесь Base — базовый адрес коммуникационного порта, а Value — символ, который требуется послать.

Процедура приема байта

```
Function ReceiveChar (Base : Word; Var Value : Char;
                      Var Status : Word) : Boolean;
Begin
  Status := Port[Base+5];
{ Если ошибка ввода/вывода (предыдущий байт не был вовремя считан, }
{ ошибка четности или ошибка синхронизации }
  If (Status and $1E <> 0) or
{ или если в буфере еще нет символа, }
     (Status and 1 = 0) then
{ тогда вернуть False. }
    ReceiveChar := False
  Else Begin
{ Считать байт из буфера ввода. }
    Value := Char(Port[Base]);
{ Вернуть True. }
    ReceiveChar := True
  End;
End;
```

Смысл параметра Base очевиден, а прочитанный байт (символ) записывается в переменную Value.

Функция возвращает True, если байт был успешно принят, в противном случае возвращает False. Чтобы определить причину непринятия символа, нужно проанализировать содержимое байта статуса коммуникационного порта, который записывается в переменную Status. Если результат логического и (And) этого байта со значением 1Eh даст отличное от нуля число, значит имела место ошибка ввода/вывода, в противном случае буфер ввода был пуст, что означает, что ошибок не было.

А теперь небольшое примечание. Читатели, знающие о возможности использования прерываний, сигнализирующих о вновь поступившем или только что отправленном символе, могут несколько удивиться тому примитивному методу, который был использован в нашей разработке. Действительно, почему бы не использовать столь удобный режим работы микроконтроллера и ПК? Ответ на этот вопрос довольно прост. Во-первых, данная разработка рассчитана для людей, которые сталкиваются с освещаемым вопросом впервые, и им нужно показать наиболее простой метод решения задачи. И во-вторых, как показала практика, коммуникационные порты ПК неустойчиво работают в режиме генерирования прерывания по поступившему и посланному символам.

8.8. Пользовательское описание программы для ПК

Мы рассмотрели фрагменты компьютерной программы, предназначенной для обмена информацией с микроконтроллером. Однако, помимо этих фрагментов, программа должна иметь еще и пользовательский интерфейс, дабы мы могли управлять (с клавиатуры или при помощи мышки) процессом обмена, читать на экране монитора результаты измерения при помощи АЦП и запоминать их в файле на винчестере. Написание такой программы — непростая проблема для тех, кто не занимался подобным программированием. Описывать, как вы должны составлять подобную программу, не входит в нашу задачу. Поэтому, дабы не лишаться возможности использовать описанный программно-аппаратный измерительный комплекс в полном объеме, мы предлагаем вам готовую пользовательскую программу, реализующую описанные (и некоторые другие) функции. Ниже приводится краткое описание этой программы и порядок работы с ней (см. **Рис. 8.4...Рис. 8.8**).

Рис. 8.4. Окно программы

Строка **Status** отображает состояние программы:

E (Error) — ошибка обмена информацией;

I (Initialization) — инициализация устройства (в начале работы и после ошибки);

S (Stopped) — обмен остановлен;

Строка **Value** отображает принятое значение из устройства: значение лежит в диапазоне 0...4095 (12-битный АЦП устройства).

Строка **Evaluated** отображает вычисленное по указанной в опциях формуле значение функции, аргументом которой является значение, принятое из устройства (см. строку Value).

Кнопка **Get 1** запрашивает одно значение из устройства.

Кнопка **Get series** запрашивает серию значений из устройства. Параметры серии см. в опциях программы.

Кнопка **Stop** останавливает обмен информацией (для нажатия этой виртуальной кнопки нужно пользоваться клавишами **Esc** и **S** клавиатуры — специфика программы).

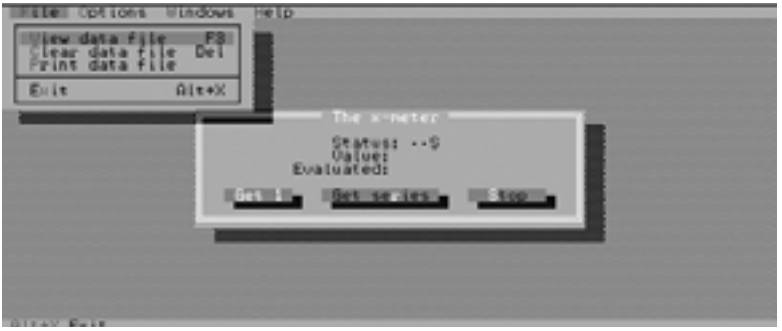


Рис. 8.5. Меню Options программы

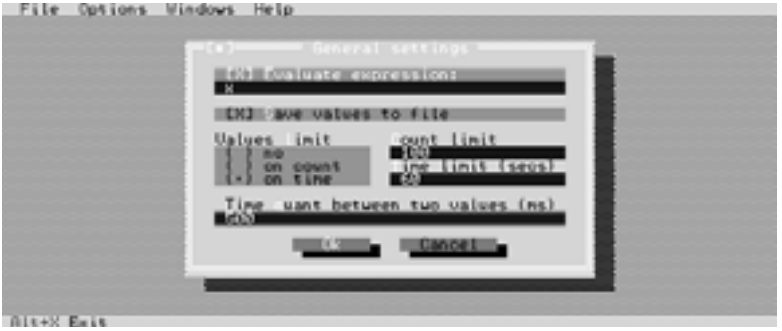


Рис. 8.6. Меню General settings

[ ] Evaluate expression — вычисление нижеследующего выражения, где параметром **x** является принятое из устройства значение; помимо аргумента **x** допустимы числа, круглые скобки, знаки сложения, вычитания, умножения, деления и возведения в степень.

[ ] Save values to file — сохранять значения в текстовый файл **SERCTRL2.DAT.**

Values limit — ограничение на длину серии значений, запрашиваемую из устройства.

no — нет ограничений, **on count** — ограничение по количеству, **on time** — ограничение по времени.

Count limit — предел серии значений в штуках.

Time limit — предел серии по времени в секундах.

Time quant between two values — временной промежуток между запросами значений в серии (в миллисекундах).

Рис. 8.7. Меню Port settings

Port — список для выбора последовательного порта для обмена информацией с устройством.

Baud rate — скорость обмена в бодах, на данный момент устройство рассчитано на 9600 бод.

Timeout for 1-st char — максимальное время ожидания из устройства первого символа, соответствующего трехсимвольной шестнадцатеричной нотации принимаемого значения (в миллисекундах).

Timeout for next chars — максимальное время ожидания из устройства следующих за первым символов (в миллисекундах).

Рис. 8.8. Меню File программы

View data file — показать содержимое файла **SERCTRL2.DAT**, в который были сохранены значения, считанные из устройства.

Clear data file — очистить файл **SERCTRL2.DAT** от значений.

8.9. Заключение

Итак, мы завершили основную часть программы нашего знакомства с микроконтроллерами. Позади восемь глав, в которых вы последовательно знакомились с внешними и внутренними аппаратными ресурсами микроконтроллера — памятью программ и данных, портами ввода/вывода, регистрами, цепями и сигналами управления и сброса, таймерами-счетчиками, системой прерываний и встроенным приемопередатчиком. Вы ознакомились со всеми командами, доступными рассматриваемым микроконтроллерам, рассмотрели многочисленные примеры их использования. По ходу вы также ознакомились с ассемблером, получили некоторые навыки работы с ним, а также ознакомились с тем, как записывать программу в микроконтроллер при помощи внешнего программатора.

Помимо этого я показал вам, как организовываются подпрограммы работы с параллельными и последовательными АЦП, индикаторами различных типов, подпрограммы обслуживания кнопок управления. Вы познакомились с тем, как использовать встроенные таймеры-счетчики, а также задействовать систему прерываний. После этого я на четырех примерах показал вам, как организованы полностью законченные программы. Конечно, как уже отмечалось, четыре примера — это немного, различных вариантов использования микроконтроллеров — десятки тысяч, и сколько примеров не приведи, все равно что-то окажется неохваченным. Поэтому задача привести десяток или два десятка примеров не ставилась, ибо принципиально это мало что изменило бы. С другой стороны, у вас перед глазами есть эти четыре примера, и вы можете, опираясь на них, осуществить свои первые разработки.

Материал, с которым вы ознакомились, очень обширен и разнообразен. Это неудивительно — чтобы «стать на ты» с микроконтроллерной техникой, вы должны освоить некоторые аспекты и навыки программирования, научиться рисовать блок-схемы программ, разбивать поставленную задачу на простые фрагменты, реализуемые при помощи подпрограмм, уметь формировать импульсы и последовательности импульсов. И все это нужно осваивать одновременно, в чем и состоит сложность пути, по которому вы сейчас идете. Но вы прошли ключевую его часть, и если она оказалась для вас посильной, то и со всем остальным вы безусловно справитесь.

Когда писалась эта книга, мне постоянно приходилось выбирать, какие материалы абсолютно необходимы, а что можно для начала и опустить. Но увы, опущенного оказалось довольно много, по крайней мере такого, что вам понадобится уже после первого знакомства с микроконтроллерами. Дабы не заставлять вас искать эти материалы в различных книгах и журналах, я собрал их во 2-ом томе, в приложениях. Знакомство с тем, что там собрано, осуществляйте по мере необходимости — эти материалы

не требуют систематического и последовательного изучения. Что-то из них вам может понадобиться сразу, что-то в отдаленном будущем, а что-то, возможно, именно вам вообще не понадобится. Но на мой взгляд все, что там приведено, должно дать ответ на любой вопрос, который может возникнуть у начинающих, и поэтому пользуйтесь приведенной информацией по мере необходимости, а также для расширения кругозора.

Литература

1. А. В. Боборыкин, Г. П. Липовецкий и др. Однокристальные микроЭВМ. — М.: МИКАП, 1994. — 400 с: илл.

2. В. Я. Нерода, В. Э. Торбинский, Е. Л. Шлыков. Однокристальные микроЭВМ MCS-51. Архитектура. — М.: изд. Диджитал Компонентс, 1995. — 164 с: илл.

3. Р. Токхайм. Микропроцессоры: курс и упражнения. Пер. с англ. М.: Энергоатомиздат, 1988. — 336 с: илл.

4. В. В. Сташин, А. В. Урусов, О. П. Мологонцева. Проектирование цифровых устройств на однокристальных микроконтроллерах.—М.: Энергоатомиздат, 1990. — 224 с: илл.

5. Дж. Коффрон. Технические средства микропроцессорных систем: Практический курс. — М.: Мир, 1983. — 344 с: илл.

6. В. В. Гребнев. Микроконтроллеры семейства AT89 фирмы Atmel (http://www.atmel.ru).

7. Ю-Чжен Лю, Г. Гибсон. Микропроцессоры семейства 8086/8088. Архитектура, программирование и проектирование микропроцессорных систем. Пер. с англ. В. Л. Григорьева. — М.: Радио и связь, 1987.

8. С. Т. Хвощ, Н. Н. Варлинский, Е. А. Попов. Микропроцессоры и микроЭВМ в системах автоматического управления. Справочник под общей редакцией С. Т. Хвощ. — Л.: Машиностроение, 1987.

9. С. Д. Погорелый, Т. Ф. Слободянюк. Программное обеспечение микропроцессорных систем. Справочник. — К.:Техніка, 1985.

10. Л. Левенталь, У. Сэйвилл. Программирование на языке ассемблера для микропроцессоров 8080 и 8085. — М.: Радио и связь, 1987.— 448 с: илл.

11. А. Л. Гуртовцев, С. В. Гудыменко. Программы для микропроцессоров: Справ. пособие. — М.: Выш. шк., 1989. — 352 с: илл.

ФРУНЗЕ А. В.

Микроконтроллеры? Это же просто!

Главный редактор *В. М. Халикеев*

Ответственный редактор *И. А. Корабельникова*
Научный редактор *С. В. Турецкий*
Технический редактор *А. И. Михалченков*
Верстальщик *А. В. Кулакова*
График *И. С. Кайнова*

Формат 60x90/16. Бумага офсетная.
Гарнитура «NewtonC». Печать офсетная.
Объем 19,5 п. л. Усл. п. л. 19,5.
Тираж 2500 экз. Код FR01. Заказ №

Издательский дом «Додэка-XXI»

105318 Москва, а/я 70
Тел./факс: (495) 366-24-29, 366-09-22
E-mail: books@dodeca.ru; red@dodeca.ru

Отпечатано с готовых диапозитивов в ОАО «Щербинская типография»
117623 Москва, ул. Типографская, д. 10

ISBN 978-5-94120-140-2

© Фрунзе А. В.
© Издательский дом «Додэка-XXI»